

Security Engineering

Build security in: architecture, hardening, identity, cloud, DevSecOps, automation and governance.

18 modules

107 lessons

77 simulated labs

18 module checkpoints

Best for: Sysadmins and developers moving into security engineering or DevSecOps.

Needs: Security 101, plus Scripting Basics from Pre-Security.

Contents

STAGE 1: DESIGN AND HARDENING

ENG-01	Secure Architecture and Threat Modeling	p. 3
ENG-02	System Hardening	p. 22
ENG-03	Identity and Access Engineering	p. 41
ENG-04	Network Security Engineering	p. 63
ENG-05	Cryptography in Practice	p. 85

STAGE 2: SECURE SOFTWARE AND PLATFORMS

ENG-06	Application Security	p. 104
ENG-07	DevSecOps and CI/CD Security	p. 129
ENG-08	Cloud Security Engineering	p. 151
ENG-09	Container and Kubernetes Security	p. 178
ENG-10	Infrastructure as Code Security	p. 200

STAGE 3: OPERATE AND GOVERN

ENG-11	Logging and Monitoring Architecture	p. 214
ENG-12	Vulnerability and Patch Programs	p. 231
ENG-13	Endpoint and Email Security	p. 248
ENG-14	Security Automation and Scripting	p. 265
ENG-15	Incident Readiness and Resilience	p. 287
ENG-16	GRC for Engineers	p. 304
ENG-17	Security Metrics and Program Management	p. 326
ENG-18	Capstone: Secure a Company	p. 337

Secure Architecture and Threat Modeling

Intermediate | 6 lessons | 4 simulated labs | checkpoint | about 4h 40m

This module teaches you to think about security before a single server is built. You will learn the threat modeling process, how to draw data flow diagrams with trust boundaries, and how to apply STRIDE, PASTA and attack trees to find and rank threats. You will then turn findings into design decisions using classic secure design principles, zero trust and defense in depth. Threats found on a whiteboard cost a fraction of what the same flaws cost after release, which is why architects and engineers use these methods on every significant design.

BY THE END YOU CAN

- + Decompose a system into a data flow diagram with trust boundaries
- + Apply STRIDE and attack trees to rank threats and pick mitigations
- + Design layered controls using zero trust and defense in depth

IN THIS MODULE

Lesson 1 Threat modeling process and DFDs

Lesson 2 Trust boundaries and attack surface

Lesson 3 STRIDE per element

Lesson 4 PASTA and attack trees

Lesson 5 Secure design patterns and principles

Lesson 6 Zero trust and defense in depth

Lab 1 Draw the DFD (Diagram builder)

Lab 2 STRIDE a login flow (Scenario)

Lab 3 Build an attack tree (Diagram builder)

Lab 4 Layer the defenses (Config editor)

Checkpoint 10 questions, 80% to pass

Threat modeling process and DFDs

Threat modeling turns vague worry into a concrete list of problems you can fix while changes are still cheap. A data flow diagram gives the whole team one shared picture to reason about.

The four questions

Threat modeling is structured thinking about what can go wrong. Adam Shostack frames it as four questions: What are we working on? What can go wrong? What are we going to do about it? Did we do a good enough job? The first question produces a model, usually a diagram. The second applies a method such as STRIDE to that diagram. The third produces mitigations, which you can avoid, reduce, transfer or accept. The fourth validates the work and feeds back into the first. The activity is iterative: you repeat it whenever the design, data or deployment changes, not once before launch.

Data flow diagram elements

A data flow diagram (DFD) uses five symbols. An external entity (a rectangle) is something outside your control that sends or receives data, such as a customer browser or a payment provider. A process (a circle) is code that transforms data, such as a web API. A data store (two parallel lines) holds data at rest, such as a database or a file bucket. A data flow (an arrow) shows data moving between the other elements and should be labeled with what it carries and over which protocol. A trust boundary (a dashed line or box) separates areas of different privilege or ownership. Start with a level 0 context diagram that shows the whole system as one process surrounded by entities. Then decompose into level 1 and level 2 diagrams until each process has one clear job.

Scoping and doing it well

A good model is small enough to finish and detailed enough to find real problems. Begin with the assets worth protecting: credentials, personal data, payment data, signing keys, availability of a key service. Draw only the flows that touch those assets at first. Involve developers, operations and a product owner, because each sees different flows. Write down assumptions explicitly, for example 'the internal network is not trusted' or 'the cloud provider patches the hypervisor'. Unstated assumptions are where most missed threats hide. Capture results as trackable tickets with an owner, a severity and a mitigation, rather than as a document nobody reopens.

When and how often

Run a full model during design, a lighter review for each significant feature, and a refresh after incidents or architecture changes. Triggers include a new external integration, a new data type, a change of identity provider, moving to a new cloud region or service, and exposure of a previously internal API. Keep the diagram in version control next to the code, in a text format such as a diagram-as-code file, so changes to the system produce a visible diff. Tools like OWASP Threat Dragon and the Microsoft Threat Modeling Tool store models in files that can be reviewed like any other artifact.

DFD IN TEXT FORM

```
[Customer browser] (entity)
  -- HTTPS login request --> [Web API] (process)
==== trust boundary: internet | DMZ ====
[Web API] -- SQL over TLS --> [(Users DB)] (store)
==== trust boundary: DMZ | data tier ====
[Web API] -- HTTPS --> [Email provider] (entity)
```

WORKED EXAMPLE

A team plans a customer portal for example.com. The level 0 diagram shows one process, Portal, with three entities: Customer, Support Agent and Payment Provider. At level 1 they split Portal into Web API, Auth Service and a Users database. They label each flow with protocol and data, such as 'password over HTTPS'. Asking 'what can go wrong with the flow from Customer to Web API?' immediately raises credential theft and tampering questions, and asking 'did we do a good enough job?' later prompts a review once the payment provider integration is added.

KEY TERMS

DFD: Data flow diagram showing processes, stores, entities, flows and boundaries.

External entity: A person or system outside your control that sends or receives data.

Data store: A place where data rests, such as a database or file share.

Trust boundary: A line where data crosses between different privilege or ownership levels.

Context diagram: Level 0 view that shows the system as a single process.

Mitigation: An action that avoids, reduces, transfers or accepts a threat.

REMEMBER

- Threat modeling asks what we are building, what can go wrong, what we will do, and whether it was enough.
- A DFD uses entities, processes, stores, flows and trust boundaries.
- Write down assumptions and track findings as owned tickets, and refresh the model when the system changes.

QUICK CHECK

1. Which question is NOT one of the four Shostack threat modeling questions?

- A) What are we working on?
- B) What can go wrong?
- C) Who should we blame?
- D) Did we do a good enough job?

2. In a DFD, a database is drawn as which element?

- A) External entity
- B) Process
- C) Data store
- D) Trust boundary

3. Which output best belongs to the 'What are we going to do about it?' step?

- A) A list of mitigations with owners
- B) The first DFD draft
- C) A penetration test report
- D) A network scan

4. When should a threat model be revisited?

- A) Only before first launch
- B) When the design, data or integrations change
- C) Never, once approved
- D) Only after a breach

Answers: 1-C 2-C 3-A 4-B

1. The four questions cover scope, threats, response and validation, not blame.
2. Databases and files holding data at rest are data stores.
3. That step decides how to avoid, reduce, transfer or accept each threat.
4. Threat modeling is iterative and tied to change.

Trust boundaries and attack surface

Most serious flaws appear where data or control crosses from a less trusted place to a more trusted one. Finding those crossings tells you where to spend validation and monitoring effort.

What a trust boundary is

A trust boundary exists wherever two parts of a system have different privilege levels, owners or assumptions about the other side. Typical boundaries include the internet and your edge network, a browser and a server, a web tier and a database tier, one tenant and another in a multi-tenant service, a user-mode process and the kernel, and your code and a third-party API. Each boundary answers the question: which principal can do what on the far side? If data or commands cross it, the receiving side must not assume the sender behaved well. Draw boundaries as dashed lines on the DFD and label them.

Entry points and attack surface

The attack surface is the sum of all places where an attacker can send input to the system or extract data from it. Entry points include public URLs, API endpoints, open ports, file upload forms, message queues, admin consoles, email addresses that are parsed automatically, and software update channels. Exit points matter too, because error pages, logs and exports can leak data. Enumerate them from the DFD: every flow that crosses a boundary toward you is an entry point. Then rank by exposure (anonymous internet versus authenticated internal) and by the privilege of the process that handles the input.

Validating at the crossing

Data crossing into a more trusted zone must be validated at the point of crossing. That means authenticating the caller, authorizing the action, checking type, length, range and format of input against an allow list, and encoding output for its destination, for example HTML encoding for a page or parameterized queries for SQL. Validation in the browser is a convenience only, since the client is on the untrusted side. Equally, data leaving a trusted zone toward a less trusted one may need filtering, so internal error details and secrets do not escape.

Reducing the surface

You cannot defend what you do not know exists, and you cannot be attacked through what you never built. Reduce the surface by removing unused endpoints and features, closing unneeded ports, disabling default accounts, restricting administrative interfaces to a management network or VPN, and splitting high-risk functions (such as file parsing) into low-privilege sandboxed processes. Measure the surface over time with an inventory of exposed hosts, APIs and ports, and treat any unexplained growth as a finding. Shadow IT and forgotten test environments are common sources of unplanned exposure.

ENTRY POINT INVENTORY

```
ID Entry point Boundary crossed Auth Exposure
E1 POST /login internet -> DMZ none public
E2 POST /upload (PDF) internet -> DMZ user public
E3 GET /admin corp VPN -> DMZ MFA restricted
E4 queue: orders-in partner -> internal mTLS partner
E5 SSH 22 on 198.51.100.7 mgmt -> host key mgmt only
```

WORKED EXAMPLE

Reviewing the inventory above, the team sees that E2 accepts files from the public internet and is handled by the same process that holds database credentials. They decide to validate file type and size at the boundary, scan files in a separate low-privilege worker, and store uploads outside the web root. E1 is anonymous by nature, so they add rate limiting and monitoring for credential stuffing. E3 is moved behind the VPN, which shrinks the attack surface without removing a feature.

KEY TERMS

Trust boundary: A line between zones with different privilege, ownership or trust.

Attack surface: The sum of entry points through which an attacker can interact with a system.

Entry point: A place where external input reaches the system.

Allow list validation: Accepting only input that matches a known-good pattern.

Input encoding: Transforming data so it is treated as data in its destination context.

Attack surface reduction: Removing or restricting features, ports and interfaces that are not needed.

REMEMBER

- Trust boundaries sit where privilege or ownership changes, and flows crossing them are entry points.
- Validate, authorize and encode at the crossing on the trusted side.
- Shrink the attack surface by removing, restricting and isolating exposed functions.

QUICK CHECK

1. Where must input validation that you rely on for security occur?

- A) In the browser only
- B) On the trusted side when data crosses the boundary
- C) Only in the database
- D) Only in logs

2. Which is an example of attack surface reduction?

- A) Adding more public endpoints
- B) Moving the admin console behind a VPN
- C) Disabling logging
- D) Reusing one password everywhere

3. What defines a trust boundary?

- A) A firewall brand
- B) A change in privilege level or ownership between two zones
- C) Any TCP connection
- D) The size of the network

4. Why do exit points matter for attack surface?

- A) They do not matter
- B) Errors, logs and exports can leak sensitive data
- C) They only affect speed
- D) They replace entry points

Answers: 1-B 2-B 3-B 4-B

1. The client is untrusted, so the receiving trusted side must validate.
2. Restricting exposure of the admin interface shrinks who can reach it.
3. Boundaries separate zones with different trust assumptions.
4. Data leaving the system can disclose secrets or internals.

STRIDE per element

STRIDE gives a repeatable checklist so you consider every kind of threat for every element, instead of only the ones you already worry about.

The six categories and the property each breaks

STRIDE, created at Microsoft, names six threat types, each the opposite of a security property. Spoofing violates authentication: pretending to be another user or system. Tampering violates integrity: modifying data or code. Repudiation violates non-repudiation: denying an action when no reliable record exists. Information disclosure violates confidentiality: exposing data to those not authorized. Denial of service violates availability: exhausting or blocking a resource. Elevation of privilege violates authorization: gaining rights beyond those granted. Memorizing the pairing lets you work backward from a required property to the threat that would break it.

STRIDE per element

Not every category applies to every DFD element, so the per-element method narrows the work. External entities are subject to spoofing and repudiation. Processes are subject to all six. Data stores are subject to tampering, information disclosure and denial of service, and to repudiation when they hold logs. Data flows are subject to tampering, information disclosure and denial of service. Walk each element on the diagram, ask which categories apply, and write at least one concrete threat per applicable category, such as 'an attacker replays a captured session cookie to the Web API' for spoofing.

Typical mitigations

Pair each category with standard controls. Spoofing: strong authentication, MFA, mutual TLS between services, signed tokens. Tampering: TLS, digital signatures, message authentication codes, file integrity monitoring, database permissions and constraints. Repudiation: secure, centralized, append-only audit logs with synchronized time and user identity. Information disclosure: encryption in transit and at rest, access control, minimal error messages, secrets management. Denial of service: rate limiting, quotas, autoscaling, timeouts and upstream filtering. Elevation of privilege: least privilege, input validation, sandboxing, separate admin paths and patching. The mitigation must act at the right element, not just somewhere nearby.

Recording and prioritizing

Record each threat as an entry: element, category, scenario, existing control, proposed mitigation, owner and priority. Prioritize by likelihood and impact using a simple scale or a scheme such as DREAD or CVSS-style scoring, and be consistent. Do not try to prove a threat is real by exploiting it; the goal is to decide what to build. Some threats will be accepted with sign-off from a risk owner, which is a valid outcome if documented. Finally, check coverage: every element and every boundary-crossing flow should have been considered against every applicable category.

STRIDE WORKSHEET ROW

```
Element : Web API (process), login handler
S: attacker guesses or replays credentials -> MFA, lockout, rate limit
T: request body altered in transit -> TLS 1.2+, server-side validation
R: user denies a password reset -> audit log with user id and time
I: stack trace in error page -> generic errors, log server-side
D: flood of login requests -> rate limit, WAF, timeouts
E: injection yields admin role -> parameterized queries, least privilege
```

WORKED EXAMPLE

A team analyzes the flow carrying order data from the Web API to an Orders database on 192.0.2.40. Flows are subject to T, I and D. For tampering, they note that an on-path attacker could alter order totals, so they require TLS with certificate validation. For information disclosure, plaintext SQL would expose customer data, which the same TLS control addresses. For denial of service, they add connection pool limits and query timeouts. One control family covers two categories, but the denial of service item needed its own control.

KEY TERMS

STRIDE: Mnemonic for Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege.

Spoofing: Pretending to be another identity, breaking authentication.

Repudiation: Denying an action when no trustworthy evidence exists.

Elevation of privilege: Gaining rights beyond those intended.

Non-repudiation: Assurance that an actor cannot credibly deny an action.

Audit log: A tamper-resistant record of who did what and when.

REMEMBER

- Each STRIDE category breaks one property: authentication, integrity, non-repudiation, confidentiality, availability, authorization.
- Apply categories per element: entities S and R, processes all, stores T I D, flows T I D.
- Record threat, control, owner and priority, and accept risk only with sign-off.

QUICK CHECK

1. Which STRIDE category violates integrity?
 - A) Spoofing
 - B) Tampering
 - C) Repudiation
 - D) Denial of service
2. Which elements are subject to spoofing and repudiation in STRIDE per element?
 - A) Data flows
 - B) Data stores
 - C) External entities
 - D) Trust boundaries
3. Which mitigation best addresses repudiation?
 - A) Rate limiting
 - B) Centralized append-only audit logging
 - C) Input length checks
 - D) Disk encryption
4. An attacker floods an API so legitimate users cannot connect. Which category is this?
 - A) Information disclosure
 - B) Tampering
 - C) Denial of service
 - D) Spoofing

Answers: 1-B 2-C 3-B 4-C

1. Tampering is unauthorized modification of data or code.
2. External entities are the sources of identity claims and actions.
3. Reliable logs provide evidence of who did what.
4. Exhausting a resource to block service is denial of service.

PASTA and attack trees

STRIDE finds categories of threats, while PASTA and attack trees connect them to attacker goals, cost and business impact so you can rank what to fix first.

PASTA: seven stages

PASTA, the Process for Attack Simulation and Threat Analysis, is a risk-centric method with seven stages. 1) Define objectives: business goals, compliance needs and impact criteria. 2) Define the technical scope: infrastructure, applications and dependencies. 3) Application decomposition: DFDs, trust boundaries, entry points and assets. 4) Threat analysis: use threat intelligence to identify realistic threat actors and techniques. 5) Vulnerability and weakness analysis: map known weaknesses, scan results and design flaws to the threats. 6) Attack modeling: build attack trees and simulate attack paths. 7) Risk and impact analysis: rank risks and select countermeasures in business terms.

Attack trees

An attack tree, popularized by Bruce Schneier, puts the attacker goal at the root and decomposes it into sub-goals. Child nodes are joined by OR when any one is enough, or AND when all are needed. Leaves are concrete actions. A root of 'steal customer data' might have OR children 'exploit the web app', 'phish an administrator' and 'steal a backup'. The node 'phish an administrator' might be AND of 'send a convincing lure', 'capture credentials' and 'bypass MFA'. The tree makes assumptions visible and shows which single action unlocks many paths.

Annotating with cost and likelihood

Each leaf can be annotated with attributes such as cost to attacker, skill required, probability of success or detection chance. Values propagate upward: an OR node takes the cheapest (minimum cost) of its children, because the attacker will choose the easiest route; an AND node takes the sum of costs, since every child is required. Likelihood combines similarly, with AND multiplying probabilities and OR taking the best option. The cheapest path to the root is where defenders should invest first, because raising cost on a path the attacker would never choose adds little.

Using the results

Turn the tree into decisions. If the cheapest path runs through phishing, strengthen MFA with phishing-resistant factors and add mail filtering rather than hardening an already strong database. Add detection at nodes where prevention is hard. Revisit the tree when new vulnerabilities, techniques or business changes appear, and keep numbers as relative estimates, not false precision. Attack trees pair well with STRIDE: STRIDE generates the list of leaf threats, and the tree arranges them into realistic attack paths with a cost view. Document the tree so reviewers can challenge the numbers.

ATTACK TREE WITH COSTS (USD)

```
GOAL: steal customer data (OR, cost = min)
1 Exploit web app SQL injection ..... 4000
2 Phish administrator (AND) ..... 1500
  2a Craft lure ..... 300
  2b Capture credentials ..... 400
  2c Bypass SMS-based MFA ..... 800
3 Steal unencrypted backup ..... 9000
CHEAPEST PATH: node 2 = 300+400+800 = 1500
```

WORKED EXAMPLE

Using the tree above, the AND node 'Phish administrator' costs 1500 (300 + 400 + 800), the web exploit costs 4000 and the backup theft costs 9000. The OR root takes the minimum, so phishing is the path to fix. Replacing SMS codes with FIDO2 security keys makes node 2c effectively very expensive, so recalculating shows the cheapest path is now the web exploit at 4000. The team then schedules code review and a WAF for that path. The tree showed that MFA, not the database, was the best investment.

KEY TERMS

PASTA: A seven-stage, risk-centric threat modeling methodology.

Attack tree: A hierarchy of attacker goals decomposed into sub-goals and actions.

OR node: A node satisfied when any one child is achieved.

AND node: A node satisfied only when all children are achieved.

Leaf: A concrete attacker action at the bottom of the tree.

Cheapest path: The lowest total cost route to the root goal.

REMEMBER

- PASTA has seven stages from business objectives to risk and impact analysis.
- Attack trees use OR (any) and AND (all) nodes with leaf costs.
- OR takes the minimum cost and AND the sum, and the cheapest path guides investment.

QUICK CHECK

1. In an attack tree, how is the cost of an OR node usually computed?

- A) Sum of children
- B) Minimum of children
- C) Maximum of children
- D) Average of children

2. How is the cost of an AND node usually computed?

- A) Minimum of children
- B) Sum of children
- C) Zero
- D) Maximum only

3. Which PASTA stage builds attack trees and simulates attack paths?

- A) Stage 1
- B) Stage 3
- C) Stage 6
- D) Stage 7

4. Why focus first on the cheapest attack path?

- A) It is the most expensive
- B) It is the route a rational attacker will choose
- C) It has no leaves
- D) It is always the longest

Answers: 1-B 2-B 3-C 4-B

1. The attacker picks the cheapest alternative.
2. All children are required, so costs add.
3. Stage 6 is attack modeling.
4. Raising cost on paths the attacker would not choose gains little.

Secure design patterns and principles

Principles from the 1970s still explain most design flaws. Applying them early prevents entire classes of vulnerabilities rather than fixing them one at a time.

Least privilege and fail-safe defaults

Least privilege means every user, process and service receives only the rights needed for its task and only for as long as needed. A web application should connect to its database with an account that can read and write its own tables, not one that can drop schemas. Fail-safe defaults (Saltzer and Schroeder) means access is denied unless explicitly granted, so a missing rule, a failed lookup or an exception results in refusal. In code, that means an authorization check that returns false on error, and in networks, a firewall policy that ends in default deny. Defaults that are open make every misconfiguration a breach.

Complete mediation

Complete mediation requires that every access to every resource be checked for authority, every time. Caching a decision or checking only at the first page of a workflow creates gaps. Classic failures include an API that checks permission in the user interface but not on the server, and an object reference such as `/invoices/1042` that is served without verifying that the caller owns invoice 1042. The remedy is a central enforcement point, such as middleware or a policy engine, that cannot be bypassed, with checks on the server side for each request and for each object.

Separation of duties and related principles

Separation of privilege or duties requires more than one party or condition for a sensitive action, such as a developer who writes code and a different person who approves production deployment, or two approvals for a large payment. It limits the damage of mistakes, insider abuse and a single compromised account. Other Saltzer and Schroeder principles help too. Economy of mechanism favors simple designs that are easier to verify. Open design means security must not depend on secret algorithms. Least common mechanism limits shared components that could carry attacks between users. Psychological acceptability asks that secure paths be usable, or people will bypass them.

Putting principles into patterns

Common patterns apply these ideas. Use a single gateway or reverse proxy for authentication and logging. Isolate untrusted processing, such as image conversion, in sandboxed low-privilege containers. Separate administrative interfaces from user interfaces on a different network path. Use a secrets manager instead of embedding keys. Prefer well-reviewed libraries for cryptography and session handling instead of custom code. Fail closed for security decisions but design availability separately, for example with redundant enforcement points, since failing closed on a single node can itself cause an outage. Document each choice in the threat model so reviewers see why it exists.

AUTHORIZATION CHECK, FAIL CLOSED

```
def can_view_invoice(user, invoice_id):
    try:
        inv = repo.get(invoice_id)
        return inv is not None and inv.owner_id == user.id
    except Exception:
        return False # deny on any error
```

WORKED EXAMPLE

A reporting service on `example.com` first used a database account with full administrator rights, and its `/reports/{id}` endpoint checked only that the user was logged in. Applying the principles, the team creates a read-only database role limited to the reporting views (least privilege), adds an ownership check on every request (complete mediation), makes the check return false on any error (fail-safe default), and requires a second approver before report definitions change in production (separation of duties).

KEY TERMS

Least privilege: Granting only the minimum rights necessary for the shortest necessary time.

Fail-safe default: A design where lack of explicit permission results in denial.

Complete mediation: Checking authorization on every access to every object.

Separation of duties: Splitting a sensitive task among multiple people or roles.

Economy of mechanism: Preferring simple designs that are easier to analyze.

Open design: Security should not rely on secrecy of the design or algorithm.

REMEMBER

- Least privilege and fail-safe defaults limit damage and make failures safe.
- Complete mediation means check every request and every object on the server.
- Separation of duties, simplicity and open design reduce single points of failure and reliance on secrecy.

QUICK CHECK

1. Which principle says that a failed authorization lookup should result in denial?
 - A) Open design
 - B) Fail-safe defaults
 - C) Economy of mechanism
 - D) Least common mechanism
2. An API checks login but not whether the caller owns the requested record. Which principle is violated?
 - A) Complete mediation
 - B) Open design
 - C) Psychological acceptability
 - D) Economy of mechanism
3. Requiring one person to write code and another to approve release is which principle?
 - A) Least privilege
 - B) Separation of duties
 - C) Fail-safe defaults
 - D) Open design
4. A database account for an app can only read and write its own tables. This applies which principle?
 - A) Least privilege
 - B) Repudiation
 - C) Obscurity
 - D) Replay protection

Answers: 1-B 2-A 3-B 4-A

1. Access is denied unless explicitly granted.
2. Every access to every object must be checked.
3. Splitting duties limits single-party abuse or error.
4. The account has only the rights needed for the task.

Zero trust and defense in depth

Modern systems cannot rely on a trusted internal network or a single control. Zero trust and layered defenses assume something will fail and make sure no one failure exposes the asset.

Zero trust and NIST SP 800-207

Zero trust shifts defense from network perimeters to resources and identities. NIST SP 800-207 lists seven tenets: all data sources and computing services are resources; all communication is secured regardless of network location; access is granted per session; access is determined by dynamic policy including client identity, application state and other attributes; the enterprise monitors the integrity and security posture of all assets; authentication and authorization are dynamic and strictly enforced before access; and the enterprise collects data to improve its posture. The core idea is never trust, always verify, and no implicit trust from network location.

Architecture components

The 800-207 logical model separates a control plane from a data plane. The Policy Engine (PE) decides whether to grant access using policy and inputs such as identity, device health and threat intelligence. The Policy Administrator (PA) establishes or tears down the session based on that decision. The Policy Enforcement Point (PEP) sits in the path of the resource and enforces it. Together the PE and PA form the Policy Decision Point. Inputs include a CDM system, identity management, SIEM, threat intelligence and data access policies. Resources are reached only through the PEP, never directly.

Defense in depth

Defense in depth places several independent controls between the attacker and the asset so that one failure is not fatal. Layers include physical and policy controls; the perimeter and network (firewalls, segmentation, DDoS protection); the host (hardening, EDR, patching, host firewall); the application (input validation, authentication, WAF, secure coding); and the data (encryption, tokenization, access control, backups). People and process, such as training and incident response, form another layer. Controls should be diverse, so that a flaw in one vendor or technique does not defeat them all, and they should include detection as well as prevention.

Combining the two

Zero trust and defense in depth complement each other. Zero trust removes the assumption that being inside the network earns access, while layering ensures that even a verified session faces more checks at each tier. A practical design might require device-compliant MFA at an identity-aware proxy, microsegmented application tiers, a hardened host with EDR, an application that checks authorization per object, and database encryption with separate key custody. Test the design with the question: if this one control fails or is bypassed, what stops the attacker next, and who will notice? If the answer is nothing, add a layer or a detection.

LAYERS FOR ONE ASSET

```
Asset: customer database (203.0.113.20)
Network : only app tier 198.51.100.0/24 may reach TCP 5432
Host : hardened image, EDR agent, no interactive logins
App : parameterized queries, per-object authorization
Data : encryption at rest, keys in separate KMS
Detect : DB audit log -> SIEM alert on bulk export
```

WORKED EXAMPLE

A firewall misconfiguration briefly exposes the database port to a wider range. Because the host accepts only connections from the application account with a certificate, the attacker cannot authenticate. Because data is encrypted with keys held in a separate KMS, a copied disk image is unreadable. Because the audit log alerts on bulk exports, the team notices the unusual query volume within minutes. No single layer decided the outcome, and a zero trust policy that requires per-session verification added another check.

KEY TERMS

Zero trust: A model that grants access per request based on identity and context, not network location.

Policy Engine: The component that decides whether access is granted.

Policy Enforcement Point: The component in the path that enforces the access decision.

Defense in depth: Using multiple independent layers of controls.

Control plane: The part of the architecture that makes and distributes decisions.

Posture: The current security health of a device or asset.

REMEMBER

- Zero trust verifies each session using identity, device and policy, with no trust from location.
- NIST 800-207 separates policy decision (PE and PA) from enforcement (PEP).
- Defense in depth layers diverse prevention and detection across network, host, app and data.

QUICK CHECK

1. What is the main change zero trust makes compared with perimeter security?
 - A) It trusts all internal traffic
 - B) It removes implicit trust based on network location
 - C) It eliminates authentication
 - D) It requires one large firewall
2. In NIST SP 800-207, which component enforces the decision in the path to the resource?
 - A) Policy Enforcement Point
 - B) Threat feed
 - C) Backup server
 - D) DNS resolver
3. Which set lists typical defense in depth layers?
 - A) Network, host, application, data
 - B) Only the perimeter
 - C) Only endpoints
 - D) Only encryption
4. Why should layered controls be diverse?
 - A) To raise costs only
 - B) So one flaw or technique does not defeat all layers
 - C) To remove detection
 - D) To avoid logging

Answers: 1-B 2-A 3-A 4-B

1. No implicit trust is given for being on an internal network.
2. The PEP enables, monitors and terminates connections.
3. Layers span network, host, app and data.
4. Independent controls avoid a common failure.

Draw the DFD

SCENARIO

You are modeling a small online store at example.com. Customers use browsers, the Web API runs in a DMZ, an Orders database sits in a private data tier, and a third-party payment provider is called over the internet. The team needs a first diagram before the design review.

OBJECTIVE

Build a DFD with the correct element types, labeled flows and trust boundaries.

STEPS IN THE SIMULATOR

- 1 **Do:** Drag a Customer onto the canvas and set type to External entity.

Simulator shows:

Node 'Customer' added as rectangle (entity).

- 2 **Do:** Add 'Web API' as a Process and 'Orders DB' as a Data store.

Simulator shows:

Circle 'Web API' and parallel-lines 'Orders DB' placed.

- 3 **Do:** Add 'Payment Provider' as an External entity.

Simulator shows:

Second rectangle placed outside any boundary.

- 4 **Do:** Draw flows: Customer -> Web API (HTTPS order), Web API -> Orders DB (SQL over TLS), Web API -> Payment Provider (HTTPS tokenized card).

Simulator shows:

Three arrows created, each with a protocol label.

- 5 **Do:** Draw a dashed boundary around the customer and another between the DMZ and the data tier.

Simulator shows:

Boundaries 'internet|DMZ' and 'DMZ|data tier' drawn.

- 6 **Do:** Click Validate.

Simulator shows:

Warning: flow Web API -> Payment Provider crosses the boundary but has no boundary marker.

- 7 **Do:** Add a boundary line separating Payment Provider from your systems and validate again.

Simulator shows:

Validation passed: 3 flows, 3 boundaries, all crossing flows labeled.

ANSWER

A valid DFD has Customer and Payment Provider as entities, Web API as a process, Orders DB as a store, three labeled flows, and trust boundaries for internet to DMZ, DMZ to data tier and your systems to the payment provider.

Hint shown in the app: Every flow that leaves your control needs a boundary line on it.

STRIDE a login flow

SCENARIO

A login DFD has four elements: User (entity), Login API (process), Credentials DB (store) and the flow carrying the password from User to Login API. The product owner wants a short list of the most relevant STRIDE threats with one mitigation each before the sprint starts.

OBJECTIVE

Match applicable STRIDE categories and mitigations to each login element.

STEPS IN THE SIMULATOR

- 1 **Do:** Select element 'User' and view its applicable categories.

Simulator shows:

Simulator lists: Spoofing, Repudiation.

- 2 **Do:** Choose Spoofing and pick a mitigation from the list.

Simulator shows:

Options: MFA, ASCII banner, longer logs. Choose MFA.

- 3 **Do:** Select element 'Login flow' (User to API).

Simulator shows:

Applicable: Tampering, Information disclosure, Denial of service.

- 4 **Do:** For Information disclosure choose 'TLS 1.2+ with certificate validation'.

Simulator shows:

Correct. Also covers Tampering for this flow.

- 5 **Do:** Select 'Login API' and add Elevation of privilege with 'Parameterized queries and least privilege'.

Simulator shows:

Accepted: process allows all six categories.

- 6 **Do:** Select 'Credentials DB' and add Information disclosure with 'Salted slow password hashes'.

Simulator shows:

Accepted: stored hashes resist offline cracking.

- 7 **Do:** Click Score.

Simulator shows:

Score: 6 of 6 applicable threat/mitigation pairs matched; no inapplicable categories chosen.

ANSWER

Spoofing and repudiation map to the User entity (MFA, audit logs); tampering, disclosure and denial of service map to the flow (TLS, rate limits); the API takes elevation of privilege (parameterized queries, least privilege); the store takes information disclosure (salted slow hashes).

Hint shown in the app: Check which categories apply to each element type before choosing mitigations.

Build an attack tree

SCENARIO

A retailer fears theft of customer data. Analysts supply three candidate routes with leaf costs: SQL injection in the web app (4000), phishing an administrator past SMS MFA (300 + 400 + 800), and stealing an unencrypted backup tape (9000). You must arrange the tree and mark the cheapest path.

OBJECTIVE

Construct the tree with correct AND/OR nodes and highlight the lowest-cost path.

STEPS IN THE SIMULATOR

- 1 **Do:** Place the root node 'Steal customer data' and set it to OR.

Simulator shows:

Root created with type OR.

- 2 **Do:** Add child 'Exploit web app' as a leaf with cost 4000.

Simulator shows:

Leaf added: cost 4000.

- 3 **Do:** Add child 'Phish administrator' and set it to AND.

Simulator shows:

AND node added, cost pending.

- 4 **Do:** Add three leaves under it: Craft lure 300, Capture credentials 400, Bypass SMS MFA 800.

Simulator shows:

AND node cost computed: 1500.

- 5 **Do:** Add child 'Steal backup' as a leaf with cost 9000.

Simulator shows:

Leaf added: cost 9000.

- 6 **Do:** Click Compute.

Simulator shows:

Root cost = $\min(4000, 1500, 9000) = 1500$.

- 7 **Do:** Click the highlight tool on the lowest path.

Simulator shows:

Path highlighted: Root -> Phish administrator.

ANSWER

The cheapest path is phishing the administrator at a total of 1500, the sum of its three AND children. The OR root takes the minimum of the three branches, so phishing resistant MFA and email controls are the first priority.

Hint shown in the app: AND nodes add their children; OR nodes pick the minimum.

Layer the defenses

SCENARIO

The customer database holds regulated data. Management asks that no single control failure expose it. A configuration panel lists twelve candidate controls and four layers: network, host, application and data. You must assign controls so each layer has at least two and one detection control is included.

OBJECTIVE

Assign controls to layers so any one failure is covered by another layer.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the controls list.

Simulator shows:

12 controls shown: security group, segmentation, EDR, OS hardening, input validation, per-object authz, encryption at rest, KMS key separation, DB audit log, WAF, host firewall, backups.

- 2 **Do:** Drag 'security group' and 'segmentation' to Network.

Simulator shows:

Network layer: 2 controls.

- 3 **Do:** Drag 'OS hardening', 'EDR' and 'host firewall' to Host.

Simulator shows:

Host layer: 3 controls.

- 4 **Do:** Drag 'input validation', 'per-object authz' and 'WAF' to Application.

Simulator shows:

Application layer: 3 controls.

- 5 **Do:** Drag 'encryption at rest' and 'KMS key separation' to Data.

Simulator shows:

Data layer: 2 controls.

- 6 **Do:** Mark 'DB audit log' as detection and attach it to Data.

Simulator shows:

Detection coverage: 1 (alerts to SIEM).

- 7 **Do:** Click Simulate failure of the network layer.

Simulator shows:

Result: host, application and data layers still block access; alert raised by audit log.

ANSWER

Each layer has at least two controls, a detection control exists, and simulating failure of any single layer leaves other layers preventing access to the data. Diverse controls and separate key custody are important.

Hint shown in the app: Put encryption keys under different custody than the data they protect.

Secure Architecture and Threat Modeling: 10 questions, 80% to pass

1. Which DFD element represents a database?
 - A) Process
 - B) Data store
 - C) Trust boundary
 - D) External entity
2. Data crossing a trust boundary into a more trusted zone should be:
 - A) Trusted automatically
 - B) Validated and authorized on the trusted side
 - C) Only validated in the browser
 - D) Ignored
3. Which STRIDE category breaks confidentiality?
 - A) Spoofing
 - B) Information disclosure
 - C) Repudiation
 - D) Elevation of privilege
4. In STRIDE per element, data flows are subject to:
 - A) Spoofing and repudiation
 - B) Tampering, information disclosure and denial of service
 - C) Only elevation of privilege
 - D) All six
5. An attack tree OR node with children costing 400, 900 and 1500 has cost:
 - A) 400
 - B) 900
 - C) 1500
 - D) 2800
6. An attack tree AND node with children costing 400, 900 and 1500 has cost:
 - A) 400
 - B) 1500
 - C) 2800
 - D) 900
7. Which PASTA stage covers risk and impact analysis?
 - A) Stage 2
 - B) Stage 4
 - C) Stage 6
 - D) Stage 7
8. An API that fails to check object ownership violates:
 - A) Open design
 - B) Complete mediation
 - C) Economy of mechanism
 - D) Separation of privilege
9. In NIST SP 800-207, the component that decides access using policy and signals is the:
 - A) Policy Enforcement Point
 - B) Policy Engine
 - C) Syslog server
 - D) DHCP server
10. What is the best test of a layered design?
 - A) Count the number of products
 - B) Ask what stops the attacker if one control fails
 - C) Use a single vendor
 - D) Remove detection

Answers: 1-B 2-B 3-B 4-B 5-A 6-C 7-D 8-B 9-B 10-B

1. A database holds data at rest.
2. The receiving side must not assume the sender is well behaved.
3. Information disclosure exposes data to unauthorized parties.
4. Flows can be modified, read or disrupted.
5. OR takes the minimum cost.
6. AND adds all required children.
7. Stage 7 is the final risk and impact stage.
8. Every access to every object needs a check.
9. The Policy Engine makes the grant or deny decision.
10. Defense in depth means no single failure exposes the asset.

System Hardening

Intermediate | 6 lessons | 4 simulated labs | checkpoint | about 4h 40m

This module shows how to make operating systems hard to attack and keep them that way. You will learn hardening principles and the CIS Benchmarks, then apply concrete settings on Linux and Windows. You will build a patch management lifecycle, enforce desired state with configuration management tools, and use gold images and compliance scanners to detect drift. Most breaches use known weaknesses and default settings, so disciplined hardening and patching remove a large share of real-world risk at low cost.

BY THE END YOU CAN

- + Apply CIS Benchmark controls to Linux and Windows systems
- + Build a patch and configuration management workflow with baselines
- + Detect configuration drift against a hardened baseline

IN THIS MODULE

Lesson 1 Hardening principles and CIS Benchmarks

Lesson 2 Linux hardening

Lesson 3 Windows hardening

Lesson 4 Patch management lifecycle

Lesson 5 Configuration management tools

Lesson 6 Baselines, drift and compliance scanning

Lab 1 Harden sshd_config (Config editor)

Lab 2 Audit a Linux host (Terminal)

Lab 3 Windows GPO review (Config editor)

Lab 4 Patch rollout plan (Scenario)

Checkpoint 10 questions, 80% to pass

Hardening principles and CIS Benchmarks

Default installations favor convenience over security. Hardening systematically removes what is not needed and tightens what remains, using agreed benchmarks instead of personal opinion.

Attack surface reduction

Hardening means configuring a system to reduce its attack surface and the impact of compromise. The core moves are consistent: install only required software, disable or remove unused services and accounts, close unneeded ports, change defaults, apply least privilege, enable logging, and keep software patched. Each service you remove is one fewer program that can contain a vulnerability, and each default password you change is one fewer guessable credential. Start from a minimal installation image, add only what the role needs, and document why each exception exists. Hardening should be repeatable and automated, not a one-time manual checklist.

What the CIS Benchmarks are

The Center for Internet Security (CIS) publishes consensus-based configuration guides called CIS Benchmarks for operating systems, cloud platforms, databases, browsers and network devices. Volunteers from industry, government and academia write and review them, so they reflect broad practical experience. Each benchmark lists numbered recommendations with a description, rationale, audit procedure, remediation steps and impact. Examples include 'Ensure SSH root login is disabled' and 'Ensure password history is configured'. Benchmarks differ from the CIS Controls, which are a prioritized program of high-level safeguards; benchmarks give the exact settings. The US DISA STIGs are a comparable, more rigid source.

Level 1 and Level 2 profiles

Benchmarks define profiles. Level 1 contains practical settings that give clear security value with little risk of reducing function, and is a sensible baseline for most systems. Level 2 is meant for environments needing defense in depth and higher assurance, and may reduce usability or break some applications, so it needs testing. Many benchmarks also separate roles, such as Windows Member Server and Domain Controller, or Workstation and Server. Some include a STIG profile that aligns with DISA requirements. Choose the profile by data sensitivity and by what the system must do, then record any recommendation you deliberately skip.

Scoring and assessment

Compliance is measured by running an assessment tool and counting passed recommendations. CIS-CAT Pro scans a host against a benchmark and reports a percentage score per section. Recommendations are marked automated, where a tool can check them, or manual, where a person must review. Scoring drives trends but not safety by itself, so read failures by risk, not only by count. Run a baseline scan on a test image, fix or document each failure, and re-scan. Always test hardening on a non-production copy first, because settings such as disabling NTLM or removing a service can break legitimate dependencies.

SCAN SUMMARY

```
Benchmark : CIS Ubuntu Linux 22.04 LTS, Level 1 Server
Host : web01.example.com
Passed 148 | Failed 22 | Manual 9 | Score 87.1 %
FAIL 5.2.8 Ensure SSH root login is disabled
FAIL 3.4.2 Ensure firewall default policy is deny
FAIL 4.1.1 Ensure auditd is installed
```

WORKED EXAMPLE

A team builds a web server image for example.com. They choose Level 1 Server because the host serves public content but holds no regulated data. The first scan scores 87.1 percent. They fix SSH root login, set a default deny firewall policy and install auditd, then re-scan to 97 percent. Three Level 1 items conflict with an old monitoring agent, so they record those as documented exceptions with an owner and a review date instead of ignoring them.

KEY TERMS

Hardening: Reducing attack surface and tightening configuration to lower risk.

CIS Benchmark: A consensus configuration guide with auditable recommendations.

Level 1 profile: Practical baseline settings with low risk of breaking function.

Level 2 profile: Stricter settings for higher assurance that may affect usability.

CIS-CAT: A CIS tool that assesses a system against a benchmark.

STIG: A DISA configuration standard used by US defense organizations.

REMEMBER

- Hardening removes unneeded software, services and defaults, and applies least privilege.
- CIS Benchmarks give consensus, auditable settings; Level 1 is the practical baseline and Level 2 is stricter.
- Scan, fix, re-scan, and document exceptions instead of silently skipping items.

QUICK CHECK

1. What distinguishes CIS Level 2 from Level 1?
 - A) It is only for Windows
 - B) Stricter settings for higher assurance that may reduce usability
 - C) It removes logging
 - D) It is optional documentation
2. Which is a core hardening step?
 - A) Enable all default services
 - B) Disable unneeded services and accounts
 - C) Reuse default passwords
 - D) Disable updates
3. What does a CIS Benchmark recommendation typically include?
 - A) Only a score
 - B) Rationale, audit and remediation steps
 - C) A vendor sales pitch
 - D) A firewall brand list
4. Why test hardening settings on a non-production copy first?
 - A) They never cause issues
 - B) Some settings can break legitimate applications
 - C) Scanners require it
 - D) Benchmarks forbid production

Answers: 1-B 2-B 3-B 4-B

1. Level 2 gives defense in depth at the cost of possible functionality impact.
2. Removing what is unneeded shrinks the attack surface.
3. Each item explains why, how to check and how to fix.
4. Restrictive settings may break dependencies.

Linux hardening

Linux servers are exposed to constant scanning and password guessing. A short list of well-understood settings blocks the most common attacks.

Securing SSH

SSH is usually the first service attackers probe on port 22. In `/etc/ssh/sshd_config` set `PermitRootLogin no` so attackers cannot target the root account directly and actions are tied to named users. Set `PasswordAuthentication no` and `PubkeyAuthentication yes` to require key-based authentication, which resists guessing. Restrict who may log in with `AllowUsers` or `AllowGroups`, set `MaxAuthTries 3` and `LoginGraceTime 30`, disable `X11Forwarding` and, where possible, require MFA. Prefer modern algorithms and remove legacy ones. Validate with `sshd -t` before reloading the service, and keep a second session open so a mistake cannot lock you out.

Kernel parameters, permissions and services

Network kernel parameters are set with `sysctl`, persisted in `/etc/sysctl.d/`. Common values: `net.ipv4.tcp_syncookies=1`, `net.ipv4.conf.all.rp_filter=1` (reverse path filtering), `net.ipv4.conf.all.accept_redirects=0`, `net.ipv4.conf.all.send_redirects=0`, `net.ipv4.ip_forward=0` on non-routers, and `kernel.randomize_va_space=2` for full ASLR. Check file permissions: `/etc/shadow` should be readable only by root and the shadow group, world-writable files should be found and justified, and a default `umask` of `027` limits new file access. List listening services with `ss -tulpn` and disable anything not required using `systemctl disable --now`.

Auditing, MAC and updates

The Linux audit framework, `auditd`, records security-relevant events using rules in `/etc/audit/rules.d/`. Typical rules watch changes to `/etc/passwd`, `/etc/shadow`, `/etc/sudoers` and `sshd_config`, and log privileged command use and time changes. Send audit logs to a central collector so a compromised host cannot erase them. Keep mandatory access control enabled: SELinux on Red Hat family systems or AppArmor on Debian and Ubuntu, in enforcing mode. Configure automatic security updates, such as `unattended-upgrades` on Debian family systems or `dnf-automatic` on Red Hat family systems, for hosts where unattended updates are acceptable.

Accounts, firewall and verification

Enforce strong authentication policy: lock or remove unused accounts, set password aging with `chage`, and configure `pam_faillock` to lock accounts after repeated failures. Give administrators individual accounts and `sudo`, with logging, rather than a shared root login. Run a host firewall such as `nftables`, `ufw` or `firewalld` with a default deny inbound policy and explicit allows. Verify the result with a scanner such as `Lynis` or `OpenSCAP`, and by checking effective settings, for example `sshd -T | grep -i permitroot` to display the running value, since an included config file can override what you edited.

TERMINAL OUTPUT

```
$ sshd -T | grep -Ei 'permitrootlogin|passwordauth|maxauthtries'
permitrootlogin no
passwordauthentication no
maxauthtries 3
$ ss -tulpn | grep LISTEN
tcp LISTEN 0 128 0.0.0.0:22 users:(("sshd"))
$ stat -c '%a %U:%G' /etc/shadow
640 root:shadow
```

WORKED EXAMPLE

An administrator hardening host `192.0.2.30` sets `PermitRootLogin no` and `PasswordAuthentication no`, but `sshd -T` still shows `passwordauthentication yes`. Searching shows a drop-in file in `/etc/ssh/sshd_config.d/` that sets it yes, and because the first obtained value wins, that file overrides the main one. They correct the drop-in, run `sshd -t`, reload the service, and confirm with `sshd -T`. The lesson is to verify the effective configuration, not just the file they edited.

KEY TERMS

PermitRootLogin: `sshd` option controlling whether root may log in directly.

sysctl: Tool and interface for reading and setting kernel parameters.

auditd: The Linux audit daemon that records security events.

SELinux: A mandatory access control system used mainly on Red Hat family systems.

AppArmor: Profile-based mandatory access control used on Debian and Ubuntu.

umask: A mask defining default permissions for newly created files.

REMEMBER

- Harden SSH: no root login, key authentication, restricted users, limited attempts.
- Set sysctl network protections, tight file permissions, and disable unneeded services.
- Use auditd, SELinux or AppArmor, default deny firewall, and verify effective settings.

QUICK CHECK

1. Which sshd_config setting disables direct root login?

- A) PermitRootLogin no
- B) RootLogin allow
- C) AllowRoot no
- D) PermitEmptyPasswords no

2. What command reveals the effective sshd configuration?

- A) sshd -T
- B) ssh-add -l
- C) ss -s
- D) sysctl -a

3. Which sysctl setting enables SYN cookies?

- A) net.ipv4.tcp_syncookies=1
- B) kernel.randomize_va_space=0
- C) net.ipv4.ip_forward=1
- D) net.ipv4.conf.all.accept_redirects=1

4. Why send auditd logs to a central collector?

- A) To make them larger
- B) A compromised host cannot easily erase remote copies
- C) Auditd requires it
- D) To disable rules

Answers: 1-A 2-A 3-A 4-B

1. PermitRootLogin no refuses root sessions.

2. sshd -T prints the active configuration including includes.

3. tcp_syncookies=1 helps resist SYN floods.

4. Remote copies preserve evidence after compromise.

Windows hardening

Windows environments are managed centrally, which allows one policy change to harden thousands of systems and to close well-known credential theft paths.

Group Policy and local policy

In Active Directory environments, Group Policy Objects (GPOs) apply settings to organizational units, giving consistent baselines. On standalone machines, the Local Security Policy (secpol.msc) and Local Group Policy editor (gpedit.msc) hold the same kinds of settings. Key areas are account policies (password length, lockout threshold), user rights assignment, security options and the advanced audit policy. Microsoft publishes Security Baselines in the Security Compliance Toolkit, and CIS provides benchmarks for each Windows version and role. Apply baselines to a pilot OU first, and use Group Policy Results or gpresult /r to confirm what actually applied.

Removing legacy protocols

Several old protocols enable credential theft or lateral movement. SMBv1 should be disabled; on servers use Set-SmbServerConfiguration -EnableSMB1Protocol \$false, and remove the feature where present. LLMNR and NetBIOS name service fall back for name resolution and can be abused to capture credential hashes; disable LLMNR through the policy Turn off multicast name resolution. Restrict NTLM using the Network security: Restrict NTLM policies, audit NTLM use first, then block where Kerberos works. Require SMB signing and, where supported, set LAN Manager authentication level to send NTLMv2 only and refuse LM and NTLM.

Endpoint protection and encryption

Microsoft Defender Antivirus should be on with cloud protection and tamper protection enabled so malware cannot switch it off. Attack Surface Reduction (ASR) rules block risky behaviors, such as Office applications creating child processes, and should be deployed first in audit mode. Enable Windows Defender Firewall with default deny inbound on all profiles. Use BitLocker with a TPM, and ideally a startup PIN on laptops, to protect data if a device is lost, and store recovery keys in Active Directory or Entra ID. Credential Guard isolates secrets using virtualization-based security. Windows LAPS gives every computer a unique, rotating local administrator password.

Logging and administration

Enable the advanced audit policy categories for logon events, account management, process creation (event 4688 with command line), and policy changes, and size the Security log appropriately. Turn on PowerShell script block logging (event 4104) and forward events to a central collector using Windows Event Forwarding or an agent. Use tiered administration so domain admin credentials never touch ordinary workstations. Remove users from the local Administrators group and manage software through a deployment tool. Check changes against a baseline, because a later GPO or local override may silently reverse your setting.

POWERSHELL CHECK

```
PS> Get-SmbServerConfiguration | Select EnableSMB1Protocol,RequireSecuritySignature
EnableSMB1Protocol RequireSecuritySignature
-----
                False True
PS> Get-BitLockerVolume C: | Select ProtectionStatus
ProtectionStatus : On
```

WORKED EXAMPLE

A domain at corp.example.com audits NTLM before blocking it. The team enables the policy 'Network security: Restrict NTLM: Audit NTLM authentication in this domain' and collects events for a month. The logs reveal an old file server at 203.0.113.60 and one reporting tool still using NTLM. They fix those, then switch the policy to deny for the rest. Auditing before enforcing prevented an outage while still removing a path commonly abused for relay and hash capture.

KEY TERMS

GPO: Group Policy Object, a set of settings applied to AD users and computers.

SMBv1: A legacy file sharing protocol that should be disabled.

LLMNR: A multicast name resolution fallback that can be abused to capture credentials.

BitLocker: Windows full-volume encryption, often backed by a TPM.

Credential Guard: Feature that uses virtualization to protect credential secrets.

LAPS: Tool that rotates unique local administrator passwords.

REMEMBER

- Use GPOs and security baselines to apply consistent policy and confirm with gpresult.
- Disable SMBv1 and LLMNR, restrict NTLM after auditing, and require SMB signing.
- Enable Defender with tamper protection, ASR rules, BitLocker, Credential Guard, LAPS and strong auditing.

QUICK CHECK

1. Where do you configure settings that apply to many domain computers at once?

- A) Registry editor on each host
- B) Group Policy Objects linked to OUs
- C) Task Manager
- D) Device Manager

2. Which action reduces the risk of hash capture through name resolution fallback?

- A) Enable LLMNR
- B) Disable LLMNR
- C) Enable SMBv1
- D) Disable logging

3. What should usually be done before blocking NTLM?

- A) Nothing
- B) Audit its use to find dependencies
- C) Delete Kerberos
- D) Disable the domain

4. What does Windows LAPS provide?

- A) Unique rotating local admin passwords
- B) Full disk encryption
- C) Name resolution
- D) Printer sharing

Answers: 1-B 2-B 3-B 4-A

1. GPOs apply centrally to scoped OUs.
2. LLMNR can be spoofed by attackers on the local network.
3. Auditing reveals applications that still need NTLM.
4. LAPS prevents one shared local admin password.

Patch management lifecycle

Attackers weaponize published vulnerabilities within days. A reliable patch process closes that window without breaking production.

Inventory and prioritization

You cannot patch what you do not know you own, so the lifecycle begins with an accurate asset and software inventory that records operating system, version, owner, criticality and exposure. Monitor vendor advisories and vulnerability feeds, then prioritize. CVSS gives severity, but also consider whether the flaw is listed in CISA's Known Exploited Vulnerabilities catalog, whether the EPSS score shows high exploitation probability, whether the system is internet-facing, and how valuable its data is. Set service-level targets, for example critical and exploited within 72 hours, high within 14 days, and routine within 30 days.

Test and staged rollout

Test patches on a representative lab or pilot group before broad release, checking boot, key applications, agents and performance. Then roll out in rings: a pilot ring of IT and volunteers (about 5 percent), a broad ring of most systems, and a last ring of critical or fragile systems that receive patches after the others prove stable. Microsoft Patch Tuesday occurs on the second Tuesday of each month, which many teams use as the start of a monthly cycle. Schedule maintenance windows, notify owners, and cluster nodes so servers can be patched one at a time without downtime.

Tools

Windows environments use WSUS or Microsoft Configuration Manager, Windows Update for Business, or Microsoft Intune with update rings and deadlines to control timing. Linux uses package managers: apt on Debian and Ubuntu, dnf or yum on Red Hat family systems, with tools like unattended-upgrades or Red Hat Satellite for control and repository snapshots. Third-party applications, browsers and firmware also need patching, and container images must be rebuilt from updated base images. Whatever the tool, use one authoritative source, report compliance per device, and flag machines that missed the cycle.

Rollback and emergency patching

Plan rollback before you patch: take VM snapshots or backups, keep previous package versions available, and know how to uninstall an update or pin a package version, for example with apt-mark hold. Define success criteria and monitoring so a bad patch is detected quickly. An emergency, or out-of-band, process handles actively exploited flaws: shorten testing, approve through an expedited change path, patch internet-facing systems first, and apply compensating controls such as a WAF rule or disabling a feature when a patch is not available. Verify afterward with a vulnerability scan and record exceptions with an expiry date.

TERMINAL OUTPUT

```
$ apt list --upgradable 2>/dev/null | head -3
openssl/jammy-updates 3.0.2-0ubuntu1.18 amd64 [upgradable]
$ sudo apt-mark hold nginx
nginx set on hold.
$ sudo apt install --only-upgrade openssl
$ needrestart -r l
```

WORKED EXAMPLE

A critical remote code execution flaw appears in a web server product on the KEV list. The team skips the normal monthly cycle. They apply the patch to the lab within hours, confirm the application works, and then patch the internet-facing hosts at 198.51.100.10 and 198.51.100.11 one at a time behind the load balancer. Internal hosts follow the next day. A scan confirms the version, and a snapshot taken before patching remains available for 24 hours as the rollback path.

KEY TERMS

Patch ring: A group of systems that receive updates in a defined order.

KEV: CISA catalog of vulnerabilities known to be exploited in the wild.

EPSS: A score estimating the probability that a vulnerability will be exploited.

WSUS: Windows Server Update Services for approving and distributing Windows updates.

Out-of-band patch: An emergency update outside the normal schedule.

Compensating control: A temporary control that reduces risk when a fix is unavailable.

REMEMBER

- Start with an inventory and prioritize using CVSS, KEV, EPSS and exposure.
- Test, then roll out in rings with maintenance windows and monitoring.
- Plan rollback, and use an expedited emergency path with compensating controls for exploited flaws.

QUICK CHECK

1. Which input helps prioritize beyond CVSS?
 - A) Inclusion in the KEV catalog
 - B) Server color
 - C) Hostname length
 - D) Vendor logo
2. Why use patch rings?
 - A) To patch everything at once
 - B) To detect problems on a small group before broad rollout
 - C) To avoid inventory
 - D) To skip testing
3. What should exist before a patch is applied?
 - A) A rollback plan
 - B) A new firewall
 - C) A new vendor
 - D) A longer password
4. Which is a compensating control for an unpatched exposed flaw?
 - A) Ignoring it
 - B) A WAF rule or disabling the vulnerable feature
 - C) Disabling logs
 - D) Rebooting daily

Answers: 1-A 2-B 3-A 4-B

1. Known exploitation raises urgency.
2. Staging limits the blast radius of a bad patch.
3. Snapshots or pinned versions allow recovery.
4. Temporary mitigation reduces exposure until a patch is installed.

Configuration management tools

Manual changes drift and cannot be audited. Configuration management describes the desired state as code so every server converges to the same secure configuration.

Desired state and idempotence

Configuration management tools let you declare the state you want, such as 'package auditd is installed, service is running, file sshd_config has these contents', rather than scripting steps. The tool compares reality with the declaration and changes only what differs. This property is called idempotence: running the same configuration twice produces the same result and makes no change the second time. Idempotence makes it safe to run configurations repeatedly to correct drift, and the definitions become documentation and an audit trail when stored in version control with code review for changes.

Ansible

Ansible is agentless: a control node connects to managed hosts over SSH (or WinRM for Windows) and runs modules described in YAML playbooks. Inventories list hosts and groups, roles package reusable tasks, and variables separate data from logic. Modules such as lineinfile, template, package, service and sysctl are idempotent. Run `ansible-playbook` with `--check --diff` to preview changes without applying them. Secrets belong in Ansible Vault or an external secrets manager, not in plain text. Ansible runs when you trigger it, so schedule it or use a controller such as AWX to run regularly.

Puppet and DSC

Puppet uses an agent on each node that periodically (by default every 30 minutes) contacts a server, retrieves a compiled catalog based on manifests written in Puppet's declarative language, and enforces it. This pull model corrects drift automatically. Windows PowerShell Desired State Configuration (DSC) describes state in configurations compiled to MOF documents and applied by the Local Configuration Manager, which can run in `ApplyAndAutoCorrect` mode. Chef and Salt are similar alternatives. Choose by team skills and platform: agentless push suits ad hoc and small estates; agent pull suits large fleets needing continuous enforcement.

Operating at scale

Store all configuration in Git, review changes through pull requests, and test in CI against disposable machines with tools such as Molecule or containers before production. Use environments (development, staging, production) with the same code and different variables. Limit who can change code and who can run it, because the configuration tool holds powerful credentials on every host. Report on runs: which nodes failed, which changed, which did not check in. Pair configuration management with image building and compliance scanning, so a hardened baseline is defined once and enforced continuously.

ANSIBLE TASK

```
- name: Disable SSH root login
  ansible.builtin.lineinfile:
    path: /etc/ssh/sshd_config
    regexp: '^#?PermitRootLogin'
    line: 'PermitRootLogin no'
  notify: reload sshd
# run: ansible-playbook harden.yml --check --diff
```

WORKED EXAMPLE

A team manages 300 Linux servers. They write a role that installs auditd, sets sshd options and applies sysctl values. The first run changes most hosts. The second run reports `changed=0` on every host, which proves idempotence. A week later an engineer manually edits `PermitRootLogin` on one host for troubleshooting. The next scheduled run reverts it and records a change for that host, so the drift is both corrected and visible in the report.

KEY TERMS

Idempotence: Applying the same configuration repeatedly yields the same state.

Playbook: An Ansible file of tasks describing desired actions in YAML.

Agentless: Managing nodes without software installed on them, as Ansible does over SSH.

Catalog: The compiled set of resources Puppet enforces on a node.

DSC: PowerShell Desired State Configuration for Windows.

Infrastructure as code: Defining infrastructure and settings in version-controlled files.

REMEMBER

- Configuration management declares desired state and applies it idempotently.
- Ansible pushes over SSH, Puppet and DSC pull and can auto-correct drift.
- Keep configuration in Git, test before production, and protect the automation credentials.

QUICK CHECK

1. What does idempotent mean for a configuration run?
 - A) It always changes files
 - B) Re-running makes no further change once the state is correct
 - C) It deletes the host
 - D) It needs a reboot
2. Which tool is agentless and uses SSH?
 - A) Ansible
 - B) Puppet agent
 - C) Windows LCM
 - D) SCCM client
3. What does --check --diff do in ansible-playbook?
 - A) Applies changes silently
 - B) Previews changes without applying them
 - C) Deletes inventories
 - D) Encrypts the vault
4. Why store configuration in Git?
 - A) Review, history and rollback of changes
 - B) To avoid testing
 - C) To remove variables
 - D) To disable idempotence

Answers: 1-B 2-A 3-B 4-A

1. Idempotent operations converge to the same state.
2. Ansible connects over SSH without an agent.
3. Check mode simulates and diff shows modifications.
4. Version control gives accountability and recoverability.

Baselines, drift and compliance scanning

A hardened baseline is only valuable if new systems start from it and existing ones stay within it. Scanning and exception handling make that measurable.

Gold images and baselines

A baseline is the approved secure configuration for a system role. A gold image is a pre-built operating system image that already contains the baseline, patches and agents. Build images with automation such as HashiCorp Packer so they are reproducible, scan them before publishing, sign or version them, and rebuild regularly to include new patches. Deploy servers from the gold image and, where possible, treat them as immutable: replace instead of modifying. Keep separate baselines for roles such as web server, database and workstation, and record the benchmark version and profile each one follows.

Compliance scanners

OpenSCAP uses SCAP content, such as the SCAP Security Guide, to evaluate a system against profiles including CIS and DISA STIG; for example `oscap xccdf eval --profile xccdf_org.ssgproject.content_profile_cis --report report.html` and a datastream file. It can also generate remediation scripts. Lynis is an open-source auditing tool that runs lynis audit system, reports warnings and suggestions, and gives a hardening index. CIS-CAT, Microsoft Security Compliance Toolkit and cloud services such as AWS Config or Azure Policy do similar work. Use scanner output as evidence, and re-scan after changes.

Drift detection

Configuration drift is any deviation from the approved baseline, caused by emergency fixes, manual edits, failed updates or compromise. Detect it through scheduled scans, configuration management reports, file integrity monitoring tools such as AIDE or Tripwire, and cloud policy engines that flag non-compliant resources. Alert on high-risk drift, for example SSH settings changed or a new local administrator. Decide for each item whether to remediate automatically, remediate through a ticket, or investigate, since unexplained drift on a critical host may indicate an attacker making changes.

Exceptions and reporting

Some recommendations cannot be met because of application or business needs. Handle them through a formal exception: describe the setting, the reason, the compensating control, the risk owner, and an expiry date for review. Track exceptions in a register so scanners can mark them as accepted rather than failed, and so they do not accumulate silently. Report compliance as trend lines and exception counts to management. A healthy program shows rising compliance, shrinking drift time, and exceptions that are reviewed and retired, not a one-time perfect score.

LYNIS EXCERPT

```
$ sudo lynis audit system --quick
Hardening index : 74 [##### ]
Warnings (2):
! No password set for GRUB boot loader [BOOT-5122]
! iptables module(s) loaded, but no rules active [FIRE-4512]
Suggestions (18):
* Harden SSH: set PermitRootLogin to NO [SSH-7440]
```

WORKED EXAMPLE

A weekly OpenSCAP scan of 40 database servers finds that db07 now fails the rule requiring SSH root login to be disabled, although it passed last week. The team checks the change log and finds no ticket. File integrity records show the `sshd_config` modified at 02:14 by an account that does not normally administer the host. They escalate to the incident process. Drift detection converted a quiet configuration change into an investigation lead.

KEY TERMS

Baseline: The approved secure configuration for a system role.

Gold image: A prebuilt hardened system image used to deploy new hosts.

Configuration drift: Deviation of a system from its approved baseline.

OpenSCAP: Open-source tooling that evaluates systems against SCAP security content.

Lynis: An open-source host auditing tool that produces a hardening index.

Exception register: A record of approved deviations with owner, reason and expiry.

REMEMBER

- Baselines and gold images make secure builds repeatable.
- Scan with OpenSCAP, Lynis or CIS-CAT and use integrity monitoring to detect drift.
- Manage exceptions with owner, compensating controls and expiry, and report trends.

QUICK CHECK

1. What is configuration drift?
 - A) Deviation from the approved baseline
 - B) A network protocol
 - C) A type of malware
 - D) A backup method
2. Which tool can evaluate a Linux host against CIS or STIG profiles using SCAP content?
 - A) OpenSCAP
 - B) Wireshark
 - C) Nmap
 - D) Burp
3. What should a documented exception include?
 - A) Only a verbal agreement
 - B) Reason, compensating control, owner and expiry
 - C) A deleted finding
 - D) Nothing
4. Why rebuild gold images regularly?
 - A) To include new patches and baseline updates
 - B) To remove version control
 - C) To avoid scanning
 - D) To increase drift

Answers: 1-A 2-A 3-B 4-A

1. Drift is any departure from the baseline.
2. OpenSCAP uses SCAP content for profile evaluation.
3. Exceptions must be accountable and time-limited.
4. Images age as vulnerabilities are published.

Harden sshd_config

SCENARIO

Host jump01.example.com at 192.0.2.50 has a default sshd_config. Audit findings say root login is permitted, passwords are accepted, and every local user can log in. You must edit the file to meet the Level 1 recommendations while keeping admins able to connect.

OBJECTIVE

Edit sshd_config to disable root login and password authentication, set modern ciphers and restrict users.

STEPS IN THE SIMULATOR

- 1 **Do:** Open /etc/ssh/sshd_config in the editor.

Simulator shows:

File loaded: PermitRootLogin yes, PasswordAuthentication yes, no AllowUsers.

- 2 **Do:** Set PermitRootLogin to no.

Simulator shows:

Line updated. Check: PASS 5.2.8.

- 3 **Do:** Set PasswordAuthentication to no and PubkeyAuthentication to yes.

Simulator shows:

Check: PASS 5.2.10 key-based authentication enforced.

- 4 **Do:** Add AllowUsers alice bob.

Simulator shows:

Check: PASS access restricted to 2 users.

- 5 **Do:** Set MaxAuthTries 3 and LoginGraceTime 30.

Simulator shows:

Both values accepted.

- 6 **Do:** Add Ciphers chacha20-poly1305@openssh.com,aes256-gcm@openssh.com and remove arcfour entries.

Simulator shows:

Check: PASS no weak ciphers offered.

- 7 **Do:** Run 'sshd -t'.

Simulator shows:

No syntax errors.

- 8 **Do:** Click Apply and reload.

Simulator shows:

Service reloaded. Scan: 6 of 6 SSH checks pass.

ANSWER

The final file contains PermitRootLogin no, PasswordAuthentication no, PubkeyAuthentication yes, AllowUsers alice bob, MaxAuthTries 3, LoginGraceTime 30 and strong ciphers only, validated with sshd -t before the reload.

Hint shown in the app: Always run sshd -t and keep an existing session open before reloading.

Audit a Linux host

SCENARIO

You have a shell on app02.example.com (192.0.2.60) and a Lynis-style audit tool. The report lists a world-readable /etc/shadow, an unneeded telnet service and a missing default firewall policy. You must read the findings and fix each failing item.

OBJECTIVE

Run the audit, interpret the findings and remediate the permission and service items.

STEPS IN THE SIMULATOR

- 1 **Do:** Type: `lynis audit system --quick`

Simulator shows:

Hardening index: 61. Warnings: 3. Suggestions: 12.

- 2 **Do:** Type: `stat -c '%a %U:%G' /etc/shadow`

Simulator shows:

644 root:root (too permissive)

- 3 **Do:** Type: `chmod 640 /etc/shadow && chgrp shadow /etc/shadow`

Simulator shows:

Permissions updated.

- 4 **Do:** Type: `ss -tulpn | grep ':23'`

Simulator shows:

tcp LISTEN 0 5 0.0.0.0:23 users: (('telnetd'))

- 5 **Do:** Type: `systemctl disable --now inetd-telnet`

Simulator shows:

Service stopped and disabled.

- 6 **Do:** Type: `ufw default deny incoming && ufw allow 22/tcp && ufw enable`

Simulator shows:

Default policy set to deny; rule for 22/tcp added.

- 7 **Do:** Type: `lynis audit system --quick`

Simulator shows:

Hardening index: 78. Warnings: 0.

ANSWER

The fixes are setting /etc/shadow to mode 640 owned by root:shadow, disabling the telnet service, and applying a default deny inbound firewall policy with only SSH allowed. After the fixes, the hardening index rises from 61 to 78 with no warnings.

Hint shown in the app: Fix warnings first, then work through suggestions by risk.

Windows GPO review

SCENARIO

A baseline for member servers in corp.example.com requires SMBv1 disabled, NTLM restricted and audit policy for logon events and process creation. A policy editor shows the current GPO state with several settings not compliant. You must toggle settings until the baseline report is green.

OBJECTIVE

Set GPO values to meet the baseline for SMBv1, NTLM and auditing.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the GPO 'Member Server Baseline'.

Simulator shows:
Compliance 4 of 10 settings.

- 2 **Do:** Under Computer Configuration, set 'SMB 1.0 server' to Disabled.

Simulator shows:
Setting updated. Check SMBv1: PASS.

- 3 **Do:** Set 'Network security: LAN Manager authentication level' to Send NTLMv2 response only, refuse LM and NTLM.

Simulator shows:
Check: PASS level 5.

- 4 **Do:** Set 'Network security: Restrict NTLM: Audit NTLM authentication in this domain' to Enable all.

Simulator shows:
Audit enabled; a note says audit before denying.

- 5 **Do:** Enable Advanced Audit Policy: Logon success and failure.

Simulator shows:
Check: PASS logon auditing.

- 6 **Do:** Enable 'Include command line in process creation events' and audit Process Creation success.

Simulator shows:
Check: PASS event 4688 with command line.

- 7 **Do:** Click Run compliance check.

Simulator shows:
Compliance 10 of 10. NTLM block deferred until audit data is reviewed.

ANSWER

The compliant policy disables SMBv1, sets NTLMv2-only authentication, enables NTLM auditing as a first step toward restriction, and turns on logon and process creation auditing including command lines.

Hint shown in the app: Audit NTLM before blocking it.

Patch rollout plan

SCENARIO

A monthly update and an emergency out-of-band patch are due. The estate has a test lab, 20 pilot laptops, 400 general workstations, 12 internet-facing servers at 198.51.100.0/28 and 6 fragile production database servers. You must order the rollout and choose rollback steps.

OBJECTIVE

Create a ring-based plan for the monthly patch and handle an actively exploited vulnerability.

STEPS IN THE SIMULATOR

- 1 **Do:** Drag 'Test lab' to Ring 0.

Simulator shows:

Accepted: validation before any production system.

- 2 **Do:** Drag 'Pilot laptops' to Ring 1.

Simulator shows:

Accepted: 5 percent pilot with daily health check.

- 3 **Do:** Drag 'General workstations' to Ring 2 and 'Database servers' to Ring 3.

Simulator shows:

Accepted: fragile systems patched last.

- 4 **Do:** Choose rollback: snapshot before patch and keep prior version available.

Simulator shows:

Rollback plan marked adequate.

- 5 **Do:** Open the emergency scenario: actively exploited flaw in the web server.

Simulator shows:

Alert: KEV listed, internet-facing hosts affected.

- 6 **Do:** Move 'Internet-facing servers' to the front of the emergency queue and add a WAF rule as a compensating control.

Simulator shows:

Accepted: exposed hosts first, mitigation in place.

- 7 **Do:** Click Evaluate plan.

Simulator shows:

Score 100: rings ordered, rollback defined, emergency path correct.

ANSWER

Monthly order is lab, pilot, general workstations, then fragile database servers, with snapshots for rollback. For the exploited flaw, patch internet-facing servers first through an expedited change and apply a compensating WAF rule until patched.

Hint shown in the app: Exposure and exploitation decide emergency order; fragility decides normal order.

System Hardening: 10 questions, 80% to pass

1. A Level 2 CIS profile is intended for:
 - A) All systems without testing
 - B) Environments needing higher assurance, with possible usability impact
 - C) Printers only
 - D) Removing logging
2. Which sshd_config pair is most important to reduce password guessing?
 - A) PasswordAuthentication no and PubkeyAuthentication yes
 - B) X11Forwarding yes and UseDNS yes
 - C) PermitEmptyPasswords yes
 - D) Port 22 and Banner none
3. Which command shows the running sshd settings including include files?
 - A) sshd -T
 - B) ssh -V
 - C) ss -a
 - D) sshd -q
4. Which Windows setting helps stop credential capture through name resolution fallback?
 - A) Disable LLMNR
 - B) Enable SMBv1
 - C) Disable BitLocker
 - D) Enable guest account
5. What is the best reason to deploy patches in rings?
 - A) To skip testing
 - B) To limit the impact of a faulty patch
 - C) To avoid inventory
 - D) To remove rollback
6. Which source tells you a vulnerability is known to be exploited?
 - A) CISA KEV catalog
 - B) A vendor logo
 - C) CIS Level 1
 - D) Lynis hardening index
7. An idempotent configuration run on a compliant host:
 - A) Makes no changes
 - B) Reinstalls everything
 - C) Reboots the host
 - D) Removes the baseline
8. Which tool is agent-based and pulls its catalog from a server?
 - A) Puppet
 - B) Ad hoc SSH loop
 - C) Ansible over SSH
 - D) Netcat
9. A scan shows db07 failing a rule it passed last week, with no change ticket. This is:
 - A) Configuration drift to investigate
 - B) A normal improvement
 - C) A gold image
 - D) A false patch
10. A proper exception record contains:
 - A) Reason, compensating control, owner and expiry
 - B) Only a verbal note
 - C) The scanner password
 - D) A deleted finding

Answers: 1-B 2-A 3-A 4-A 5-B 6-A 7-A 8-A 9-A 10-A

1. Level 2 adds defense in depth and may affect function.
2. Key-only authentication eliminates password guessing.
3. sshd -T prints the effective configuration.
4. LLMNR can be spoofed to capture hashes.
5. Small early rings reveal problems before wide release.
6. KEV lists vulnerabilities exploited in the wild.
7. The state already matches, so nothing changes.
8. Puppet agents periodically retrieve and apply catalogs.
9. Unexplained drift may indicate error or compromise.
10. Exceptions must be accountable and reviewed.

Identity and Access Engineering

Intermediate | 7 lessons | 5 simulated labs | checkpoint | about 5h 35m

Identity is now the main security perimeter, and this module teaches you to engineer it. You will study directory services and Active Directory, access control models, SAML and OIDC single sign-on, and token validation. You will plan MFA rollouts that resist phishing and fatigue attacks, control privileged access with vaulting and just-in-time elevation, and manage service accounts and secrets without hardcoded credentials. Stolen or over-privileged credentials are involved in most breaches, so getting identity right removes the most common attack path.

BY THE END YOU CAN

- + Configure SSO using SAML and OIDC and read their tokens
- + Design MFA, PAM and service account controls
- + Store and rotate secrets without hardcoding them

IN THIS MODULE

Lesson 1 Directory services and AD architecture

Lesson 2 AuthN, AuthZ and access models

Lesson 3 SAML federation

Lesson 4 OAuth 2.0 and OIDC

Lesson 5 MFA rollout

Lesson 6 Privileged access management

Lesson 7 Service accounts and secrets

Lab 1 Decode a SAML assertion (Web proxy)

Lab 2 Inspect an OIDC token (Code editor)

Lab 3 MFA rollout plan (Scenario)

Lab 4 Audit AD groups (File explorer)

Lab 5 Remove hardcoded secrets (Code editor)

Checkpoint 10 questions, 80% to pass

Directory services and AD architecture

Nearly every enterprise login, permission and policy depends on a directory. Understanding its structure shows where attackers aim and how administrators should protect it.

LDAP and the directory tree

A directory is a hierarchical database of identities and resources. The Lightweight Directory Access Protocol (LDAP) is the standard way to query and modify it. Entries are identified by a distinguished name (DN), for example `cn=Alice Example,ou=Finance,dc=corp,dc=example,dc=com`, built from a relative name and its parent containers. LDAP listens on TCP 389, with StartTLS available, and LDAPS on 636; the global catalog uses 3268 and 3269. A client performs a bind to authenticate, then searches with a base DN, scope and filter. Anonymous or simple binds without encryption expose credentials, so require signing and TLS.

Kerberos in Active Directory

Active Directory (AD) combines LDAP, DNS and Kerberos. Kerberos, on port 88, uses tickets so passwords are not sent across the network. A user authenticates to the Key Distribution Center on a domain controller and receives a Ticket Granting Ticket (TGT), encrypted with the secret of the `krbtgt` account. To use a service, the client presents the TGT and gets a service ticket for that service principal name (SPN). Clocks must agree within five minutes by default. Attacks such as Kerberoasting, which cracks service ticket secrets of weak service account passwords, and golden tickets, which forge TGTs using the `krbtgt` secret, show why service passwords and the `krbtgt` account need protection.

Domains, OUs and groups

An AD forest is the security boundary, containing one or more domains that share a schema and trust each other. Within a domain, organizational units (OUs) organize objects and are the targets for Group Policy and delegated administration. Groups hold users and other groups. Security groups grant permissions; distribution groups only email. Scopes are domain local, global and universal. A common practice, AGDLP, places Accounts into Global groups, nests them into Domain Local groups, and assigns Permissions to those local groups. Privileged groups such as Domain Admins, Enterprise Admins and Schema Admins grant control over the whole directory and must be kept tiny and monitored.

Tiered administration

The tier model separates administrative credentials by what they control. Tier 0 covers domain controllers, the identity system and anything with control over them. Tier 1 covers servers and applications. Tier 2 covers user workstations and devices. An administrator in a higher tier never logs on to a lower tier, because credentials left in memory on a less trusted machine can be stolen and used upward. Use separate admin accounts, privileged access workstations, and the Protected Users group for sensitive accounts, and use authentication policy silos to restrict where Tier 0 accounts may sign in.

LDAP SEARCH AND DN

```
$ ldapsearch -H ldaps://dc01.corp.example.com -D "cn=svc-ldap,ou=Service,dc=corp,dc=example,dc=com" -W \
-b "ou=Finance,dc=corp,dc=example,dc=com" "(sAMAccountName=aexample)" memberOf
dn: CN=Alice Example,OU=Finance,DC=corp,DC=example,DC=com
memberOf: CN=GG-Finance-Users,OU=Groups,DC=corp,DC=example,DC=com
```

WORKED EXAMPLE

An engineer reviews `corp.example.com`. A server admin account, `adm-bob`, is a member of Domain Admins and also signs in to ordinary workstations to fix printers. This breaks tiering, since a credential used on a Tier 2 machine can be harvested from memory. The fix is a separate Tier 1 account for servers, no logon rights for admins on workstations, Domain Admins membership limited to a few Tier 0 accounts, and use of the Protected Users group.

KEY TERMS

LDAP: Protocol for querying and modifying directory entries.

Distinguished name: The unique path identifying a directory entry.

TGT: Ticket Granting Ticket issued by the Kerberos KDC after authentication.

SPN: Service principal name that identifies a Kerberos service.

AGDLP: Accounts to Global groups to Domain Local groups to Permissions.

Tier 0: The set of assets that control the identity system.

REMEMBER

- LDAP queries a directory tree, and Kerberos on port 88 uses TGTs and service tickets.
- AD organizes forests, domains, OUs and groups; use AGDLP and protect privileged groups.
- Use tiered administration so privileged credentials stay on trusted systems.

QUICK CHECK

1. Which port does Kerberos use?
 - A) 88
 - B) 389
 - C) 636
 - D) 3389
2. What encrypts the TGT in Active Directory?
 - A) The krbtgt account secret
 - B) The user's email
 - C) The firewall key
 - D) The SPN list
3. Why should Tier 0 admins avoid signing in to workstations?
 - A) It is slower
 - B) Credentials in memory can be stolen on less trusted systems
 - C) Workstations lack screens
 - D) It disables Kerberos
4. Which secure LDAP port is standard?
 - A) 389
 - B) 636
 - C) 110
 - D) 25

Answers: 1-A 2-A 3-B 4-B

1. Kerberos uses TCP and UDP 88.
2. The krbtgt secret protects TGTs, so it must be guarded.
3. Higher privilege credentials must not be exposed on weaker tiers.
4. LDAPS uses TCP 636.

AuthN, AuthZ and access models

Authentication proves who someone is, and authorization decides what they may do. Choosing the right model and keeping permissions current prevents privilege creep.

Authentication versus authorization

Authentication (AuthN) establishes identity by verifying something the user knows, has or is. Authorization (AuthZ) decides what an authenticated identity may do. A system can authenticate correctly yet authorize badly, as in an application where any logged-in user can open any account number. Accounting records what happened. Keep the concepts apart in design: identity providers handle AuthN, while each application or a central policy service handles AuthZ per request. Never treat the presence of a valid session as permission, and check authorization on the server for each action and object.

RBAC, ABAC and ReBAC

Role-based access control (RBAC) grants permissions to roles, and users get roles. It is simple to audit but can suffer role explosion when many special cases arise. Attribute-based access control (ABAC) evaluates policies over attributes of the user, resource, action and context, such as 'allow if department equals finance and device is compliant and time is business hours'. It is flexible and fine-grained, but policies are harder to review. Relationship-based access control (ReBAC), used in systems inspired by Google Zanzibar, grants access from relationships, such as 'viewer of a folder is viewer of files inside'. Many organizations combine models.

Least privilege and the identity lifecycle

Least privilege gives users only the access needed. Enforce it through the joiner, mover, leaver (JML) process. Joiners receive a role-based starting set from HR data. Movers must lose access tied to the old job, because accumulated permissions, called privilege creep, are a major source of excess access. Leavers are disabled on their last day or sooner, with sessions and tokens revoked and owned resources reassigned. Automate through HR-driven provisioning with SCIM or an identity governance tool, and periodically run access recertification campaigns where managers confirm or remove each person's entitlements.

Separation of duties and review

Separation of duties prevents a single person from completing a risky process alone, for example creating and approving a vendor payment. Define conflicting role pairs and have the governance tool block or flag them. Review privileged and sensitive entitlements more often than ordinary ones, such as quarterly for administrators and annually for standard access. Log every grant and revocation with approver and reason. Orphaned accounts, shared accounts and accounts without owners should be found and removed. Measure the process using time to deprovision, percentage of accounts with an owner, and the number of overdue reviews.

POLICY EXAMPLES

```
RBAC : user alice -> role Finance-Analyst -> permission reports:read
ABAC : allow if user.dept == "finance"
           and device.compliant == true
           and resource.classification <= "internal"
ReBAC: folder:Q3#viewer@group:finance#member
       doc:budget#parent@folder:Q3 -> finance can view doc:budget
```

WORKED EXAMPLE

When Dana moves from Support to Finance at example.com, the mover process assigns the Finance-Analyst role but does not remove her old Support-Admin role. Six months later an access review shows she can both approve refunds and issue them. Recertification catches the conflict. The company adds a mover trigger that removes prior role assignments and a separation-of-duties rule that blocks the two roles from coexisting.

KEY TERMS

Authentication: Verifying the identity of a user or system.

Authorization: Deciding what an authenticated identity may do.

RBAC: Access control by assigning permissions to roles.

ABAC: Access control by evaluating attributes and context.

ReBAC: Access control derived from relationships between entities.

Privilege creep: Accumulation of access beyond what a current role needs.

REMEMBER

- AuthN proves identity; AuthZ checks permission for every action.
- RBAC is simple, ABAC is flexible, ReBAC follows relationships, and many systems mix them.
- Automate joiner, mover and leaver, recertify access, and enforce separation of duties.

QUICK CHECK

1. Which describes authorization?
 - A) Proving who you are
 - B) Deciding what an identity may do
 - C) Encrypting a disk
 - D) Resolving a hostname
2. Which model evaluates user, resource and context attributes?
 - A) ABAC
 - B) RBAC only
 - C) MAC only
 - D) DAC only
3. What is privilege creep?
 - A) Faster logins
 - B) Accumulated access not needed for a current role
 - C) A network attack
 - D) A password policy
4. What process most directly fixes excess entitlements over time?
 - A) Access recertification
 - B) More logging
 - C) Bigger firewalls
 - D) Longer passwords

Answers: 1-B 2-A 3-B 4-A

1. Authorization governs permitted actions.
2. ABAC policies use attributes.
3. Movers keep old permissions unless removed.
4. Managers review and remove unneeded access.

SAML federation

SAML is still the backbone of enterprise SSO for many SaaS applications. Reading an assertion correctly is how you find misconfigurations and forged responses.

The SP-initiated flow

SAML 2.0 federates identity between an Identity Provider (IdP) and a Service Provider (SP). In the SP-initiated flow a user visits the application, which has no session. The SP creates an AuthnRequest and redirects the browser to the IdP, usually with the HTTP-Redirect binding. The IdP authenticates the user, then builds a SAML Response containing an Assertion and returns it to the browser as an auto-submitted form using the HTTP-POST binding to the SP's Assertion Consumer Service (ACS) URL. The SP validates the response and creates a session. Metadata files exchange entity IDs, certificates and endpoints in advance.

Anatomy of an assertion

An assertion contains an Issuer (the IdP entity ID), a Subject with a NameID and SubjectConfirmationData, Conditions with NotBefore, NotOnOrAfter and an AudienceRestriction, an AuthnStatement saying when and how the user authenticated, and an AttributeStatement with values such as email and groups. SubjectConfirmationData holds Recipient, which must equal the ACS URL, InResponseTo, which should match the original request ID, and its own NotOnOrAfter. The XML is signed using XML Digital Signature, with a signature over the Response, the Assertion, or both, and the SP holds the IdP's public certificate.

What the SP must validate

The SP must verify the signature with the trusted IdP certificate and make sure the signature covers the element it actually uses, because XML signature wrapping attacks insert unsigned content next to a validly signed element. It must also check Issuer, check that the Audience equals its own entity ID, Recipient equals its ACS URL, the current time lies within NotBefore and NotOnOrAfter (allowing small clock skew), and InResponseTo matches an outstanding request. It should reject replayed assertion IDs. Accepting unsigned responses or ignoring audience lets an attacker reuse assertions meant for other applications.

Operating SAML safely

Use a maintained SAML library rather than writing XML parsing yourself, and disable external entity processing to avoid XXE. Prefer signing both the response and assertion where supported, and encrypt assertions that carry sensitive attributes. Plan certificate rollover, since an expired IdP signing certificate breaks all logins at once, by publishing the new certificate in metadata before switching. Keep assertion lifetimes short, a few minutes. Map groups to roles carefully and validate that attribute values come from the signed assertion. Log failed validations with reason so attacks and misconfigurations can be distinguished.

SAML ASSERTION (TRIMMED)

```
<saml:Assertion ID="_a1b2" IssueInstant="2026-03-02T09:00:00Z">
  <saml:Issuer>https://idp.example.com/metadata</saml:Issuer>
  <saml:Subject><saml:NameID>alice@example.com</saml:NameID>
    <saml:SubjectConfirmationData Recipient="https://app.example.com/saml/acs"
      InResponseTo="_req77" NotOnOrAfter="2026-03-02T09:05:00Z"/></saml:Subject>
  <saml:Conditions NotBefore="2026-03-02T08:59:30Z" NotOnOrAfter="2026-03-02T09:05:00Z">
    <saml:AudienceRestriction><saml:Audience>https://app.example.com/sp</saml:Audience>
  </saml:AudienceRestriction></saml:Conditions></saml:Assertion>
```

WORKED EXAMPLE

A reviewer examines logs for app.example.com. The SP accepted a response whose Audience is https://other.example.com/sp and whose signature element was missing. Both are failures: the audience is not this application, and an unsigned response cannot be trusted. Anyone able to tamper with the browser POST could set NameID to an administrator. The fix is to require a valid signature from the IdP certificate, reject mismatched audience, recipient and expired NotOnOrAfter values, and add replay detection.

KEY TERMS

IdP: Identity Provider that authenticates users and issues assertions.

SP: Service Provider that consumes assertions to grant access.

ACS: Assertion Consumer Service, the SP endpoint receiving SAML responses.

NotOnOrAfter: Timestamp after which an assertion or confirmation is invalid.

Audience restriction: Limits the assertion to a named service provider.

XML signature wrapping: Attack that moves signed content to confuse validation.

REMEMBER

- SP-initiated SAML redirects with an AuthnRequest and returns a signed Response by POST to the ACS.
- Assertions carry Issuer, Subject, Conditions, Audience, NotOnOrAfter and attributes.
- Validate signature, audience, recipient, time and InResponseTo, and use a maintained library.

QUICK CHECK

1. Which binding commonly carries the SAML Response to the SP?

- A) HTTP-POST
- B) FTP
- C) SMTP
- D) DNS

2. What should an SP check in the Audience?

- A) It equals its own entity ID
- B) It equals the IdP name
- C) It is empty
- D) It matches any value

3. What does NotOnOrAfter protect against?

- A) Use of stale assertions
- B) Slow networks
- C) Weak ciphers
- D) Large attributes

4. What is the main defense against XML signature wrapping?

- A) Verify the signature covers the exact element you consume, using a maintained library
- B) Use shorter URLs
- C) Disable HTTPS
- D) Trust any signed element

Answers: 1-A 2-A 3-A 4-A

1. The IdP posts the response to the ACS.
2. Audience restriction binds the assertion to this SP.
3. It limits the time an assertion can be used.
4. The processed content must be the signed content.

OAuth 2.0 and OIDC

OAuth 2.0 and OpenID Connect power modern web, mobile and API logins. Correct flow choice and token validation are what separate secure integrations from account takeovers.

Roles and the difference between OAuth and OIDC

OAuth 2.0 is an authorization framework in which a client obtains an access token from an authorization server to call a resource server on behalf of a resource owner. It does not by itself say who the user is. OpenID Connect (OIDC) adds an identity layer: when the scope openid is requested, the authorization server also returns an ID token, a JSON Web Token (JWT) asserting who authenticated and how. Scopes such as profile, email and custom API scopes limit what the access token permits. Access tokens are for APIs; ID tokens are for the client and should not be sent to APIs.

Authorization code flow with PKCE

The recommended flow is authorization code with Proof Key for Code Exchange (PKCE, RFC 7636). The client generates a random code_verifier and sends its SHA-256 hash as code_challenge with code_challenge_method=S256 in the authorization request, along with client_id, redirect_uri, scope, a random state and, for OIDC, a nonce. After authentication the server redirects back with a code. The client exchanges the code at the token endpoint, supplying the code_verifier, and the server checks it matches the challenge. An attacker who steals the code cannot redeem it. Use exact redirect URI matching, and avoid the implicit flow.

Validating the ID token

A client must validate the ID token before trusting it. Check the signature using keys from the provider's JWKS endpoint, selecting by kid, and allow only expected algorithms; never accept alg none or let the token header choose a weaker algorithm. Check iss equals the expected issuer, aud contains your client_id, exp has not passed, iat is reasonable, and nonce matches the value sent, which prevents replay. Verify state on the redirect to stop cross-site request forgery. Resource servers validate access tokens similarly, checking audience and scope for each request, or call the introspection endpoint for opaque tokens.

Common mistakes

Frequent errors include using a loose redirect URI pattern that lets attackers capture codes, storing tokens in places scripts can read, requesting broad scopes, skipping nonce or state, and trusting claims from a token whose signature was not verified. Keep access tokens short-lived, use refresh token rotation, and bind tokens to the client where possible. Treat the sub claim together with iss as the stable user identifier, not email, which can change. Public clients such as single-page and mobile applications cannot keep secrets, so rely on PKCE rather than a client secret. Log token validation failures for monitoring.

DECODED ID TOKEN CLAIMS

```
{
  "iss": "https://login.example.com",
  "sub": "u-4821",
  "aud": "client-web-01",
  "exp": 1767225900,
  "iat": 1767225600,
  "nonce": "n-0S6_WzA2Mj",
  "email": "alice@example.com"
}
```

header: {"alg": "RS256", "kid": "2026-01"}

WORKED EXAMPLE

A single-page app at app.example.com receives an ID token. The developer's code only base64-decodes the payload and reads email. A reviewer points out that without signature verification, anyone can craft a token. The fix: fetch the JWKS, verify RS256 against the kid, require iss to be https://login.example.com, aud to equal client-web-01, exp in the future and nonce to match the stored value, and reject the token if any check fails.

KEY TERMS

PKCE: Extension that binds an authorization code to the client that requested it.

ID token: A JWT asserting the user's authentication result in OIDC.

JWKS: The set of public keys published to verify token signatures.

Nonce: A random value that ties an ID token to a specific request.

Scope: A string that limits what an access token allows.

State: A random value that protects the redirect from forgery.

REMEMBER

- OAuth authorizes API access; OIDC adds an ID token for authentication.
- Use authorization code with PKCE, exact redirect URIs, state and nonce.
- Verify signature, iss, aud, exp and nonce, and reject alg none.

QUICK CHECK

1. What does PKCE protect against?
 - A) Redeeming a stolen authorization code
 - B) Weak passwords
 - C) Large cookies
 - D) DNS spoofing
2. Which claim must match your client_id?
 - A) aud
 - B) iss
 - C) kid
 - D) iat
3. What should a validator do with a token whose alg is none?
 - A) Accept it
 - B) Reject it
 - C) Cache it
 - D) Decode only
4. Which token is meant to be sent to an API?
 - A) Access token
 - B) ID token
 - C) State value
 - D) Nonce

Answers: 1-A 2-A 3-B 4-A

1. The verifier proves the redeemer started the flow.
2. aud identifies the intended recipient.
3. Unsigned tokens must never be trusted.
4. ID tokens are for the client; access tokens are for APIs.

MFA rollout

Passwords alone fail against phishing and credential stuffing. A well-planned MFA program raises the cost of attack without overwhelming users or help desks.

Factor types and strength

Multi-factor authentication combines at least two of something you know, have or are. Time-based one-time passwords (TOTP, RFC 6238) from an authenticator app generate a code every 30 seconds; they are good against password reuse but can be phished by a fake login page. Push approvals are convenient but susceptible to fatigue prompts and blind approval. SMS and voice codes are the weakest, exposed to SIM swapping and interception. FIDO2 and WebAuthn security keys and passkeys use public key cryptography bound to the site's origin, so a lookalike domain cannot obtain a valid response, making them phishing-resistant.

Planning phased enrollment

Roll out in phases. Start with administrators and privileged users, then remote access and email, then high-value applications, then everyone. Run a pilot to find usability problems. Communicate early with clear instructions, support hours and deadlines. Prefer registering at a moment of strong identity proof, such as onboarding or from a trusted network, to stop an attacker enrolling their own device on a compromised account. Use conditional access to require MFA more strictly for risky sign-ins, such as new devices or impossible travel, and enforce phishing-resistant methods first for Tier 0 and finance roles.

Recovery and help desk

Recovery is the weak point attackers target, because a social engineered reset bypasses MFA. Issue backup codes or a second registered factor at enrollment. Require identity verification for resets, such as a manager approval or video check, and never reset solely on a caller's claim. Notify the user and the security team when factors change, and delay or limit sensitive actions after a reset. Document break-glass accounts for outages of the MFA service itself, protected by long random passwords and alerts on use. Measure enrollment coverage, method distribution and help desk reset volume.

Mitigating MFA fatigue

MFA fatigue, or push bombing, floods a user with approval requests until they accept one. Defenses include number matching, where the user must type a number shown on the login screen, showing application and location context in the prompt, rate limiting prompts and locking after repeated denials, and alerting when a user denies several prompts or reports one as not theirs. Train users to report unexpected prompts instead of approving them, and make reporting easy. Prefer phishing-resistant methods over push where possible. Review sign-in logs for patterns such as many denials followed by a success from a new location.

SIGN-IN LOG EXCERPT

```
2026-04-11T02:14:07Z user=bob@example.com mfa=push result=denied ip=203.0.113.88
2026-04-11T02:14:41Z user=bob@example.com mfa=push result=denied ip=203.0.113.88
2026-04-11T02:15:12Z user=bob@example.com mfa=push result=denied ip=203.0.113.88
2026-04-11T02:15:50Z user=bob@example.com mfa=push result=approved ip=203.0.113.88
ALERT: 3 denials then approval from new location -> block and review
```

WORKED EXAMPLE

A company enrolls 2,000 staff. Phase 1 gives FIDO2 keys to 40 administrators. Phase 2 enrolls remote access users with authenticator apps and number-matching push. Phase 3 covers everyone, with registration only from the office network or after a manager-verified link. The help desk script requires video verification for resets. After deployment, an alert fires on a user who denied three prompts at 2 a.m. and then approved one, and the security team blocks the session and resets the password.

KEY TERMS

TOTP: A time-based one-time password generated every 30 seconds.

FIDO2: Standards for public key authentication using security keys or passkeys.

WebAuthn: The web API that lets sites use FIDO2 credentials.

Number matching: A push approval requiring the user to enter a displayed number.

MFA fatigue: Repeated prompts meant to trick a user into approving.

Break-glass account: An emergency account used if normal authentication fails.

REMEMBER

- TOTP, push and SMS vary in strength; FIDO2 and WebAuthn resist phishing.
- Roll out in phases starting with admins, and enroll under strong identity proof.
- Secure recovery, and counter fatigue with number matching, limits and alerts.

QUICK CHECK

1. Which factor type is the most phishing-resistant?
 - A) SMS code
 - B) FIDO2 security key
 - C) Email code
 - D) Security question
2. What is a common defense against push bombing?
 - A) Number matching
 - B) Longer logos
 - C) More SMS
 - D) Disabling logs
3. Who should receive MFA first in a phased rollout?
 - A) Privileged users
 - B) Guests only
 - C) Printers
 - D) No one
4. Why is the reset process an attack target?
 - A) It is rarely used
 - B) A social engineered reset can bypass MFA
 - C) It encrypts data
 - D) It needs a firewall

Answers: 1-B 2-A 3-A 4-B

1. FIDO2 binds the response to the site origin.
2. The user must enter a number shown on the login screen.
3. Administrator accounts carry the highest risk.
4. Weak recovery undermines strong factors.

Privileged access management

Privileged accounts can change or destroy everything. PAM limits how long, how widely and how invisibly those powers can be used.

Vaulting and rotation

Privileged access management (PAM) controls accounts with elevated rights, such as domain admins, root, database administrators and cloud owners. A vault stores privileged credentials, encrypted and with strong access control. Users check out a credential through an approval workflow, or the PAM system injects it into a session without revealing it. After use, the credential is rotated automatically, so a stolen copy is soon useless. Shared and default accounts such as root or sa are brought under the vault, and each checkout is tied to a named person and a ticket. Discovery tools find unmanaged privileged accounts.

Just-in-time elevation

Standing privilege, where an account holds admin rights permanently, gives attackers a constant target. Just-in-time (JIT) elevation grants rights only when needed and only for a set time, for example 60 minutes, after a request and approval. Examples include Microsoft Entra Privileged Identity Management, time-limited sudo rules, and cloud role assumption with short sessions. Requests can require MFA, a ticket reference and a justification, and can be scoped to a single system. The goal is zero standing privilege for most administrators, with permanent rights reserved for a few audited accounts.

Session recording and monitoring

A PAM gateway or bastion can proxy SSH, RDP and database sessions, recording keystrokes and screens, and logging commands. Recordings give accountability, speed investigations and support compliance. Real-time monitoring can alert on risky commands or terminate a session. Protect recordings from tampering and restrict who may view them, since they can contain sensitive data. Forward PAM events to the SIEM and correlate them with change tickets so activity without a matching ticket stands out. Note that recording a session does not stop an attack; it must be paired with approvals, least privilege and alerting.

Break-glass accounts

Break-glass or emergency accounts restore access when normal systems, such as the identity provider or PAM vault, are unavailable. Create a small number, such as two, with long random passwords stored offline in a sealed, access-controlled location, excluded from conditional access policies that depend on the failed system but protected by other controls. Alert immediately on any use, and review each use. Test the process periodically, then rotate the credentials after every use. Without monitoring, a break-glass account becomes a silent backdoor, so treat each sign-in as an incident until verified.

PAM CHECKOUT RECORD

```
request : id=PAM-5521 user=ops-carol target=db-prod-01 (192.0.2.90)
approval : approver=mgr-dave ticket=CHG-1043 duration=60m
session : proxied SSH, recording=on, start=14:02 end=15:01
rotate : account dbadmin password rotated at 15:02
event : forwarded to SIEM, matched to CHG-1043
```

WORKED EXAMPLE

A database administrator at example.com previously kept a permanent dbadmin password in a shared spreadsheet. After PAM adoption, the account password is vaulted and unknown to people. When Carol needs to fix an issue she opens ticket CHG-1043, gets approval, and the PAM gateway opens a recorded session for 60 minutes. Afterward the password rotates. Months later an auditor can show who accessed the database, when, why, and what commands ran.

KEY TERMS

PAM: Controls and tools for managing and monitoring privileged accounts.

Vault: Secure store for privileged credentials with controlled checkout.

JIT access: Granting elevated rights only when needed and for a limited time.

Standing privilege: Permanent administrative rights held by an account.

Session recording: Capturing privileged sessions for review and accountability.

Break-glass account: Emergency account used only when normal access fails.

REMEMBER

- Vault privileged credentials, check them out with approval and rotate after use.
- Use just-in-time elevation to approach zero standing privilege.
- Record sessions, forward events to the SIEM, and monitor break-glass accounts closely.

QUICK CHECK

1. What is the main benefit of just-in-time elevation?

- A) Reduces standing privilege
- B) Removes MFA
- C) Disables logging
- D) Speeds up DNS

2. Why rotate a credential after each checkout?

- A) A copied credential becomes useless
- B) To use more storage
- C) To remove audit
- D) To change usernames

3. What must accompany break-glass accounts?

- A) Alerts on every use
- B) No password
- C) Public sharing
- D) Daily use

4. Does session recording alone prevent misuse?

- A) Yes, fully
- B) No, it supports accountability but needs other controls
- C) Yes, for cloud only
- D) It replaces MFA

Answers: 1-A 2-A 3-A 4-B

1. Rights exist only for the approved time.
2. Rotation limits the value of a stolen secret.
3. Use must be detected and reviewed.
4. Recording is detective and must pair with preventive controls.

Service accounts and secrets

Applications need credentials too, and hardcoded passwords in code or scripts are among the most common and damaging mistakes.

Why service accounts are risky

Service accounts let applications and scripts authenticate without a person. They often have static passwords that never expire, broad permissions, no MFA and no owner, which makes them attractive targets and hard to audit. In AD, weak passwords on service accounts with SPNs enable Kerberoasting. Reduce risk by inventorying each account with an owner and purpose, scoping permissions to least privilege, forbidding interactive logon, and monitoring for use from unusual hosts. Where possible replace them with identities the platform manages, so no human ever knows or types a password.

Managed identities and short-lived credentials

Managed identities remove stored secrets. Windows group Managed Service Accounts (gMSA) have passwords of 240 characters that the domain rotates automatically. In cloud platforms, workloads receive an identity such as an AWS IAM role, an Azure managed identity or a Google Cloud service account bound to the workload, and obtain short-lived tokens from a metadata or token service. Kubernetes can issue projected service account tokens, and OIDC workload identity federation lets pipelines such as CI jobs exchange a signed token for cloud access without long-lived keys. Short lifetimes of minutes to hours limit stolen token value.

Secrets management and rotation

A secrets manager such as HashiCorp Vault, AWS Secrets Manager or Azure Key Vault stores secrets encrypted, authenticates the requesting application, authorizes access by policy, and logs every read. Vault can create dynamic secrets, such as a database user generated on request with a lease of one hour that is revoked on expiry, so no shared password exists. Static secrets should be rotated automatically on a schedule and immediately after suspected exposure. Applications fetch secrets at start or on demand, hold them in memory, and handle rotation without restarting where possible.

Keeping secrets out of code

Never hardcode credentials in source, container images, configuration committed to Git, or chat. Once committed, a secret persists in history and should be treated as compromised and rotated. Use pre-commit and CI secret scanning, for example gitleaks or GitHub secret scanning, to catch leaks. Inject secrets at runtime through the platform's secret store, mounted files or a Vault agent, and note that plain environment variables can leak through process listings, crash dumps and logs, so apply care. Separate secrets per environment, grant each application access to only its own secrets, and audit reads for anomalies.

BEFORE AND AFTER

```
# BEFORE (bad)
DB_PASS = "Summer2025!" # hardcoded, in Git history
# AFTER
import hvac, os
c = hvac.Client(url="https://vault.example.com:8200")
c.auth.approle.login(role_id=os.environ["ROLE_ID"], secret_id=os.environ["SECRET_ID"])
cred = c.secrets.database.generate_credentials("orders-ro") # 1h lease
db_user, db_pass = cred["data"]["username"], cred["data"]["password"]
```

WORKED EXAMPLE

A nightly backup script on host 192.0.2.70 contains a domain administrator password. A review finds it in a Git repository, so the password is rotated at once and treated as exposed. The team replaces the account with a gMSA limited to the backup share, removes the secret from history, adds gitleaks to CI, and retrieves the one remaining database secret from Vault at runtime. The job now has no human-known password and only the rights it needs.

KEY TERMS

Service account: A non-human identity used by applications or scripts.

gMSA: Group Managed Service Account with automatically rotated password.

Managed identity: A platform-provided workload identity with no stored secret.

Dynamic secret: A credential generated on demand with a limited lease.

Secret scanning: Automated detection of credentials in code and repositories.

Workload identity federation: Exchanging a trusted token for cloud access without static keys.

REMEMBER

- Inventory service accounts, scope them tightly and prevent interactive logon.
- Prefer managed identities, gMSA and short-lived tokens over static credentials.
- Use a secrets manager with rotation and scanning, and never hardcode or commit secrets.

QUICK CHECK

1. What should you do after discovering a secret committed to Git?
 - A) Delete the file only
 - B) Treat it as compromised and rotate it
 - C) Ignore it
 - D) Make the repo private only
2. What is a benefit of a gMSA?
 - A) Passwords rotate automatically and are not known to people
 - B) It disables Kerberos
 - C) It needs MFA prompts
 - D) It is always interactive
3. What does a dynamic secret provide?
 - A) A short-lived credential created on request
 - B) A permanent shared password
 - C) A public key
 - D) An email alias
4. Which is the best way to give a cloud workload access without long-lived keys?
 - A) Managed identity or workload identity federation
 - B) Hardcoded access key
 - C) Shared admin password
 - D) Public bucket

Answers: 1-B 2-A 3-A 4-A

1. History retains the secret, so it must be rotated.
2. The domain manages a long random password.
3. Leases limit exposure and are revoked at expiry.
4. Short-lived tokens replace stored keys.

Decode a SAML assertion

SCENARIO

A web proxy has intercepted a SAML POST to `https://app.example.com/saml/acs` during a security test of a fictional training app. Support reports a user could log in as another person. You must inspect the decoded response and identify why the SP accepted it.

OBJECTIVE

Spot the missing signature validation and the incorrect audience in the assertion.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the intercepted POST and select the SAMLResponse parameter.

Simulator shows:

Base64 value shown, 2,314 characters.

- 2 **Do:** Click Decode.

Simulator shows:

XML Response displayed with one Assertion.

- 3 **Do:** Inspect the Issuer element.

Simulator shows:

Issuer: `https://idp.example.com/metadata` (expected).

- 4 **Do:** Look for a `ds:Signature` element.

Simulator shows:

No `ds:Signature` found in Response or Assertion.

- 5 **Do:** Check Conditions and AudienceRestriction.

Simulator shows:

Audience: `https://other.example.com/sp` (expected `https://app.example.com/sp`).

- 6 **Do:** Check NotOnOrAfter against the clock.

Simulator shows:

NotOnOrAfter: `09:05:00Z`; current `09:41:12Z`. Expired.

- 7 **Do:** Open the SP validation log.

Simulator shows:

Log: signature check skipped (config `verify_signature=false`).

- 8 **Do:** Click Submit findings.

Simulator shows:

Findings recorded: 3 defects.

ANSWER

The assertion has no signature, its audience names a different service provider, and it is past NotOnOrAfter, yet the SP accepted it because signature verification is disabled. Enable signature verification, enforce audience, recipient and time checks, and add replay detection.

Hint shown in the app: Check signature, audience and time before looking at attributes.

Inspect an OIDC token

SCENARIO

A single-page app validates ID tokens with a small JavaScript function. A tester reports that a token with header alg set to none was accepted. The editor shows the validation code and a decoded token. You must find and fix the validation bug.

OBJECTIVE

Correct the token validation so it enforces signature, algorithm and claim checks.

STEPS IN THE SIMULATOR

- 1 **Do:** Open validate.js and the sample token.

Simulator shows:

Token header: {alg:none}; payload shows iss, aud, exp, nonce.

- 2 **Do:** Run the existing test suite.

Simulator shows:

FAIL: unsigned token accepted (expected rejection).

- 3 **Do:** Read the verifyToken function.

Simulator shows:

Line 12: algorithm taken from the token header: `jwt.verify(t, key, {algorithms: [header.alg]}).`

- 4 **Do:** Change to a fixed allow list: `algorithms: ['RS256']`.

Simulator shows:

Line 12 updated.

- 5 **Do:** Add checks: iss must equal `https://login.example.com` and aud must contain `client-web-01`.

Simulator shows:

Lines 14-15 added.

- 6 **Do:** Add checks for exp and for nonce equal to the stored value.

Simulator shows:

Lines 16-17 added.

- 7 **Do:** Run the test suite again.

Simulator shows:

PASS 8 of 8: alg none rejected, wrong aud rejected, expired rejected, wrong nonce rejected.

ANSWER

The bug was letting the token header choose the verification algorithm, which allowed alg none. The fix uses a fixed RS256 allow list with JWKS keys and verifies iss, aud, exp and nonce, so unsigned or mismatched tokens are rejected.

Hint shown in the app: Never take the algorithm from the token you are verifying.

MFA rollout plan

SCENARIO

Example Corp has 1,500 staff: 30 administrators, 120 finance users, 200 remote contractors and 1,150 general staff. The help desk is small, and a recent phishing campaign targeted finance. You must choose factors per group, order enrollment and define safe recovery.

OBJECTIVE

Build a phased MFA plan with appropriate factors and a secure recovery process.

STEPS IN THE SIMULATOR

- 1 **Do:** Assign factors to Administrators.

Simulator shows:

Option chosen: FIDO2 security keys. Rated: phishing-resistant.

- 2 **Do:** Assign factors to Finance.

Simulator shows:

Option chosen: FIDO2 or passkeys. Rated: strong, addresses recent phishing.

- 3 **Do:** Assign factors to Contractors.

Simulator shows:

Option chosen: authenticator app with number matching. Rated: acceptable.

- 4 **Do:** Assign factors to General staff.

Simulator shows:

Option chosen: authenticator app, passkeys optional. Rated: acceptable.

- 5 **Do:** Set enrollment order: Admins, Finance, Contractors, General.

Simulator shows:

Order accepted: highest risk first.

- 6 **Do:** Set recovery: backup codes at enrollment, manager and video verification for resets.

Simulator shows:

Recovery rated: strong.

- 7 **Do:** Add alert: 3 denied prompts followed by an approval.

Simulator shows:

Fatigue detection enabled.

- 8 **Do:** Click Evaluate.

Simulator shows:

Score 95: SMS avoided, privileged users first, recovery verified.

ANSWER

Use FIDO2 keys for administrators and finance, authenticator apps with number matching for others, enroll highest risk first, protect recovery with identity verification, and alert on repeated denials. Avoid SMS where possible.

Hint shown in the app: Match factor strength to the risk of the group.

Audit AD groups

SCENARIO

You are given an exported view of corp.example.com groups in a file explorer. Domain Admins should contain only a few Tier 0 accounts, but an audit suggests stale and nested memberships grant extra privilege. You must browse and flag problems.

OBJECTIVE

Find stale admins and nested privilege paths and flag them for removal.

STEPS IN THE SIMULATOR

- 1 **Do:** Open Groups > Domain Admins.

Simulator shows:

Members: adm-alice, adm-bob, svc-backup, jdoe-old, group: IT-Helpdesk.

- 2 **Do:** Open the account properties of jdoe-old.

Simulator shows:

Last logon: 412 days ago. Status: enabled.

- 3 **Do:** Flag jdoe-old as stale admin.

Simulator shows:

Flag added.

- 4 **Do:** Open svc-backup.

Simulator shows:

Service account with password never expires, last set 5 years ago. SPN present.

- 5 **Do:** Flag svc-backup as over-privileged service account.

Simulator shows:

Flag added with note: use gMSA with limited rights.

- 6 **Do:** Open IT-Helpdesk and view its members.

Simulator shows:

Members include nested group Contractors-Temp (14 users).

- 7 **Do:** Trace the path.

Simulator shows:

Path: Contractors-Temp -> IT-Helpdesk -> Domain Admins. 14 users have domain admin.

- 8 **Do:** Flag the nested path and submit.

Simulator shows:

3 findings submitted.

ANSWER

Three findings: jdoe-old is an enabled stale admin, svc-backup is an over-privileged service account with an old never-expiring password, and nested groups give 14 temporary contractors Domain Admin rights through IT-Helpdesk. Remove each.

Hint shown in the app: Expand every nested group; effective membership is what counts.

Remove hardcoded secrets

SCENARIO

A deployment script `backup.py` at `example.com` contains a database password and an API key in plain text and is stored in a Git repository. You must remove them and read secrets safely at runtime using environment-injected values and a Vault lookup.

OBJECTIVE

Replace hardcoded credentials with injected secrets and a Vault lookup, and ensure scanning catches regressions.

STEPS IN THE SIMULATOR

- 1 **Do:** Open `backup.py`.

Simulator shows:

Line 4: `DB_PASS = 'Summer2025!'` Line 5: `API_KEY = 'ak_live_example123'`.

- 2 **Do:** Run the secret scanner.

Simulator shows:

2 findings: generic password, API key.

- 3 **Do:** Replace `DB_PASS` with a Vault lookup using `hvac` and an `AppRole`.

Simulator shows:

Code now calls `client.secrets.kv.v2.read_secret_version(path='backup/db')`.

- 4 **Do:** Read `ROLE_ID` and `SECRET_ID` from environment variables injected by the platform.

Simulator shows:

`os.environ['ROLE_ID']` and `os.environ['SECRET_ID']` in use.

- 5 **Do:** Remove `API_KEY` and read it from the same Vault path.

Simulator shows:

No literal secrets remain in the file.

- 6 **Do:** Run the scanner again.

Simulator shows:

0 findings in working tree. Warning: 2 findings in Git history.

- 7 **Do:** Click Rotate exposed secrets.

Simulator shows:

Both old secrets marked revoked; new values issued.

ANSWER

The script should hold no literal secrets: it authenticates to Vault with injected `AppRole` values and reads the database password and key at runtime. Because the old values were committed, rotate them. Add pre-commit and CI scanning to catch regressions.

Hint shown in the app: Removing a secret from the file is not enough if it was committed; rotate it.

Identity and Access Engineering: 10 questions, 80% to pass

1. Which port does Kerberos use by default?
 - A) 88
 - B) 443
 - C) 161
 - D) 514
2. Why should Tier 0 admin accounts avoid workstation logons?
 - A) Credentials can be stolen from less trusted machines
 - B) It blocks LDAP
 - C) It breaks DNS
 - D) It disables GPO
3. Which model grants access based on user, resource and context attributes?
 - A) ABAC
 - B) Role list only
 - C) ACL on files only
 - D) Discretionary only
4. Which SAML check ensures the assertion was meant for this application?
 - A) Audience restriction
 - B) The Issuer color
 - C) NameID length
 - D) ACS file size
5. In SAML, why is an unsigned response dangerous?
 - A) Anyone could alter the NameID
 - B) It uses too much bandwidth
 - C) It breaks TLS
 - D) It changes DNS
6. In OIDC, the nonce is used to:
 - A) Bind the ID token to the original request and prevent replay
 - B) Encrypt the token
 - C) Hash the password
 - D) Select the scope
7. Which ID token claim must contain your client_id?
 - A) aud
 - B) kid
 - C) jti
 - D) scope
8. Which MFA method is most phishing-resistant?
 - A) FIDO2 or WebAuthn
 - B) SMS
 - C) Email OTP
 - D) Security questions
9. What is the best description of just-in-time privilege?
 - A) Admin rights granted temporarily when needed
 - B) Permanent admin for all
 - C) Shared password
 - D) Disabled MFA
10. What must you do with a secret that was committed to Git?
 - A) Rotate it
 - B) Leave it
 - C) Rename the file
 - D) Tag the commit

Answers: 1-A 2-A 3-A 4-A 5-A 6-A 7-A 8-A 9-A 10-A

1. Kerberos uses port 88.
2. Tiering contains credential theft.
3. ABAC evaluates attributes in policy.
4. Audience binds the assertion to the SP.
5. Without a signature, tampering is undetectable.
6. The client compares nonce with the value it sent.
7. aud names the intended audience.
8. WebAuthn credentials are origin-bound.
9. JIT limits standing privilege.
10. Git history keeps it, so treat it as exposed.

Network Security Engineering

Intermediate | 7 lessons | 5 simulated labs | checkpoint | about 5h 35m

This module teaches how to design the controls that protect a network. You will compare firewall types and write ordered allow-list rules, build segmented zones, and apply microsegmentation to stop lateral movement. You will then choose among VPN, ZTNA, WAF and network access control for specific needs, and defend DNS and absorb DDoS attacks. By the end you can read a network diagram, judge where each control belongs, and explain the trade-offs to colleagues and auditors.

BY THE END YOU CAN

- + Design firewall rules and segmented network zones
- + Choose VPN, ZTNA, WAF and NAC controls for a given need
- + Defend DNS and absorb DDoS attacks

IN THIS MODULE

Lesson 1 Firewall types and rule design

Lesson 2 Network segmentation and zones

Lesson 3 Microsegmentation

Lesson 4 VPN and ZTNA

Lesson 5 WAF and reverse proxies

Lesson 6 DNS security and DDoS defense

Lesson 7 Network access control

Lab 1 Write firewall rules (Config editor)

Lab 2 Segment the network (Diagram builder)

Lab 3 Tune a WAF (Log viewer)

Lab 4 Spot DNS abuse (Packet viewer)

Lab 5 Pick the right access control (Scenario)

Checkpoint 10 questions, 80% to pass

Firewall types and rule design

A firewall is only as good as its rules, and most real-world firewall failures are rule design failures, not product failures.

Stateless packet filters

A stateless packet filter judges every packet on its own, using only header fields: source and destination address, protocol, and port, as in a classic router access control list. It keeps no memory of earlier packets. That makes it very fast and simple, but it cannot tell whether a packet belongs to a conversation your user started. To allow replies to outbound web browsing, a stateless filter must permit inbound traffic from any source port 443 to high ports, which also lets an attacker craft packets that look like replies. Stateless ACLs still have a place on routers, switch ports and cloud subnet lists, where speed matters and the policy is coarse.

Stateful inspection and NGFW

A stateful firewall tracks connections in a state table. When an inside host at 192.0.2.10 opens a TCP connection to 203.0.113.5 port 443, the firewall records the address pair, ports, and TCP state. Return packets that match an entry are allowed automatically, and unsolicited inbound packets are dropped. A next-generation firewall (NGFW) adds application identification, user identity from directory integration, intrusion prevention signatures, URL categories and optional TLS decryption. Rules can then say 'finance group may use the banking application' instead of 'any host may use port 443'. The cost is more CPU, more licensing, and a decryption policy that needs privacy and legal review.

Rule order and default deny

Most firewalls evaluate rules top to bottom and apply the first match, so order is part of the policy. Specific rules go above general ones, and an explicit deny for known-bad traffic can sit near the top. The last rule is a default deny that drops and logs everything not previously allowed. This allow-list model is safer than a deny-list, because you enumerate what is needed instead of guessing what is dangerous. Beware shadowed rules, where an earlier broad rule makes a later one unreachable, and any rule whose source and destination are both 'any'. Such rules are usually emergency fixes that nobody removed.

Designing and maintaining a rule set

Start from data flows, not from ports. For a three-tier application, the load balancer accepts TCP 443 from the internet, the web tier reaches the application tier on one defined port, and only the application tier reaches the database on TCP 5432. Name objects after their role, add a comment with a ticket number and an owner to each rule, and set an expiry for temporary ones. Log denies and sampled allows to a central collector. Review rules at least quarterly, using hit counters to find unused rules, and test changes in a staging firewall or with a change window and rollback plan.

ORDERED ALLOW-LIST (GENERIC SYNTAX)

```
1 allow tcp any -> 198.51.100.10:443 # public LB
2 allow tcp 198.51.100.10 -> 10.1.2.0/24:8443 # web to app
3 allow tcp 10.1.2.0/24 -> 10.1.3.5:5432 # app to db
4 allow udp 10.1.0.0/16 -> 192.0.2.53:53 # DNS
99 deny any any -> any any log # default deny
```

WORKED EXAMPLE

A retail company's cloud-adjacent datacenter has a rule 'allow any any' added during a late-night outage and never removed. Months later a scan shows the database reachable from the guest Wi-Fi. A review replaces it with five role-based rules: the load balancer on TCP 443, web to app on 8443, app to database on 5432, DNS to the internal resolver, and a logged default deny. Hit counters show two older rules never matched and are deleted. The rule count shrinks while the exposed surface shrinks with it.

KEY TERMS

Stateless filter: Firewall that checks each packet alone using header fields, with no connection memory.

Stateful inspection: Tracking connection state so return traffic is allowed automatically.

NGFW: Firewall adding application, user and threat awareness to stateful filtering.

Default deny: Final rule that drops and logs anything not explicitly allowed.

Shadowed rule: A rule that can never match because an earlier rule already catches its traffic.

REMEMBER

- Stateful firewalls track connections; stateless filters judge single packets.
- Rules are evaluated in order, ending in a logged default deny.
- Design from data flows, document each rule, and review with hit counters.

QUICK CHECK

1. Why is a stateless filter weaker for return traffic?
 - A) It cannot read IP addresses
 - B) It cannot tell replies from crafted packets so must allow broad inbound ranges
 - C) It encrypts all packets
 - D) It only works on UDP
2. In first-match processing, where should specific rules go?
 - A) Below general rules
 - B) Above general rules
 - C) Anywhere, order never matters
 - D) Only after the default deny
3. What does an NGFW add over a stateful firewall?
 - A) Layer 1 repeaters
 - B) Application and user awareness plus integrated threat prevention
 - C) Faster physical ports only
 - D) Removal of the need for rules
4. Which design is best for a database server?
 - A) Allow any to port 5432
 - B) Allow only the application tier to port 5432, deny the rest with logging
 - C) Allow the whole internal network on all ports
 - D) Disable logging for speed

Answers: 1-B 2-B 3-B 4-B

1. Without a state table, replies are allowed by broad port patterns that attackers can imitate.
2. A broad rule above would match first and make the specific rule unreachable.
3. NGFWs identify applications and users and often include IPS and URL filtering.
4. Least-open ports and a named source implement allow-list design.

Network segmentation and zones

Flat networks let one compromised laptop reach everything; zones turn a breach into a contained incident.

Why segment

Segmentation divides a network into zones with different trust levels and controls the traffic between them. Without it, a single phished workstation can scan the whole environment, reach file servers and move laterally to the domain controller. With it, the attacker meets a firewall at every boundary, and each crossing is logged and limited. Segmentation also reduces compliance scope: PCI DSS, for example, lets you shrink the cardholder data environment if it is isolated from the rest of the network. Smaller zones also cut broadcast noise and make troubleshooting easier, so the security benefit comes with an operational one.

Typical zones

A common design has an untrusted internet zone, a DMZ (demilitarized zone) holding public-facing servers such as web proxies and mail gateways, an internal user zone, a server or data zone, and a separate management zone for administrative interfaces, jump hosts and backup systems. Guest Wi-Fi gets its own isolated zone with internet-only access. Operational technology (OT), such as factory controllers, belongs in its own zone behind an industrial DMZ, following the Purdue model idea that business IT and control systems should exchange only brokered data. Each zone has a clear purpose and an owner.

VLANs and ACLs between zones

On switches, a VLAN (IEEE 802.1Q) creates a separate Layer 2 broadcast domain, and a trunk port carries several VLANs tagged with a 12-bit ID. VLANs alone do not enforce policy, because a Layer 3 switch will route between them freely. Traffic between zones must pass a firewall or a router ACL. Avoid VLAN hopping risks by disabling DTP negotiation, setting unused ports to an unused access VLAN and shutting them down, and changing the native VLAN away from VLAN 1. Keep management interfaces off user VLANs and reachable only from the management zone.

Designing the flows

Draw the allowed flows as a matrix: rows are source zones, columns are destinations, and each cell lists the permitted ports or 'none'. Most cells should be empty. Users reach the DMZ proxy on 443, the DMZ reaches only specific internal application ports, and nothing in the DMZ may initiate to user networks. The management zone may reach everything on SSH or HTTPS, but only from hardened jump hosts with MFA. Plan egress too: servers rarely need the open internet. Document the matrix, turn it into firewall rules, and test it by attempting forbidden connections.

ZONE FLOW MATRIX (EXCERPT)

```
src\dst DMZ USERS DATA MGMT
INTERNET 443 none none none
USERS 443 - 443,445 none
DMZ - none 8443 (app) none
MGMT 22,443 22 22,443 -
```

WORKED EXAMPLE

A manufacturer's office laptops and its packaging-line controllers share one 10.0.0.0/16 network. After a malware outbreak in the office halts the line, engineers create four zones: office, servers, DMZ and an OT zone. A firewall between office and OT permits only a historian server in an industrial DMZ to pull data on one defined port. The next infection stays in the office zone and the packaging line keeps running, because no path existed from laptops to the controllers.

KEY TERMS

DMZ: Zone for public-facing services that limits what an attacker can reach if they are compromised.

VLAN: Logical Layer 2 broadcast domain defined by an 802.1Q tag.

Trunk port: Switch port carrying multiple VLANs using tags.

OT zone: Network segment holding industrial control systems, isolated from business IT.

Flow matrix: Table of which source zones may reach which destination zones and on which ports.

REMEMBER

- Zones group assets by trust and purpose; firewalls control crossings.
- VLANs separate broadcast domains but need ACLs or firewalls for policy.
- A flow matrix turns design into testable rules.

QUICK CHECK

1. What does a DMZ primarily do?
 - A) Speeds up the internet link
 - B) Hosts public services in a buffer zone so a breach there does not reach internal systems
 - C) Stores backups
 - D) Replaces the firewall
2. Why are VLANs alone not enough for policy?
 - A) VLANs encrypt traffic
 - B) Routing between VLANs is allowed by default unless a firewall or ACL blocks it
 - C) VLANs only work on Wi-Fi
 - D) They cap bandwidth
3. Which hardening step reduces VLAN hopping risk?
 - A) Leaving DTP on auto
 - B) Disabling DTP and moving the native VLAN off VLAN 1
 - C) Using VLAN 1 for management
 - D) Enabling all unused ports
4. Who should reach the management zone's admin interfaces?
 - A) Any user VLAN
 - B) Only hardened jump hosts with MFA
 - C) The internet
 - D) Guest Wi-Fi

Answers: 1-B 2-B 3-B 4-B

1. The DMZ isolates exposed servers from the inside network.
2. A Layer 3 device routes between VLANs, so policy needs ACLs or a firewall.
3. Turning off trunk negotiation and changing the native VLAN blocks common hopping tricks.
4. Administrative access is restricted to controlled, authenticated paths.

Microsegmentation

Perimeter zones do not stop movement inside a zone, so modern designs apply policy at the workload itself.

Limiting east-west movement

North-south traffic crosses the data center boundary, while east-west traffic moves between servers inside it. Attackers who gain a foothold depend on east-west paths to reach valuable systems, using SMB, RDP, SSH or database ports. Traditional zone firewalls rarely see this traffic because it stays within a VLAN or subnet. Microsegmentation places enforcement at the individual host or workload, so that even two servers in the same subnet cannot talk unless a policy says so. The goal is to make each workload its own small zone with only the connections its function requires.

Where enforcement lives

Policy can be enforced in several places. Host firewalls such as nftables, iptables or Windows Defender Firewall, managed centrally, work on any platform. Hypervisor-based tools filter at the virtual switch, independent of the guest operating system. Cloud security groups and Kubernetes NetworkPolicy objects enforce at the virtual NIC or pod. Agent-based products program host firewalls from a central controller. Each option trades coverage, visibility and operational effort. Agentless network approaches are easier to deploy but see less context, while host agents see process and user detail.

Tags, labels and identity

Static IP-based rules break in dynamic environments where containers and VMs change address constantly. Microsegmentation therefore writes policy against labels: role=web, app=billing, env=prod. A rule might read 'workloads labelled role=web may connect to role=app on TCP 8443'. Newer designs add cryptographic workload identity, such as SPIFFE identities with mutual TLS in a service mesh, so that the policy follows verified identity rather than an address that could be spoofed. Good labelling depends on an accurate inventory, so tag hygiene becomes a security task.

Rolling it out safely

Never start with enforcement. First observe in a monitor-only mode, build a map of real flows over several weeks, and group them into applications. Write allow policies for the observed legitimate flows and then enforce on low-risk applications before critical ones. Begin by blocking the highest-risk lateral protocols, such as SMB and RDP between workstations, and by isolating crown-jewel systems like domain controllers. Keep a break-glass procedure, alert on denied flows because they may reveal scanning, and re-run the flow mapping after each application release to keep policy current.

KUBERNETES NETWORKPOLICY (EXAMPLE)

```
kind: NetworkPolicy
spec:
  podSelector: {matchLabels: {role: db}}
  policyTypes: [Ingress]
  ingress:
    - from:
      - podSelector: {matchLabels: {role: app}}
      ports: [{protocol: TCP, port: 5432}]
```

WORKED EXAMPLE

An attacker compromises a reporting server at 10.20.4.15 and tries to reach file servers on TCP 445 in the same subnet. Because the flow was never in the observed map, the host-based policy drops the connections and logs 40 denied attempts in two minutes. The security team sees the spike in their dashboard and isolates the server. Without microsegmentation, the same scan would have succeeded because the packets never crossed a firewall.

KEY TERMS

East-west traffic: Traffic between internal workloads, as opposed to traffic entering or leaving the network.

Microsegmentation: Per-workload policy that restricts which systems may communicate.

Label-based policy: Rules written against tags such as role or environment instead of IP addresses.

Service mesh: Infrastructure layer providing workload identity and mutual TLS between services.

Monitor-only mode: Observing and logging flows without blocking, used before enforcement.

REMEMBER

- Microsegmentation makes each workload its own zone.
- Use labels or identity rather than IPs for dynamic environments.
- Observe, then enforce gradually, starting with lateral protocols and crown jewels.

QUICK CHECK

1. What problem does microsegmentation address?
 - A) Slow internet links
 - B) Lateral movement inside a zone
 - C) Weak Wi-Fi passwords
 - D) Missing backups
2. Why write policy using labels instead of IP addresses?
 - A) Labels are encrypted
 - B) Workloads change addresses often and labels follow the workload
 - C) IPs are illegal to use
 - D) Labels need no inventory
3. What is the safest first rollout step?
 - A) Enforce deny everywhere
 - B) Observe real flows in monitor-only mode
 - C) Delete the perimeter firewall
 - D) Disable logging
4. Which tool enforces pod-level policy in Kubernetes?
 - A) NetworkPolicy
 - B) DHCP snooping
 - C) Spanning tree
 - D) NAT64

Answers: 1-B 2-B 3-B 4-A

1. It restricts east-west connections that perimeter firewalls never see.
2. Dynamic infrastructure makes IP rules brittle; labels track role.
3. Observation avoids breaking legitimate application flows.
4. NetworkPolicy objects define allowed pod traffic, given a CNI that supports them.

VPN and ZTNA

Remote access is a main entry point for attackers, so choosing between a network-level tunnel and per-application access is a core design decision.

IPsec, WireGuard and SSL VPN

IPsec uses IKEv2 on UDP 500 and 4500 to negotiate keys and ESP (protocol 50) to protect packets. It is a mature standard, widely supported on routers and ideal for site-to-site tunnels. WireGuard is a small modern protocol on a configurable UDP port, using Curve25519, ChaCha20-Poly1305 and static public keys; its short codebase eases auditing, and it connects quickly. SSL or TLS VPNs, such as OpenVPN or vendor portals, run over TCP or UDP 443 and traverse restrictive networks easily. Whatever the protocol, use modern ciphers, certificate or key-based authentication, MFA for users, and prompt patching, since VPN gateways are heavily targeted.

Full tunnel, split tunnel and the VPN weakness

With a full tunnel, all client traffic goes through the corporate network, allowing inspection but adding load and latency. Split tunneling sends only corporate destinations through the VPN, which saves bandwidth but lets the client bridge the internet and the internal network at once. The deeper weakness of a traditional VPN is that it grants network-level access: once connected, a user or an attacker with stolen credentials often sees a whole subnet. A compromised contractor laptop becomes a doorway. Restrict VPN pools with firewall rules, and give each user group only the destinations it needs.

Zero trust network access

ZTNA applies the zero trust principle: never trust based on network location, and verify every request. A ZTNA broker authenticates the user with SSO and MFA, checks device posture such as disk encryption and patch level, and then grants access to one specific application, not to a network. Applications are not exposed to the internet or to client scanning, because connectors make outbound-only connections to the broker. Policy can use identity, group, device health, location and risk score, and can be re-evaluated continuously. Access is logged per application, which improves visibility over a flat VPN.

Choosing between them

Use IPsec or WireGuard for site-to-site links and for administrators who need broad network access to specific management segments, protected by MFA and jump hosts. Use ZTNA for employees and contractors who need a handful of web or SSH applications, particularly on unmanaged devices where you want to avoid network-level reach. Many organizations run both during migration: ZTNA for new access, VPN for legacy protocols that need full network connectivity. Evaluate failure modes too: a single broker is a dependency, so plan high availability, and keep a tested emergency path for administrators.

WIREGUARD CLIENT (DOCUMENTATION VALUES)

```
[Interface]
Address = 10.8.0.2/32
PrivateKey = <client-private-key>
[Peer]
PublicKey = <server-public-key>
Endpoint = vpn.example.com:51820
AllowedIPs = 10.20.0.0/24 # split tunnel
```

WORKED EXAMPLE

A consulting firm gives 30 contractors VPN access to the whole 10.20.0.0/16 network. One contractor's personal laptop is infected and the attacker uses the open tunnel to scan file servers. The firm moves contractors to ZTNA: each person can reach only the ticketing web app and one Git server, and only from devices that pass a posture check. The next stolen credential yields access to two applications instead of a whole subnet, and the broker logs every request.

KEY TERMS

IKEv2: Protocol that negotiates keys and security associations for IPsec.

ESP: IPsec payload protocol (IP protocol 50) that encrypts and authenticates packets.

Split tunnel: VPN setup that sends only some traffic through the tunnel.

ZTNA: Access model granting per-application access after verifying identity and device.

Device posture: Health signals such as encryption, patch level and EDR status used in access decisions.

REMEMBER

- VPNs give network access; ZTNA gives per-application access.
- Use MFA, modern ciphers and rapid patching on any gateway.
- Choose by need: site links and admins use VPN, general users and contractors use ZTNA.

QUICK CHECK

1. Which VPN type is most common for site-to-site tunnels?

- A) IPsec
- B) Telnet
- C) SMTP
- D) FTP

2. What is the main weakness of a traditional VPN?

- A) It cannot encrypt
- B) It grants broad network-level access once connected
- C) It needs no authentication
- D) It only supports IPv6

3. What does ZTNA grant access to?

- A) The entire LAN
- B) Specific applications after identity and device checks
- C) Only DNS
- D) The router's console

4. Which is a risk of split tunneling?

- A) The client may bridge the internet and the internal network
- B) All traffic is inspected
- C) The VPN stops working
- D) Keys are shared

Answers: 1-A 2-B 3-B 4-A

1. IPsec is the standard for gateway-to-gateway tunnels.
2. Stolen credentials can expose a whole subnet.
3. ZTNA is application-level, not network-level.
4. A compromised internet-side process can reach the internal network through the client.

WAF and reverse proxies

A web application firewall shields web apps from common attacks and buys time for fixes, but only if it is tuned.

What a WAF does

A web application firewall inspects HTTP and HTTPS at Layer 7. It looks for patterns of attacks such as SQL injection, cross-site scripting, path traversal, command injection and protocol abuse, and can block, challenge, or log them. It also enforces limits on request size, methods and rate. A WAF does not fix vulnerable code; it reduces exposure while developers patch, which is called virtual patching. Network firewalls pass TCP 443 without understanding what is inside the request, and that is exactly the gap a WAF fills.

Reverse proxy placement

A WAF usually runs as a reverse proxy: clients connect to it, it terminates TLS, inspects the decrypted request, and forwards clean traffic to the origin servers over a new connection. This hides the origin addresses, centralizes certificates, and allows caching and load balancing. Deployment options include a cloud service in front of the site, a virtual appliance, or a module such as ModSecurity on NGINX or Apache. The origin must accept traffic only from the WAF's address ranges, otherwise attackers bypass it by connecting directly. Pass the real client address in X-Forwarded-For and have the application log it.

OWASP Core Rule Set and modes

The OWASP Core Rule Set (CRS) is a free, maintained set of generic detection rules for ModSecurity-compatible engines. It uses anomaly scoring: each matching rule adds points, and the request is blocked when the total reaches a threshold, such as 5 for inbound traffic. Paranoia levels 1 to 4 trade coverage for false positives; level 1 suits most sites. Deploy in detection-only mode first, collecting logs for one to two weeks of real traffic, and only then switch to blocking. Skipping this phase usually blocks legitimate customers on day one.

Tuning false positives

A false positive occurs when legitimate traffic trips a rule, for example a rich-text editor posting HTML that resembles script injection. Read the audit log to find the rule ID, the matched variable, and the URL. Prefer narrow exclusions: disable rule 941100 only for the argument 'body' on the path /editor/save, not globally. Record the reason, owner and date for each exclusion. Review blocks regularly, watch for real attacks hidden among noise, and retest after application releases. Combine the WAF with rate limiting, bot controls and secure coding, since a WAF can be evaded by determined attackers.

MODSECURITY AUDIT ENTRY (SHORTENED)

```
[client 203.0.113.50] ModSecurity: Access denied with code 403
[id "942100"] [msg "SQL Injection Attack Detected via libinjection"]
[data "Matched Data: s&l found within ARGS:q: 1' or '1'='1"]
[uri "/search"] [hostname "shop.example.com"]
[anomaly_score 5]
```

WORKED EXAMPLE

A shop turns on blocking mode directly after installing a WAF. Within hours, support tickets arrive: customers cannot save product reviews. The log shows rule 941100 firing on ARGS:review because customers use angle brackets. The team switches back to detection mode, adds a narrow exclusion for that single parameter on /reviews/post, and keeps every other rule active. Meanwhile the same logs show a genuine SQL injection probe from 203.0.113.50, correctly blocked. Tuning removed noise without removing protection.

KEY TERMS

WAF: Layer 7 filter that inspects HTTP requests for web attacks.

Virtual patching: Using WAF rules to block an exploit until the code can be fixed.

OWASP CRS: Community rule set for ModSecurity-compatible WAFs using anomaly scoring.

Anomaly score: Cumulative points from matching rules that trigger blocking at a threshold.

False positive: Legitimate traffic wrongly flagged as an attack.

REMEMBER

- A WAF filters web attacks at Layer 7 and enables virtual patching.
- Deploy as a reverse proxy, lock the origin, run detection first, then block.
- Tune with narrow, documented exclusions and keep fixing the code.

QUICK CHECK

1. Why start a WAF in detection-only mode?
 - A) It is faster
 - B) To learn real traffic and find false positives before blocking customers
 - C) Blocking is not supported
 - D) It disables TLS
2. What stops attackers from bypassing a cloud WAF?
 - A) Hiding the website name
 - B) Restricting the origin to accept only WAF address ranges
 - C) Disabling logging
 - D) Using HTTP only
3. What is the best way to handle a false positive?
 - A) Turn off the WAF
 - B) Create a narrow exclusion for the rule, parameter and path
 - C) Delete CRS
 - D) Allow all traffic from the user
4. A WAF primarily operates at which layer?
 - A) Layer 2
 - B) Layer 4 only
 - C) Layer 7
 - D) Layer 1

Answers: 1-B 2-B 3-B 4-C

1. Observation prevents legitimate users from being blocked.
2. If the origin is open, direct connections skip the WAF.
3. Targeted exclusions keep protection elsewhere.
4. It inspects application-layer HTTP content.

DNS security and DDoS defense

DNS is both a critical dependency and a favorite abuse channel, and availability attacks target both it and everything else.

DNS integrity: DNSSEC, DoT and DoH

Classic DNS on UDP and TCP port 53 is unauthenticated and unencrypted, so an attacker can forge responses, a problem known as cache poisoning. DNSSEC adds digital signatures: zones sign records with RRSIG, publish keys in DNSKEY, and a validating resolver checks the chain of trust up to the root. It proves authenticity but does not provide privacy. DNS over TLS (DoT, TCP 853) and DNS over HTTPS (DoH, TCP 443) encrypt the client-to-resolver hop. Enterprises should run validating internal resolvers, and decide deliberately whether to allow or block DoH to outside resolvers, which can bypass filtering.

Controlling DNS abuse

Attackers use DNS for command and control, data exfiltration, and tunneling, encoding data in long subdomain labels such as a1b2c3.tunnel.example.net. Defenses start with forcing all clients to use approved resolvers and blocking outbound port 53 and known DoH endpoints from elsewhere. A response policy zone (RPZ) lets a resolver return NXDOMAIN or a sinkhole address for malicious domains from threat feeds. Monitor for high query rates, unusual TXT record volume, very long names and high-entropy labels. Disable open recursion, split internal and external DNS, and restrict zone transfers (AXFR) to named secondary servers.

DDoS types and absorbing volume

Distributed denial of service attacks exhaust resources and fall into three groups. Volumetric attacks, such as DNS or NTP amplification, saturate bandwidth, using spoofed source addresses so that a small query yields a large reply. Protocol attacks, such as SYN floods, exhaust connection tables. Application attacks, such as HTTP floods, consume server work with seemingly valid requests. Volume cannot be defeated by a single site's link, so the main defenses are scale: anycast, which announces the same IP from many locations so traffic spreads, and scrubbing centers, which divert traffic through filters and return only clean packets. Content delivery networks absorb and cache.

Local controls and readiness

Several local measures help. SYN cookies let a server answer SYN packets without allocating state until the final ACK arrives, defeating SYN floods. Rate limiting and response rate limiting (RRL) on DNS servers cap abuse. Ingress filtering following BCP 38 drops spoofed source addresses at network edges. Applications need timeouts, caching and autoscaling limits. Prepare a runbook: contact details for the upstream provider or scrubbing service, thresholds for activating mitigation, a baseline of normal traffic, and a communication plan. Test it with a controlled exercise, since the first real attack is a poor time to learn the process.

SIGNS IN A DNS CAPTURE (DOCUMENTATION VALUES)

```
Q TXT 4f3a9c1b8e7d2a6f.t.example.net len=64
Q TXT 91c0b7aa55e1d3f2.t.example.net len=64
... 600 queries/min from 192.0.2.77, all unique labels
vs. normal host: 15 queries/min, repeated names
```

WORKED EXAMPLE

A university DNS server begins answering thousands of queries per second from spoofed sources, sending large replies at a victim. Logs show a flood of ANY queries for the same domain from addresses all over the internet. The team disables open recursion, enables response rate limiting, and restricts recursive service to campus ranges. Traffic to the server falls within minutes. Separately, a monitoring rule flags host 192.0.2.77 for hundreds of unique long TXT queries per minute, which turns out to be a tunneling tool.

KEY TERMS

DNSSEC: Signatures that let resolvers verify DNS data is authentic and unmodified.

RPZ: Resolver policy that blocks or redirects queries to malicious domains.

Amplification: Reflected attack where a small spoofed query triggers a much larger response.

Anycast: Announcing one IP address from many locations so load and attacks are distributed.

SYN cookies: Technique that avoids storing state for half-open TCP connections.

REMEMBER

- DNSSEC proves authenticity; DoT and DoH add privacy for the client hop.
- Control resolvers, use RPZ, and monitor for tunneling patterns.
- DDoS needs scale (anycast, scrubbing) plus local controls like SYN cookies and rate limits.

QUICK CHECK

1. What does DNSSEC provide?
 - A) Encryption of queries
 - B) Authenticity and integrity of DNS data
 - C) Faster lookups
 - D) Hiding the resolver
2. Which is a sign of DNS tunneling?
 - A) Few queries to popular sites
 - B) Many long, unique, high-entropy subdomain queries from one host
 - C) Standard A records
 - D) Short TTLs on a CDN
3. How do SYN cookies defend against a SYN flood?
 - A) By encrypting SYN packets
 - B) By avoiding state allocation until the final ACK validates the client
 - C) By blocking all TCP
 - D) By raising TTL
4. What is a main benefit of anycast for DDoS?
 - A) It hides the IP address
 - B) It spreads attack traffic across many locations
 - C) It stops spoofing automatically
 - D) It removes the need for DNS

Answers: 1-B 2-B 3-B 4-B

1. It signs records but does not encrypt them.
2. Data is encoded into labels, producing unusual lengths and uniqueness.
3. The server keeps no half-open connection entries.
4. Traffic is absorbed by the nearest of many sites.

Network access control

Network access control decides which devices and users are allowed onto the wired and wireless network at all, before any firewall sees them.

The 802.1X model

IEEE 802.1X is port-based access control with three roles. The supplicant is the client device, the authenticator is the switch port or wireless access point, and the authentication server is typically RADIUS. Until authentication succeeds, the port passes only EAPOL frames, which carry the Extensible Authentication Protocol. The authenticator relays the exchange to the RADIUS server, using UDP 1812 for authentication and 1813 for accounting, and the server answers Access-Accept or Access-Reject. The shared secret between the authenticator and the server must be long and unique per device. RadSec wraps RADIUS in TLS for untrusted paths.

EAP methods and authorization

The EAP method defines the credentials. EAP-TLS uses client and server certificates and is the strongest, because it needs no passwords to steal. PEAP with MSCHAPv2 and EAP-TTLS tunnel a password exchange inside TLS, easier to deploy but exposed if clients do not validate the server certificate. After authentication the RADIUS server returns attributes that set authorization, such as the VLAN ID (Tunnel-Private-Group-ID), a downloadable ACL, or a security group tag. Thus the same switch port can place a finance user on one VLAN and a printer on another, based on identity.

Posture and quarantine

Network access control (NAC) products add posture assessment: they check that a device has an up-to-date operating system, active endpoint protection, disk encryption and no prohibited software. Devices that fail are moved to a quarantine or remediation VLAN with access only to patch servers and a portal. Unknown devices land in a restricted guest VLAN. Posture can be checked by an agent, by a managed-device certificate, or by integration with an MDM or EDR. Define the policy clearly, with exceptions approved and expired, or help desks will be swamped on day one.

Handling devices that cannot do 802.1X

Printers, cameras and IoT gear often lack 802.1X supplicants. For these, MAC Authentication Bypass (MAB) sends the device's MAC address to RADIUS as the identity. MAC addresses are easily spoofed, so combine MAB with profiling, which uses DHCP fingerprints, OUI vendor data and traffic behavior to classify devices, and place them in restricted VLANs with tight ACLs. Monitor for a device whose profile changes, such as a printer that begins acting like a laptop. Roll out in monitor mode first, then low-impact mode, then closed mode, and keep fail-open rules for critical ports during RADIUS outages.

SWITCH PORT 802.1X (GENERIC SYNTAX)

```
interface Gi1/0/12
 authentication port-control auto
 dot1x pae authenticator
 mab
 authentication event fail action authorize vlan 999 # quarantine
 radius server RAD1 address 192.0.2.40 auth-port 1812 acct-port 1813
```

WORKED EXAMPLE

A visitor plugs a laptop into an open meeting-room jack. The switch port, set to 802.1X with MAB fallback, sees no supplicant and no known MAC, so RADIUS returns an Access-Accept into guest VLAN 999 with internet access only. Later an employee laptop with an expired antivirus signature authenticates using EAP-TLS but fails the posture check and is moved to a remediation VLAN that reaches only the update server. After patching, the device is re-evaluated and returned to the corporate VLAN automatically.

KEY TERMS

Supplicant: Client software or device that requests network access under 802.1X.

Authenticator: Switch or access point that enforces the port state and relays EAP.

RADIUS: AAA protocol on UDP 1812/1813 used to authenticate and authorize network access.

EAP-TLS: Certificate-based EAP method providing mutual authentication.

MAB: MAC Authentication Bypass: using a device's MAC as its identity when 802.1X is unavailable.

REMEMBER

- 802.1X uses supplicant, authenticator and RADIUS server to gate ports.
- EAP-TLS is strongest; RADIUS returns VLAN or ACL authorization.
- Posture checks and quarantine VLANs handle unhealthy and unmanaged devices; use MAB with profiling.

QUICK CHECK

1. Which device in 802.1X enforces the port state?
 - A) Supplicant
 - B) Authenticator
 - C) DNS server
 - D) Load balancer
2. Which EAP method is generally strongest?
 - A) EAP-TLS
 - B) PEAP with weak passwords
 - C) Plain PAP
 - D) Open authentication
3. What happens to a device that fails a posture check?
 - A) Full access
 - B) It is moved to a quarantine VLAN with remediation access
 - C) The switch reboots
 - D) DNS is disabled
4. Why is MAB weaker than 802.1X?
 - A) MAC addresses are easy to spoof
 - B) It uses certificates
 - C) It encrypts traffic
 - D) It needs RADIUS secrets

Answers: 1-B 2-A 3-B 4-A

1. The switch or access point acts as the authenticator.
2. EAP-TLS uses certificates for mutual authentication.
3. Quarantine limits it to patch and portal resources.
4. A MAC is not a secret, so profiling and tight ACLs should accompany MAB.

Write firewall rules

SCENARIO

You are the network engineer for a fictional store, example.com. A three-tier application is moving to a new subnet set. The load balancer is 198.51.100.10, the web tier is 10.1.1.0/24, the app tier is 10.1.2.0/24 and the database is 10.1.3.5. The existing firewall has a single 'allow any any' rule. You must replace it with an ordered, least-open rule set ending in a logged default deny.

OBJECTIVE

Build ordered rules with default deny and least-open ports for a three-tier application.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the config editor and view the current rule list.

Simulator shows:

One rule appears: allow any any, with logging disabled.

- 2 **Do:** Delete the 'allow any any' rule.

Simulator shows:

The rule list is empty and the implicit policy shows as permit.

- 3 **Do:** Add rule 1: allow TCP from any to 198.51.100.10 port 443.

Simulator shows:

Rule 1 appears with a comment field.

- 4 **Do:** Add rule 2: allow TCP from 198.51.100.10 to 10.1.1.0/24 port 8080, rule 3: web to app 10.1.2.0/24 port 8443.

Simulator shows:

Rules 2 and 3 are listed in order below rule 1.

- 5 **Do:** Add rule 4: allow TCP from 10.1.2.0/24 to 10.1.3.5 port 5432.

Simulator shows:

The editor notes the database is reachable only from the app tier.

- 6 **Do:** Add rule 5: allow UDP and TCP from 10.1.0.0/16 to resolver 192.0.2.53 port 53.

Simulator shows:

DNS is permitted to one resolver.

- 7 **Do:** Add a final rule: deny any any, logging enabled, and run the validator.

Simulator shows:

Validator reports no shadowed rules and a default deny present. Score 100 percent.

ANSWER

Five allow rules: internet to load balancer 443, load balancer to web 8080, web to app 8443, app to database 5432, internal DNS to one resolver, then a logged deny any any as the last rule. No rule uses 'any' for both source and destination.

Hint shown in the app: Trace one request from the internet to the database and allow only each hop.

Segment the network

SCENARIO

A fictional manufacturer, Example Works, has one flat network holding office laptops, ERP servers, a public website, factory controllers and guest Wi-Fi. After a malware scare, you are asked to redraw it with zones and firewalls and to mark which flows are allowed between zones. The goal is that a laptop infection cannot reach the controllers, and that every allowed path is deliberate and documented.

OBJECTIVE

Place zones and firewalls on a diagram and mark the allowed flows between them.

STEPS IN THE SIMULATOR

- 1 Do:** Open the diagram builder with the unzoned asset list.

Simulator shows:

Assets appear in a single grey box.

- 2 Do:** Create zones: Internet, DMZ, Office, Servers, OT, Management, Guest.

Simulator shows:

Seven labelled zones appear on the canvas.

- 3 Do:** Drag each asset into its zone: website to DMZ, ERP to Servers, controllers to OT.

Simulator shows:

Assets snap into zones and take zone colors.

- 4 Do:** Place a firewall between each pair of zones that must communicate.

Simulator shows:

Firewall icons connect the zones; unconnected pairs show no path.

- 5 Do:** Draw allowed flows: Office to DMZ 443, DMZ to Servers 8443, Management to all on 22 and 443.

Simulator shows:

Arrows are added with port labels.

- 6 Do:** Add an industrial DMZ with a historian and allow it to pull from OT on one port; draw no Office to OT arrow.

Simulator shows:

The checker marks the Office to OT path as blocked.

- 7 Do:** Set Guest to internet-only and run the checker.

Simulator shows:

Checker reports all required flows present and no forbidden paths. Score 100 percent.

ANSWER

Seven zones with firewalls at every boundary. Allowed flows: Office to DMZ 443, DMZ to Servers on the app port, Management to all zones via jump host, Servers to industrial DMZ historian, historian to OT on one port. No direct Office to OT or Guest to internal flows.

Hint shown in the app: Ask which zone pairs genuinely need to talk and leave every other pair without an arrow.

Tune a WAF

SCENARIO

The WAF for shop.example.com was just switched from detection to blocking, and support is receiving complaints. The log viewer holds one hour of WAF events with rule IDs, matched data, client addresses and URLs. Some entries are real attacks from outside addresses; others are normal customer actions tripping generic rules. You need to decide which are which and write narrow exclusions for the false positives.

OBJECTIVE

Separate real attacks from false positives in WAF logs and write targeted exclusions.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the log viewer and sort events by rule ID.

Simulator shows:

Rules 942100, 941100 and 920350 dominate the list.

- 2 **Do:** Filter rule 942100 and read the matched data.

Simulator shows:

Entries show payloads like 1' or '1'='1 from 203.0.113.50 hitting /search. These are real attacks.

- 3 **Do:** Filter rule 941100 and read the matched data.

Simulator shows:

Matches are angle brackets in ARGS:review on /reviews/post from many customers.

- 4 **Do:** Mark 203.0.113.50 events as true positives and keep the rule active.

Simulator shows:

The viewer tags the group as attack and leaves the rule enabled.

- 5 **Do:** Mark the review events as false positives and open the exclusion editor.

Simulator shows:

A form asks for rule ID, target and path.

- 6 **Do:** Create an exclusion for rule 941100, target ARGS:review, path /reviews/post, with a reason.

Simulator shows:

The exclusion is saved with a note and date.

- 7 **Do:** Replay the hour of traffic against the new config.

Simulator shows:

Review posts pass, SQL injection probes remain blocked. Score 100 percent.

ANSWER

Rule 942100 hits from 203.0.113.50 are real SQL injection probes and stay blocked. Rule 941100 on ARGS:review at /reviews/post is a false positive. Add an exclusion limited to that rule, argument and path, with a documented reason, not a global disable.

Hint shown in the app: Look at the matched data: a payload that tries to change query logic is an attack; ordinary text is not.

Spot DNS abuse

SCENARIO

A capture from the resolver at 192.0.2.53 is loaded into the packet viewer. It contains normal lookups, one host making an unusual pattern of queries to a domain called t.example.net, and a suspicious response that arrives with a different transaction ID and source than the real server. You must find the tunneling host, identify the spoofed response, and recommend controls.

OBJECTIVE

Find DNS tunneling and a spoofed response in a capture and propose DNS controls.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the capture and apply the filter dns.

Simulator shows:

About 3,000 DNS packets are listed.

- 2 **Do:** Sort by source and count queries per host.

Simulator shows:

Host 192.0.2.77 sent 600 queries per minute; others sent about 15.

- 3 **Do:** Inspect the queries from 192.0.2.77.

Simulator shows:

All are TXT queries with unique 64-character labels under t.example.net.

- 4 **Do:** Flag this host as DNS tunneling and note the high entropy of the labels.

Simulator shows:

The viewer records the finding.

- 5 **Do:** Filter for responses to a query of www.example.com and compare transaction IDs.

Simulator shows:

Two responses appear: one from 192.0.2.53 and one from 203.0.113.99 with a mismatched ID, arriving first.

- 6 **Do:** Mark the 203.0.113.99 response as spoofed.

Simulator shows:

The viewer shows the cache entry would be poisoned.

- 7 **Do:** Select controls: force approved resolvers, block outbound 53, enable DNSSEC validation, add RPZ and rate limits.

Simulator shows:

Checker accepts the control set. Score 100 percent.

ANSWER

Host 192.0.2.77 is tunneling through long unique TXT queries to t.example.net. The response from 203.0.113.99 with a wrong transaction ID is a spoof. Controls: approved resolvers only, block outbound port 53, DNSSEC validation, RPZ sinkhole, rate limits and query monitoring.

Hint shown in the app: Compare queries per host first, then compare transaction IDs and sources on responses.

Pick the right access control

SCENARIO

Example Corp has five remote-access needs from different groups. For each, you must decide whether site-to-site IPsec or WireGuard VPN, ZTNA, or NAC with 802.1X is the best fit, and give a reason for each choice. The scenario simulator presents the requirements one at a time and scores your choice and the justification you select from a list. Think about where each decision is enforced.

OBJECTIVE

Given five remote-access requirements, choose between VPN, ZTNA and NAC with reasons.

STEPS IN THE SIMULATOR

- 1 **Do:** Read requirement 1: link the branch office to headquarters all day.

Simulator shows:

Options: VPN, ZTNA, NAC.

- 2 **Do:** Choose a site-to-site IPsec VPN.

Simulator shows:

Correct. A permanent gateway-to-gateway tunnel is a VPN use case.

- 3 **Do:** Read requirement 2: contractors on personal laptops need only the ticketing app.

Simulator shows:

The simulator lists device and scope details.

- 4 **Do:** Choose ZTNA with a posture check.

Simulator shows:

Correct. Per-application access limits the reach of unmanaged devices.

- 5 **Do:** Requirement 3: unknown devices plugged into office wall jacks. Choose NAC with 802.1X and a guest VLAN.

Simulator shows:

Correct. The control acts at the port.

- 6 **Do:** Requirement 4: admins need SSH to servers in a management segment. Choose VPN with MFA and a jump host.

Simulator shows:

Correct. Broad but tightly scoped network access.

- 7 **Do:** Requirement 5: staff need one web HR app from anywhere. Choose ZTNA.

Simulator shows:

Correct. Score 5 of 5.

ANSWER

Branch link: IPsec site-to-site VPN. Contractors: ZTNA with posture check. Wall jacks: NAC with 802.1X. Admin SSH: VPN with MFA and jump host. HR web app: ZTNA. Match the control to where the decision is made: gateway, application, or port.

Hint shown in the app: Ask whether the need is network-wide, per application, or at the physical port.

Network Security Engineering: 10 questions, 80% to pass

1. A firewall rule set processes rules top to bottom with first match. Which placement is correct?

- A) General allow above specific deny
- B) Specific rules above general ones with a default deny last
- C) Default deny first
- D) Random order

2. Which firewall tracks connection state?

- A) Stateless ACL
- B) Stateful firewall
- C) Hub
- D) Patch panel

3. A DMZ is used to:

- A) Store backups
- B) Host public-facing services separate from internal systems
- C) Replace VLANs
- D) Encrypt disks

4. Which control restricts lateral movement between servers in the same subnet?

- A) Microsegmentation
- B) Larger subnet
- C) Wi-Fi
- D) DNS caching

5. Which is the strongest reason to choose ZTNA over a traditional VPN for contractors?

- A) Per-application access limits exposure from stolen credentials
- B) It uses no encryption
- C) It needs no identity
- D) It opens the whole LAN

6. Why run a WAF in detection mode first?

- A) To find false positives before blocking real users
- B) To avoid TLS
- C) To make attacks succeed
- D) Blocking is unavailable

7. Which DNS feature validates that records were not forged?

- A) DNSSEC
- B) DHCP
- C) ARP
- D) NAT

8. What best describes anycast for DDoS defense?

- A) One IP announced from many locations to spread traffic
- B) Hiding behind NAT
- C) A type of VPN
- D) Filtering by MAC

9. In 802.1X, which element relays EAP to RADIUS?

- A) Authenticator
- B) Supplicant
- C) Client browser
- D) DNS

10. A device fails a posture check. What should NAC do?

- A) Grant full access
- B) Place it in a quarantine VLAN with remediation access
- C) Disable the switch
- D) Ignore it

Answers: 1-B 2-B 3-B 4-A 5-A 6-A 7-A 8-A 9-A 10-B

1. First-match logic requires specific rules before broad ones and a final deny.
2. Stateful firewalls keep a state table so replies are allowed automatically.
3. It limits what a compromised public server can reach.
4. Host or workload-level policy blocks east-west connections.
5. ZTNA does not give network-level reach.
6. Learning traffic prevents outages.
7. DNSSEC signatures prove authenticity.
8. Anycast distributes load and attack traffic.
9. The switch or access point is the authenticator.
10. Quarantine limits the device to fixing its health state.

Cryptography in Practice

Intermediate | 6 lessons | 4 simulated labs | checkpoint | about 4h 40m

This module covers the practical side of cryptography that engineers touch every day: configuring TLS safely, running a certificate lifecycle without outages, protecting keys with a KMS and HSM, rotating and destroying them, managing secrets, and avoiding the implementation mistakes behind most real breaches. The goal is to use well-vetted tools correctly, because strong algorithms still fail when keys leak, certificates expire or modes are misused.

BY THE END YOU CAN

- + Configure TLS securely and manage certificate lifecycles
- + Design key management with KMS, HSM and rotation
- + Identify and fix common cryptographic implementation mistakes

IN THIS MODULE

Lesson 1 TLS configuration

Lesson 2 PKI and certificate lifecycle

Lesson 3 KMS and envelope encryption

Lesson 4 HSMs and key rotation

Lesson 5 Secrets management

Lesson 6 Common implementation mistakes

Lab 1 Fix a TLS config (Config editor)

Lab 2 Triage an expired cert (Terminal)

Lab 3 Envelope encryption flow (Diagram builder)

Lab 4 Review crypto code (Code editor)

Checkpoint 10 questions, 80% to pass

TLS configuration

A weak TLS configuration exposes traffic to downgrade and interception attacks, and it is one of the first things auditors and scanners check.

Protocol versions

TLS 1.3 (RFC 8446) is the current standard and TLS 1.2 (RFC 5246) is still acceptable when configured carefully. SSL 2.0, SSL 3.0, TLS 1.0 and TLS 1.1 are deprecated and must be disabled because they have known design flaws such as POODLE and BEAST. PCI DSS and most government baselines require TLS 1.2 or higher. TLS 1.3 also shortens the handshake to one round trip and removes legacy features like static RSA key exchange, compression and renegotiation, which removes entire classes of attack.

Cipher suites and forward secrecy

A cipher suite names the key exchange, authentication, bulk cipher and hash. Prefer AEAD ciphers such as AES-GCM and ChaCha20-Poly1305, which provide encryption and integrity together. Use ephemeral key exchange (ECDHE or DHE) so every session gets a fresh key, giving forward secrecy: stealing the server private key later does not decrypt recorded traffic. Remove RC4, 3DES, export suites, NULL and anonymous suites, and CBC suites where possible. TLS 1.3 only allows five AEAD suites, so there is little to misconfigure.

HSTS and related headers

HTTP Strict Transport Security tells browsers to use HTTPS only for a site. The header Strict-Transport-Security: max-age=31536000; includeSubDomains blocks protocol downgrade and SSL stripping after the first visit. The preload directive lets a domain be hard-coded into browser lists, but removal is slow, so test with a short max-age first. Also redirect port 80 to 443, disable TLS compression, and enable OCSP stapling. Never combine HSTS with a certificate you cannot renew, because users get a hard failure with no click-through option.

Testing the result

Configuration is only trusted once measured. testssl.sh, ssllscan, nmap with the ssl-enum-ciphers script and SSL Labs all enumerate protocols, suites, certificate details and known vulnerabilities such as Heartbleed or ROBOT. Run a scan in CI or on a schedule against every public endpoint, and compare results to a written baseline such as the Mozilla server-side TLS guidelines. Test internal services too, since load balancers, databases and message brokers often keep old defaults long after the web tier is fixed.

In practice, roll changes out in stages. Start by logging the protocol and cipher negotiated by real clients for a week, then check which clients would break if you removed a suite. Most organizations find that old protocols are used only by a handful of scanners, monitoring agents or legacy devices, which can be upgraded or isolated behind a gateway. Record the baseline in version control, review it at least yearly, and treat any exception as a tracked risk with an owner and an end date.

CONFIG EXCERPT (NGINX)

```
ssl_protocols TLSv1.2 TLSv1.3;
ssl_prefer_server_ciphers off;
ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-RSA-CHACHA20-POLY1305;
ssl_session_tickets off;
ssl_stapling on;
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
```

WORKED EXAMPLE

A scan of shop.example.com on 203.0.113.10 shows TLS 1.0 enabled and a 3DES suite. The engineer sets ssl_protocols to TLSv1.2 and TLSv1.3, replaces the cipher list with ECDHE and GCM suites, adds HSTS and reruns testssl.sh. The report now shows no deprecated protocols, forward secrecy for all suites, and a clean grade. A canary on legacy clients is checked first so the change does not surprise an old partner integration.

KEY TERMS

AEAD: Authenticated encryption with associated data, which gives confidentiality and integrity in one operation.

Forward secrecy: Property where compromise of a long-term key does not expose past session keys.

HSTS: A response header that forces browsers to use HTTPS for a domain.

ECDHE: Elliptic-curve Diffie-Hellman with ephemeral keys, used for forward-secret key exchange.

Cipher suite: A named combination of key exchange, authentication, cipher and hash used in a TLS session.

REMEMBER

- Allow TLS 1.2 and 1.3 only, with AEAD and ECDHE suites.
- Add HSTS after confirming certificates renew reliably.
- Scan continuously with testssl.sh and compare to a baseline.

QUICK CHECK

1. Which protocol set is the best baseline today?
 - A) TLS 1.0 and 1.1
 - B) TLS 1.2 and 1.3
 - C) SSL 3.0 and TLS 1.2
 - D) Only TLS 1.2 with RC4
2. What does forward secrecy protect against?
 - A) Decrypting recorded traffic after a server key is stolen
 - B) Weak passwords
 - C) Cross-site scripting
 - D) Expired certificates
3. What does HSTS do?
 - A) Validates OCSP responses
 - B) Blocks CORS requests
 - C) Encrypts cookies at rest
 - D) Makes browsers use HTTPS only for the domain
4. Which tool enumerates a server's protocols and cipher suites?
 - A) cosign
 - B) testssl.sh
 - C) gitleaks
 - D) Grype

Answers: 1-B 2-A 3-D 4-B

1. TLS 1.2 and 1.3 are the supported modern protocols; older ones have known flaws.
2. Ephemeral keys mean the stolen long-term key cannot derive past session keys.
3. The header instructs browsers to refuse plain HTTP for the site for max-age seconds.
4. testssl.sh tests protocols, suites and known TLS vulnerabilities.

PKI and certificate lifecycle

Expired or mis-issued certificates cause some of the most common and most embarrassing outages, and a trusted-but-wrong certificate enables interception.

Chains and trust

A public key infrastructure binds public keys to identities through signed certificates (X.509). A server sends its leaf certificate and one or more intermediates; the client validates the chain up to a root CA in its trust store, checks signatures, validity dates, name match against the Subject Alternative Name field, key usage and revocation status. A missing intermediate on the server is a classic cause of errors that work in one browser and fail in another. Roots stay offline; intermediates sign leaf certificates daily.

Issuance and ACME

Automatic Certificate Management Environment (RFC 8555) lets a client such as certbot, acme.sh or a cert-manager controller request certificates from a CA like Let's Encrypt without human steps. The client proves control of a domain through HTTP-01, DNS-01 or TLS-ALPN-01 challenges; wildcard names need DNS-01. Public certificate lifetimes are shrinking, with 90 days common and the industry moving toward shorter limits, so manual renewal does not scale. Automation, plus monitoring of the renewal job itself, is the control that prevents expiry.

Revocation and transparency

When a private key is exposed, the certificate should be revoked. Clients check status through CRLs or OCSP; OCSP stapling lets the server attach a signed, fresh response to the handshake, saving a lookup and protecting client privacy. Certificate Transparency (RFC 6962 and 9162) requires CAs to publish issued certificates to public append-only logs. Defenders monitor those logs for unexpected certificates for their domains, and CAA DNS records restrict which CAs may issue for a name.

Preventing expiry outages

Inventory every certificate, including internal CAs, load balancers, VPN gateways and code-signing certs, and record an owner for each. Alert at 30, 14 and 7 days before expiry using an independent probe, not only the renewal tool. Deploy renewals through the same pipeline as other config and reload services automatically. Keep private keys unique per certificate, store them with restrictive permissions or in a KMS or HSM, and rehearse emergency re-issuance so replacement takes minutes, not days.

Internal PKI deserves the same discipline as public certificates. Private CAs issue certificates for services, devices and mutual TLS, and their roots are trusted by every internal client, so their compromise is severe. Keep the root offline, use short-lived leaf certificates, and automate issuance through ACME-compatible tools or a service mesh. Document who may request which names, and log every issuance. When a CA must be replaced, plan a cross-signed overlap period so that clients are updated before the old certificate expires.

TERMINAL OUTPUT

```
$ openssl s_client -connect app.example.com:443 -servername app.example.com </dev/null | openssl x509
-noout -dates -subject -issuer
notBefore=Sep 1 00:00:00 2026 GMT
notAfter=Nov 30 23:59:59 2026 GMT
subject=CN = app.example.com
issuer=C = US, O = Example CA, CN = Example Issuing CA 1
```

WORKED EXAMPLE

A partner reports that api.example.com fails for older clients while browsers work. The engineer runs openssl s_client and sees the server sends only the leaf, with no intermediate. Modern browsers fetch the intermediate themselves, but strict clients cannot. The fix is to configure the web server with the full chain file (leaf plus intermediate). An alert is added so the monitoring probe validates the complete chain, not only the leaf expiry date.

KEY TERMS

X.509: The standard format for public key certificates.

Intermediate CA: A CA certificate signed by a root that issues leaf certificates.

ACME: A protocol for automated certificate issuance and renewal.

OCSP stapling: Server attaches a signed revocation status response to the TLS handshake.

CT log: A public append-only log of issued certificates used to detect mis-issuance.

REMEMBER

- Serve the full chain and validate it externally.
- Automate renewal with ACME and monitor the job and expiry dates.
- Use CAA and CT monitoring to detect mis-issuance.

QUICK CHECK

1. Why do some clients fail while browsers succeed with the same server?

- A) The server omitted an intermediate certificate
- B) The leaf is too new
- C) HSTS is enabled
- D) TLS 1.3 is on

2. Which ACME challenge is required for wildcard certificates?

- A) DNS-01
- B) TLS-ALPN-01
- C) Email
- D) HTTP-01

3. What does OCSP stapling provide?

- A) Private key escrow
- B) Automatic renewal
- C) A fresh signed revocation response sent by the server
- D) Faster key generation

4. How can a domain owner watch for unexpected issuance?

- A) Turn off OCSP
- B) Monitor Certificate Transparency logs
- C) Disable CAA
- D) Use longer lifetimes

Answers: 1-A 2-A 3-C 4-B

1. Browsers can fetch missing intermediates; strict clients cannot, so the chain must be complete.
2. Wildcard issuance requires proving control through a DNS TXT record.
3. The server includes a recent CA-signed status so clients need no separate lookup.
4. CT logs list every publicly trusted certificate, revealing rogue issuance.

KMS and envelope encryption

Encrypting data is easy; protecting and controlling the keys is the hard part, and a KMS makes key use auditable and policy-controlled.

What a KMS does

A key management service, such as AWS KMS, Azure Key Vault or Google Cloud KMS, creates and stores key-encryption keys (called KEKs, CMKs or KMS keys) and performs cryptographic operations on request. The key material never leaves the service in plaintext. Callers authenticate with their cloud identity and every Encrypt, Decrypt and GenerateDataKey call is logged. This replaces keys stored in config files with a central, revocable, auditable control point. Disabling a key immediately blocks decryption of everything protected beneath it.

Envelope encryption

KMS APIs limit payload size, and sending bulk data over the network is slow. Envelope encryption solves this: the application asks the KMS for a data encryption key (DEK), receiving a plaintext copy and a copy wrapped by the master key. It encrypts the data locally with the plaintext DEK using AES-256-GCM, then discards the plaintext key and stores the wrapped DEK beside the ciphertext. To read, the app sends the wrapped DEK to the KMS to unwrap. Each object can have its own DEK, limiting the blast radius.

Key policies and separation

Access to keys should be separate from access to data. A storage administrator who can read the encrypted bucket should not be able to call Decrypt on its key. Write key policies with least privilege, using conditions such as encryption context, which binds ciphertext to metadata like a tenant ID so that a blob cannot be decrypted in the wrong context. Separate the roles that administer keys from the roles that use them, and require a second approval for key deletion, which should have a waiting period.

Operational concerns

Plan regions and availability, since a regional KMS outage can stop decryption. Use multi-region keys or replicated wrapped DEKs when needed. Watch KMS request quotas under heavy load and cache DEKs briefly with limits. Alert on unusual Decrypt volume, calls from unexpected principals and scheduled deletions. Use customer-managed keys where regulation demands control, and cloud-managed keys for lower-risk data. Always back up wrapped DEKs with the data, since they are useless without the master key but required to use it.

Testing is part of the design. Verify that an application role without kms:Decrypt cannot read a sample object, that disabling the key stops reads, and that restoring it brings them back. Rehearse the case where a region is unavailable. Check that backups include the wrapped DEKs and the key version metadata. Finally, review CloudTrail or the equivalent log to confirm that the encryption context appears on each call, because missing context often signals code that bypasses your standard library.

LOG EXCERPT

```
{"eventName": "GenerateDataKey", "userIdentity": {"arn": "arn:aws:iam::111122223333:role/app-writer"},
  "requestParameters": {"keyId": "alias/orders", "keySpec": "AES_256",
    "encryptionContext": {"tenant": "t-1001"}}, "sourceIPAddress": "198.51.100.20"}
```

WORKED EXAMPLE

An orders service stores customer files. On upload it calls GenerateDataKey with encryption context tenant=t-1001, encrypts the file with AES-GCM, stores ciphertext plus wrapped DEK in object storage, and wipes the plaintext key. The storage admins have no kms:Decrypt permission, so a copied bucket is useless to them. During an incident the security team disables the key, and every file becomes unreadable until the key is re-enabled.

KEY TERMS

KMS: A managed service that stores keys and performs cryptographic operations without exposing key material.

DEK: A data encryption key used to encrypt actual data, usually per object.

KEK: A key-encryption key that wraps other keys.

Envelope encryption: Encrypting data with a DEK and protecting the DEK with a KMS key.

Encryption context: Additional authenticated metadata that must match at decrypt time.

REMEMBER

- Use envelope encryption with per-object DEKs.
- Separate key administration, key use and data access.
- Log and alert on KMS activity and key deletion.

QUICK CHECK

1. What is stored next to the ciphertext in envelope encryption?
 - A) The plaintext DEK
 - B) The wrapped DEK
 - C) The KMS master key
 - D) The IAM password
2. Why use envelope encryption?
 - A) To avoid sending bulk data to the KMS and limit key exposure
 - B) It removes the need for keys
 - C) It makes TLS faster
 - D) KMS cannot encrypt anything
3. Which design best separates duties?
 - A) One shared admin account
 - B) Keys in source control
 - C) Storage admins also hold Decrypt
 - D) Different roles administer data and use keys
4. What does encryption context add?
 - A) Automatic rotation
 - B) Faster encryption
 - C) Bound metadata required at decryption and logged
 - D) A larger key

Answers: 1-B 2-A 3-D 4-C

1. Only the wrapped DEK is stored; the plaintext copy is discarded after use.
2. Local encryption with a DEK handles big data while the KMS protects the small key.
3. Splitting key and data access means one compromised role cannot read plaintext.
4. The context is authenticated and must match, tying ciphertext to its intended use.

HSMs and key rotation

High-value keys need hardened storage and a lifecycle plan, so that compromise, expiry or staff changes never lead to lost or exposed keys.

HSMs and FIPS 140-3

A hardware security module is a tamper-resistant device that generates, stores and uses keys without exporting them. Validation comes from FIPS 140-3 (which replaced 140-2, using ISO/IEC 19790 requirements). Level 1 is basic software or hardware correctness; Level 2 adds tamper evidence and role-based authentication; Level 3 adds tamper resistance, identity-based authentication and zeroization on attack; Level 4 adds protection against environmental attacks. Many cloud KMS services use HSMs underneath, while dedicated options such as AWS CloudHSM give single-tenant control.

Choosing HSM or software

Software key stores are cheaper and easy to scale, but keys exist in server memory and can be copied by an attacker with host access. HSMs trade cost and complexity for strong key protection, audit evidence and compliance fit, for example for certificate authority roots, payment keys and code-signing. Use PKCS#11, KMIP or cloud APIs to integrate. Remember an HSM protects the key, not the application: if an attacker can call the HSM as the application, they can still misuse the key, so access control and logging remain essential.

Rotation

Rotation limits how much data one key protects and shortens the exposure window after a leak. Define a crypto period by risk and algorithm, such as annual automatic rotation for KMS keys. With envelope encryption, rotating the master key does not require re-encrypting data if old key versions remain available for decryption; new data uses the new version. Rotate immediately after suspected compromise, staff departure with key access, or algorithm weakness. Rotation of data keys may require re-encryption, so plan the migration.

Escrow, backup and destruction

Document the full key lifecycle: generation, distribution, use, storage, backup, rotation, archive and destruction (see NIST SP 800-57). Use split knowledge and dual control for root keys, for example M-of-N smart cards held by different people, and keep offline backups in separate sites. Escrow supports recovery but creates a target, so protect it as strongly as the key. Destruction should be verified and logged: zeroize HSM objects, delete KMS keys after a waiting period, and record who approved. Destroying a key crypto-shreds all data under it.

Rotation fails most often because nobody can find where a key is used. Keep an inventory that maps each key to the applications, data stores and certificates that depend on it, along with an owner and next rotation date. Test rotation in a staging environment first, measuring how many systems cache old keys. Automate the common cases, and keep a documented manual path for the rare ones, such as keys embedded in partner integrations or in hardware devices.

POLICY EXCERPT

```
Key: payments-root
Storage: FIPS 140-3 Level 3 HSM
Crypto period: 12 months (auto-rotate)
Backup: 3-of-5 quorum, 2 sites
Deletion: dual approval + 30 day wait
Audit: all use events to SIEM
```

WORKED EXAMPLE

A company's code-signing key is moved from a build server file to an HSM requiring two operators for activation. After a build engineer leaves, the team rotates the signing certificate, revokes the old one and updates verification policies. The old key's destruction ceremony is witnessed, logged and approved by the security lead. Later, auditors accept the HSM validation certificate and the logs as evidence for the control.

KEY TERMS

HSM: A tamper-resistant device that performs cryptographic operations and protects keys from export.

FIPS 140-3: A US and Canadian standard defining security levels 1 to 4 for cryptographic modules.

Crypto period: The time span a key is authorized for use.

Split knowledge: A control where no single person knows or holds the whole key or secret.

Zeroization: Securely erasing key material from a module.

REMEMBER

- HSMs protect key material; FIPS 140-3 levels describe the strength of that protection.
- Rotate on schedule and on events such as compromise or departures.
- Document backup, escrow and verified destruction.

QUICK CHECK

1. What distinguishes FIPS 140-3 Level 3 from Level 2?
 - A) Tamper resistance and zeroization on attack
 - B) No physical security
 - C) Software-only operation
 - D) Only documentation
2. With envelope encryption, what happens when the master key rotates?
 - A) New data uses the new version and old versions still decrypt old data
 - B) Data becomes unreadable
 - C) DEKs are shown in plaintext
 - D) All data must be re-encrypted immediately
3. What does an HSM not prevent?
 - A) Physical tampering
 - B) Unauthorized duplication
 - C) Key export in plaintext
 - D) Misuse of the key by a compromised authorized application
4. Which is a good control for a root key?
 - A) Sent by email
 - B) One administrator knows it
 - C) M-of-N quorum with dual control
 - D) Stored in a wiki

Answers: 1-A 2-A 3-D 4-C

1. Level 3 adds tamper resistance and identity-based authentication, with active zeroization.
2. Key versions are retained so older wrapped DEKs still unwrap.
3. If an attacker can invoke the application's permissions, they can still use the key.
4. Quorum and dual control remove single points of trust.

Secrets management

Passwords, API tokens and database credentials leak easily when kept in code or config, so they need a managed, audited home.

What counts as a secret

Secrets are credentials that grant access: database passwords, API keys, OAuth client secrets, SSH keys, signing keys and tokens. Common failures include secrets in source code, container images, environment dumps, wikis and chat messages. The principle is to keep secrets out of artifacts and fetch them at runtime from a dedicated store, using the workload's own identity to authenticate. Tools include HashiCorp Vault, AWS Secrets Manager, Azure Key Vault and Google Secret Manager. Encrypted files in git, like SOPS, are acceptable only with strict key control.

Dynamic secrets and leases

Static secrets live for months and are shared widely. Vault's dynamic secrets engines instead create a unique database user or cloud credential on demand. Each secret has a lease with a TTL, can be renewed up to a maximum, and is automatically revoked at expiry or when revoked manually. If a credential leaks, it is short-lived and traceable to the requesting identity. Where dynamic secrets are not possible, rotate static secrets automatically, for example with managed rotation functions for database passwords.

Authentication to the store

The hardest problem is the first secret. Avoid a bootstrap password in config; prefer platform identity. Kubernetes service accounts, cloud instance roles and OIDC tokens let a workload prove who it is, and Vault AppRole with response wrapping handles other cases. Policies grant each identity access only to its paths. Humans use SSO with MFA and short sessions. Unseal keys for Vault should be split using Shamir's secret sharing or delegated to a cloud KMS auto-unseal.

Audit and operations

Enable audit devices so every read, write and login is logged with the identity, path and time, and send logs to a separate system the secrets admin cannot edit. Alert on mass reads, access from new networks and use of root tokens. Plan for high availability and backups, and test restore. Prevent exposure in logs by masking, and never pass secrets in command-line arguments, which appear in process lists. Keep a runbook to revoke and rotate quickly after exposure.

Adoption is a people problem as much as a technical one. Give developers a simple library or sidecar that fetches secrets so that the secure path is the easiest path. Scan repositories and images for leaked secrets to find what remains outside the store, and migrate in priority order, starting with production database and cloud credentials. Measure progress through the number of static secrets older than ninety days and the mean time to rotate after an exposure drill.

TERMINAL OUTPUT

```
$ vault read database/creds/orders-ro
Key Value
lease_id database/creds/orders-ro/Xk3pQ9aExample
lease_duration 1h
username v-orders-ro-8fj2
password <redacted>
```

WORKED EXAMPLE

A web service previously read DB_PASSWORD from a .env file committed to a private repo. The team switches to Vault: the pod authenticates with its Kubernetes service account, requests database/creds/orders-ro, and receives a one-hour credential. A leaked log containing the password is harmless two hours later. The audit log shows exactly which pod requested it, and the old shared password is rotated and removed from git history.

KEY TERMS

Dynamic secret: A credential generated on demand for one client with a limited life.

Lease: A time-limited grant on a secret that can be renewed or revoked.

AppRole: A Vault auth method for machines using a role ID and secret ID.

Auto-unseal: Using a cloud KMS or HSM to unseal Vault without manual key entry.

Audit device: A Vault backend that records every request and response metadata.

REMEMBER

- Fetch secrets at runtime using workload identity.
- Prefer dynamic, leased credentials; rotate static ones.
- Audit every access and have a revoke-and-rotate runbook.

QUICK CHECK

1. What is the main benefit of dynamic secrets?
 - A) They never expire
 - B) Short-lived unique credentials limit leak impact
 - C) They are stored in code
 - D) They remove need for authentication
2. How should a workload authenticate to a secrets manager?
 - A) With its platform identity such as a service account or role
 - B) With a shared admin token
 - C) Anonymously
 - D) With a password in a config file
3. Why avoid secrets in command-line arguments?
 - A) They slow the CLI
 - B) They cannot be encrypted
 - C) They are visible in process listings
 - D) They are too long
4. What must audit logs be protected from?
 - A) Rotation
 - B) Being read by SIEM
 - C) Modification by the secrets administrators
 - D) Compression

Answers: 1-B 2-A 3-C 4-C

1. Each credential is unique, expires, and traces to the requester.
2. Platform identity avoids a bootstrap secret.
3. Other users and logs can see process arguments.
4. Independent log storage prevents tampering by those who control secrets.

Common implementation mistakes

Most real cryptographic failures come from misuse of sound algorithms, not from broken math, so spotting the patterns is a core engineering skill.

Modes, nonces and IVs

ECB mode encrypts identical blocks to identical ciphertext, leaking patterns, as in the well-known penguin image. Use an authenticated mode such as AES-GCM or ChaCha20-Poly1305. These modes need a unique nonce for each key: reusing a nonce with GCM can reveal the XOR of plaintexts and let attackers forge messages. For CBC, a static or predictable IV also leaks data. Generate nonces randomly with a secure source (12 bytes for GCM) or use a counter, store them with the ciphertext, and consider misuse-resistant constructions like AES-GCM-SIV.

Passwords and hashing

Fast hashes such as MD5, SHA-1 and plain SHA-256 are wrong for passwords because attackers can try billions of guesses per second on GPUs. Use a slow, salted, memory-hard function: Argon2id, scrypt or bcrypt, or PBKDF2 with a high iteration count where FIPS rules require it. A unique random salt defeats rainbow tables. A server-side pepper held in a KMS adds protection. Use constant-time comparison for secrets and MACs. MD5 and SHA-1 are also broken for signatures and collision resistance, so prefer SHA-256 or SHA-3.

Randomness and keys

Predictable random numbers produce predictable keys and tokens. Use the operating system CSPRNG: secrets module or os.urandom in Python, crypto.randomBytes in Node.js, SecureRandom in Java, never Math.random or rand(). Hardcoded keys in source or images are discovered through repositories and decompilation, and cannot be rotated without redeploying. Load keys from a KMS or secrets manager. Do not derive keys from low-entropy strings with a single hash, and do not invent your own protocol or cipher.

Review and prevention

Search code for the patterns: AES/ECB, DES, RC4, MD5, SHA1, hardcoded byte arrays, fixed IV variables and non-crypto random calls. Static analysis rules in Semgrep and CodeQL detect many of these. Prefer high-level libraries such as libsodium, Tink or cryptography's Fernet that choose safe defaults and manage nonces for you. Add unit tests for tamper detection, check TLS certificate validation is never disabled (verify=False), and require a cryptography review for new designs, using the OWASP Cryptographic Storage Cheat Sheet as a guide.

Prevention works best when built into the pipeline. Add a lint rule that blocks banned algorithms, a secret scanner that catches embedded keys, and a code owner requirement on files that import cryptographic libraries. Keep a short internal guide listing approved libraries and parameters, such as AES-256-GCM, Argon2id parameters and the minimum RSA or ECC key sizes. When developers have a vetted recipe to copy, they stop inventing their own, and reviews become faster and more consistent.

CODE REVIEW FINDING

```
cipher = AES.new(KEY, AES.MODE_ECB) # mistake: ECB
KEY = b'0123456789abcdef' # mistake: hardcoded key
token = str(random.randint(0,10**6)) # mistake: non-crypto RNG
pw_hash = hashlib.md5(pw.encode()).hexdigest() # mistake: fast hash
```

WORKED EXAMPLE

A reviewer finds the four lines above in a session module. The fix replaces ECB with AES-GCM using a fresh 12-byte nonce stored with the ciphertext, loads the key from a secrets manager, uses secrets.token_urlsafe(32) for tokens, and stores passwords with Argon2id and a per-user salt. A unit test then confirms that a modified ciphertext raises an authentication error.

KEY TERMS

ECB: A block cipher mode where identical plaintext blocks give identical ciphertext blocks.

Nonce: A number used once per key that must never repeat in modes like GCM.

Argon2id: A memory-hard password hashing function recommended for new systems.

CSPRNG: A cryptographically secure pseudorandom number generator.

Salt: A unique random value added to each password before hashing.

REMEMBER

- Use authenticated modes with unique nonces.
- Hash passwords with Argon2id, bcrypt or scrypt, never fast hashes.
- Use CSPRNGs and managed keys, never hardcoded ones.

QUICK CHECK

1. Why is ECB mode unsafe?
 - A) It is too slow
 - B) Identical blocks produce identical ciphertext revealing patterns
 - C) It needs a long key
 - D) It does not use a key
2. What is the danger of reusing a GCM nonce under one key?
 - A) Plaintext relationships leak and forgery becomes possible
 - B) Nothing
 - C) Key expiry
 - D) Slower decryption
3. Which is best for storing passwords?
 - A) Argon2id with unique salt
 - B) Base64
 - C) MD5 with salt
 - D) SHA-256 once
4. Which source should generate session tokens?
 - A) The user ID
 - B) Math.random
 - C) The system time
 - D) The OS CSPRNG

Answers: 1-B 2-A 3-A 4-D

1. ECB leaks structure because it has no chaining or nonce.
2. Nonce reuse breaks both confidentiality and authentication.
3. Argon2id is deliberately slow and memory-hard.
4. Only a CSPRNG gives unpredictable values.

Fix a TLS config

SCENARIO

You maintain the nginx front end for portal.example.com on 203.0.113.25. A scan flagged TLS 1.0, a 3DES suite and a missing HSTS header. Edit the server block so it meets the company baseline without breaking modern clients. The change must not break the payment partner that still connects with TLS 1.2 and ECDHE ciphers. A security review is scheduled for this afternoon, and the reviewer will rerun the scanner against the staging host before approving the change for production.

OBJECTIVE

Produce an nginx TLS block allowing only secure protocols and ciphers with HSTS enabled.

STEPS IN THE SIMULATOR

- 1 **Do:** Open /etc/nginx/conf.d/portal.conf and inspect the ssl_protocols line.

Simulator shows:

```
ssl_protocols TLSv1 TLSv1.1 TLSv1.2;
```

- 2 **Do:** Change ssl_protocols to TLSv1.2 TLSv1.3.

Simulator shows:

```
Line updated. Validator: legacy protocols removed.
```

- 3 **Do:** Replace ssl_ciphers with ECDHE AES-GCM and CHACHA20 suites.

Simulator shows:

```
Validator: no CBC, 3DES or RC4 suites remain. Forward secrecy: OK.
```

- 4 **Do:** Add ssl_stapling on; and ssl_session_tickets off;

Simulator shows:

```
Validator: stapling enabled; tickets disabled.
```

- 5 **Do:** Add add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;

Simulator shows:

```
Validator: HSTS present, max-age >= 31536000.
```

- 6 **Do:** Click Test config then Run scan.

Simulator shows:

```
nginx: configuration test is successful. Scan grade: A. Findings: 0 critical.
```

ANSWER

The final block sets ssl_protocols TLSv1.2 TLSv1.3, an ECDHE plus AEAD cipher list, stapling on, session tickets off, and an HSTS header with a one-year max-age. The scan shows no deprecated protocols, no weak ciphers and forward secrecy everywhere.

Hint shown in the app: Remove protocols first, then ciphers, and do not forget the HSTS header uses the always keyword.

Triage an expired cert

SCENARIO

Users of intranet.example.com (198.51.100.40) report certificate errors starting at midnight. The leaf certificate was renewed last week, so the cause is unclear. Use `openssl s_client` to examine the chain and decide what to renew. The on-call engineer has shell access to a bastion host and the public certificate files, but not to the private CA. The helpdesk is escalating, and management wants a clear root cause and a plan to prevent a repeat of this outage before the end of the day.

OBJECTIVE

Identify which certificate in the chain expired and plan its renewal.

STEPS IN THE SIMULATOR

- Do:** Run: `openssl s_client -connect 198.51.100.40:443 -showcerts </dev/null`
Simulator shows:
Certificate chain: 0 s:CN=intranet.example.com; 1 s:CN=Example Issuing CA 2. Verify return code: 10 (certificate has expired)
- Do:** Run: `openssl s_client ... | openssl x509 -noout -dates` for certificate 0.
Simulator shows:
notBefore=Sep 28 2026 notAfter=Dec 27 2026
- Do:** Extract certificate 1 and run `openssl x509 -noout -dates -subject`.
Simulator shows:
subject=CN=Example Issuing CA 2 notAfter=Oct 5 2026 00:00:00 GMT (EXPIRED)
- Do:** Run: `openssl verify -CAfile root.pem -untrusted inter.pem leaf.pem`
Simulator shows:
error 10 at 1 depth lookup: certificate has expired
- Do:** Check the internal CA inventory for who owns Issuing CA 2.
Simulator shows:
Owner: pki-team@example.com. Renewal job: disabled since migration.
- Do:** Draft the renewal plan: renew intermediate, redeploy full chain, re-enable alerting.
Simulator shows:
Plan saved. Alert threshold set to 30/14/7 days.

ANSWER

The expired certificate is the intermediate Example Issuing CA 2, not the leaf. The plan: have the PKI team renew the intermediate, deploy the new chain to the server, verify with `s_client` returning code 0, and restore expiry monitoring covering every certificate in the chain.

Hint shown in the app: Read the verify return code and then look at each certificate in the chain, not just the first.

Envelope encryption flow

SCENARIO

A document service must encrypt uploaded files using a KMS key named alias/docs. Arrange the building blocks of the flow so plaintext keys never persist and decryption can be audited. Auditors will review the design, so each step must show where the plaintext key exists, where the wrapped key is stored, and which call produces a KMS audit event. The service runs in a single region with a customer-managed key and no manual key handling at all.

OBJECTIVE

Order the steps of envelope encryption correctly.

STEPS IN THE SIMULATOR

- 1 **Do:** Drag 'Request data key' from the app to the KMS.

Simulator shows:

Connector created: App -> KMS GenerateDataKey(alias/docs).

- 2 **Do:** Place 'Receive plaintext DEK and wrapped DEK' next.

Simulator shows:

Node added. KMS returns two copies of the key.

- 3 **Do:** Add 'Encrypt file locally with AES-256-GCM using plaintext DEK'.

Simulator shows:

Node added. Ciphertext + nonce + tag produced.

- 4 **Do:** Add 'Discard plaintext DEK from memory'.

Simulator shows:

Node added. Warning cleared: plaintext key not persisted.

- 5 **Do:** Add 'Store ciphertext and wrapped DEK together in object storage'.

Simulator shows:

Node added. Object metadata: wrapped_dek, nonce.

- 6 **Do:** Add the read path: 'Send wrapped DEK to KMS Decrypt' then 'Decrypt file'. Click Validate.

Simulator shows:

Validation passed: 6/6 steps in a correct order. KMS audit event on each decrypt.

ANSWER

Correct order: request data key; receive plaintext and wrapped DEK; encrypt locally; discard plaintext DEK; store ciphertext with wrapped DEK. To read, send the wrapped DEK to KMS to unwrap, then decrypt locally. KMS logs every unwrap.

Hint shown in the app: The plaintext key is used once and then thrown away; only the wrapped copy is stored.

Review crypto code

SCENARIO

A teammate submitted a Python function that encrypts customer notes. The code uses MD5 to derive a key, AES in ECB mode, and a fixed IV. Review it and replace the unsafe parts. The module is used by a customer notes feature that stores personal data, so the pull request is blocking the release. The team lead expects a clear explanation of each flaw, a corrected implementation, and a test showing that tampering with stored data is detected after the change.

OBJECTIVE

Replace weak hashing, ECB and static IV with safe alternatives.

STEPS IN THE SIMULATOR

- 1 **Do:** Open `notes_crypto.py` and read the `encrypt` function.

Simulator shows:

```
key = hashlib.md5(PASSWORD).digest(); AES.new(key, AES.MODE_ECB); IV = b'0000000000000000'
```

- 2 **Do:** Flag line 3 (MD5 key derivation) and replace with `script` or load a random 32-byte key from the secrets manager.

Simulator shows:

Linter: weak-hash resolved.

- 3 **Do:** Flag the ECB mode and change to `AES.MODE_GCM`.

Simulator shows:

Linter: insecure-mode resolved.

- 4 **Do:** Remove the static IV and generate `nonce = os.urandom(12)` per message.

Simulator shows:

Linter: static-iv resolved. Nonce reuse test: pass.

- 5 **Do:** Return `nonce + ciphertext + tag`, and decrypt with verification.

Simulator shows:

Tamper test: modified ciphertext raises `InvalidTag`.

- 6 **Do:** Click Run checks.

Simulator shows:

4/4 checks passed. Static scan: 0 high findings.

ANSWER

The corrected code gets a random 32-byte key from the secrets manager (or a `script` derived key), uses AES-GCM, generates a fresh 12-byte nonce for every message stored with the ciphertext, and verifies the tag on decrypt. MD5, ECB and the static IV are gone.

Hint shown in the app: Three separate flaws: key derivation, cipher mode, and the IV or nonce.

Cryptography in Practice: 10 questions, 80% to pass

1. Which TLS setting provides forward secrecy?
 - A) Static RSA key exchange
 - B) ECDHE key exchange
 - C) TLS compression
 - D) RC4
2. What does a missing intermediate certificate usually cause?
 - A) Failures on strict clients while some browsers work
 - B) HSTS errors
 - C) Weak ciphers
 - D) Slow handshakes
3. Which ACME challenge supports wildcards?
 - A) TLS-ALPN-01
 - B) FTP-01
 - C) HTTP-01
 - D) DNS-01
4. What is the DEK in envelope encryption?
 - A) A TLS session ticket
 - B) The key that encrypts the data
 - C) The root CA
 - D) A password hash
5. Why separate key admins from data admins?
 - A) It speeds up backups
 - B) One compromised role cannot both access ciphertext and decrypt it
 - C) KMS requires it for pricing
 - D) To avoid logging
6. A HSM validated at FIPS 140-3 Level 3 adds which feature over Level 2?
 - A) Software-only keys
 - B) No authentication
 - C) Longer certificates
 - D) Tamper resistance and zeroization
7. What is the best response to a leaked static API key?
 - A) Rename the variable
 - B) Hash it
 - C) Wait for expiry
 - D) Revoke it, rotate it and review audit logs
8. Which function is appropriate for password storage?
 - A) CRC32
 - B) SHA-1
 - C) MD5
 - D) Argon2id
9. What happens if a GCM nonce is reused with the same key?
 - A) Nothing
 - B) Confidentiality and integrity are broken
 - C) Only speed drops
 - D) The key rotates
10. Which tool tests a server's TLS protocols and ciphers?
 - A) Gripe
 - B) Semgrep
 - C) cosign
 - D) testssl.sh

Answers: 1-B 2-A 3-D 4-B 5-B 6-D 7-D 8-D 9-B 10-D

1. Ephemeral Diffie-Hellman creates per-session keys.
2. Clients must build a chain to a trusted root.
3. Wildcards need a DNS TXT proof.
4. A data encryption key does the bulk encryption.
5. Separation of duties limits damage.
6. Level 3 resists tampering physically.
7. Revoke and rotate immediately and check for misuse.
8. Slow memory-hard hashes resist cracking.
9. Nonce reuse exposes plaintext relationships and forgery.
10. testssl.sh enumerates protocol and cipher support.

Application Security

Intermediate | 8 lessons | 6 simulated labs | checkpoint | about 6h 30m

This module teaches how to build and test software that resists common attacks. You will connect security to every phase of the SDLC, write code that validates input, encodes output and controls access correctly, and review code with a checklist. You will then use SAST, DAST and software composition analysis, triage their output, and set up a champions program with remediation SLAs. These skills address the OWASP Top 10 flaws that cause most real application breaches.

BY THE END YOU CAN

- + Write code that avoids OWASP Top 10 flaws
- + Run and triage SAST, DAST and SCA results
- + Operate a secure SDLC with reviews and security champions

IN THIS MODULE

Lesson 1 Secure SDLC and requirements

Lesson 2 Input validation and output encoding

Lesson 3 Authentication and session handling

Lesson 4 Access control and API security

Lesson 5 Secure code review

Lesson 6 SAST and DAST

Lesson 7 SCA and dependency risk

Lesson 8 Security champions program

Lab 1 Fix SQL injection (Code editor)

Lab 2 Fix XSS output (Code editor)

Lab 3 Review a pull request (Code editor)

Lab 4 Triage SAST findings (Scenario)

Lab 5 Exploit an IDOR in sandbox (Web proxy)

Lab 6 Dependency audit (Terminal)

Checkpoint 10 questions, 80% to pass

Secure SDLC and requirements

Fixing a design flaw after release costs many times more than catching it in requirements, so security has to be built into every phase.

Security in each phase

A secure software development lifecycle adds security activities to each phase rather than a single test at the end. In requirements, define security and privacy requirements, abuse cases and compliance needs. In design, perform threat modeling and choose proven components. In implementation, follow coding standards, use linters and peer review. In testing, run SAST, DAST, software composition analysis and targeted penetration tests. In release and operations, harden configuration, monitor, and have a vulnerability response process. Each phase produces evidence that auditors and customers can review.

Security requirements

Good security requirements are specific and testable. Instead of 'the app must be secure', write 'all passwords are stored with Argon2id', 'every endpoint enforces authorization server-side' or 'sessions expire after 15 minutes of inactivity'. The OWASP Application Security Verification Standard (ASVS) offers a ready catalogue in three levels, where level 1 suits low-risk apps and level 3 suits high-value systems such as banking. Abuse cases, written as 'as an attacker I want to...', help teams think about misuse beside normal user stories.

Maturity models

OWASP SAMM and BSIMM help organizations measure and improve their program. SAMM is a prescriptive model with five business functions (governance, design, implementation, verification, operations), each with security practices scored from 0 to 3. BSIMM is descriptive: it reports what real firms do across more than a hundred measured activities, allowing comparison to peers. Microsoft SDL and NIST SSDF (SP 800-218) are further reference frameworks. Choose one, assess where you are, and pick a few improvements per quarter instead of attempting everything at once.

Making it stick

Adoption depends on integrating security into tools developers already use. Provide secure templates, definitions of done that include security tasks, and automated checks in pull requests. Classify applications by risk so that a public payment service receives deeper review than an internal wiki. Track measurable outcomes such as the share of apps with threat models, median time to fix high severity bugs and the count of findings escaping to production.

Governance matters too. Name an owner for each application, set a policy for exceptions with expiry dates and report metrics to leadership each quarter. When security teams act as enablers who supply guardrails and paved roads, developers cooperate. When they act only as gatekeepers at the end, teams route around them and late findings cause conflict and release delays. Review the program each year against your chosen model, retire activities that produce little value and add new ones as the threat landscape and technology stack change.

REQUIREMENT EXAMPLE

```
REQ-AUTH-07: Passwords are hashed with Argon2id (m=64MiB, t=3, p=1) and a unique 16-byte salt.  
Test: unit test reads stored hash prefix; DAST confirms lockout after 10 failed attempts.  
Source: ASVS v4 2.4.1  
Owner: identity team  
Phase: design review, SAST rule
```

WORKED EXAMPLE

A team plans a customer portal for example.com. During requirements they classify it as high risk because it holds personal data, choose ASVS level 2, and write abuse cases such as 'a user views another customer's invoice'. Threat modeling in design leads to a requirement for object-level authorization checks. Pull requests run SAST, and a release gate needs no open high findings. A SAMM assessment later shows verification as the weakest area, so the team funds automated DAST next.

KEY TERMS

SDLC: The sequence of phases used to plan, build, test and operate software.

ASVS: The OWASP Application Security Verification Standard, a list of testable security requirements.

Abuse case: A scenario describing how an attacker could misuse a feature.

SAMM: The OWASP Software Assurance Maturity Model for measuring a security program.

BSIMM: A descriptive study of software security activities in real organizations.

REMEMBER

- Add a security activity to every SDLC phase.
- Write specific, testable requirements, using ASVS as a catalogue.
- Use SAMM or BSIMM to measure progress and choose improvements.

QUICK CHECK

1. What makes a security requirement good?
 - A) It is vague and broad
 - B) It is specific and testable
 - C) It is written after release
 - D) It only mentions firewalls
2. Which model is descriptive and based on observed practices of real firms?
 - A) ASVS
 - B) CVSS
 - C) STIX
 - D) BSIMM
3. When is threat modeling most cost effective?
 - A) Only at penetration test
 - B) Never
 - C) After production incidents
 - D) During design
4. What is the purpose of risk-classifying applications?
 - A) To change programming language
 - B) To skip testing
 - C) To match security depth to the application's risk
 - D) To avoid documentation

Answers: 1-B 2-D 3-D 4-C

1. Testable requirements can be verified in design reviews and tests.
2. BSIMM reports what participating organizations actually do.
3. Design-time findings are cheapest to fix.
4. High-risk apps get deeper controls and reviews.

Input validation and output encoding

Injection and cross-site scripting remain among the most common web flaws, and both come from mixing untrusted data with code.

Treat all input as untrusted

Any data crossing a trust boundary can be hostile: form fields, query strings, headers, cookies, uploaded files, JSON bodies and even data read from a database or third-party API. Validate on the server, because client-side checks are only a usability aid. Prefer allow-list validation, which defines what is valid (type, length, range, format, enumerated values) and rejects everything else, over block-lists that attackers bypass with encodings. Canonicalize input before validating it, and validate the whole value, such as using a full-match regular expression with anchors.

Parameterized queries

SQL injection happens when user input is concatenated into a query string and changes its structure. The fix is parameterized queries, also called prepared statements, where the SQL text and the data travel separately and the database never parses data as code. In Python this is `cursor.execute('SELECT * FROM users WHERE id = %s', (uid,))`; in Java it is `PreparedStatement`. ORMs usually parameterize for you, but raw query features can still be injectable. For parts that cannot be parameterized, such as column names, map user choices to an allow-list of constants. Apply the same approach to LDAP, OS commands and NoSQL queries, and avoid shell calls entirely when a library function exists.

Context-aware output encoding

Cross-site scripting occurs when untrusted data is rendered as active content in a page. The defense is encoding at output time, matched to the context: HTML body, HTML attribute, JavaScript, URL and CSS each need a different encoder. Modern template engines such as Jinja2, React and Razor encode HTML by default, so the main risk lies in escape hatches such as `innerHTML`, `dangerouslySetInnerHTML` or the `safe` filter. When rich HTML must be accepted, sanitize it with a maintained library such as `DOMPurify` rather than writing regular expressions.

Defense in depth

Add layers behind the primary fix. A Content Security Policy such as `default-src 'self'; script-src 'self'` limits what injected scripts can do. Mark session cookies `HttpOnly` so scripts cannot read them. Run database accounts with least privilege so an injection cannot drop tables. Return generic error messages so attackers learn nothing about queries.

Other input risks deserve attention: path traversal in file names, unrestricted file upload, XML external entities, server-side request forgery and unsafe deserialization. Each has the same cause, trusting input, and the same shape of fix: constrain input to a known-good set, avoid interpreting it as code, and give the process the minimum permissions it needs. Build shared helper libraries for validation and encoding so teams reuse tested code, and add unit tests with hostile strings such as quotes, angle brackets, null bytes and very long values to catch regressions quickly. Document which layer owns each check so that validation is neither forgotten nor duplicated inconsistently across services. Re-test whenever a framework is upgraded, because default behaviors can change between versions.

CODE COMPARISON

```
# vulnerable
q = "SELECT * FROM users WHERE name = '" + name + "'"
# safe
cur.execute("SELECT * FROM users WHERE name = %s", (name,))
# template: autoescaped output
<p>Hello {{ user_name }}</p>
```

WORKED EXAMPLE

A search page builds SQL by joining the search term. Test input of a single quote causes a database error, which signals injection. The developer changes the query to a prepared statement, restricts the sort column to the values `name` or `date`, and limits the term to 50 characters using an allow-list pattern. The results page also printed the term unescaped, so the template is changed to autoescape and a CSP header is added. Retesting shows the quote is handled as plain text.

KEY TERMS

Allow-list: Validation that accepts only values known to be good.

Parameterized query: A query where the SQL structure and the values are sent separately.

Output encoding: Converting special characters so data renders as text in a given context.

XSS: Cross-site scripting, where untrusted data runs as script in a victim's browser.

CSP: Content Security Policy, a header restricting sources of scripts and other content.

REMEMBER

- Validate with allow-lists on the server.
- Use parameterized queries for every database call.
- Encode output for its context and add CSP as a backup.

QUICK CHECK

1. What is the primary defense against SQL injection?
 - A) Client-side validation
 - B) Parameterized queries
 - C) Hiding error pages
 - D) Longer passwords
2. Why is allow-list validation preferred over block-lists?
 - A) It rejects everything not explicitly valid, resisting encoding tricks
 - B) It encrypts data
 - C) It removes the need for encoding
 - D) It is faster to write
3. Which action best prevents stored XSS in a comment display?
 - A) Using GET not POST
 - B) Limiting length to 5
 - C) Context-aware output encoding
 - D) Hashing the comment
4. Where must validation be enforced?
 - A) In CSS
 - B) Only in the browser
 - C) On the server
 - D) Only in the database

Answers: 1-B 2-A 3-C 4-C

1. Separating code and data stops input changing the query.
2. Block-lists miss variants that attackers craft.
3. Encoding at output renders data as inert text.
4. Clients can be bypassed, so servers must validate.

Authentication and session handling

Weak credential storage or session handling lets attackers take over accounts at scale, even when the rest of the application is sound.

Password storage and policy

Store passwords only as salted hashes made by a slow, memory-hard function: Argon2id first choice, then scrypt or bcrypt (work factor at least 10, with its 72-byte input limit in mind). Never encrypt passwords reversibly or use a fast hash. NIST SP 800-63B advises long passphrases (minimum 8, allow at least 64 characters), screening against breached-password lists, no forced periodic rotation and no composition rules. Rate-limit login attempts per account and per source, add lockout or delay, and respond with identical messages for unknown users and wrong passwords to avoid account enumeration.

Multi-factor and recovery

Support multi-factor authentication through TOTP apps, hardware keys or passkeys (WebAuthn/FIDO2), which resist phishing far better than SMS codes. Password reset and account recovery are frequent weak points: use single-use, high-entropy tokens that expire in minutes, are stored hashed, and are sent to the verified channel only. Notify users of security events, and require re-authentication before sensitive actions such as changing email or adding a payment method. Log successes and failures with user ID and source address for detection.

Session cookies

After login, generate a new session identifier from a CSPRNG (at least 128 bits of entropy) to prevent session fixation, and keep it server-side or in a signed, validated token. Set cookie flags: Secure so it travels only over HTTPS, HttpOnly so scripts cannot read it, and SameSite=Lax or Strict to limit cross-site sending. Use the __Host- prefix where possible. Apply idle and absolute timeouts, invalidate the session on logout and on password change, and never place session IDs in URLs.

CSRF defenses

Cross-site request forgery tricks a logged-in browser into sending a state-changing request. Defend with anti-CSRF tokens that are unpredictable and tied to the session, checked on every POST, PUT, PATCH and DELETE; SameSite cookies as a second layer; and verification of Origin or Referer headers. Never change state with GET. For APIs using bearer tokens in an Authorization header rather than cookies, CSRF is much less of a concern, but CORS must still be configured narrowly.

Check also for session-related flaws in single sign-on code: validate the OIDC state and nonce, check token signature, issuer and audience, and prefer a maintained identity provider or library rather than building authentication yourself. Test the flow end to end with a proxy: confirm cookie flags in responses, try replaying an old session after logout, submit a form without the token, and verify that repeated failures trigger throttling. Document the results as part of the release checklist so that later refactors do not silently remove these protections. Review third-party identity components regularly, since a library update can change defaults such as token lifetimes.

HTTP RESPONSE HEADER

```
Set-Cookie: __Host-sid=Zm9vYmFyRXhhbXBsZQ; Path=/; Secure; HttpOnly; SameSite=Lax; Max-Age=900
Cache-Control: no-store
```

WORKED EXAMPLE

A login endpoint is reviewed. It stores unsalted SHA-256 hashes, issues the same session ID before and after login, and sets a cookie without flags. The fix migrates to Argon2id by rehashing at the next successful login, regenerates the session ID on authentication, sets Secure, HttpOnly and SameSite=Lax, adds a CSRF token to the profile form, and applies a rate limit of ten attempts per account per fifteen minutes. A DAST scan then confirms the cookie flags are present.

KEY TERMS

Argon2id: A memory-hard password hashing function resistant to GPU cracking.

Session fixation: An attack where the victim is made to use an attacker-chosen session ID.

HttpOnly: A cookie flag that blocks JavaScript access to the cookie.

SameSite: A cookie attribute limiting when cookies are sent on cross-site requests.

CSRF token: A secret per-session value that proves a request came from the real site.

REMEMBER

- Hash passwords with Argon2id, bcrypt or scrypt and rate-limit login.
- Regenerate sessions at login and set Secure, HttpOnly and SameSite.
- Use CSRF tokens and phishing-resistant MFA.

QUICK CHECK

1. Which cookie flag stops JavaScript from reading the session cookie?

- A) Secure
- B) HttpOnly
- C) SameSite
- D) Path

2. What should happen to the session ID at login?

- A) It is regenerated
- B) It is put in the URL
- C) It is shortened
- D) It stays the same

3. Which factor is most phishing resistant?

- A) Security questions
- B) Email link
- C) SMS code
- D) Passkey or FIDO2 key

4. What does a CSRF token protect against?

- A) Directory listing
- B) SQL injection
- C) Forged cross-site state-changing requests
- D) Weak passwords

Answers: 1-B 2-A 3-D 4-C

1. HttpOnly hides the cookie from document.cookie.
2. Regeneration prevents session fixation.
3. WebAuthn binds credentials to the genuine origin.
4. The attacker's site cannot know the token value.

Access control and API security

Broken access control is the top risk in the OWASP Top 10, and it is invisible to scanners that cannot know who should see what.

Server-side authorization

Authentication proves who you are; authorization decides what you may do. Enforce it on the server for every request, by default deny, using a central mechanism such as middleware, policy engine or framework annotations rather than scattered if statements. Hiding a button in the user interface is not a control. Check both the function (may this role call this endpoint) and the object (may this user access this record). Log denials, and write automated tests that attempt access as other users and roles.

IDOR and object-level checks

Insecure direct object reference, called broken object-level authorization (BOLA) in the OWASP API Top 10, occurs when a request such as GET /api/invoices/1042 returns the record without verifying that it belongs to the caller. Attackers simply change the number. Fix by scoping queries to the authenticated principal (WHERE id = ? AND owner_id = ?), or by calling an authorization check on every load. Unpredictable identifiers such as UUIDs help reduce guessing but do not replace the check.

Mass assignment and function-level access

Mass assignment occurs when a framework binds all request fields to a model, letting an attacker add role=admin or price=0 to a JSON body. Use allow-listed data transfer objects, mark sensitive fields as non-bindable, and never trust client-supplied roles. Broken function-level authorization appears when administrative endpoints such as /api/admin/users are reachable by ordinary users because only the menu was hidden. Deny by default, map each route to required roles, and review any endpoint that is undocumented or deprecated, since old API versions often lack new checks.

Wider API hardening

Beyond authorization, authenticate with short-lived OAuth 2.0 tokens, validate scopes, and enforce rate limits and quotas to resist scraping and abuse. Validate request bodies against a schema (OpenAPI), cap payload size, and return only the fields needed, to avoid excessive data exposure. Keep an inventory of every API and version, since shadow and zombie endpoints are common. Use TLS, set strict CORS rules, and log request IDs for tracing.

The OWASP API Security Top 10 is a good checklist for design reviews, and it maps to automated tests: run a suite that logs in as user A and tries every request against user B's resources and expects 403 or 404. Add a policy matrix listing each role and endpoint, then generate tests from it, so a new route without a rule fails the build. Review logs for repeated 403 and 404 responses on sequential IDs, which signal enumeration attempts, and alert on them. Teach developers to ask the question 'what stops another user from sending this request?' during every design discussion and code review.

REQUEST AND RESPONSE

```
GET /api/invoices/1042 HTTP/1.1
Authorization: Bearer eyJ...(user-2201)

HTTP/1.1 200 OK <- vulnerable: invoice belongs to user-3307
HTTP/1.1 404 Not Found <- fixed: query scoped to owner_id
```

WORKED EXAMPLE

During testing on a staging site, user alice (id 2201) opens her invoice 1042, then changes the URL to 1043 and sees another customer's invoice. The code loads by ID only. The fix is to query by both invoice ID and the caller's account, returning 404 if no row exists. A second finding shows PATCH /api/profile accepts an isAdmin field; the team replaces the model binding with a DTO listing name and email only, then adds automated cross-user tests.

KEY TERMS

IDOR/BOLA: Accessing another user's object by changing an identifier because no ownership check exists.

Mass assignment: Binding unintended request fields to internal model properties.

Deny by default: Access is refused unless a rule explicitly allows it.

Function-level authorization: Checking that a caller's role permits invoking a particular operation.

Scope: An OAuth permission limiting what a token may do.

REMEMBER

- Authorize every request on the server with default deny.
- Scope object lookups to the caller to stop IDOR.
- Use DTOs and role mapping to prevent mass assignment and function-level gaps.

QUICK CHECK

1. What is the correct fix for an IDOR?
 - A) Use longer URLs
 - B) Verify object ownership on the server for each request
 - C) Hide the ID in the UI
 - D) Block GET
2. What is mass assignment?
 - A) Binding all request fields to a model including sensitive ones
 - B) Bulk encryption
 - C) Assigning many roles
 - D) Sending many requests
3. Why is hiding an admin menu item not a control?
 - A) Browsers cache menus
 - B) Admins dislike it
 - C) Menus load slowly
 - D) The endpoint can still be called directly
4. Which test detects broken object-level authorization?
 - A) Measuring latency
 - B) Logging in as user A and requesting user B's objects
 - C) Port scanning
 - D) Checking TLS ciphers

Answers: 1-B 2-A 3-D 4-B

1. The server must check the caller is allowed to access that record.
2. Attackers add fields like role=admin to the body.
3. Authorization must be enforced at the endpoint.
4. Cross-user requests show whether ownership is enforced.

Secure code review

Review finds logic and design flaws that tools miss, and a checklist makes it consistent across reviewers and teams.

Review approach

Start from risk. Read the change description, identify entry points that handle untrusted data, and trace data from source (request, file, queue) to sink (database, command, HTML, outbound call). Prioritize authentication, authorization, cryptography, input handling and error paths. Review small changes of a few hundred lines at a time, as defect detection falls with size. Combine tool output from linters and SAST with human reasoning, and keep notes about what was checked so gaps are visible. Authors should explain the security intent in the pull request.

Checklist: injection and authorization

For injection, look for string concatenation in SQL, shell calls, template rendering with raw output, file path joins, XML parsers with external entities enabled and deserialization of untrusted data such as Java `ObjectInputStream` or Python `pickle`. For authorization, ask on every endpoint: who is the caller, where is the check, is it on the object as well as the function, and does the default deny? Watch for TODO comments that disable checks, debug backdoors and role values taken from request bodies.

Checklist: crypto, secrets and errors

Look for hardcoded credentials, weak algorithms (MD5, SHA-1, ECB, DES), non-secure random number use, disabled certificate validation and homemade crypto. For errors and logging, check that exceptions do not leak stack traces or SQL to users, that failures deny access rather than allow it (fail closed), that sensitive values such as passwords, tokens and card numbers are not logged, and that security events are recorded. Verify resource limits, such as timeouts and file size caps, to avoid denial of service.

Culture and process

Make review part of the merge process: require approval from a second person, and from a security champion or code owner for sensitive paths such as authentication and payments. Keep feedback specific, explain why it matters, and suggest a fix. Track recurring findings and turn them into Semgrep rules, training topics or library helpers so they cannot reappear.

Reviewers should also look for what is missing: a test for the failure path, a check after a refactor that removed a middleware, or a new dependency added without review. Measure review coverage and time to merge so the process does not become a bottleneck; fast, focused reviews are followed, slow ones are bypassed. Rotate reviewers occasionally so fresh eyes catch blind spots, and keep a library of real examples from past findings. New developers learn faster from concrete before and after diffs than from abstract guidelines. Ask the author to mark risky areas in the description so reviewers know where to spend limited time.

DIFF EXCERPT

```
- if (user.canView(doc)) { return doc; }
+ return docRepo.findById(req.params.id); // authz check removed
+ log.info("login", username, password); // secret logged
+ String q = "SELECT * FROM t WHERE c='" + c + "'";
```

WORKED EXAMPLE

A reviewer opens a pull request adding document sharing. Tracing the `id` parameter from the controller to the repository, she notices the old `canView` check was removed during refactoring, so any logged-in user can fetch any document. She also spots a log line that records the password on failed login. She comments with the line numbers, proposes restoring the check in a service method, and asks for a test where a second user receives 403. The author fixes both and the merge proceeds.

KEY TERMS

Source and sink: The point where untrusted data enters and the dangerous operation where it is used.

Fail closed: On error, the system denies access rather than granting it.

Code owner: A person or team who must approve changes to specified files.

Deserialization: Rebuilding objects from bytes, risky when the data is untrusted.

Secure code review checklist: A list of security questions applied to each change.

REMEMBER

- Trace untrusted data from source to sink.
- Use a checklist for injection, authz, crypto and error handling.
- Feed recurring findings into rules, libraries and training.

QUICK CHECK

1. What does a reviewer trace to find injection?
 - A) Commit timestamps
 - B) Data from untrusted source to dangerous sink
 - C) File sizes
 - D) Variable names
2. Which behavior is fail closed?
 - A) Deny request if auth service errors
 - B) Skip checks in debug
 - C) Log and continue
 - D) Allow request if auth service errors
3. Which finding is most serious in a diff?
 - A) A renamed variable
 - B) An extra blank line
 - C) A comment typo
 - D) A removed authorization check
4. How can recurring review findings be prevented?
 - A) Disable linting
 - B) Ignore them
 - C) Convert them into automated rules and training
 - D) Increase PR size

Answers: 1-B 2-A 3-D 4-C

1. Taint flow shows where input reaches a risky operation.
2. Failures should deny access.
3. Removing the check exposes data to any user.
4. Automation makes the fix permanent across the codebase.

SAST and DAST

Automated testing scales security checks across thousands of builds, but only if results are tuned so developers trust them.

SAST

Static application security testing analyzes source code or bytecode without running it. Tools such as Semgrep (pattern-based rules that look like the code), CodeQL (queries over a code database that track data flow) and SonarQube find injection, unsafe functions, hardcoded secrets and misconfigurations early, in the IDE or pull request. Strengths: line-level locations, early feedback and full code coverage. Weaknesses: false positives, limited insight into runtime configuration and business logic, and difficulty with frameworks they do not model. Language and framework support varies, so verify coverage.

DAST

Dynamic application security testing probes a running application from the outside, like an attacker. OWASP ZAP and Burp Suite crawl pages, fuzz parameters and check for reflected XSS, injection symptoms, missing headers and weak cookies. DAST sees the deployed configuration and has fewer false positives for reachable flaws, but it finds issues late, needs a working environment and test accounts, and may miss paths it cannot crawl. Run it against staging or an isolated test instance, never against systems you are not authorized to test, with authenticated scans for realistic coverage.

Tuning and triage

Untuned tools bury teams in noise. Start with a small high-confidence ruleset and widen it gradually. For each finding decide: true positive (fix), false positive (suppress with a documented reason, ideally a code comment or baseline file), or risk accepted (with owner and expiry). Write custom Semgrep rules for in-house frameworks, mark sanitizers so data flow tools stop flagging safe paths, and set the pipeline to fail only on new high severity findings, not on existing debt. Review suppressions regularly.

Pipeline placement

Place checks where they give fast feedback: SAST and secret scanning on every pull request, SCA on every build, DAST nightly or on release candidates, and manual testing for high-risk features. Output results in SARIF format so they appear in the code host as annotations. Track metrics: mean time to remediate, false positive rate, and percentage of findings fixed within SLA.

Remember that tools complement but do not replace design review and testing. IAST agents and fuzzers add runtime insight, and bug bounty programs add outside perspective. Combine several methods because each finds different classes of defects. Revisit the setup quarterly. Check that new repositories are onboarded automatically, that scan times stay short enough for pull requests, and that scanner versions and rule packs are updated. Compare findings from different tools, since overlap builds confidence while differences reveal coverage gaps, and use bug reports from testers to write new custom rules for future scans.

TOOL OUTPUT

```
$ semgrep --config p/owasp-top-ten src/
src/orders.py
python.django.security.injection.sql.sql-injection
line 42: cursor.execute("SELECT ... " + order_id)
Findings: 1 (1 blocking) Ran 312 rules on 58 files
```

WORKED EXAMPLE

A team enables Semgrep and sees 240 findings. They set the gate to block only new high-confidence findings, then triage the backlog: twelve are real SQL injections, forty are test files and are excluded by path, and the rest are false positives from a custom sanitizer, for which a rule is added. A nightly ZAP scan on the staging copy of the app finds a missing Content-Security-Policy header and a session cookie without the Secure flag. Both are fixed and verified in the next run.

KEY TERMS

SAST: Analysis of source code without executing it.

DAST: Testing a running application from the outside.

False positive: A reported issue that is not a real vulnerability.

SARIF: A standard JSON format for static analysis results.

Baseline: A recorded set of existing findings so only new ones fail a build.

REMEMBER

- SAST reads code; DAST probes a running app; use both.
- Tune rules, baseline old findings and gate on new high severity issues.
- Track false positives and fix times to keep tools trusted.

QUICK CHECK

1. Which is a strength of SAST?
 - A) Finds issues with exact code locations early
 - B) Sees deployed server config
 - C) Needs no source
 - D) Never reports false positives
2. Which tool is a typical DAST scanner?
 - A) CodeQL
 - B) Gype
 - C) gitleaks
 - D) OWASP ZAP
3. What is a good way to reduce noise from a new SAST rollout?
 - A) Raise every finding to critical
 - B) Scan only once a year
 - C) Disable it
 - D) Block only new high-confidence findings and tune rules
4. Where should a documented suppression of a false positive be kept?
 - A) In the production database
 - B) In a chat
 - C) Near the code or in a reviewed baseline file with a reason
 - D) Nowhere

Answers: 1-A 2-D 3-D 4-C

1. SAST works on code and points to lines.
2. ZAP tests running web applications.
3. Gradual gates keep trust while debt is reduced.
4. Reasoned, reviewed suppressions are auditable.

SCA and dependency risk

Most application code is third-party, so a vulnerable library can expose an application that has no flaws of its own.

What SCA does

Software composition analysis identifies the open source components in an application and matches them against vulnerability databases such as the National Vulnerability Database (CVE IDs), GitHub Advisory Database and OSV. Tools include OWASP Dependency-Check, Dependabot, Renovate, Snyk, Trivy and Grype. They read lockfiles and manifests (package-lock.json, poetry.lock, pom.xml) and report vulnerable versions, license issues and sometimes abandoned packages. A software bill of materials lists all components in a standard format such as CycloneDX or SPDX.

Reading the data

A CVE entry gives a description, affected version ranges and a CVSS score. CVSS describes severity, not risk: ask whether the vulnerable function is reachable, whether the component is exposed to untrusted input and whether a fix exists. The EPSS score estimates the probability of exploitation, and CISA's Known Exploited Vulnerabilities catalog lists flaws seen in active attacks, which should be fixed first. Reachability analysis and VEX statements can mark a CVE as not affected when the vulnerable code is never called.

Transitive risk and pinning

Your direct dependency pulls in many others, and a flaw in a deep transitive package still affects you, as the Log4Shell case (CVE-2021-44228) showed. Use commands such as `npm ls`, `pip show` or `mvn dependency:tree` to find which direct package brings it in, then upgrade the parent or override the version. Commit lockfiles and pin exact versions with integrity hashes so builds are repeatable and a malicious new release is not pulled in automatically, while keeping update bots running so pinning does not mean stagnation.

Policy and process

Set a policy: block builds on known exploited or critical reachable flaws, set remediation SLAs by severity, and approve new dependencies with checks on maintenance, popularity, license and typosquatting. Prefer fewer, well-maintained libraries and remove unused ones. Automate update pull requests and run the test suite on them so upgrades are low effort.

When no fix exists, use mitigations: disable the vulnerable feature, add a WAF rule, restrict network exposure, replace the library, or document an accepted risk with an expiry date. Subscribe to advisories for your key components and keep an inventory so you can answer the question 'where do we use this package?' within minutes. Include build tools, container base images and infrastructure modules in the inventory, since they also bring vulnerable code. Re-scan deployed artifacts regularly, because new CVEs appear after release and a build that was clean last month may be vulnerable today. Keep a record of exceptions and their review dates.

TERMINAL OUTPUT

```
$ grype dir:. --only-fixed
NAME INSTALLED FIXED-IN TYPE VULNERABILITY SEVERITY
lodash 4.17.15 4.17.21 npm CVE-2021-23337 High
express 4.16.0 4.19.2 npm CVE-2024-29041 Medium
Packages scanned: 214 Findings: 2
```

WORKED EXAMPLE

A scan flags a High finding in the transitive package `minimist`, brought in by an old build tool. `npm ls` shows the path. The tool is a dev dependency used only in CI, so runtime risk is low, but the team upgrades the parent package anyway, regenerates the lockfile and reruns the scan, which now reports zero findings. For a second CVE with no fix, they document that the vulnerable function is never called and record a VEX statement with a review date.

KEY TERMS

SCA: Identification of third-party components and their known vulnerabilities.

Transitive dependency: A package required indirectly through another dependency.

Lockfile: A file recording exact dependency versions and hashes.

EPSS: A score estimating the likelihood that a CVE will be exploited.

KEV: CISA's catalog of vulnerabilities known to be exploited.

REMEMBER

- Scan dependencies and generate SBOMs every build.
- Prioritize by reachability, exposure, EPSS and KEV, not CVSS alone.
- Pin versions, commit lockfiles and keep update automation running.

QUICK CHECK

1. Why can CVSS alone mislead prioritization?
 - A) It measures severity, not exposure or reachability
 - B) It is always zero
 - C) It lists patches
 - D) It encrypts packages
2. What is a transitive dependency?
 - A) A package you wrote
 - B) A test file
 - C) A license
 - D) A package your dependency requires
3. Why commit lockfiles?
 - A) To reduce testing
 - B) To disable updates
 - C) To hide versions
 - D) To make builds repeatable with exact versions
4. Which catalog identifies vulnerabilities seen in real attacks?
 - A) PEP index
 - B) CISA KEV
 - C) Alexa list
 - D) RFC index

Answers: 1-A 2-D 3-D 4-B

1. Risk also depends on reachability, exposure and exploitation.
2. It is indirectly included through another package.
3. Locked versions avoid surprise upgrades.
4. KEV lists exploited vulnerabilities to patch first.

Security champions program

Security teams cannot review every change, so trained developers inside each team multiply their reach and keep practices alive.

Why champions

A security champion is a developer or tester who volunteers to be the first point of contact for security within their team, in addition to their normal job. The role scales a small central team: champions triage tool findings, run quick design discussions, remind peers of standards and bring issues upward. They are not a replacement for the security team or the person who fixes everything. Executive sponsorship and explicit time allocation, often 10 to 20 percent, determine whether the program survives beyond the first enthusiasm.

Building the network

Pick volunteers from every product team and aim for roughly one per eight to ten developers. Hold a regular community meeting every two to four weeks, run a chat channel, and share new threats, post-incident learning and tool updates. Offer progressive training: secure coding labs on the OWASP Top 10, threat modeling workshops, capture-the-flag events and certifications. Recognize contributions through career credit, badges and visibility to management, since non-monetary rewards sustain participation.

Threat model reviews and tooling

Champions lead lightweight threat modeling in design sessions using STRIDE and a data flow diagram, and bring complex cases to the central team. They own the backlog of security findings for their team, check that SAST and SCA results are triaged, and review pull requests touching authentication, authorization and crypto. Give them templates, checklists and dashboards so the work is quick and consistent, and security tools configured with team-level views.

Vulnerability SLAs and metrics

Agree service level targets for fixing vulnerabilities by severity, for example critical in 7 days, high in 30, medium in 90, with shorter times for known exploited issues, and define the exception process with expiry dates. Report metrics such as findings by age, SLA compliance, training completion, threat models completed and champion coverage by team. Review them in leadership meetings.

Watch for failure modes: champions becoming bottlenecks, burnout from unpaid extra work, and loss of momentum when sponsors change. Rotate duties, refresh the roster each year, and tie the program to the secure SDLC maturity goals so its value is visible in measurable improvement. Start small with a pilot of two or three teams, learn what works, and publish a short charter describing responsibilities, time commitment and escalation paths before expanding to the whole organization. Collect feedback from champions and their managers each quarter and adjust the program, because success depends on the role being useful to developers and not merely another compliance task.

SLA POLICY EXCERPT

```
Severity Fix within Escalate at
Critical 7 days day 3 (CISO)
High 30 days day 20 (Eng director)
Medium 90 days day 60 (Team lead)
KEV-listed 48 hours immediately
```

WORKED EXAMPLE

A company with twelve product teams recruits one champion per team and gives each four hours a week. The first quarter includes a threat modeling workshop and a secure coding lab. Champions then run a one-hour STRIDE session for each new feature. The security team publishes a dashboard showing that high severity findings fixed within 30 days rose from 55 to 82 percent after six months, and the champion roster is presented at the engineering all-hands.

KEY TERMS

Security champion: A team member who acts as the local security advocate and liaison.

Threat model: A structured analysis of what can go wrong in a design.

STRIDE: A threat classification: spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege.

SLA: An agreed time limit for fixing issues of a given severity.

Sponsor: An executive who supplies backing and time for the program.

REMEMBER

- Recruit and train champions with sponsor backing and time.
- Use champions for threat models, triage and review of sensitive code.
- Set SLAs and report metrics to keep the program credible.

QUICK CHECK

1. What is the main purpose of a champions program?
 - A) Replace the security team
 - B) Scale security knowledge into development teams
 - C) Eliminate testing
 - D) Reduce headcount
2. Which element most sustains a champions program?
 - A) Secrecy
 - B) No training
 - C) Random assignment
 - D) Sponsor support and allocated time
3. What does STRIDE help with?
 - A) Scanning ports
 - B) Packaging code
 - C) Classifying threats during design
 - D) Hashing passwords
4. Why define vulnerability SLAs?
 - A) To change CVSS
 - B) To set expected remediation times and escalation
 - C) To hide issues
 - D) To rename CVEs

Answers: 1-B 2-D 3-C 4-B

1. Champions extend reach and keep practices consistent.
2. Without time and backing, volunteers burn out.
3. It gives categories for what could go wrong.
4. Clear deadlines drive accountability.

Fix SQL injection

SCENARIO

The orders service for example.com builds a SQL query by joining a customer-supplied search term. A tester sent a single quote and received a raw database error from a staging server at 192.0.2.30. Fix the function, then replay the attack input to prove that the weakness is closed. The release goes out tomorrow, and the product owner wants proof that the issue is closed. Use the sandbox tests and the static scan results as evidence for the change ticket, and keep the fix small so it can be reviewed quickly by a second developer.

OBJECTIVE

Replace string-built SQL with a parameterized statement and confirm the attack fails.

STEPS IN THE SIMULATOR

- 1 **Do:** Open orders.py and read find_orders(name).

Simulator shows:

```
query = "SELECT id,total FROM orders WHERE customer='" + name + "'"
```

- 2 **Do:** Run the test input: ' OR '1'='1

Simulator shows:

Result: 3,412 rows returned. Test status: VULNERABLE.

- 3 **Do:** Rewrite the call as cursor.execute("SELECT id,total FROM orders WHERE customer = %s", (name,)).

Simulator shows:

Editor: syntax OK. Static check: sql-concat rule cleared.

- 4 **Do:** Add an allow-list check: name must match `^[A-Za-z .-]{1,50}$`.

Simulator shows:

Validator: input constraint present.

- 5 **Do:** Replay ' OR '1'='1 against the fixed code.

Simulator shows:

Result: 0 rows. The input is treated as a literal customer name.

- 6 **Do:** Run the unit tests and static scan.

Simulator shows:

12/12 tests pass. Scan: 0 high findings.

ANSWER

The function uses a parameterized query with the value passed separately, plus a length and character allow-list. The injection payload now returns zero rows because it is treated as data, and the SAST rule no longer flags the code.

Hint shown in the app: Keep the SQL text constant and pass the variable as a parameter.

Fix XSS output

SCENARIO

A profile page template prints the user's display name and a bio inside HTML, an attribute and an inline script. A tester stored a script tag in the bio and it executed in another user's browser. You must apply the correct encoder in each context and add a CSP header. The page serves several thousand users, so the fix must not break legitimate formatting such as plain paragraphs. The security team will retest the stored payload after your change, and the CSP must not block the site's own bundled scripts.

OBJECTIVE

Remove the XSS by context-aware encoding and add a Content-Security-Policy.

STEPS IN THE SIMULATOR

- 1 **Do:** Open profile.html and find the three output points.

Simulator shows:

1: `<p>{{ bio|safe }}</p>` 2: `<input value="{{ name }}">` 3: `<script>var n='{{ name }}';</script>`

- 2 **Do:** Remove the safe filter so the bio is HTML-escaped.

Simulator shows:

Preview: `<script>` now displays as text.

- 3 **Do:** Ensure the attribute uses quoted autoescaped output.

Simulator shows:

Linters: attribute context OK.

- 4 **Do:** Replace the inline script with data attribute plus JSON via tojson filter, or move it to an external file.

Simulator shows:

Linters: no raw output inside script context.

- 5 **Do:** Add header Content-Security-Policy: default-src 'self'; script-src 'self'.

Simulator shows:

Header present. Inline scripts blocked in browser test.

- 6 **Do:** Replay the stored payload.

Simulator shows:

Result: payload displayed as harmless text, no alert. Console: no CSP violations from legitimate scripts.

ANSWER

HTML body output is autoescaped, the attribute is quoted and escaped, the script context no longer receives raw input, and a CSP of default-src 'self'; script-src 'self' blocks inline scripts. The stored payload no longer executes.

Hint shown in the app: Different contexts need different encoders; the safe filter is the usual culprit.

Review a pull request

SCENARIO

A colleague opened a pull request adding a document export feature to the internal portal. You are the reviewer and have a checklist for authorization and deserialization. Annotate the diff with the issues you find and request specific changes before approving. The author is a new team member, so comments should be specific and constructive, with a clear explanation of each risk and the expected fix. Use the review checklist for injection, authorization, cryptography and error handling, and block the merge only for real findings.

OBJECTIVE

Identify missing authorization and unsafe deserialization in the diff and annotate them.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the diff for `ExportController.java` and read the new endpoint.

Simulator shows:

```
@GetMapping("/export/{docId}") returns docService.load(docId)
```

- 2 **Do:** Check whether the endpoint verifies the caller owns the document.

Simulator shows:

No ownership or role check found. Annotation slot opened.

- 3 **Do:** Add comment: Missing object-level authorization; require owner or role check.

Simulator shows:

Comment saved on line 18, severity High.

- 4 **Do:** Open `ImportService.java` and find the `readObject` call.

Simulator shows:

```
ObjectInputStream.readObject() on uploaded file bytes
```

- 5 **Do:** Add comment: Unsafe deserialization of untrusted data; use JSON with a schema or an allow-list filter.

Simulator shows:

Comment saved on line 44, severity High.

- 6 **Do:** Click Request changes.

Simulator shows:

Review submitted: 2 blocking findings. Author notified.

ANSWER

Two blocking findings: the export endpoint lacks an ownership check (broken object-level authorization), and the import service deserializes uploaded bytes with `ObjectInputStream` (unsafe deserialization). Requested fixes are an authorization check on every load and a safe data format such as validated JSON.

Hint shown in the app: Look for what happens to the `id` parameter and where uploaded bytes end up.

Triage SAST findings

SCENARIO

A nightly Semgrep scan of the billing service reports eight findings. You are on call for the product team and must classify each as true positive, false positive or risk accepted, with a short reason, before the release meeting. Consider reachability, test-only code and compensating controls. Each decision will be recorded in the findings tracker and reviewed by the security champion. Suppress only with a written reason, and make sure that any accepted risk has a named owner and an expiry date so it will be revisited rather than forgotten.

OBJECTIVE

Classify each finding correctly with a documented reason.

STEPS IN THE SIMULATOR

- Do:** Open finding 1: SQL concat in invoices.py line 88, input from request.

Simulator shows:

Context: user-controlled param reaches query. Classify as TRUE POSITIVE, high.

- Do:** Open finding 2: hardcoded password in tests/fixtures.py.

Simulator shows:

Context: dummy value used only by unit tests. Classify as FALSE POSITIVE with reason 'test fixture'.

- Do:** Open finding 3: use of MD5 for cache key naming.

Simulator shows:

Context: no security purpose. Classify as FALSE POSITIVE with reason 'non-security hash'.

- Do:** Open finding 4: yaml.load without SafeLoader on config file read at startup from a locked directory.

Simulator shows:

Context: file not user controlled. Classify as RISK ACCEPTED, owner platform team, expires in 90 days.

- Do:** Open finding 5: missing CSRF token on POST /api/refund.

Simulator shows:

Context: cookie session auth, no token. Classify as TRUE POSITIVE, high.

- Do:** Submit classifications.

Simulator shows:

Saved. 2 true positives (fix before release), 2 false positives (suppressed with reasons), 1 risk accepted (expiry set). Remaining 3 queued.

ANSWER

The SQL concatenation and the missing CSRF protection are true positives to fix before release. The hardcoded test password and MD5 cache key are false positives suppressed with reasons. The unsafe YAML load on a locked config file is risk accepted with an owner and expiry date.

Hint shown in the app: Ask whether untrusted input can reach the code and whether it has a security purpose.

Exploit an IDOR in sandbox

SCENARIO

In an authorized training sandbox at shop.example.com, you are logged in as user alice. Using the intercepting proxy, test whether the order lookup endpoint checks ownership. Then write down the fix. Only the sandbox application is in scope and all data is fictional. Treat the exercise as a professional test: stay inside the agreed scope, record each request you send, and do not store or share any data you see. The goal is a clear report that developers can act on, including the exact fix.

OBJECTIVE

Confirm an IDOR by changing an object ID, then specify the server-side fix.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the proxy history and select GET /api/orders/5001 (your order).

Simulator shows:

```
200 OK {"id":5001,"owner":"alice","total":42.00}
```

- 2 **Do:** Send the request to Repeater and change the ID to 5002.

Simulator shows:

```
Request edited.
```

- 3 **Do:** Click Send.

Simulator shows:

```
200 OK {"id":5002,"owner":"bob","total":310.50,"address":"1 Sample St"}
```

- 4 **Do:** Try IDs 5003 and 5004.

Simulator shows:

```
Both return 200 with other users' orders. Finding: broken object-level authorization.
```

- 5 **Do:** Record the fix: scope the query by owner and return 404 when not found.

Simulator shows:

```
Notes saved: WHERE id=? AND owner_id=session.user_id
```

- 6 **Do:** Apply the fix in the sandbox and resend ID 5002.

Simulator shows:

```
404 Not Found. Own order 5001 still returns 200.
```

ANSWER

Changing the order ID returns other users' data, which proves an IDOR (BOLA). The fix is a server-side check that the order belongs to the authenticated user, implemented by adding the owner to the query, returning 404 for others, and adding automated cross-user tests.

Hint shown in the app: Only the ID changes in the request, so the server must compare the record owner to the caller.

Dependency audit

SCENARIO

A Node.js service for the example.com web store has a package-lock.json. Before release, the security team requires an SCA scan and upgrade plan. Run the scanner, read the results and choose the least disruptive upgrade for each vulnerable package, noting transitive dependencies. The release manager needs a short list of changes with their risk. Avoid major version jumps unless required, run the tests after every change, and note any package that cannot be upgraded so a mitigation or exception can be recorded.

OBJECTIVE

Scan a lockfile and choose upgrade paths for the vulnerable packages.

STEPS IN THE SIMULATOR

- 1 **Do:** Run: `grype dir:. --only-fixed`

Simulator shows:

3 findings: lodash 4.17.15 (High, fixed 4.17.21); minimist 1.2.0 (Critical, fixed 1.2.6); express 4.16.0 (Medium, fixed 4.19.2)

- 2 **Do:** Run: `npm ls minimist`

Simulator shows:

app@1.0.0 -> mkdirp-tool@0.5.1 -> minimist@1.2.0 (transitive)

- 3 **Do:** Run: `npm ls lodash`

Simulator shows:

app@1.0.0 -> lodash@4.17.15 (direct)

- 4 **Do:** Run: `npm install lodash@4.17.21 express@4.19.2`

Simulator shows:

Updated 2 packages. Lockfile rewritten.

- 5 **Do:** Add an overrides entry forcing minimist to 1.2.6, then `npm install`.

Simulator shows:

Resolved minimist@1.2.6 for mkdirp-tool.

- 6 **Do:** Rerun grype and the test suite.

Simulator shows:

0 findings. 48/48 tests pass.

ANSWER

Upgrade the direct dependencies lodash and express, and use an override for the transitive minimist so it resolves to 1.2.6 or later. Commit the updated lockfile, rerun the scan and tests, and keep update automation enabled for future advisories.

Hint shown in the app: Find which direct package pulls in the transitive one with `npm ls`.

Application Security: 10 questions, 80% to pass

1. Which SDLC activity should occur during design?
 - A) Threat modeling
 - B) Penetration test of production only
 - C) Patching after release
 - D) Log deletion
2. What is the best defense against SQL injection?
 - A) Client-side checks
 - B) Hiding error text
 - C) Using GET
 - D) Parameterized queries
3. Which cookie attribute limits cross-site sending?
 - A) Domain
 - B) Expires
 - C) SameSite
 - D) Path
4. A user changes /api/orders/5001 to 5002 and sees another customer's order. What is this?
 - A) SSRF
 - B) IDOR/BOLA
 - C) XSS
 - D) CSRF
5. What is mass assignment?
 - A) Binding unintended request fields to a model
 - B) Mass emailing
 - C) Bulk hashing
 - D) A DDoS
6. Which is true for DAST?
 - A) Reads source only
 - B) Only scans dependencies
 - C) Cannot find XSS
 - D) Tests a running application from outside
7. What should you do with a SAST finding in test-only fixture code?
 - A) Ignore without record
 - B) Raise to critical
 - C) Classify as false positive with a documented reason
 - D) Delete the tool
8. Why is CVSS alone not enough to prioritize dependency fixes?
 - A) It lists fixes
 - B) It ignores reachability and exposure
 - C) It is secret
 - D) It is always low
9. Which password storage approach is correct?
 - A) Argon2id with unique salt
 - B) Plain SHA-256
 - C) Reversible encryption
 - D) MD5
10. What keeps a security champions program sustained?
 - A) Surprise audits
 - B) No metrics
 - C) Removing training
 - D) Sponsor backing, time and recognition

Answers: 1-A 2-D 3-C 4-B 5-A 6-D 7-C 8-B 9-A 10-D

1. Threat modeling finds flaws before code exists.
2. Parameters separate data from code.
3. SameSite controls whether cookies are sent cross-site.
4. Missing object-level authorization.
5. Attackers add fields such as role.
6. DAST works against deployed behavior.
7. Document and suppress with reasoning.
8. Context determines real risk.
9. Slow salted hashes resist cracking.
10. Time and support prevent burnout.

DevSecOps and CI/CD Security

Advanced | 7 lessons | 5 simulated labs | checkpoint | about 6h 25m

This module focuses on securing the path from commit to production. You will map pipeline attack points, harden runners, tokens and workflow files, find and respond to leaked secrets, sign and verify artifacts with Sigstore, produce and scan SBOMs, and understand SLSA and supply chain attacks such as dependency confusion. Finally you will enforce rules automatically with policy as code. These skills reduce the chance that a compromised build, dependency or credential becomes a compromise of production.

BY THE END YOU CAN

- + Find and fix weaknesses in CI/CD pipelines
- + Generate SBOMs and sign artifacts
- + Enforce security rules with policy as code

IN THIS MODULE

Lesson 1 CI/CD threat model

Lesson 2 Pipeline hardening

Lesson 3 Secrets in repos

Lesson 4 Artifact signing and provenance

Lesson 5 SBOM and vulnerability data

Lesson 6 Supply chain security and SLSA

Lesson 7 Policy as code gates

Lab 1 Harden a workflow file (Config editor)

Lab 2 Scan repo for secrets (Terminal)

Lab 3 Verify a signature (Terminal)

Lab 4 Read an SBOM (File explorer)

Lab 5 Write a Rego rule (Code editor)

Checkpoint 10 questions, 80% to pass

CI/CD threat model

The build pipeline holds the keys to production, so compromising it can be more damaging than compromising any single application.

Why pipelines are targets

A CI/CD system checks out code, downloads dependencies, runs scripts with credentials and publishes artifacts that are later deployed with high privileges. An attacker who controls any step can insert malicious code that is signed and trusted by the normal process, as happened in the SolarWinds build compromise and the Codecov bash uploader incident. Pipelines also concentrate secrets: cloud keys, registry tokens, signing keys and database passwords. Treat the pipeline as a production system with its own threat model, owners and monitoring, rather than as developer tooling.

Attack points in the flow

Map the flow and list attack points. Source: stolen developer credentials, malicious pull requests from forks, force pushes and unprotected branches. Build: poisoned build scripts, compromised third-party actions or plugins, and injected environment variables. Runners: shared or persistent runners where one job can read another's files, and self-hosted runners exposed to the internet. Dependencies: malicious or typosquatted packages. Registries: overwritten tags and weak push permissions. Deploy: over-privileged cloud credentials and missing approvals. Each point suggests a control.

Frameworks and categories

The OWASP Top 10 CI/CD Security Risks names categories such as insufficient flow control, inadequate identity and access management, dependency chain abuse, poisoned pipeline execution, insufficient credential hygiene and insecure system configuration. Poisoned pipeline execution means an attacker changes pipeline definitions, or inputs they influence, so that the CI system runs their commands. The SLSA framework and NIST SSDF address the same area from the supply chain perspective. Use these lists as review checklists and for tabletop exercises with the platform team.

Building the model

Draw a data flow diagram with trust boundaries: developer workstation, source host, CI control plane, runners, artifact registry, deployment target. For each flow apply STRIDE and note which credentials are present. Rank findings by impact, for example a runner that holds production admin keys is critical. Then choose controls: branch protection, mandatory reviews, ephemeral runners, least-privilege tokens, protected environments with approvals, signed artifacts and logging of all pipeline changes.

Keep the model alive. Update it when you add a new tool, repository or deployment target, and review it after incidents elsewhere in the industry. Ask 'if this runner were fully compromised, what could it reach?' and use the answer to shrink credentials and network access until the blast radius is acceptable.

THREAT TABLE

Stage	Threat	Control
Source	Force push to main	Branch protection, signed commits
Build	Malicious 3rd-party action	Pin to commit SHA, allow-list
Runner	Persistent shared runner	Ephemeral runners, isolation
Registry	Tag overwrite	Immutable tags, digest pinning
Deploy	Long-lived admin key	OIDC, short-lived roles

WORKED EXAMPLE

A team maps their pipeline for the shop.example.com service and finds that pull requests from forks run with access to a deploy token, that the self-hosted runner is shared with another project, and that the registry allows overwriting the latest tag. They rank the fork token issue as critical, change workflows so fork builds run without secrets, move to ephemeral runners, and enable immutable tags. Their threat model document is attached to the platform runbook and reviewed twice a year.

KEY TERMS

Poisoned pipeline execution: Attack that modifies pipeline definitions or inputs so the CI system runs attacker commands.

Runner: The machine or container that executes pipeline jobs.

Blast radius: The amount of damage possible if a component is compromised.

Trust boundary: A line where data or control passes between components of different trust.

Ephemeral runner: A clean, single-use build environment destroyed after the job.

REMEMBER

- Treat CI/CD as a production system holding powerful secrets.
- Map attack points from source through build, runners, registry and deploy.
- Use STRIDE and the OWASP CI/CD risks to choose controls.

QUICK CHECK

1. What is poisoned pipeline execution?
 - A) Altering pipeline definitions or inputs so CI runs attacker commands
 - B) Slow builds
 - C) Corrupt backups
 - D) Expired tokens
2. Why are shared persistent runners risky?
 - A) They are always offline
 - B) They cost more
 - C) They use TLS
 - D) One job can leave data or malware for another
3. Which control reduces the damage of a compromised runner?
 - A) Disabling logs
 - B) Sharing it widely
 - C) Giving it admin keys
 - D) Least-privilege, short-lived credentials
4. Why does the pipeline need its own threat model?
 - A) It is not networked
 - B) It holds secrets and can change what runs in production
 - C) It stores marketing data
 - D) It has no users

Answers: 1-A 2-D 3-D 4-B

1. The attacker abuses the pipeline's own execution authority.
2. State persists across jobs, so secrets and files can leak.
3. Smaller privileges limit the blast radius.
4. Its privileges make it a high-value target.

Pipeline hardening

Hardening shrinks what a compromised job, action or token can do, and removes the long-lived keys that attackers love to steal.

Runners and isolation

Use ephemeral runners that start clean for each job and are destroyed after it, so a compromised job cannot persist or read another job's workspace. Prefer hosted runners for public repositories, because self-hosted runners on public repos can run code from forks. Place self-hosted runners in isolated networks with egress filtering, run them without root, and keep images patched. Separate runners for trusted and untrusted code, and never run untrusted pull request code on a runner that can reach production networks or internal secrets.

Restricting tokens and permissions

CI systems issue an automatic token to each run, such as GITHUB_TOKEN. Set the default to read-only and grant write scopes per job, for example permissions: contents: read at workflow level and id-token: write only on the deploy job. Use protected environments that require approval and restrict which branches may deploy. Require reviews and status checks on main, and block secrets from fork pull requests. Be careful with triggers such as pull_request_target that run in the base repository context and expose secrets.

Pinning dependencies

Third-party actions and plugins execute inside your pipeline, so a tag like v3 that is moved by a compromised maintainer can inject code, as happened with the tj-actions/changed-files incident in 2025. Pin actions to a full 40-character commit SHA, review updates through an automated bot such as Dependabot or Renovate, and allow only approved actions through organization policy. Pin container base images by digest, and verify downloaded tools by checksum rather than by piping curl into a shell.

OIDC instead of static keys

Long-lived cloud keys stored as CI secrets are a major leak risk. With OpenID Connect federation, the CI provider issues a short-lived signed token that states the repository, branch and workflow. AWS, Azure and GCP can be configured to trust that token and issue temporary credentials only if claims match, for example only the main branch of example-org/shop may assume the deploy role. The credentials last minutes and there is no secret to rotate or leak.

Protect against script injection too. Never place untrusted values such as pull request titles or branch names directly into a shell command with expression syntax; pass them through environment variables and quote them. Log pipeline configuration changes, alert on new secrets or runners, and treat the workflow files as code requiring review by code owners.

WORKFLOW EXCERPT

```
permissions:
  contents: read
jobs:
  deploy:
    permissions:
      id-token: write
      contents: read
    steps:
      - uses: actions/checkout@b4ffde65f46336ab88eb53be808477a3936bae11 # v4.1.1
      - uses: aws-actions/configure-aws-credentials@<full-sha>
        with: { role-to-assume: arn:aws:iam::111122223333:role/deploy-shop }
```

WORKED EXAMPLE

A workflow for shop.example.com uses actions at floating tags, has write-all token permissions and stores an AWS access key in secrets. The engineer pins each action to a commit SHA, sets read-only defaults with write scope only where needed, creates an IAM role trusting the OIDC provider for the main branch only, deletes the stored key, and enables environment approval for production. The next run assumes the role through OIDC and the old key is revoked.

KEY TERMS

GITHUB_TOKEN: The automatic per-run token a workflow uses to call the repository API.

OIDC federation: Trust relationship where cloud accounts accept short-lived identity tokens from the CI provider.

SHA pinning: Referencing a dependency by an immutable commit hash instead of a movable tag.

Protected environment: A deployment target requiring approvals and branch restrictions.

Script injection: Untrusted text in an expression being interpreted as shell commands.

REMEMBER

- Use ephemeral, isolated runners.
- Restrict tokens and pin third-party actions by SHA.
- Use OIDC federation rather than stored cloud keys.

QUICK CHECK

1. Why pin actions to a commit SHA?
 - A) Tags can be moved to malicious code
 - B) SHA is shorter
 - C) It speeds builds
 - D) It disables logs
2. What is the main benefit of OIDC federation for deploys?
 - A) Faster compile
 - B) Larger artifacts
 - C) Free storage
 - D) No long-lived cloud key to steal
3. What should the default token permission be?
 - A) Admin
 - B) None and no checkout
 - C) Write-all
 - D) Read-only with per-job elevation
4. Which is a risk with pull_request_target?
 - A) It blocks merges
 - B) It can expose secrets to code from forks
 - C) It disables tests
 - D) It encrypts logs

Answers: 1-A 2-D 3-D 4-B

1. A commit hash is immutable, tags are not.
2. Credentials are temporary and tied to workflow claims.
3. Least privilege per job.
4. It runs with base repo permissions on untrusted input.

Secrets in repos

Once a secret is committed, it lives in git history and in every clone, and automated bots find new leaks within minutes.

How secrets leak

Developers commit API keys, passwords, private keys and .env files by accident, paste them in config examples or leave them in test fixtures. Even after a file is deleted, the secret remains in earlier commits, forks and clones. Attackers and bots continuously scan public repositories and respond within minutes to cloud keys, so a leaked key can be abused almost immediately. Private repositories are not safe either, since access is wide and repositories sometimes become public. Treat any committed secret as compromised.

Detection tools

Gitleaks and TruffleHog scan the working tree and the entire git history using regular expressions, entropy checks and, for TruffleHog, verification calls that test whether a credential is still live. GitHub secret scanning and push protection can block known token formats at push time. Run scans in three places: locally before commit, in CI on every pull request, and periodically across all repositories and history. Tune with an allow-list file for known false positives such as documented example keys, and keep that file reviewed.

Prevention with hooks

A pre-commit hook runs the scanner on staged changes before a commit is created, stopping the leak at its source. The pre-commit framework makes it easy to share a configuration, for example a gitleaks hook in .pre-commit-config.yaml. Hooks can be skipped with --no-verify, so back them with server-side push protection and CI checks. Keep secrets out of code by design: use a secrets manager or environment injection, add .env to .gitignore, and ship a .env.example containing only placeholders.

Response: revoke and rotate

When a real secret is found, rotate first. Revoke or disable the credential, issue a new one, deploy it, and then investigate use of the old one through provider logs such as CloudTrail. Removing the secret from history with git filter-repo or BFG is optional hygiene and does not make the secret safe again, because forks and caches may keep copies. Record the incident, find the root cause and fix the process that allowed it.

Classify each finding: a verified live credential is a true leak that needs rotation; a revoked, expired or placeholder value is a false positive; a test key for a sandbox may only need removal. Measure the number of open leaks, time to rotate, and the share of repos protected by hooks and push protection.

TERMINAL OUTPUT

```
$ gitleaks detect --source . --log-opts="--all"
Finding: AWS_SECRET_ACCESS_KEY=*****
RuleID: aws-access-token
File: config/deploy.env
Commit: 3f9a1c2 Author: dev1@example.com Date: 2026-03-02
leaks found: 1
```

WORKED EXAMPLE

A scan of the history of the example-org billing repo finds an AWS key in config/deploy.env from a commit made months ago, though the file was deleted. TruffleHog reports it as verified, meaning still active. The team deactivates the key in IAM, creates a role for the service, reviews CloudTrail for calls made with the old key from unknown addresses, and adds a gitleaks pre-commit hook plus push protection so it cannot recur.

KEY TERMS

Secret scanning: Automated detection of credentials in code, history or artifacts.

Pre-commit hook: A script run by git before a commit is created.

Push protection: Server-side blocking of pushes that contain recognized secrets.

Verified secret: A detected credential confirmed to be still valid.

Rotation: Replacing a credential with a new one and invalidating the old one.

REMEMBER

- Treat committed secrets as compromised.
- Scan history with gitleaks or TruffleHog and block at commit and push.
- Rotate first, then investigate use and clean up.

QUICK CHECK

1. What is the first action after a real key is committed?
 - A) Delete the file only
 - B) Revoke and rotate the credential
 - C) Rewrite history and stop
 - D) Rename the repo
2. Why scan git history and not only the current files?
 - A) History is faster
 - B) Files are encrypted
 - C) It is required by git
 - D) Old commits still contain the secret
3. Why back up pre-commit hooks with CI and push protection?
 - A) CI is slower
 - B) Hooks encrypt code
 - C) Hooks can be skipped or not installed
 - D) Hooks are illegal
4. What does TruffleHog verification add?
 - A) Opens a PR
 - B) Tests whether a found credential is active
 - C) Encrypts the repo
 - D) Rewrites history

Answers: 1-B 2-D 3-C 4-B

1. Deleting the file does not remove the key from clones.
2. Deleted secrets remain retrievable in earlier commits.
3. Local controls are optional, server checks are enforced.
4. Verified secrets are prioritized for rotation.

Artifact signing and provenance

Signatures let deployment systems prove an artifact came from your pipeline and was not altered, closing a major gap between build and run.

Why sign

A container image or binary is just bytes in a registry. If an attacker gains registry access or tampers in transit, a plain tag cannot reveal it. Signing binds an artifact's digest to an identity so consumers can verify who produced it and that it is unchanged. Together with provenance, a signed statement describing how and where it was built, signatures let admission controllers reject anything not built by your trusted pipeline. This is a core requirement of SLSA and of NIST SSDF guidance.

Sigstore and cosign

Sigstore is an open project providing free signing infrastructure. Cosign signs and verifies container images and other artifacts. In keyless mode, the signer authenticates through OIDC (for example a GitHub Actions workflow identity), Fulcio issues a short-lived certificate for that identity, and the signature plus certificate are recorded in the Rekor transparency log, so no long-lived private key must be protected. Key-based signing with a KMS-held key is also supported. The signature is stored in the registry beside the image and refers to its digest.

Verification and attestations

Signing is useless without verification. Verify before deploy: `cosign verify --certificate-identity <workflow identity> --certificate-oidc-issuer https://token.actions.githubusercontent.com image@sha256:<digest>`. In Kubernetes, enforce this with admission controllers such as Kyverno, Sigstore policy-controller or Gatekeeper with external data, rejecting unsigned or wrongly signed images. Always refer to images by digest, not tag. Attestations, in the in-toto format, attach claims such as SLSA provenance or an SBOM to the artifact and are verified the same way.

Operational practices

Sign in the pipeline immediately after the build, using the same job that produced the artifact, and never sign artifacts built on developer laptops. Restrict which identities are trusted: a policy that accepts any valid Sigstore signature is weak, whereas one that accepts only the release workflow of your repository is strong. Monitor the transparency log for signatures using your identity, and plan for key or identity rotation.

Also consider signing commits and tags with SSH or GPG keys or gitsign, to protect the source side. Document how to handle verification failures: block the deploy, alert the owner, and investigate whether the registry, pipeline or artifact was altered before overriding with a break-glass process.

TERMINAL OUTPUT

```
$ cosign verify registry.example.com/shop/api@sha256:9f2c...e41 \
  --certificate-identity https://github.com/example-org/shop/.github/workflows/release.yml@refs/heads/main \
  --certificate-oidc-issuer https://token.actions.githubusercontent.com
Verification for registry.example.com/shop/api@sha256:9f2c...e41 --
The cosign claims were validated
The transparency log claims were verified
```

WORKED EXAMPLE

A cluster policy requires images from registry.example.com to carry a cosign signature from the release workflow of example-org/shop on the main branch. A developer builds a test image on a laptop and pushes it to the registry using a stolen token. The admission controller rejects the deployment because the signature identity does not match. The team investigates how the token was exposed, revokes it and rotates registry credentials.

KEY TERMS

Cosign: A Sigstore tool for signing and verifying container images and artifacts.

Keyless signing: Signing with a short-lived certificate tied to an OIDC identity instead of a stored key.

Rekor: Sigstore's public transparency log of signatures.

Provenance: A signed record of how, where and from what an artifact was built.

Admission controller: A Kubernetes component that accepts or rejects workloads at deploy time.

REMEMBER

- Sign artifacts in the pipeline and verify before deploy.
- Use cosign keyless or KMS keys and trust specific identities.
- Deploy by digest and enforce with admission control.

QUICK CHECK

1. What does keyless signing remove?
 - A) The need to protect a long-lived private key
 - B) The need for registries
 - C) The need for tests
 - D) All verification
2. Where should signature verification happen?
 - A) Never
 - B) Only in dev
 - C) After incident
 - D) Before deployment, enforced by policy
3. Why refer to images by digest?
 - A) Tags are encrypted
 - B) Digests are faster to pull
 - C) Tags can be moved but digests are content-bound
 - D) Digests are shorter
4. What does a provenance attestation state?
 - A) DNS records
 - B) How and where the artifact was built
 - C) The user's password
 - D) The CVE list only

Answers: 1-A 2-D 3-C 4-B

1. A short-lived certificate is issued per signing.
2. Admission or deploy gates must refuse unverified artifacts.
3. A digest identifies exact content.
4. It records build inputs and builder identity.

SBOM and vulnerability data

When a new critical flaw is announced, an accurate inventory lets you answer 'are we affected?' in minutes instead of days.

What an SBOM is

A software bill of materials is a machine-readable list of the components in a piece of software, with names, versions, suppliers, licenses and relationships. Two main formats are used: SPDX (an ISO/IEC 5962 standard, strong on licensing and compliance) and CycloneDX (from OWASP, designed for security use cases). US Executive Order 14028 and related guidance pushed SBOMs into procurement requirements. An SBOM is generated per build and stored with the artifact, ideally attached as a signed attestation.

Generating with Syft

Syft by Anchore generates SBOMs from container images, directories and archives, for example `syft registry.example.com/shop/api:1.4.2 -o cyclonedx-json > sbom.json`. It reads package manager databases and lockfiles for many ecosystems. Generate during the build from the exact artifact being shipped, since an SBOM created from source may differ from the final image with its OS packages. Include transitive dependencies, and record the component hash and package URL (purl) so that matching to vulnerability data is accurate.

Scanning with Gripe

Gripe scans an image, directory or SBOM against vulnerability databases such as NVD, GitHub advisories and distribution feeds: `gripe sbom:./sbom.json`. Findings show package, installed and fixed versions and severity. Scanning the stored SBOM lets you re-check old releases against new CVEs without rebuilding. Use options like `--fail-on high` to gate builds, and `--only-fixed` to focus on actionable issues. Rescan deployed images on a schedule, since the vulnerability database changes daily.

VEX and prioritization

Many reported CVEs do not affect a product because the vulnerable code is not present or reachable. A Vulnerability Exploitability eXchange (VEX) statement, available in CycloneDX VEX, CSAF and OpenVEX, says that a product is `not_affected`, `affected`, `fixed` or `under_investigation` for a CVE, with a justification such as `vulnerable_code_not_in_execute_path`. Suppliers publish VEX to reduce noise for customers. Combine VEX with EPSS and CISA KEV data to prioritize.

To answer 'where is package X used?', store SBOMs in a searchable system such as Dependency-Track, which ingests CycloneDX files and shows affected projects per component. Keep SBOMs for every released version, and verify their completeness by sampling, since inaccurate SBOMs create false confidence.

TERMINAL OUTPUT

```
$ syft registry.example.com/shop/api:1.4.2 -o cyclonedx-json=sbom.json
$ gripe sbom:sbom.json --fail-on high
NAME INSTALLED FIXED-IN VULNERABILITY SEVERITY
openssl 3.0.2 3.0.13 CVE-2024-0727 Medium
libxml2 2.9.13 2.9.14 CVE-2022-29824 High
1 error occurred: discovered vulnerabilities at or above severity "high"
```

WORKED EXAMPLE

A new critical flaw is announced in the logging library log4j-core. The security team queries Dependency-Track for the component and sees three services use it, one directly and two through a transitive dependency of an internal framework. They prioritize the internet-facing service, rebuild with the fixed version, regenerate the SBOM and publish a VEX statement 'not_affected' for the batch job that bundles the library but runs without the vulnerable lookup feature.

KEY TERMS

SBOM: A machine-readable inventory of software components.

CycloneDX: An OWASP SBOM format focused on security use cases.

SPDX: An ISO standard SBOM format emphasizing licensing and provenance.

VEX: A statement of whether a product is affected by a particular vulnerability.

purl: Package URL, a standard way to identify a package across ecosystems.

REMEMBER

- Generate SBOMs per build in SPDX or CycloneDX.
- Scan SBOMs with Gripe and rescan as new CVEs appear.
- Use VEX, EPSS and KEV to focus on the vulnerabilities that matter.

QUICK CHECK

1. What does a VEX statement communicate?
 - A) Whether a product is actually affected by a CVE
 - B) The product price
 - C) The build time
 - D) A password
2. Which tool generates an SBOM from a container image?
 - A) Gitleaks
 - B) Conftest
 - C) ZAP
 - D) Syft
3. Why generate the SBOM from the final artifact?
 - A) It avoids hashing
 - B) It removes dependencies
 - C) It matches what is actually shipped, including OS packages
 - D) It is shorter
4. What lets you re-check old releases against new CVEs without rebuilding?
 - A) Disabling scanning
 - B) Stored SBOMs scanned with Gripe
 - C) Deleting images
 - D) Changing tags

Answers: 1-A 2-D 3-C 4-B

1. It states exploitability status such as `not_affected`.
2. Syft catalogs packages in images and directories.
3. Source-only lists miss base image content.
4. Scanning the SBOM uses current vulnerability data.

Supply chain security and SLSA

Attackers increasingly target what you depend on rather than you directly, so you must understand and reduce trust in upstream code.

The supply chain problem

Modern software uses hundreds of open source packages and many build tools, each maintained by people and systems outside your control. Attacks include compromised maintainer accounts, malicious updates (event-stream, 2018), hijacked build systems (SolarWinds, 2020), and malicious packages designed to be installed by mistake. Defending requires visibility (SBOMs), integrity (signatures and provenance), and process controls over which dependencies enter your builds. No single tool covers it, so use layers.

SLSA levels

Supply-chain Levels for Software Artifacts (SLSA, pronounced salsa) is a framework from the OpenSSF. In version 1.0 the Build track has levels: Build L1 requires that provenance exists, documenting how the artifact was built; Build L2 requires a hosted build platform that signs the provenance, making forgery harder; Build L3 requires a hardened build platform with isolated builds and protected signing keys, so a compromised project cannot tamper with provenance. Consumers check provenance against expectations before using an artifact.

Dependency confusion and typosquatting

Dependency confusion exploits package managers that prefer a public package over an internal one with the same name: an attacker publishes a higher version of an internal package name such as example-internal-utils to the public registry and builds fetch it. Typosquatting uses names resembling popular ones, like requeusts. Mitigations: reserve or namespace internal names, configure registries so internal scopes resolve only to the private feed, use a proxy that curates packages, and review new dependencies for age, maintainers and download history.

Mitigations that work

Commit lockfiles with integrity hashes and install with commands that respect them (npm ci, pip install --require-hashes). Pin versions, disable install scripts when not needed, and delay adopting brand-new releases through a cooling-off period. Use a private proxy registry such as Artifactory or Nexus that caches vetted packages. Scorecard from OpenSSF rates repository security practices, and 2FA enforcement on registry accounts reduces maintainer takeovers.

Plan for incident response: know which services contain a compromised package through SBOMs, be able to rebuild from clean sources, rotate any secrets the build had access to during the exposure window, and publish a customer notice when needed.

CONFIG EXCERPT (NPM)

```
# .npmrc
@example-internal:registry=https://npm.example.com/
registry=https://proxy.example.com/npm-public/
ignore-scripts=true
# CI
npm ci --ignore-scripts
```

WORKED EXAMPLE

A developer accidentally runs pip install requeusts and gets a package that sends environment variables to a remote host. The company's proxy registry only serves packages from an approved list, so the install is blocked and the request triggers a review. Separately, security discovers that the build configuration for an internal library could fall back to the public index. They scope internal names to the private feed and require hashes, preventing a dependency confusion attack.

KEY TERMS

SLSA: A framework of levels for supply chain integrity of build artifacts.

Dependency confusion: Tricking a build into fetching a public package instead of the intended internal one.

Typosquatting: Publishing malicious packages with names similar to popular ones.

Lockfile: A record of exact versions and hashes of all dependencies.

Provenance: Verifiable information about how an artifact was produced.

REMEMBER

- Understand upstream risks: compromised maintainers, build systems and malicious packages.
- Aim for SLSA provenance and increasing build integrity levels.
- Use lockfiles, scoped registries and curated proxies against confusion and typosquatting.

QUICK CHECK

1. What does SLSA Build L2 add over L1?
 - A) Hosted build platform that signs provenance
 - B) No provenance
 - C) Manual builds
 - D) Plain text logs
2. What is dependency confusion?
 - A) A slow build
 - B) A merge conflict
 - C) A license mismatch
 - D) A build fetches a public package with the same name as an internal one
3. Which control helps most against typosquatting?
 - A) Disabling HTTPS
 - B) Removing tests
 - C) Curated proxy registry and dependency review
 - D) Bigger servers
4. Why use npm ci with a lockfile?
 - A) It removes dependencies
 - B) It installs exact locked versions and fails on mismatch
 - C) It updates all packages
 - D) It skips checks

Answers: 1-A 2-D 3-C 4-B

1. Signed provenance from a hosted builder makes forgery harder.
2. Higher public version numbers can win resolution.
3. Allow-listing blocks look-alike packages.
4. The install is reproducible.

Policy as code gates

Written security rules are ignored under deadline pressure; rules expressed as code run automatically and consistently on every build and deployment.

Concept

Policy as code expresses security and compliance rules in a machine-evaluated language stored in version control, so they can be reviewed, tested and applied automatically. Typical rules: images must come from approved registries, containers must not run as root, storage must be encrypted, Terraform must not open port 22 to 0.0.0.0/0, and all resources must carry an owner tag. Policies run at multiple gates: in the developer's editor, in CI against code and plans, and at admission time in the platform as a last defence.

OPA and Rego

Open Policy Agent (OPA) is a general-purpose policy engine, and Rego is its declarative language. A policy receives JSON input and produces decisions. A common pattern defines deny rules that add a message when a condition holds: deny contains msg if { input.kind == "Pod"; some c in input.spec.containers; not startswith(c.image, "registry.example.com/"); msg := sprintf("image %s is from an unapproved registry", [c.image]) }. If the deny set is not empty, the request fails. Rego supports unit tests with opa test, which should be written for allowed and denied examples.

Conftest and gatekeepers

Conftest runs Rego policies against structured files such as Kubernetes manifests, Terraform plan JSON, Dockerfiles and CI configuration: conftest test deployment.yaml -p policy/. In Kubernetes, OPA Gatekeeper and Kyverno enforce rules as admission controllers, with audit mode that reports existing violations. Cloud tools such as Checkov, tfsec and AWS CloudFormation Guard fill similar roles for infrastructure code. Start in warn or audit mode, measure false positives, then move to enforcement.

Governance of policies

Treat policies like software: code review, versioned releases, unit tests and a clear owner. Provide helpful messages that tell developers what to change, and link to documentation. Allow exceptions through a recorded, time-limited process, for example an annotation approved by security, rather than disabling the policy. Report metrics on denials, exceptions and policy coverage.

Balance strictness with developer experience: gates that block constantly will be bypassed, while gates that never fail provide no protection. Introduce new rules gradually with notice periods, and test policies against real manifests from existing services before enforcing them.

REGO POLICY

```
package kubernetes.admission

deny contains msg if {
  input.request.kind.kind == "Pod"
  some c in input.request.object.spec.containers
  not startswith(c.image, "registry.example.com/")
  msg := sprintf("image %s is not from an approved registry", [c.image])
}
```

WORKED EXAMPLE

The platform team writes a Rego rule denying images outside registry.example.com and another denying containers that run as root. They add opa test cases for compliant and violating pods, run Conftest in CI against all Kubernetes manifests, and deploy Gatekeeper in audit mode first. The audit shows two legacy services violating the rule. After the owners fix them, the team switches Gatekeeper to enforce, so a manifest pulling docker.io/library/nginx is rejected with a clear message.

KEY TERMS

Policy as code: Security or compliance rules written in executable form and version-controlled.

Rego: The declarative policy language of Open Policy Agent.

Conftest: A tool that tests structured configuration files against Rego policies.

Admission control: A gate that accepts or rejects workloads when they are created in a cluster.

Audit mode: A mode that reports violations without blocking.

REMEMBER

- Express rules as code and apply them in editors, CI and admission.
- Use OPA/Rego, Conftest, Gatekeeper or Kyverno with unit tests.
- Roll out in audit mode, give clear messages and track exceptions.

QUICK CHECK

1. What language does OPA use for policies?
 - A) Rego
 - B) YAML only
 - C) SQL
 - D) Bash
2. Which tool tests Terraform plan JSON or manifests against Rego in CI?
 - A) Syft
 - B) Cosign
 - C) TruffleHog
 - D) Conftest
3. Why start policies in audit or warn mode?
 - A) To skip testing
 - B) To avoid logs
 - C) To measure impact and false positives before blocking
 - D) To hide violations
4. What is a good way to handle policy exceptions?
 - A) Delete logs
 - B) A recorded, time-limited approval
 - C) Disable the policy
 - D) Ignore them

Answers: 1-A 2-D 3-C 4-B

1. Rego is OPA's declarative policy language.
2. Conftest evaluates files with Rego policies.
3. Gradual enforcement avoids breaking delivery.
4. Exceptions should be visible and expire.

Harden a workflow file

SCENARIO

The release workflow for example-org/shop deploys to AWS. A review found three problems: third-party actions referenced by floating tags, default write-all token permissions and an AWS secret key written directly into the YAML. You must harden the file without breaking the deployment, using least privilege and OIDC.

OBJECTIVE

Fix the CI YAML by pinning actions, limiting permissions and removing the plaintext secret.

STEPS IN THE SIMULATOR

- 1 **Do:** Open `.github/workflows/release.yml`.

Simulator shows:

Lines flagged: `uses: actions/checkout@v4`; `permissions absent`; `env: AWS_SECRET_ACCESS_KEY: wJa1r...EXAMPLE`

- 2 **Do:** Replace `actions/checkout@v4` with the full commit SHA and a version comment.

Simulator shows:

Linters: `unpinned-action resolved (1 of 2)`.

- 3 **Do:** Pin the second action, `aws-actions/configure-aws-credentials`, to its SHA.

Simulator shows:

Linters: `unpinned-action resolved (2 of 2)`.

- 4 **Do:** Add top-level permissions: `contents: read`, and permissions `id-token: write` on the deploy job only.

Simulator shows:

Linters: `token permissions minimal`.

- 5 **Do:** Delete the `env` secret line and add `role-to-assume: arn:aws:iam::111122223333:role/deploy-shop`.

Simulator shows:

Linters: `plaintext secret removed`. Reminder: revoke the exposed key.

- 6 **Do:** Click Validate workflow.

Simulator shows:

0 errors, 0 warnings. Policy check: PASS.

ANSWER

The fixed workflow pins both actions to full commit SHAs, sets `contents: read` at the top with `id-token: write` only on the deploy job, and assumes an AWS role through OIDC instead of using a stored key. The exposed key must also be revoked and rotated.

Hint shown in the app: Three separate fixes: pins, permissions and the secret. Remember that deleting a secret from a file does not make it safe.

Scan repo for secrets

SCENARIO

The example-org/billing repository is going to be moved to a wider-access organization. Before the move, security must scan the full git history for secrets, separate real leaks from false positives and prepare a rotation plan. Use the scanner and provider information shown in the terminal.

OBJECTIVE

Find the true leaks in history and plan their rotation.

STEPS IN THE SIMULATOR

- 1 Do:** Run: `gitleaks detect --source . --log-opts="--all"`

Simulator shows:

3 leaks found: aws-access-token (config/deploy.env, commit 3f9a1c2); generic-api-key (docs/example.md); stripe-key (tests/fixtures.py)

- 2 Do:** Run: `trufflehog git file://. --only-verified`

Simulator shows:

1 verified: AWS key AKIAEXAMPLE... (still active)

- 3 Do:** Inspect docs/example.md.

Simulator shows:

Value is 'YOUR_API_KEY_HERE'. Classified: false positive (placeholder).

- 4 Do:** Inspect tests/fixtures.py.

Simulator shows:

Value is a Stripe test-mode key sk_test_.... Classified: low risk, remove from history, rotate for hygiene.

- 5 Do:** Check provider logs for the AWS key.

Simulator shows:

CloudTrail: 14 calls in the last 30 days, 2 from 203.0.113.77 (not a company address).

- 6 Do:** Record the plan.

Simulator shows:

Plan saved: 1) disable AWS key, 2) create role and deploy via OIDC, 3) investigate 203.0.113.77 activity, 4) add pre-commit hook and push protection.

ANSWER

One true, verified leak: the AWS key in config/deploy.env, with suspicious use from 203.0.113.77, so rotate immediately and investigate. The Stripe test key is low risk and the docs placeholder is a false positive. Add gitleaks hooks and push protection to prevent recurrence.

Hint shown in the app: Verified means the credential still works, which makes it the urgent one.

Verify a signature

SCENARIO

Your deployment job is about to roll out `registry.example.com/shop/api`. Policy states that only images signed by the release workflow of `example-org/shop` on main may be deployed. Run cosign verification on two candidate images, and refuse deployment for the one that fails.

OBJECTIVE

Verify image signatures and block deployment when verification fails.

STEPS IN THE SIMULATOR

- 1 **Do:** Run: `cosign verify registry.example.com/shop/api@sha256:9f2c1a...` with the expected identity and issuer flags.

Simulator shows:

Verification succeeded. The cosign claims were validated. Certificate identity:
.../release.yml@refs/heads/main

- 2 **Do:** Run the same command for `registry.example.com/shop/api@sha256:77ab03...`

Simulator shows:

Error: no matching signatures: none of the expected identities matched

- 3 **Do:** Run: `cosign tree` for the second image.

Simulator shows:

No signatures or attestations found for this digest.

- 4 **Do:** Check the registry audit log for the second digest.

Simulator shows:

Pushed by user `svc-test` at 02:14 from 198.51.100.60 (not the CI runner).

- 5 **Do:** Set the deploy job result for the second image to `BLOCKED` and file an incident.

Simulator shows:

Deployment halted. Ticket `SEC-2041` opened with the digest and push details.

- 6 **Do:** Deploy the verified image by digest.

Simulator shows:

Deploy proceeded: `shop-api@sha256:9f2c1a...` verified and rolled out.

ANSWER

The first image verifies with the correct workflow identity and is deployed by digest. The second has no valid signature and was pushed by an unexpected account from outside the pipeline, so deployment is refused and an incident is opened to review credential exposure.

Hint shown in the app: The identity and issuer flags define who is trusted; a missing signature means block.

Read an SBOM

SCENARIO

A critical CVE has been announced for the package libwebp 1.2.0. The company keeps CycloneDX SBOMs for each service in a shared folder. Browse the files, locate the vulnerable component, work out whether it is a direct or transitive dependency and which service ships it.

OBJECTIVE

Find the vulnerable transitive package in the SBOMs and trace the affected service.

STEPS IN THE SIMULATOR

- 1 **Do:** Open /sboms and list the files.

Simulator shows:

api.cdx.json, web.cdx.json, worker.cdx.json, reports.cdx.json

- 2 **Do:** Open web.cdx.json and search for libwebp.

Simulator shows:

Found: pkg:deb/debian/libwebp7@1.2.0 (bom-ref: libwebp7)

- 3 **Do:** Open the dependencies section for libwebp7.

Simulator shows:

dependsOn of: imagemagick -> pkg:deb/debian/imagemagick@6.9

- 4 **Do:** Check which direct component requires imagemagick.

Simulator shows:

thumbnailer-lib@2.3.1 (direct dependency of the web service)

- 5 **Do:** Search the other SBOMs.

Simulator shows:

api.cdx.json: not found; worker.cdx.json: not found; reports.cdx.json: libwebp7@1.2.0 found

- 6 **Do:** Write the finding.

Simulator shows:

Saved: web and reports services ship libwebp 1.2.0 transitively via thumbnailer-lib/imagemagick.

ANSWER

The vulnerable libwebp 1.2.0 is a transitive dependency, pulled in by imagemagick, which is required by thumbnailer-lib in the web service; the reports service also contains it. The api and worker services are unaffected. Prioritize patching the internet-facing web service.

Hint shown in the app: Follow the dependency graph backwards from the vulnerable component to the direct package.

Write a Rego rule

SCENARIO

The platform team wants to stop workloads from pulling images from outside the company registry `registry.example.com`. Write a Rego rule that denies Pods using unapproved registries, then test it against one compliant and one non-compliant sample input using the built-in tester.

OBJECTIVE

Write and test a Rego rule that denies unapproved image registries.

STEPS IN THE SIMULATOR

- 1 **Do:** Create `policy/registry.rego` with package `kubernetes.admission`.

Simulator shows:

File created. Syntax OK.

- 2 **Do:** Add: deny contains msg if { some c in input.request.object.spec.containers; not startswith(c.image, "registry.example.com/"); msg := sprintf("image %s is not from an approved registry", [c.image]) }

Simulator shows:

Compiled. 1 rule defined.

- 3 **Do:** Load sample input A: image `registry.example.com/shop/api:1.4.2`.

Simulator shows:

Result: deny = [] (allowed).

- 4 **Do:** Load sample input B: image `docker.io/library/nginx:latest`.

Simulator shows:

Result: deny = ["image docker.io/library/nginx:latest is not from an approved registry"]

- 5 **Do:** Test an input with two containers where only the sidecar is unapproved.

Simulator shows:

Result: one deny message for the sidecar. The rule iterates all containers.

- 6 **Do:** Click Run opa test.

Simulator shows:

3/3 tests passed.

ANSWER

The rule iterates over every container, checks that the image starts with `registry.example.com/`, and adds a clear deny message otherwise. Tests show compliant pods are allowed, the nginx image is denied, and a bad sidecar is also caught.

Hint shown in the app: Use some to iterate containers and startswith to compare the registry prefix.

DevSecOps and CI/CD Security: 10 questions, 80% to pass

1. What is a key risk of persistent shared runners?
 - A) State and secrets can leak between jobs
 - B) They run too fast
 - C) They cannot clone
 - D) They use IPv6
2. Why pin a GitHub Action to a commit SHA?
 - A) SHAs are readable
 - B) It disables caching
 - C) It speeds checkout
 - D) Tags can be moved to malicious code
3. What is the advantage of OIDC federation for deployment?
 - A) More logs
 - B) Larger runners
 - C) Short-lived credentials without stored cloud keys
 - D) Longer passwords
4. After a real secret is committed, what comes first?
 - A) Wait
 - B) Revoke and rotate it
 - C) Rewrite history only
 - D) Rename the file
5. What does cosign keyless signing rely on?
 - A) OIDC identity and short-lived certificates logged in Rekor
 - B) A password in git
 - C) A shared admin key
 - D) Plain checksums
6. What does an SBOM provide?
 - A) A firewall rule
 - B) A password list
 - C) A TLS cert
 - D) An inventory of components and versions
7. What does a VEX statement say?
 - A) Where the cluster is
 - B) Who owns a repo
 - C) Whether a product is affected by a CVE
 - D) How to bill customers
8. Which scenario is dependency confusion?
 - A) A bad merge
 - B) A public package with a higher version replaces an internal package
 - C) A merge conflict
 - D) A slow build
9. What does SLSA Build L2 require?
 - A) Hosted build platform that signs provenance
 - B) Manual builds
 - C) No records
 - D) Only a README
10. Which tool tests configuration files against Rego policies in CI?
 - A) Syft
 - B) Gitleaks
 - C) Cosign
 - D) Conftest

Answers: 1-A 2-D 3-C 4-B 5-A 6-D 7-C 8-B 9-A 10-D

1. Jobs can read leftovers from earlier jobs.
2. Commit hashes are immutable.
3. No long-lived key to leak.
4. Assume it is compromised and replace it.
5. Fulcio issues short-lived certificates tied to an identity.
6. It enables fast impact analysis.
7. It states affected, not_affected, fixed or under_investigation.
8. The resolver prefers the public package.
9. Signed provenance from a hosted builder.
10. Conftest evaluates structured files with Rego.

Cloud Security Engineering

Advanced | 9 lessons | 6 simulated labs | checkpoint | about 7h 45m

This module shows how to secure the three major clouds with one consistent design. You will start from the shared responsibility model and common attack paths, then read and tighten IAM policies, design network and data protections, and enable logging and detection. You will use CSPM to find and prioritize misconfigurations, plan a multi-account landing zone with guardrails, and map equivalent services across AWS, Azure and GCP so you can apply the same principles on any platform.

BY THE END YOU CAN

- + Design secure IAM and network controls in AWS, Azure and GCP
- + Configure encryption, logging and CSPM in the cloud
- + Plan a multi-account landing zone

IN THIS MODULE

Lesson 1 Shared responsibility and cloud threat model

Lesson 2 Cloud IAM fundamentals

Lesson 3 IAM least privilege in practice

Lesson 4 Cloud network controls

Lesson 5 Data protection and encryption

Lesson 6 Logging and detection

Lesson 7 CSPM and misconfiguration

Lesson 8 Landing zones and multi-account design

Lesson 9 Cloud pattern comparison

Lab 1 Fix an IAM policy (Config editor)

Lab 2 Trace a privilege escalation (Scenario)

Lab 3 Lock down a bucket (Config editor)

Lab 4 Hunt in CloudTrail (Log viewer)

Lab 5 Triage CSPM findings (Mock SIEM)

Lab 6 Design a landing zone (Diagram builder)

Checkpoint 10 questions, 80% to pass

Shared responsibility and cloud threat model

Most cloud breaches come from the customer's side of the line, so you must know exactly where that line sits for each service type.

The shared responsibility model

Cloud providers secure the cloud, and customers secure what they put in it. The provider always owns the physical data centers, hardware, hypervisor and core network. The customer always owns its data, its identities and access decisions, and its configuration choices. Everything in between shifts with the service model. The exact split is published by AWS, Microsoft and Google in their documentation, and auditors expect you to cite it. The practical lesson is that moving to the cloud transfers some operational work but never transfers accountability for your data and who can reach it.

IaaS, PaaS and SaaS

With IaaS, such as EC2, Azure Virtual Machines or Compute Engine, you manage the guest operating system, patching, host firewall, applications, and data. With PaaS, such as managed databases or app platforms, the provider patches the operating system and runtime, while you manage application code, configuration, identities and data. With SaaS, such as a mail or CRM service, the provider runs nearly everything, and you still manage users, MFA, sharing settings, and data classification. Serverless functions sit near PaaS. A frequent mistake is assuming 'managed' means 'secured': a managed database with a public endpoint and a weak password is still your failure.

Common cloud attack paths

Attackers rarely break the hypervisor. They use exposed credentials, such as access keys committed to a public repository or leaked in CI logs, phished console logins without MFA, over-permissive roles, and publicly readable storage. A common chain starts with a vulnerable web application, uses server-side request forgery to reach the instance metadata service at 169.254.169.254, steals temporary role credentials, and then uses the role's permissions to read storage or create new users. Other paths include exposed management ports, unpatched workloads, misconfigured Kubernetes dashboards, and compromised third-party integrations with broad cross-account access.

Building a cloud threat model

A cloud threat model lists assets such as data stores, identities, keys and pipelines, then maps entry points, trust boundaries and likely abuse paths. Useful boundaries include the internet to the account, one account to another, a workload to the control plane, and a CI system to production. For each path, note the preventive control, the detective control, and the response. Prioritize identity, because the control plane is reachable from anywhere with valid credentials. Use frameworks such as the MITRE ATT&CK Cloud matrix and the cloud provider's well-architected security pillar to make sure categories are not forgotten, and revisit the model after major architecture changes.

RESPONSIBILITY BY MODEL (SUMMARY)

Layer	IaaS	PaaS	SaaS
Data, identity	customer	customer	customer
Application	customer	customer	provider
OS, patching	customer	provider	provider
Hypervisor, HW	provider	provider	provider

WORKED EXAMPLE

A startup stores customer exports in a managed object storage bucket. A developer tests with a public-read setting and forgets to revert it. A security researcher finds the bucket by searching for its predictable name and notifies the company. The provider's data centers and storage service were fully secure, but the bucket configuration, which is the customer's responsibility, exposed 80,000 records. The fix is a block-public-access setting at the account level plus an alert when a policy makes a bucket public.

KEY TERMS

Shared responsibility model: Division of security duties between cloud provider and customer, varying by service type.

Control plane: APIs and consoles used to create and manage cloud resources.

Instance metadata service: Local endpoint at 169.254.169.254 that serves instance details and temporary credentials.

SSRF: Server-side request forgery: tricking a server into making requests on the attacker's behalf.

Trust boundary: Point where data or control passes between zones of different trust.

REMEMBER

- Providers secure the cloud; customers secure what they configure and store.
- Responsibility shifts by IaaS, PaaS and SaaS but data and identity stay with you.
- Leaked keys, weak identity and public storage are the main attack paths.

QUICK CHECK

1. In IaaS, who patches the guest operating system?
 - A) The provider
 - B) The customer
 - C) The ISP
 - D) Nobody
2. Which responsibility never moves to the provider, even in SaaS?
 - A) Physical security
 - B) Data and identity management
 - C) Hypervisor patching
 - D) Power and cooling
3. What does SSRF commonly target in cloud workloads?
 - A) The metadata service for temporary credentials
 - B) The monitor's power supply
 - C) Physical disks
 - D) DNS root servers
4. Why prioritize identity in a cloud threat model?
 - A) It is cheaper
 - B) The control plane is reachable from anywhere with valid credentials
 - C) Identity is provider owned
 - D) Networks no longer exist

Answers: 1-B 2-B 3-A 4-B

1. The customer manages everything above the hypervisor in IaaS.
2. Customers always own data classification, users and access settings.
3. Reaching 169.254.169.254 can yield role credentials.
4. Stolen credentials give direct control-plane access.

Cloud IAM fundamentals

Every cloud action is an API call authorized by IAM, so reading policies correctly is the most important cloud security skill.

Principals, policies and roles

A principal is who acts: a user, a group member, a service account, or a workload assuming a role. Policies are documents that allow or deny actions on resources. In AWS, identity-based policies attach to users or roles and resource-based policies attach to things like S3 buckets. A role has no permanent password; a principal assumes it through AWS STS and receives temporary credentials. Azure uses Microsoft Entra ID identities with role-based access control assignments at a scope, and GCP binds members to roles on resources through IAM policies. Prefer roles and workload identities over long-lived users.

Reading an IAM policy

An AWS policy statement has Effect, Action, Resource and optional Condition. Evaluation follows fixed logic: all requests start denied, an explicit Deny always wins, and an Allow works only if no boundary blocks it. A statement with Action 's3:*' and Resource '*' lets the principal do anything with every bucket in the account. Watch for NotAction, wildcards in the Principal element, and conditions that look protective but are not. In Azure, a role assignment pairs a principal, a role definition and a scope; in GCP, check whether a role is basic (Owner, Editor, Viewer), which is too broad, or predefined and narrower.

Trust relationships

A role has two policies in effect: a permission policy saying what it can do, and a trust policy saying who can assume it. A trust policy that names Principal AWS '*' with no condition lets anyone with an AWS account assume the role. Cross-account access should name the exact account or role and, for third parties, require an ExternalId to prevent the confused deputy problem. Federation uses OIDC or SAML so that workloads, such as GitHub Actions, assume roles without stored secrets. Always review trust policies as carefully as permissions, because they decide who can start the chain.

Organization-level boundaries

Above individual accounts, organization policies set limits that no local administrator can override. AWS Service Control Policies (SCPs) in AWS Organizations define the maximum permissions available within accounts and OUs; they never grant access by themselves. Azure uses management groups with Azure Policy and role assignment inheritance, and GCP uses Organization Policy constraints and folder-level IAM. Effective permission is the intersection of what an identity policy allows and what these guardrails permit. Typical SCPs deny leaving the organization, disabling logging, or using unapproved regions, which gives security teams a safety net even when workload teams make mistakes.

AWS POLICY STATEMENT (DOCUMENTATION ACCOUNT)

```
{
  "Effect": "Allow",
  "Action": ["s3:GetObject"],
  "Resource": "arn:aws:s3:::example-reports/*",
  "Condition": {
    "StringEquals": {
      "aws:PrincipalAccount": "111122223333"
    }
  }
}
```

WORKED EXAMPLE

A reviewer finds a role named 'DevHelper' whose trust policy allows Principal AWS '*'. Any AWS account in the world could ask to assume it. Its permission policy allows 's3:*' on all resources. The reviewer changes the trust policy to the specific build role in account 111122223333, narrows the permission to s3:GetObject on one bucket path, and adds an SCP that denies disabling logging organization-wide. The same role now has a small, auditable blast radius.

KEY TERMS

Principal: The identity, such as a user, role or service account, making a request.

Trust policy: Policy on a role stating who may assume it.

SCP: Organization policy that sets the maximum permissions in member accounts.

STS: Service issuing temporary credentials for assumed roles.

Explicit deny: A Deny statement that overrides any Allow.

REMEMBER

- Policies allow or deny actions; explicit deny wins and defaults are deny.
- Read trust policies as well as permission policies.
- Organization guardrails cap what any account administrator can do.

QUICK CHECK

1. In AWS evaluation, what overrides an Allow?

- A) An explicit Deny
- B) A tag
- C) A DNS entry
- D) A comment

2. What does a role's trust policy define?

- A) What the role may do
- B) Who may assume the role
- C) Which region it runs in
- D) Its billing owner

3. An SCP by itself can:

- A) Grant access to a user
- B) Limit the maximum permissions in accounts
- C) Create buckets
- D) Encrypt data

4. Why use ExternalId with third-party roles?

- A) To speed up login
- B) To prevent the confused deputy problem
- C) To encrypt traffic
- D) To skip MFA

Answers: 1-A 2-B 3-B 4-B

1. Explicit denies always win.
2. The trust policy names the principals allowed to assume the role.
3. SCPs only restrict; identity policies must still grant.
4. It ensures the vendor assumes the role only on behalf of the right customer.

IAM least privilege in practice

Permissions grow over time; deliberate techniques shrink them back to what is actually used.

Starting small and measuring use

Least privilege means granting only the actions needed, on only the resources needed, for only as long as needed. In practice teams start too broad because it is easier. The fix is measurement. AWS IAM Access Analyzer can generate a policy from CloudTrail activity and report unused access, and IAM last-accessed data shows which services a role has not touched in 90 days. Azure offers Entra permissions management and access reviews, and GCP offers the IAM Recommender, which suggests removing unused roles. Treat these suggestions as input to review, and test changes in a non-production account first.

Conditions and scoping

Conditions narrow a permission without rewriting it. Typical examples limit actions to a source IP range, require `aws:MultiFactorAuthPresent` to be true for sensitive calls, restrict to a region with `aws:RequestedRegion`, or require a resource tag to match the principal's tag, an approach known as attribute-based access control (ABAC). Replace `'Resource: *'` with specific ARNs wherever the service allows it. Where the action cannot be scoped, use the condition keys the service provides. Avoid broad actions like `iam:PassRole` on all roles, because it lets a principal hand powerful roles to services it controls.

Permission boundaries and escalation paths

A permission boundary is a managed policy that sets the maximum permissions an IAM user or role can ever have, even if broader policies are attached. It lets you delegate: developers may create roles for their applications, but only roles carrying the boundary, so they cannot create an administrator. Without such controls, privilege escalation paths appear, for example `iam:CreatePolicyVersion`, `iam:AttachUserPolicy`, `iam:PassRole` combined with `lambda:CreateFunction`, or `sts:AssumeRole` into a more powerful role. Review who can change IAM, since the ability to write policy is effectively administrator access.

Short-lived credentials and break-glass

Static access keys are the most commonly leaked credential. Replace them with temporary credentials: IAM roles for EC2 and containers, IAM Identity Center or Entra ID sign-in with SSO for humans, and OIDC federation for pipelines so no secret is stored. Set session durations to hours, not days. Where keys are unavoidable, rotate them at least every 90 days, alert on keys older than that, and scan code repositories for secrets. Keep one or two break-glass administrator accounts, protected by hardware MFA, excluded from conditional access outages, monitored with alerts on every use.

SCOPED POLICY WITH MFA AND REGION CONDITIONS

```
{
  "Effect": "Allow",
  "Action": ["ec2:StopInstances", "ec2:StartInstances"],
  "Resource": "arn:aws:ec2:eu-west-1:111122223333:instance/*",
  "Condition": {
    "Bool": {
      "aws:MultiFactorAuthPresent": "true"
    },
    "StringEquals": {
      "aws:ResourceTag/team": "web"
    }
  }
}
```

WORKED EXAMPLE

A developer role has `'ec2:*'` and `'iam:*'` on all resources. Access Analyzer shows that in 90 days the role only called `StartInstances`, `StopInstances` and `DescribeInstances` on instances tagged `team=web`. The engineer writes a policy limited to those actions, requires MFA, and attaches a permission boundary that blocks IAM changes. A tester then confirms that the developer can still restart web servers, while attempts to create users or touch database instances are denied and appear in the audit log.

KEY TERMS

Least privilege: Granting only the minimum permissions needed, scoped by action, resource and time.

Permission boundary: Policy that caps the maximum permissions an identity may have.

ABAC: Access control using attributes such as tags to decide permissions.

Privilege escalation: Gaining higher permissions than originally granted, often through IAM misconfigurations.

Break-glass account: Emergency administrator identity with strict protection and monitoring.

REMEMBER

- Measure real use and trim permissions with Access Analyzer or recommender tools.
- Use conditions, tags and boundaries to scope and delegate safely.
- Prefer short-lived credentials and protect break-glass accounts.

QUICK CHECK

1. Which tool builds a policy from observed CloudTrail activity?
 - A) IAM Access Analyzer
 - B) Route 53
 - C) CloudFront
 - D) Glacier
2. What does a permission boundary do?
 - A) Grants permissions
 - B) Caps the maximum permissions of a principal
 - C) Encrypts keys
 - D) Blocks the internet
3. Which approach best avoids leaked access keys?
 - A) Emailing keys securely
 - B) Roles, SSO and OIDC federation with temporary credentials
 - C) Longer keys
 - D) Storing keys in code
4. Why is iam:PassRole on all roles dangerous?
 - A) It slows APIs
 - B) It lets a principal give a powerful role to a service it controls
 - C) It disables logs
 - D) It needs MFA

Answers: 1-A 2-B 3-B 4-B

1. Access Analyzer can generate least-privilege policies from logs.
2. It sets the ceiling regardless of attached policies.
3. Temporary credentials remove long-lived secrets.
4. That can become a path to administrator permissions.

Cloud network controls

The network is a second line of defense: well-designed cloud networks keep services private and make data theft harder.

VPC and VNet segmentation

A virtual private cloud (AWS VPC, Azure VNet, GCP VPC) is a software-defined network divided into subnets. Public subnets have a route to an internet gateway; private subnets do not, and reach the internet through a NAT gateway if at all. A common layout puts load balancers in public subnets, application servers in private subnets, and databases in isolated subnets with no internet route, spread across at least two availability zones. Use non-overlapping CIDR ranges, such as 10.10.0.0/16, to allow future peering. Separate production and development into different VPCs or accounts, not only subnets.

Security groups, NACLs and firewalls

AWS security groups are stateful, attach to network interfaces, and only allow rules; their default for inbound is deny. Network ACLs are stateless, apply at the subnet, support allow and deny, and are numbered rules. Azure uses network security groups and Application Security Groups, while GCP uses VPC firewall rules with network tags or service accounts as targets. Reference security groups by ID instead of IP ranges, so that 'app tier may reach database tier on 5432' remains true when instances change. Never open SSH 22 or RDP 3389 to 0.0.0.0/0; use Session Manager, Azure Bastion or IAP instead.

Private endpoints and service access

Cloud services such as storage and databases are reachable over public endpoints by default. Private connectivity keeps traffic off the internet: AWS PrivateLink and gateway endpoints, Azure Private Link and private endpoints, and GCP Private Service Connect and Private Google Access. Combined with a policy on the service, for example a bucket policy that denies requests not coming from a given VPC endpoint (aws:SourceVpce), this prevents access from stolen credentials used outside the network. Private DNS zones must resolve the service name to the private address, which is a common misconfiguration during rollout.

Egress control and inspection

Attackers need an outbound path for command and control and data theft, so egress deserves as much design as ingress. Route outbound traffic from private subnets through a central inspection point: a firewall such as AWS Network Firewall or Azure Firewall, or a proxy with a domain allow list. Block unneeded ports, use DNS firewalls to filter malicious domains, and restrict which services may send traffic to the internet. Enable VPC Flow Logs, NSG flow logs and VPC Flow Logs in GCP to record accepted and rejected connections. A hub-and-spoke or transit gateway design gives one place to enforce and log shared controls.

SECURITY GROUP REFERENCE BY ID (GENERIC)

```
sg-web : inbound 443 from 0.0.0.0/0 (via load balancer only)
sg-app : inbound 8443 from sg-web
sg-db : inbound 5432 from sg-app
sg-db : outbound none except sg-app responses (stateful)
Admin : no 22/3389 open; use Session Manager / Bastion / IAP
```

WORKED EXAMPLE

An engineer discovers that a database security group allows TCP 5432 from 0.0.0.0/0 because of a test rule. Flow logs show repeated rejected and accepted attempts from addresses like 203.0.113.80. The engineer replaces the rule with a source of the application security group, moves the database to an isolated subnet, adds a private endpoint for backups to storage, and routes outbound traffic through a firewall with a domain allow list. The next scan of the account finds no reachable database port.

KEY TERMS

VPC: Isolated virtual network in a cloud account, divided into subnets.

Security group: Stateful, allow-only virtual firewall attached to network interfaces.

NACL: Stateless subnet-level access control list with allow and deny rules.

PrivateLink: Service providing private access to cloud services without internet exposure.

Egress control: Restricting and inspecting outbound traffic.

REMEMBER

- Segment with public, private and isolated subnets across availability zones.
- Reference security groups by ID and avoid open admin ports.
- Use private endpoints and inspect egress, with flow logs enabled.

QUICK CHECK

1. Which statement about security groups is correct?
 - A) They are stateless
 - B) They are stateful and only allow
 - C) They apply only to subnets and support deny
 - D) They need an internet gateway
2. Why avoid opening SSH to 0.0.0.0/0?
 - A) It slows the instance
 - B) It exposes the service to internet-wide scanning and brute force
 - C) It breaks DNS
 - D) It disables logs
3. What do private endpoints achieve?
 - A) Public bucket listing
 - B) Service access without crossing the public internet
 - C) Faster NAT
 - D) Free storage
4. Why design egress controls?
 - A) To stop outbound command and control and data theft
 - B) To speed downloads
 - C) To remove firewalls
 - D) To avoid DNS

Answers: 1-B 2-B 3-B 4-A

1. AWS security groups are stateful allow lists on interfaces.
2. Use managed access paths like Session Manager instead.
3. Traffic stays on the provider's private network.
4. Blocking outbound paths limits attacker options.

Data protection and encryption

Encryption and storage policies are the last barrier protecting data when other controls fail, and public exposure remains the most common cloud data breach.

Encryption at rest with KMS

Cloud storage services encrypt data at rest using keys managed in a key management service: AWS KMS, Azure Key Vault and Managed HSM, and Google Cloud KMS. Envelope encryption is the standard pattern: a data key encrypts the data, and a master key in the KMS encrypts the data key. Choose between provider-owned keys, customer-managed keys (CMKs) you control through key policy, and customer-supplied keys. Customer-managed keys let you restrict who can decrypt, audit every use, and disable a key to cut off access. Enable automatic annual rotation, separate key administrators from key users, and use per-environment keys.

Encryption in transit and secrets

Require TLS 1.2 or newer for all service endpoints. Bucket policies can deny requests where `aws:SecureTransport` is false, and storage accounts can enforce secure transfer. Use managed certificates and automatic renewal. Application secrets such as database passwords belong in a secrets manager (AWS Secrets Manager, Azure Key Vault, GCP Secret Manager) with rotation and access logging, not in environment variables baked into images or source code. For highly sensitive workloads, consider confidential computing and hardware security modules, but only after the basics of access control are in place.

Bucket and storage policies

Object storage breaches usually come from permissive access, not weak ciphers. AWS S3 Block Public Access can be turned on for an account and for each bucket, and overrides ACLs and policies that would grant public access. Disable legacy ACLs with the Bucket Owner Enforced setting, grant access through policies to named roles, and use versioning plus Object Lock or MFA Delete for protection against deletion and ransomware. Azure uses storage account settings to disallow anonymous blob access and shared key authorization, and GCP uses uniform bucket-level access and public access prevention.

Classification, backup and monitoring

Not all data needs the same protection. Classify data, for example public, internal, confidential and regulated, and apply stronger controls to the higher classes. Tools such as Amazon Macie, Microsoft Purview and Google Sensitive Data Protection discover personal and financial data in storage. Keep backups in a separate account with restricted delete rights, and test restores. Enable access logging on sensitive buckets, set lifecycle policies to remove obsolete data, and alert on policy changes that open access. Data that you do not keep cannot be stolen, so deletion schedules are a security control too.

BUCKET POLICY: REQUIRE TLS AND DENY DELETES OF LOGS

```
{
  "Effect": "Deny",
  "Principal": "*",
  "Action": "s3:*",
  "Resource": [
    "arn:aws:s3:::example-data",
    "arn:aws:s3:::example-data/*"
  ],
  "Condition": {
    "Bool": {
      "aws:SecureTransport": "false"
    }
  }
}
```

plus: Block Public Access = ON, default encryption = aws:kms,
versioning = enabled, Object Ownership = BucketOwnerEnforced

WORKED EXAMPLE

A company stores invoices in a bucket encrypted with a customer-managed KMS key. A former contractor's leaked credentials are used to list the bucket, but the key policy allows decryption only to the invoice-processing role, so reads fail with `AccessDenied` and each attempt appears in KMS logs. The team enables account-wide Block Public Access, enforces TLS in the bucket policy, and turns on versioning. Even a successful deletion could now be undone, and the exposure surface is far smaller.

KEY TERMS

Envelope encryption: Encrypting data with a data key that is itself encrypted by a master key in a KMS.

Customer-managed key: KMS key whose policy and lifecycle the customer controls.

Block Public Access: S3 setting that overrides policies and ACLs that would make data public.

Object Lock: Feature that makes stored objects immutable for a retention period.

Secrets manager: Service that stores, rotates and audits access to credentials.

REMEMBER

- Use KMS envelope encryption and customer-managed keys for sensitive data.
- Enable Block Public Access, TLS enforcement and versioning on storage.
- Classify data, back up to a separate account, and delete what is not needed.

QUICK CHECK

1. What is envelope encryption?

- A) A data key encrypts data and a master key encrypts the data key
- B) Sending data in an envelope
- C) Encrypting only emails
- D) Hashing passwords

2. What does S3 Block Public Access do?

- A) Overrides settings that would expose data publicly
- B) Encrypts objects
- C) Deletes old data
- D) Opens buckets

3. Why prefer customer-managed keys for sensitive data?

- A) They cost nothing
- B) You control key policy, auditing and can disable them
- C) They remove TLS
- D) They are unbreakable

4. Where should database passwords be stored?

- A) In source code
- B) In a secrets manager with rotation
- C) In a public wiki
- D) In bucket names

Answers: 1-A 2-A 3-B 4-B

1. The KMS key protects the data keys, which protect the data.
2. It prevents public ACLs and policies from taking effect.
3. Control and visibility over decryption is the benefit.
4. Secrets managers add rotation and audit trails.

Logging and detection

You cannot investigate what you did not record; control-plane logs are the foundation of cloud detection.

The control-plane logs

Each cloud records management API calls. AWS CloudTrail logs who called what, from which IP, with which user agent, and whether it succeeded; management events are free for one copy in the default event history of 90 days, but you need a trail to keep them longer. Azure's Activity Log records subscription-level operations, supplemented by Entra ID sign-in and audit logs. GCP Cloud Audit Logs include Admin Activity, which is always on, Data Access, which is off by default for most services, and System Event logs. Turn on organization-wide trails and enable data events for sensitive buckets and key operations.

Resource and network logs

Control-plane logs miss what happens inside resources. Add VPC Flow Logs, DNS query logs such as Route 53 Resolver logs, load balancer and WAF logs, object storage access logs, and database audit logs. For instances, ship operating system logs with an agent. Azure Monitor diagnostic settings route resource logs to a destination, and GCP uses log sinks. Include Kubernetes audit logs for clusters. Choose retention by need: many organizations keep 90 days hot for search and one year or more archived to satisfy compliance, using lifecycle rules to cut cost.

Centralize and protect the logs

Send logs from every account to a dedicated logging account that workload administrators cannot modify. In AWS, an organization trail writes to an S3 bucket in the log archive account with log file validation and Object Lock or a deny policy against deletion. An attacker's first move is often to stop logging, so alert on StopLogging, DeleteTrail, and changes to log sinks, and use SCPs to forbid them. Forward logs to a SIEM or a cloud-native tool such as Amazon Security Hub and GuardDuty, Microsoft Sentinel, or Google Security Command Center and Chronicle.

Detections that matter

Start with high-signal events: root account use, console login without MFA, creation of access keys, changes to IAM policies and trust relationships, disabling of logging or encryption, security group changes opening 0.0.0.0/0, public bucket policy changes, and API calls from unusual regions or from Tor exit nodes. Managed detection services such as GuardDuty add machine-learning findings like credential use from an unusual location. Write detections as queries, test them against past incidents, and document the response steps. Tune out noise so alerts remain trusted, and check that logging continues, with an alert if a source stops sending.

CLOUDTRAIL EVENT (SHORTENED)

```
{
  "eventTime": "2026-03-04T02:14:09Z",
  "eventName": "CreateAccessKey",
  "userIdentity": {
    "type": "IAMUser",
    "userName": "svc-build"
  },
  "sourceIPAddress": "203.0.113.77",
  "requestParameters": {
    "userName": "svc-build"
  },
  "additionalEventData": {
    "MFAUsed": "No"
  }
}
```

WORKED EXAMPLE

An alert fires at 02:14 UTC: a ConsoleLogin for user 'dana-test' succeeded with MFAUsed 'No' from 203.0.113.77, and minutes later CreateAccessKey ran for the same user. The analyst searches the central log archive, sees the same address attempt ListBuckets, and disables the user, revokes sessions, and deletes the new key. Because the organization trail writes to a locked log account, the attacker could not remove the evidence, and the full timeline is available for the incident report.

KEY TERMS

CloudTrail: AWS service recording management and data API activity.

Activity Log: Azure record of subscription-level control-plane operations.

Audit Logs: GCP logs of admin and data access API activity.

Log archive account: Dedicated account holding immutable centralized logs.

GuardDuty: AWS managed threat detection service analyzing logs for suspicious activity.

REMEMBER

- Enable organization-wide control-plane logs plus resource, flow and DNS logs.
- Centralize into an immutable log account and alert on tampering.
- Detect root use, no-MFA logins, key creation and policy or network changes.

QUICK CHECK

1. Which AWS service records API calls?
 - A) CloudTrail
 - B) CloudFront
 - C) Route 53
 - D) Athena
2. Why send logs to a separate account?
 - A) To protect them from tampering by workload admins or attackers
 - B) To save electricity
 - C) To avoid encryption
 - D) To speed up VMs
3. Which event should always raise an alert?
 - A) StopLogging or DeleteTrail
 - B) ListRegions
 - C) DescribeTags
 - D) GetObject on public data
4. In GCP, which audit log type is on by default?
 - A) Admin Activity
 - B) Data Access for all services
 - C) VPC billing
 - D) Debug logs

Answers: 1-A 2-A 3-A 4-A

1. CloudTrail logs management and data events.
2. Isolated storage keeps evidence trustworthy.
3. Disabling logging is a common attacker step.
4. Admin Activity logs are always enabled; Data Access is mostly opt-in.

CSPM and misconfiguration

Cloud environments change daily, so posture must be checked continuously and fixed in order of real risk.

What CSPM does

Cloud security posture management (CSPM) tools continuously compare your cloud configuration with benchmarks and policies. They use read-only API access to inventory resources across accounts and clouds, and report findings such as public buckets, security groups open to 0.0.0.0/0, unencrypted volumes and databases, missing MFA on privileged users, and disabled logging. Examples include AWS Security Hub and Config, Microsoft Defender for Cloud, Google Security Command Center, and third-party products. Checks usually map to frameworks like the CIS Foundations Benchmarks, which many auditors accept as evidence.

Common misconfigurations

A short list explains most incidents. Publicly readable storage, open database or management ports, unencrypted snapshots shared too widely, root or global administrator accounts without MFA, access keys that are old or unused, over-privileged service accounts, default network settings left unchanged, and disabled audit logs. Also common are public container registries with secrets inside images, exposed Kubernetes API servers, and instance metadata v1 still enabled, which makes SSRF attacks easier; IMDSv2 requires a session token. CSPM finds these quickly, but the better outcome is preventing them from being deployed at all.

Prioritizing findings

A large account can produce thousands of findings, and treating them equally leads to alert fatigue. Prioritize by combining severity with context: is the resource internet-reachable, does it hold sensitive data, does its role allow privilege escalation, and is there evidence of exploitation? An open security group on an empty test instance is lower risk than a public bucket holding customer records. Many tools show attack paths, chaining an exposed server with a powerful role and sensitive storage. Assign each finding an owner and a due date based on risk, and track mean time to remediate.

Fixing and preventing at scale

Remediate at the source. Fix the infrastructure-as-code template, such as Terraform or CloudFormation, so the problem does not return, and scan templates in the pipeline with tools like Checkov or cfn-guard before deployment. Use preventive guardrails, for example SCPs and Azure Policy deny effects, to block public storage outright. Where fast response is needed, automate safe remediation, such as removing a public ACL or tagging the owner, but test it, as automation can break production. Record accepted risks with an expiry date. Measure posture trends, since a falling count of critical findings shows the program works.

CSPM FINDING RECORD (EXAMPLE)

```
id: FIND-1042 severity: CRITICAL
resource: bucket example-customer-exports (acct 111122223333)
issue: public read via bucket policy; data class: PII
exposure: internet; owner: team-billing
fix: enable Block Public Access; patch Terraform module
due: 24h
```

WORKED EXAMPLE

A CSPM dashboard lists 2,300 findings. The analyst filters to internet-exposed resources and finds 14 items: a public bucket with personal data, three databases with open ports, and ten stale test instances. Ranking by exposure and data sensitivity puts the bucket first, then the databases. The team enables Block Public Access, restricts security groups, updates the Terraform module that created them, and adds a policy that denies public buckets in the organization. The next scan shows zero of the 14 findings and no new ones.

KEY TERMS

CSPM: Tooling that continuously checks cloud configuration against security benchmarks.

CIS Benchmark: Consensus configuration guidelines for hardening cloud platforms.

IMDSv2: Session-token-based instance metadata service that resists SSRF credential theft.

Infrastructure as code: Defining infrastructure in version-controlled templates.

Attack path: Chain of weaknesses that together lead an attacker to a valuable asset.

REMEMBER

- CSPM compares configuration to benchmarks such as CIS across accounts.
- Prioritize by exposure, data sensitivity and attack path, not count.
- Fix templates and add preventive guardrails so issues do not return.

QUICK CHECK

1. What is the main job of CSPM?
 - A) Continuously detect cloud misconfigurations against benchmarks
 - B) Replace firewalls
 - C) Encrypt email
 - D) Train staff
2. Which finding deserves top priority?
 - A) Open port on an empty test host
 - B) Public bucket holding customer personal data
 - C) Missing tag
 - D) Old naming convention
3. Why scan infrastructure-as-code templates in pipelines?
 - A) To prevent misconfigurations before deployment
 - B) To slow releases
 - C) To disable CSPM
 - D) To hide logs
4. What does IMDSv2 improve?
 - A) Disk speed
 - B) Resistance of metadata credentials to SSRF theft
 - C) DNS caching
 - D) Billing

Answers: 1-A 2-B 3-A 4-B

1. It inventories and checks posture continuously.
2. Internet exposure plus sensitive data creates the highest risk.
3. Shifting left stops issues recurring.
4. It requires a session token, blocking simple SSRF requests.

Landing zones and multi-account design

A landing zone gives every new workload a secure, governed starting point so security does not depend on each team's memory.

Why multiple accounts

An account (AWS), subscription (Azure) or project (GCP) is the strongest isolation boundary the provider offers for permissions, quotas, billing and blast radius. Putting everything in one account means one compromised role may reach every workload. Separate accounts for production, development, security tooling, logging, networking and shared services limit damage and simplify access decisions. Differing compliance needs, such as payment data, are another reason to isolate. Multi-account design adds overhead, such as cross-account roles and central networking, which is why automation and standard baselines are needed.

Organizations, OUs and guardrails

AWS Organizations groups accounts into organizational units (OUs), such as Security, Infrastructure, Workloads-Prod, Workloads-Dev and Sandbox. Azure uses management groups and GCP uses folders in the same way. Policy attaches at the OU and is inherited: SCPs deny actions like leaving the organization, disabling CloudTrail or GuardDuty, deleting the log archive, creating IAM users, or using regions you do not operate in. Guardrails are preventive if they block, or detective if they report, as with AWS Config rules. Keep the management account nearly empty, with very few people allowed in it.

Baseline accounts and services

A well-known pattern, used by AWS Control Tower, defines core accounts. The management account runs Organizations and billing. A log archive account stores immutable audit logs. A security tooling or audit account hosts GuardDuty, Security Hub and read-only access for responders, with delegated administration. A network account holds transit gateways, shared DNS and egress inspection. Workload accounts are created from a template through an account factory, arriving with logging, encryption defaults, baseline roles, networking and CSPM already attached. Azure landing zones and GCP's foundation blueprint follow the same logic.

Operating the landing zone

Build the landing zone with infrastructure as code, in version control, with review and automated tests, so changes are traceable. Use central identity: one identity provider with SSO, permission sets mapped to groups, and no local IAM users. Detect drift when someone changes a baseline control manually, and re-apply it. Plan for sandbox accounts with budgets and tight limits so experimentation does not touch production. Create an exception process, since teams will have valid needs outside the baseline. Document ownership of each account and test your recovery, including loss of the identity provider and a locked-out management account.

ORGANIZATION LAYOUT (ILLUSTRATIVE)

```
Root
|- Security OU: log-archive, security-tooling
|- Infrastructure OU: network, shared-services
|- Workloads OU: prod OU (strict SCPs), dev OU
|- Sandbox OU: budget-limited, no prod connectivity
SCPs: deny leave-org, deny StopLogging, region allow-list
```

WORKED EXAMPLE

A growing company runs everything in one AWS account, and a developer's leaked key lets an intruder delete log files and instances. The company builds a landing zone: separate production, development, log archive and security accounts under distinct OUs. An SCP now blocks stopping CloudTrail, an organization trail writes to the locked log archive, and new accounts come from a template with GuardDuty and encryption defaults. A similar leak later is confined to one development account, with logs intact.

KEY TERMS

Landing zone: Pre-configured, governed multi-account environment into which workloads are deployed.

OU: Organizational unit: a group of accounts that share policies.

Guardrail: Preventive or detective control applied across accounts.

Account factory: Automated process that creates new accounts with a standard baseline.

Drift: Divergence of live configuration from the intended baseline.

REMEMBER

- Use multiple accounts to isolate workloads, logging and security tooling.
- OUs and SCPs or policies apply inherited guardrails.
- Create accounts from automated baselines and detect drift.

QUICK CHECK

1. Why use separate accounts for prod and dev?
 - A) To limit blast radius and simplify permissions
 - B) It is required by TCP
 - C) To avoid logging
 - D) To remove IAM
2. Where should immutable audit logs live?
 - A) In the workload account
 - B) In a dedicated log archive account
 - C) On developer laptops
 - D) In a public bucket
3. What do SCPs attached to an OU do?
 - A) Grant access
 - B) Set maximum permissions for all accounts below
 - C) Encrypt data
 - D) Create VPCs
4. What is the purpose of an account factory?
 - A) Create accounts with a standard secure baseline
 - B) Mine data
 - C) Store passwords
 - D) Replace DNS

Answers: 1-A 2-B 3-B 4-A

1. Accounts are a strong isolation boundary.
2. Separation prevents workload admins from altering evidence.
3. They enforce guardrails with inheritance.
4. New accounts start with controls already in place.

Cloud pattern comparison

The three major clouds solve the same problems with different names, and mapping them lets you apply one security design everywhere.

Identity and access

The concepts match but the vocabulary differs. AWS uses IAM users, roles, policies, IAM Identity Center and Organizations with SCPs. Azure uses Microsoft Entra ID for identities, Azure RBAC for resources, managed identities for workloads, Conditional Access for sign-in policy, and management groups with Azure Policy. GCP uses Cloud Identity or Workspace, IAM roles bound to members at organization, folder, project or resource level, service accounts, and Organization Policy constraints. A key difference: Azure separates directory roles from resource roles, while GCP inherits policy downward through its resource hierarchy, so a grant at a folder reaches every project below.

Network, data and key services

Networking maps as VPC, security group and NACL in AWS; VNet, network security group and Azure Firewall in Azure; VPC, firewall rules and Cloud Armor in GCP. Private access is PrivateLink, Private Link and Private Service Connect respectively. Object storage is S3, Blob Storage and Cloud Storage. Key management is KMS, Key Vault and Cloud KMS, and secrets are Secrets Manager, Key Vault and Secret Manager. Note one detail with direct security consequences: AWS VPCs are regional, while GCP VPCs are global, so a single GCP network can span regions and firewall rules apply across them.

Monitoring and posture services

Audit logging is CloudTrail, Activity Log with Entra logs, and Cloud Audit Logs. Threat detection is GuardDuty, Microsoft Defender for Cloud with Sentinel as the SIEM, and Security Command Center with Chronicle. Posture and compliance are Security Hub with Config, Defender for Cloud secure score, and Security Command Center findings. Web and DDoS protection are AWS WAF and Shield, Azure Front Door WAF and DDoS Protection, and Cloud Armor. Using a multi-cloud CSPM or a common SIEM can give one view, but native tools often integrate best with provider-specific signals, so many teams use both.

Choosing patterns for workloads

For a public web application, place a CDN and WAF in front of a load balancer, run the application in private subnets with a role-based identity, keep the database in an isolated subnet with private endpoints, and encrypt with customer-managed keys. For a data lake, use private storage with Block Public Access equivalents, fine-grained access, classification scanning and complete data access logs. For containers, use managed Kubernetes with private nodes, workload identity, network policies and image scanning. For serverless, scope each function's role tightly. Across clouds, keep the principles identical and write controls once as policy as code.

SERVICE MAPPING (SUMMARY)

Concept	AWS	Azure	GCP
Identity	IAM/Identity Ctr	Entra ID	Cloud Identity/IAM
Guardrails	SCP	Azure Policy	Org Policy
Keys	KMS	Key Vault	Cloud KMS
Audit log	CloudTrail	Activity Log	Cloud Audit Logs
Posture	Security Hub	Defender for Cloud	Security Cmd Center

WORKED EXAMPLE

A company acquires a startup running on GCP while its own workloads are on AWS. The security engineer maps controls: SCPs correspond to Organization Policy constraints, CloudTrail to Cloud Audit Logs with Data Access logs enabled, and KMS to Cloud KMS. She notes the GCP default service account has the broad Editor role and replaces it with dedicated service accounts. Logs from both clouds flow to one SIEM with the same detections, such as key creation and public storage, written once and translated per provider.

KEY TERMS

Managed identity: Azure workload identity that removes the need to store credentials.

Organization Policy: GCP service applying constraints across an organization, folder or project.

Cloud Armor: GCP WAF and DDoS service.

Security Command Center: GCP security and risk platform for findings and posture.

Policy as code: Expressing controls in versioned, testable code applied across environments.

REMEMBER

- Identity, network, data, logging and posture each have an equivalent on every cloud.
- Watch for real differences such as global versus regional VPCs and hierarchy inheritance.
- Apply the same design principles and express them as policy as code.

QUICK CHECK

1. Which service is Azure's equivalent of AWS SCPs for guardrails?

- A) Azure Policy with management groups
- B) Azure Disk
- C) Key Vault
- D) Cloud Armor

2. Which statement about VPC scope is correct?

- A) AWS VPCs are global and GCP VPCs regional
- B) AWS VPCs are regional and GCP VPCs are global
- C) Both are physical
- D) Neither exists

3. What is the GCP equivalent of AWS KMS?

- A) Cloud KMS
- B) Chronicle
- C) Cloud Armor
- D) BigQuery

4. Why keep principles identical across clouds?

- A) It lets you write controls once and apply them consistently
- B) Clouds are identical
- C) It avoids logging
- D) It removes IAM

Answers: 1-A 2-B 3-A 4-A

1. Policy and management groups provide hierarchical guardrails.
2. This difference affects network and firewall design.
3. Cloud KMS provides managed keys on GCP.
4. Consistent principles reduce gaps; only implementation names differ.

Fix an IAM policy

SCENARIO

A fictional analytics team at example.com has a role used by a reporting job. Its policy allows every action on every resource, which the last audit flagged as critical. Logs show the job only reads objects from one bucket, example-reports, and writes results to one prefix. Your task is to rewrite the policy with scoped actions, specific resources and conditions, and then run the policy validator in the config editor.

OBJECTIVE

Replace a wildcard policy with scoped actions, resources and conditions.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the config editor and read the current policy.

Simulator shows:

Statement: Effect Allow, Action *, Resource *.

- 2 **Do:** Remove the wildcard action and add s3:GetObject and s3:ListBucket.

Simulator shows:

The editor flags ListBucket as needing a bucket-level resource.

- 3 **Do:** Set the Resource for GetObject to arn:aws:s3:::example-reports/input/*.

Simulator shows:

The validator accepts an object-level ARN.

- 4 **Do:** Add a second statement allowing s3:PutObject on arn:aws:s3:::example-reports/output/*.

Simulator shows:

Write access is limited to the output prefix.

- 5 **Do:** Add a condition requiring aws:SourceVpc to equal the job's VPC.

Simulator shows:

Condition block appears in the statement.

- 6 **Do:** Add an explicit Deny for iam:* on all resources.

Simulator shows:

The validator notes a guardrail against escalation.

- 7 **Do:** Run the validator and Access Analyzer comparison.

Simulator shows:

No wildcard warnings; unused permissions: none. Score 100 percent.

ANSWER

Allow s3:GetObject and ListBucket on the example-reports input path, allow s3:PutObject only on the output prefix, add a VPC source condition, and deny iam:*. No wildcard actions or resources remain, and the job still works because it only used those calls.

Hint shown in the app: Take the actions and resources from what the logs show the job really did.

Trace a privilege escalation

SCENARIO

In a fictional account for example.com, a low-privilege role named 'Intern-Dev' is used by a trainee. Investigators suspect it could become an administrator. The scenario simulator shows the role's policies and the other roles in the account. Your job is to follow the permissions step by step, find the chain that leads to admin, and then choose the changes that break it without stopping legitimate work.

OBJECTIVE

Follow an IAM path from low-privilege role to admin and break the chain.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the scenario and review Intern-Dev's attached policies.

Simulator shows:

Permissions include iam:PassRole, lambda:CreateFunction and lambda:InvokeFunction on '*'.

- 2 **Do:** List the roles in the account.

Simulator shows:

A role 'Admin-Automation' with AdministratorAccess is trusted by lambda.amazonaws.com.

- 3 **Do:** Trace the path: create a function, pass Admin-Automation to it, invoke it.

Simulator shows:

The map highlights the chain ending in admin permissions.

- 4 **Do:** Mark iam:PassRole on '*' as the key weakness.

Simulator shows:

The simulator confirms it is the pivot step.

- 5 **Do:** Restrict PassRole to the single role 'Intern-Lambda-Exec' with a condition on iam:PassedToService.

Simulator shows:

The chain turns grey.

- 6 **Do:** Add a permission boundary to Intern-Dev denying iam:* and attach it.

Simulator shows:

A second route via IAM policy edits is closed.

- 7 **Do:** Add an alert on CreateFunction using privileged roles and rerun the test.

Simulator shows:

No path to admin remains. Score 100 percent.

ANSWER

The chain is Intern-Dev, iam:PassRole on all roles, creating a Lambda with Admin-Automation, then invoking it for admin rights. Break it by limiting PassRole to one low-privilege role, adding a permission boundary, and alerting on functions created with privileged roles.

Hint shown in the app: Look for permissions that let you hand a powerful role to a service you control.

Lock down a bucket

SCENARIO

A fictional bucket, example-customer-exports, holds exported invoices for example.com. A routine scan found that it is readable by anyone, stores objects unencrypted, and has no access logging. You edit the bucket configuration in the config editor to enable protection settings and a policy, then run the checker which tests anonymous access, encryption and whether access logs are actually delivered to the central archive.

OBJECTIVE

Enable block public access, encryption and logging on a storage bucket policy.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the bucket configuration.

Simulator shows:

Block Public Access is off, encryption is none, logging is off.

- 2 **Do:** Turn on all four Block Public Access settings.

Simulator shows:

The status banner changes from Public to Not public.

- 3 **Do:** Set default encryption to SSE-KMS with the key alias invoice-key.

Simulator shows:

New objects will be encrypted with the customer-managed key.

- 4 **Do:** Set Object Ownership to Bucket owner enforced.

Simulator shows:

ACLs are disabled.

- 5 **Do:** Enable versioning and server access logging to the log archive bucket.

Simulator shows:

Logging target accepted.

- 6 **Do:** Add a bucket policy denying requests when aws:SecureTransport is false.

Simulator shows:

The policy validator shows one Deny statement.

- 7 **Do:** Run the checker, including an anonymous read test.

Simulator shows:

Anonymous read denied, encryption enforced, logs delivered. Score 100 percent.

ANSWER

Enable all Block Public Access settings, default SSE-KMS with a customer-managed key, Bucket Owner Enforced ownership, versioning and access logging to the log archive, and a bucket policy denying non-TLS requests. The anonymous read test then fails as intended.

Hint shown in the app: Check public access first, then encryption, then logging and TLS.

Hunt in CloudTrail

SCENARIO

The security team received a report about odd activity in a fictional account. The log viewer contains a day of CloudTrail events with timestamps, event names, user names, source IPs and MFA fields. You must find a console login that did not use MFA, the access key that was created shortly afterward, and the source address, then decide what containment steps should follow.

OBJECTIVE

Find the console login without MFA and the key creation that followed.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the log viewer and filter eventName = ConsoleLogin.

Simulator shows:
Twelve logins are listed.

- 2 **Do:** Add a filter for MFAUsed = No.

Simulator shows:
One result: user dana-test at 02:14 UTC from 203.0.113.77.

- 3 **Do:** Note the source IP and user, then filter all events from 203.0.113.77.

Simulator shows:
Events include ListBuckets and CreateAccessKey.

- 4 **Do:** Open the CreateAccessKey event.

Simulator shows:
A new key was created for dana-test at 02:19 UTC.

- 5 **Do:** Check for later use of the new key.

Simulator shows:
GetObject calls from the same IP appear at 02:31 UTC.

- 6 **Do:** Select containment actions: disable user, delete key, revoke sessions, rotate related secrets.

Simulator shows:
The checker marks the actions correct.

- 7 **Do:** Add a recommendation to require MFA via policy condition.

Simulator shows:
Report generated. Score 100 percent.

ANSWER

User dana-test logged in at 02:14 UTC from 203.0.113.77 without MFA, then created an access key at 02:19 and used it to read objects. Contain by disabling the user, deleting the key, revoking sessions and enforcing MFA through an IAM condition.

Hint shown in the app: Filter by ConsoleLogin and the MFA field, then pivot on the source IP.

Triage CSPM findings

SCENARIO

A fictional company's posture tool has produced 40 findings across two accounts. The SIEM view lists each with severity, resource, internet exposure and data classification. You have time to fix three issues today. You must sort the findings by exposure and sensitivity, choose the top three, and state the fix for each, avoiding the temptation to simply sort by the tool's default severity.

OBJECTIVE

Sort misconfiguration alerts by exposure and data sensitivity and fix the top three.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the SIEM findings view.

Simulator shows:

40 findings appear sorted by default severity.

- 2 **Do:** Add columns for exposure and data class.

Simulator shows:

Each row shows Internet or Internal and Public, Internal or PII.

- 3 **Do:** Filter to Exposure = Internet.

Simulator shows:

Eleven findings remain.

- 4 **Do:** Sort by data class and select the PII items.

Simulator shows:

A public bucket and a database with port 3306 open to 0.0.0.0/0 rise to the top.

- 5 **Do:** Select the third item: an access key for an admin user, 400 days old.

Simulator shows:

It is marked as an escalation risk.

- 6 **Do:** Assign fixes: Block Public Access, restrict the security group to the app tier, rotate and remove the key.

Simulator shows:

Each fix is attached with owner and due date.

- 7 **Do:** Submit the triage.

Simulator shows:

Checker agrees with the top three. Score 100 percent.

ANSWER

Top three: the public bucket with personal data, the database open to the internet, and the stale admin access key. Fixes are Block Public Access, a security group limited to the application tier, and key rotation or removal. Rank by exposure plus data sensitivity, not by count.

Hint shown in the app: Ask whether the internet can reach it and what the damage would be.

Design a landing zone

SCENARIO

A fictional company, Example Retail, is moving to AWS and needs a multi-account design. It has production and development workloads, a security team that needs central tooling, and an auditor who requires tamper-proof logs. In the diagram builder you lay out the organization, OUs, accounts and guardrail policies, and then the checker evaluates isolation, tamper-proof logging and control coverage.

OBJECTIVE

Lay out OUs, accounts and guardrail policies for prod, dev and security tooling.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the diagram builder with an empty organization root.

Simulator shows:

A management account icon is shown.

- 2 **Do:** Create OUs: Security, Infrastructure, Workloads-Prod, Workloads-Dev and Sandbox.

Simulator shows:

Five OUs are added under the root.

- 3 **Do:** Add accounts: log-archive and security-tooling under Security; network under Infrastructure.

Simulator shows:

Account icons appear in their OUs.

- 4 **Do:** Add prod and dev workload accounts under the matching OUs.

Simulator shows:

The checker notes separate blast radii.

- 5 **Do:** Attach SCPs: deny StopLogging, deny leaving the organization, region allow-list at the root.

Simulator shows:

Policy badges appear on the root.

- 6 **Do:** Attach a stricter SCP to Workloads-Prod denying IAM user creation.

Simulator shows:

The badge shows inheritance to prod accounts.

- 7 **Do:** Connect the organization trail to log-archive and run the checker.

Simulator shows:

Immutable logging, isolation and guardrails all pass. Score 100 percent.

ANSWER

Root with Security, Infrastructure, Workloads-Prod, Workloads-Dev and Sandbox OUs. Accounts: log-archive, security-tooling, network, prod and dev. Root SCPs block disabling logging, leaving the organization and unapproved regions; prod adds stricter limits. The organization trail writes to log-archive.

Hint shown in the app: Separate logs, security tooling and workloads, then attach guardrails at the highest sensible level.

Cloud Security Engineering: 10 questions, 80% to pass

1. In an IaaS deployment, who is responsible for patching the guest operating system?
 - A) The customer
 - B) The cloud provider
 - C) The ISP
 - D) The auditor
2. What happens in AWS policy evaluation when an explicit Deny and an Allow both apply?
 - A) Allow wins
 - B) Deny wins
 - C) They cancel
 - D) The older wins
3. Which trust policy is most dangerous?
 - A) Principal limited to one named role
 - B) Principal AWS '*' with no condition
 - C) Principal limited to the same account
 - D) Principal with ExternalId
4. Which technique best reduces unused permissions?
 - A) Granting Administrator
 - B) Generating policies from CloudTrail activity with Access Analyzer
 - C) Sharing credentials
 - D) Disabling logs
5. What does a permission boundary do?
 - A) Sets the maximum permissions an identity can have
 - B) Opens ports
 - C) Rotates keys
 - D) Encrypts buckets
6. Which is the safest way to give administrators shell access to cloud servers?
 - A) Open port 22 to the internet
 - B) Use Session Manager, Bastion or IAP
 - C) Share a private key
 - D) Use password only
7. What does S3 Block Public Access provide?
 - A) Overrides policies and ACLs that would make data public
 - B) Faster downloads
 - C) Cross-region copy
 - D) Automatic backups
8. Why centralize logs in a separate account?
 - A) To prevent tampering by workload administrators or attackers
 - B) To lower IP costs
 - C) To skip retention
 - D) To avoid MFA
9. How should CSPM findings be prioritized?
 - A) By exposure, data sensitivity and attack path
 - B) Alphabetically
 - C) By oldest
 - D) Randomly
10. What is the primary benefit of a multi-account landing zone?
 - A) Smaller blast radius and consistent guardrails
 - B) Fewer logs
 - C) No need for IAM
 - D) Free compute

Answers: 1-A 2-B 3-B 4-B 5-A 6-B 7-A 8-A 9-A 10-A

1. IaaS customers manage everything above the hypervisor.
2. An explicit Deny always overrides an Allow.
3. It lets any AWS account attempt to assume the role.
4. Observed usage informs minimal policies.
5. It caps permissions even if broader policies are attached.
6. Managed access paths avoid public admin ports.
7. It is a guardrail against accidental public exposure.
8. Attackers often try to delete logs in the compromised account.
9. Context decides real risk.
10. Isolation and inherited guardrails limit the damage and ensure baseline controls.

Container and Kubernetes Security

Advanced | 7 lessons | 5 simulated labs | checkpoint | about 6h 25m

Containers and Kubernetes now run much of the world's software, and they bring their own security model. This module explains how containers are isolated and where that isolation is thin, how to build and scan trustworthy images, how the Kubernetes control plane can be attacked, and how RBAC, pod security, admission control and network policies shrink the blast radius. It closes with runtime detection using Falco, seccomp and AppArmor, and with safe handling of secrets.

BY THE END YOU CAN

- + Scan and harden container images
- + Configure Kubernetes RBAC, network and pod security
- + Detect runtime threats and protect secrets in clusters

IN THIS MODULE

Lesson 1 Container security fundamentals

Lesson 2 Image security and scanning

Lesson 3 Kubernetes architecture and attack surface

Lesson 4 RBAC in Kubernetes

Lesson 5 Pod security and admission control

Lesson 6 Network policies

Lesson 7 Runtime security and secrets

Lab 1 Scan an image (Terminal)

Lab 2 Harden a Dockerfile (Code editor)

Lab 3 Fix over-broad RBAC (Config editor)

Lab 4 Write a NetworkPolicy (Config editor)

Lab 5 Investigate a Falco alert (Log viewer)

Checkpoint 10 questions, 80% to pass

Container security fundamentals

A container is not a small virtual machine, and treating it like one leads to dangerous assumptions. Knowing what actually isolates a container tells you where escapes happen.

What a container really is

A container is an ordinary Linux process (or group of processes) started with restrictions. Those restrictions come from kernel features, not from a separate operating system. The container image supplies a root filesystem and the application, but at runtime every container on a host runs on the same host kernel. That is the core security difference from a virtual machine, where a hypervisor separates guest kernels.

Because the kernel is shared, a kernel vulnerability or a mis-granted privilege can let a process in one container affect the host or its neighbors. Container security therefore means narrowing what that process can see, use and call, and keeping the host kernel patched.

Namespaces, cgroups and capabilities

Namespaces limit what a process can see. The main types are pid (process IDs), net (network interfaces and routing), mnt (mount points), uts (hostname), ipc, user (UID mapping) and cgroup. A container gets its own set of these, so it sees only its own processes and interfaces. Control groups (cgroups) limit what a process can consume: CPU, memory, process count and I/O. Without cgroup limits, one container can starve the host.

Linux capabilities split the all-powerful root user into about forty separate privileges, such as CAP_NET_BIND_SERVICE or CAP_SYS_ADMIN. Container runtimes drop most capabilities by default. A container run with `--privileged` regains nearly all of them plus device access, which removes most isolation and is a common cause of escapes.

Common weaknesses and baseline controls

Frequent mistakes include running as root inside the container, mounting the Docker socket (`/var/run/docker.sock`) into a container, using `--privileged`, sharing the host PID or network namespace, mounting sensitive host paths, and omitting resource limits. Mounting the Docker socket gives that container full control of the host's runtime.

Baseline controls are simple: run as a non-root user, drop all capabilities and add back only what is needed, use a read-only root filesystem, set memory and CPU limits, enable the default seccomp profile, keep the host OS and kernel updated, and consider user namespaces so root in the container maps to an unprivileged host UID.

Putting it into practice

Start by inventorying how your workloads are launched. Run `docker inspect` or `kubectl get pods -o yaml` across a sample of services and look for `privileged: true`, `hostPath` volumes, `hostNetwork` and missing resource limits. Most organizations find that a small number of legacy workloads account for nearly all of the risky settings. Fix those first, then encode the safe defaults in templates, Helm charts or admission policy so new services inherit them automatically. This turns a one-time cleanup into a lasting standard and gives auditors a clear, testable baseline to check against.

TERMINAL OUTPUT

```
$ docker run --rm -d --name web --memory 256m --cap-drop ALL \
  --read-only --user 10001:10001 example/web:1.4
$ docker inspect web --format '{{.HostConfig.Privileged}} {{.HostConfig.CapDrop}}'
false [ALL]
$ ps -o pid,user,cmd -C nginx # host view of the same process
PID USER CMD
4121 10001 nginx: master process
```

WORKED EXAMPLE

A team finds a container that mounts `/var/run/docker.sock` so a monitoring tool can list containers. A reviewer points out that any code execution in that container can start a new privileged container and mount the host root filesystem. The fix is to replace the socket mount with a read-only metrics endpoint or a socket proxy that allows only list calls, and to run the tool as non-root with all capabilities dropped.

KEY TERMS

Namespace: A kernel feature that gives a process its own isolated view of a resource such as processes or network.

cgroup: A kernel feature that limits and accounts for CPU, memory and I/O used by a group of processes.

Capability: One of the fine-grained privileges that make up root power on Linux.

Privileged container: A container started with nearly all capabilities and host device access, removing most isolation.

Container escape: An attack in which code in a container gains access to the host or other containers.

REMEMBER

- Containers are processes isolated by namespaces, cgroups and capabilities on a shared kernel.
- Privileged mode, the Docker socket and host namespaces remove isolation.
- Run non-root, drop capabilities, limit resources and patch the host kernel.

QUICK CHECK

1. Which statement about containers and the kernel is correct?
 - A) Each container runs its own kernel
 - B) All containers on a host share the host kernel
 - C) Containers use a hypervisor for isolation
 - D) Containers cannot see any kernel features
2. Which kernel feature limits how much memory a container can use?
 - A) Namespaces
 - B) Capabilities
 - C) cgroups
 - D) seccomp
3. Why is mounting `/var/run/docker.sock` into a container dangerous?
 - A) It slows the network
 - B) It grants control over the host container runtime
 - C) It disables cgroups
 - D) It removes the image layers
4. Which setting best reduces a container's privileges by default?
 - A) `--privileged`
 - B) `--pid=host`
 - C) `--cap-drop ALL` then add back needed ones
 - D) Run as root for compatibility

Answers: 1-B 2-C 3-B 4-C

1. Containers are isolated processes on the shared host kernel, unlike VMs.
2. cgroups account for and cap resources such as memory and CPU.
3. Access to the socket lets the container start privileged containers and take over the host.
4. Dropping all capabilities and adding only what is required follows least privilege.

Image security and scanning

Every container starts from an image, so a vulnerable or tampered image becomes a vulnerable production workload. Securing the supply of images is the cheapest place to prevent problems.

Building minimal, non-root images

A smaller image has fewer packages and therefore fewer vulnerabilities and fewer tools an attacker can use. Choose a minimal base such as a slim, Alpine or distroless image, and use multi-stage builds so compilers and build tools never reach the final image. Add a USER instruction with a numeric non-root UID, avoid installing shells and package managers in runtime images, and never copy secrets into a layer.

Remember that image layers are permanent. A secret added in one layer and deleted in the next is still recoverable from the earlier layer. Pass build-time secrets with BuildKit secret mounts, and read runtime secrets from the environment or a secret store. Add a HEALTHCHECK so orchestrators can detect a broken process.

Scanning with Trivy and reading results

Scanners such as Trivy, Gype or Clair compare installed OS packages and application dependencies against vulnerability databases such as the NVD and distribution advisories. Trivy can also scan Dockerfiles and Kubernetes manifests for misconfiguration, detect embedded secrets, and produce an SBOM. Output lists the package, the installed version, the CVE ID, severity and the fixed version when one exists.

Triage by severity, by whether a fix exists, and by whether the vulnerable code is reachable. A critical finding with a fixed version is usually solved by rebuilding on a patched base image. Run scans in CI to fail builds above a threshold, and rescan stored images regularly, because new CVEs appear after an image was built.

Trusted registries, digests and signing

Tags such as :latest or :1.4 are mutable; the owner can point them at different content later. A digest (sha256:...) identifies exact content and cannot be changed. Deploy by digest, or at least pin tags and verify them. Pull only from approved private registries and mirror public base images into them after scanning.

Image signing with Sigstore cosign lets you attach a signature and attestation to an image, and an admission controller can refuse unsigned images. Generating an SBOM and provenance record (for example in SLSA style) shows what went into an image and which pipeline built it, supporting both audits and rapid response when a new CVE is announced.

Putting it into practice

Make scanning part of the normal delivery path rather than a separate audit. A common pattern is to scan on every pull request, fail the build for critical findings that have a fix, and publish the scan report with the image. Then run a nightly job that rescans everything in the registry and opens tickets for new findings. Keep an exceptions list with owners and expiry dates for findings that cannot be fixed immediately, and review it regularly so accepted risk does not quietly become permanent.

TERMINAL OUTPUT

```
$ trivy image --severity HIGH,CRITICAL example/web:1.4
example/web:1.4 (debian 11.9)
Total: 2 (HIGH: 1, CRITICAL: 1)
  Library Vulnerability Severity Installed Fixed
  libssl1 CVE-2024-0001 CRITICAL 1.1.1n-0 1.1.1w-0
  zlib1g CVE-2024-0002 HIGH 1.2.11 1.2.13
$ trivy image --exit-code 1 --severity CRITICAL example/web:1.4
```

WORKED EXAMPLE

A Trivy scan of example/web:1.4 reports one critical OpenSSL finding with a fixed version. The team updates the FROM line to the current patched base, rebuilds with a multi-stage Dockerfile, sets USER 10001, and re-runs the scan, which now reports zero critical issues. The pipeline pushes the image, signs it with cosign and records its digest. The deployment manifest references the digest, so a later tag change cannot swap the content.

KEY TERMS

Distroless image: A minimal image containing only the application and runtime libraries, without a shell or package manager.

Multi-stage build: A Dockerfile technique that compiles in one stage and copies only the result into a small final image.

Digest: A sha256 hash that identifies exact image content and cannot be reassigned.

SBOM: A software bill of materials listing the components inside an image.

cosign: A Sigstore tool for signing and verifying container images.

REMEMBER

- Use minimal, non-root images and multi-stage builds; never bake in secrets.
- Scan in CI and on a schedule; triage by severity, fix availability and reachability.
- Pull from trusted registries, pin digests and verify signatures.

QUICK CHECK

1. Why deploy by digest instead of by tag?

- A) Digests are shorter
- B) Tags can be moved to different content, digests cannot
- C) Digests are faster to pull
- D) Tags are not supported by registries

2. A secret copied in one layer and deleted in the next is:

- A) Gone permanently
- B) Still recoverable from the earlier layer
- C) Encrypted by the registry
- D) Hidden by cgroups

3. What does a multi-stage build mainly achieve?

- A) Faster CPU
- B) Keeping build tools out of the final image
- C) Encrypting layers
- D) Disabling the registry

4. A scan shows a critical CVE with a fixed version in the base image. Best first action?

- A) Ignore it
- B) Rebuild on a patched base and rescan
- C) Delete the scanner
- D) Switch the tag to :latest

Answers: 1-B 2-B 3-B 4-B

1. A digest pins exact content, preventing silent replacement.
2. Layers are immutable, so earlier layers keep the data.
3. Only the built artifacts are copied to the small final stage.
4. Updating the base package set removes the vulnerability.

Kubernetes architecture and attack surface

You cannot defend a cluster without knowing its components and what each one trusts. Most Kubernetes compromises abuse an exposed component or an over-privileged identity.

Control plane and node components

The control plane consists of the API server (kube-apiserver), etcd, the scheduler and the controller manager. The API server is the front door: every kubectl command, controller and kubelet talks to it over HTTPS, usually port 6443. etcd is the key-value database holding all cluster state, including Secrets. Each worker node runs a kubelet (port 10250), a container runtime and kube-proxy.

Whoever can write to etcd or reach an unauthenticated kubelet API effectively controls the cluster. Pods run on nodes, and nodes trust the control plane. The attack surface therefore includes the API server's network exposure, the authentication it accepts, the kubelet API, etcd, the container runtime and the workloads themselves.

Authentication, authorization and exposure risks

Requests to the API server pass through authentication (certificates, tokens, OIDC), then authorization (RBAC is the standard mode), then admission control. Typical weaknesses are an API server reachable from the internet, anonymous authentication left enabled, the kubelet configured with `--anonymous-auth=true` and authorization mode `AlwaysAllow`, etcd reachable without mutual TLS, and the legacy insecure port.

Defenses are to keep the API server on private networks or behind an allow-list, disable anonymous access, require client certificates for etcd and the kubelet, encrypt etcd traffic and disk, and use managed control planes where the provider hardens these pieces. CIS Kubernetes Benchmark checks, run with kube-bench, test for these settings.

Service accounts and in-cluster identity

Every pod runs with a service account, and by default a token for it can be mounted at `/var/run/secrets/kubernetes.io/serviceaccount`. If the application is compromised, the attacker can use that token to call the API server with whatever rights the account holds. Older clusters used long-lived token Secrets; modern ones use short-lived, audience-bound projected tokens.

Create dedicated service accounts per workload, set `automountServiceAccountToken: false` when the pod does not call the API, and grant minimal RBAC. Also watch for pods that can reach the cloud metadata service at `169.254.169.254`, because node or workload cloud credentials there extend a cluster compromise into the cloud account.

Putting it into practice

Use a benchmark to make hardening measurable. Run kube-bench against the control plane and worker nodes, and a configuration scanner such as kube-score or Trivy's Kubernetes mode against manifests. Record the baseline result, fix failures in priority order, and rerun after every upgrade because new versions can change defaults. If you use a managed service such as EKS, GKE or AKS, remember the shared responsibility split: the provider secures the control plane, while you remain responsible for nodes, workloads, RBAC and network exposure.

TERMINAL OUTPUT

```
$ kubectl get --raw /version
{"major":"1","minor":"30"}
$ kubectl -n shop get pod web-7d9 -o jsonpath='{.spec.serviceAccountName}'
default
$ kubectl -n shop get pod web-7d9 -o jsonpath='{.spec.automountServiceAccountToken}'

$ curl -sk https://192.0.2.10:10250/pods
Unauthorized
```

WORKED EXAMPLE

During a review of a lab cluster at `192.0.2.10`, kube-bench flags that the kubelet accepts anonymous requests. A test request to the kubelet pods endpoint returns a list of pods instead of `Unauthorized`. The team sets `anonymous-auth` to false, switches authorization mode to `Webhook`, restricts port 10250 to the control plane subnet with a firewall rule, and re-runs the benchmark to confirm the checks now pass.

KEY TERMS

kube-apiserver: The control plane component that exposes the Kubernetes API and validates and processes requests.

etcd: The distributed key-value store that holds all cluster state, including Secrets.

kubelet: The agent on each node that starts and monitors pods and exposes a node API.

Service account: A Kubernetes identity used by pods to authenticate to the API server.

kube-bench: A tool that checks a cluster against the CIS Kubernetes Benchmark.

REMEMBER

- Know the control plane parts: API server, etcd, scheduler, controllers, kubelet.
- Keep the API server, kubelet and etcd authenticated, encrypted and not exposed.
- Treat service account tokens as credentials and minimize their rights.

QUICK CHECK

1. Which component stores all cluster state including Secrets?
 - A) kube-proxy
 - B) etcd
 - C) scheduler
 - D) kubelet
2. What is the risk of kubelet anonymous authentication being enabled?
 - A) Slower scheduling
 - B) Unauthenticated callers can use the node API
 - C) Pods cannot start
 - D) Logs are deleted
3. How can you reduce the impact of a compromised pod's service account token?
 - A) Grant cluster-admin
 - B) Disable automount when not needed and grant minimal RBAC
 - C) Share one account across namespaces
 - D) Store the token in the image
4. The order of API request processing is:
 - A) Admission, authorization, authentication
 - B) Authentication, authorization, admission control
 - C) Authorization, authentication, admission
 - D) Admission, authentication, authorization

Answers: 1-B 2-B 3-B 4-B

1. etcd is the backing datastore for the API server.
2. The kubelet API can list pods and even run commands if left open.
3. No token means no API access, and minimal RBAC limits what a token can do.
4. Identity is established first, then permissions checked, then admission policies applied.

RBAC in Kubernetes

RBAC decides who can do what in the cluster. Over-broad bindings, especially cluster-admin, are among the most common ways a small compromise becomes a full one.

Roles, ClusterRoles and bindings

RBAC is additive: there are no deny rules, only grants. A Role grants permissions within one namespace; a ClusterRole defines permissions cluster-wide or can be reused across namespaces. A RoleBinding attaches a Role (or ClusterRole) to subjects inside a namespace, and a ClusterRoleBinding attaches a ClusterRole to subjects across the whole cluster. Subjects are users, groups or service accounts.

A rule combines apiGroups, resources and verbs such as get, list, watch, create, update, patch and delete. Prefer namespaced Roles and RoleBindings. Use ClusterRoleBindings only for true cluster-level needs such as a node monitoring agent.

Dangerous permissions to avoid

Wildcards (* in resources or verbs) and the built-in cluster-admin role give unlimited power. Some narrow-looking permissions are also dangerous: reading Secrets exposes credentials; create on pods lets a user run a pod with a powerful service account or privileged settings; the verbs escalate, bind and impersonate let a subject raise its own rights; and access to pods/exec allows running commands in containers.

Review these routinely. A developer who can only create pods in a namespace may still be able to read any Secret that namespace's service accounts can access by mounting it. Combine RBAC with admission policy to restrict what those pods may contain.

Designing and auditing least privilege

Start from the workload's needs: list the exact API calls an operator or application makes, and write a Role with only those resources and verbs. Bind to a dedicated service account, not to default. Use groups from your identity provider for humans so access follows team membership, and remove standing admin rights in favor of time-limited elevation.

Audit with kubectl auth can-i, for example kubectl auth can-i list secrets --as=system:serviceaccount:shop:web -n shop, and kubectl auth can-i --list to see all permissions. Tools such as rbac-tool or KubiScan can find risky bindings. Enable API server audit logging so changes to roles and bindings, and sensitive reads, are recorded.

Putting it into practice

Run a periodic RBAC review the way you would review firewall rules. Export all RoleBindings and ClusterRoleBindings, list who holds cluster-admin, and compare against the people and services that truly need it. Look for bindings that reference users or service accounts that no longer exist, and for the system:authenticated and system:unauthenticated groups, which should never receive meaningful rights. Keep roles in version control and require review for changes, so every grant has a ticket and an owner.

CONFIG

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: {name: cm-reader, namespace: shop}
rules:
- apiGroups: [""]
  resources: ["configmaps"]
  verbs: ["get", "list"]
---
kind: RoleBinding
metadata: {name: web-cm-reader, namespace: shop}
subjects: [{kind: ServiceAccount, name: web, namespace: shop}]
roleRef: {kind: Role, name: cm-reader, apiGroup: rbac.authorization.k8s.io}
```

WORKED EXAMPLE

An audit shows a ClusterRoleBinding giving the service account shop:web the cluster-admin role because a developer wanted the app to read one ConfigMap. The fix is to delete the binding, create the namespaced Role cm-reader with get and list on configmaps, bind it to the account, and verify with `kubectl auth can-i list secrets --as=system:serviceaccount:shop:web -n shop`, which should now return no.

KEY TERMS

Role: A set of permissions that applies within a single namespace.

ClusterRoleBinding: A binding that grants a ClusterRole to subjects across all namespaces.

Least privilege: Granting only the permissions needed to perform a task.

Escalate verb: A permission that lets a subject grant rights it does not itself hold.

kubectl auth can-i: A command that checks whether a subject may perform an action.

REMEMBER

- RBAC is additive; prefer namespaced Roles and dedicated service accounts.
- Avoid wildcards, cluster-admin, secrets read, pods/exec, escalate, bind and impersonate.
- Audit with `kubectl auth can-i` and API audit logs.

QUICK CHECK

1. Which statement about Kubernetes RBAC is true?
 - A) It supports explicit deny rules
 - B) It is additive: only grants exist
 - C) Roles apply cluster-wide
 - D) Bindings cannot reference service accounts
2. A RoleBinding that references a ClusterRole grants those permissions:
 - A) In all namespaces
 - B) Only in the binding's namespace
 - C) Only to nodes
 - D) To anonymous users
3. Which permission can let a user run code with another account's rights?
 - A) get on configmaps
 - B) create on pods
 - C) list on namespaces
 - D) watch on events
4. How do you test what a service account may do?
 - A) `kubectl top`
 - B) `kubectl auth can-i --as=system:serviceaccount:ns:name`
 - C) `kubectl logs`
 - D) `kubectl cordon`

Answers: 1-B 2-B 3-B 4-B

1. RBAC has no deny; permissions are the union of grants.
2. A RoleBinding scopes the ClusterRole to its own namespace.
3. A new pod can use a powerful service account or privileged settings.
4. `can-i` with `--as` impersonates the subject for the check.

Pod security and admission control

RBAC controls who may create pods, but pod settings decide how dangerous those pods can be. Admission policies enforce safe settings automatically for every workload.

securityContext and Pod Security Standards

A pod or container securityContext sets runtime privileges: runAsNonRoot, runAsUser, allowPrivilegeEscalation: false, readOnlyRootFilesystem: true, capabilities drop: [ALL], privileged: false and seccompProfile type RuntimeDefault. These settings turn the baseline controls from the first lesson into declarative configuration.

Kubernetes defines three Pod Security Standards. Privileged is unrestricted. Baseline blocks known escalations such as privileged containers, host namespaces and hostPath volumes. Restricted follows current hardening practice: non-root, no privilege escalation, dropped capabilities and a seccomp profile. Aim for Restricted for ordinary applications.

Pod Security Admission

Pod Security Admission is built into the API server and replaced the removed PodSecurityPolicy. You apply it with namespace labels such as pod-security.kubernetes.io/enforce=restricted, plus audit and warn modes. Enforce rejects violating pods, audit records violations in the audit log, and warn returns a message to the user.

A practical rollout starts with warn and audit on the target level, fixes the workloads that would fail, then moves to enforce. System namespaces that need privileges, such as a CNI or storage driver, are given a looser level and tightly controlled access.

Policy engines: Kyverno and Gatekeeper

Admission controllers intercept API requests after authorization. Validating webhooks can reject a request and mutating webhooks can modify it. Policy engines package this: Kyverno writes policies as Kubernetes YAML, while OPA Gatekeeper uses constraint templates written in Rego. ValidatingAdmissionPolicy, using CEL expressions, is a built-in alternative.

Typical policies require images from an approved registry, forbid the latest tag, require resource limits and labels, block hostPath and hostNetwork, and verify image signatures. Be aware of failure modes: a webhook that is down with failurePolicy Fail can block all deployments, while Ignore can silently skip checks.

Putting it into practice

Treat policy and admission as a product with a lifecycle. Write each policy with a clear message so developers learn what to fix, publish exceptions through a pull request rather than ad hoc edits, and test policies in a staging cluster with realistic manifests. Track how many violations occur per team over time. A falling count shows that the standards are being absorbed, while a rising count after a platform change shows where documentation or templates need work.

CONFIG

```
securityContext:
  runAsNonRoot: true
  runAsUser: 10001
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
  capabilities: {drop: ["ALL"]}
  seccompProfile: {type: RuntimeDefault}
# namespace label
# pod-security.kubernetes.io/enforce: restricted
```

WORKED EXAMPLE

The shop namespace is labeled with enforce=restricted. A developer deploys a pod without runAsNonRoot and receives a rejection message naming the violated fields. They add the securityContext above, and the pod is admitted. Separately, a Kyverno policy rejects any image not starting with registry.example.com/, so a pod pulling an unvetted public image is denied with a clear policy message.

KEY TERMS

securityContext: Pod and container settings that define privilege and access controls at runtime.

Pod Security Standards: Three predefined levels: Privileged, Baseline and Restricted.

Admission controller: API server stage that can validate or modify requests after authorization.

Kyverno: A Kubernetes-native policy engine that uses YAML policies.

Gatekeeper: An OPA-based admission controller using constraint templates written in Rego.

REMEMBER

- Set securityContext: non-root, no escalation, drop capabilities, RuntimeDefault seccomp.
- Use Pod Security Admission labels, staged from warn to enforce.
- Use Kyverno, Gatekeeper or ValidatingAdmissionPolicy for custom rules.

QUICK CHECK

1. Which Pod Security Standard is the most restrictive?
 - A) Privileged
 - B) Baseline
 - C) Restricted
 - D) Default
2. What replaced PodSecurityPolicy?
 - A) Pod Security Admission
 - B) RBAC
 - C) NetworkPolicy
 - D) kube-proxy
3. Which securityContext field stops a process gaining more privileges than its parent?
 - A) readOnlyRootFilesystem
 - B) allowPrivilegeEscalation: false
 - C) hostPID
 - D) runAsGroup
4. A safe way to roll out enforce mode is to:
 - A) Enable enforce everywhere immediately
 - B) Use warn and audit first, fix workloads, then enforce
 - C) Disable admission
 - D) Only use Privileged

Answers: 1-C 2-A 3-B 4-B

1. Restricted enforces current hardening best practice.
2. PodSecurityPolicy was removed and replaced by built-in Pod Security Admission.
3. It sets no_new_privs, blocking setuid-style escalation.
4. Warn and audit reveal breakage before enforcement blocks deployments.

Network policies

By default every pod in a cluster can talk to every other pod. Network policies are how you break that flat network into segments and limit lateral movement.

Default behavior and how policies work

Kubernetes networking is open by default: any pod can reach any other pod and service, in any namespace. A NetworkPolicy selects pods with a podSelector and then defines allowed ingress and egress traffic. Once a pod is selected by any policy for a direction, only traffic matching a rule is allowed in that direction. Policies are additive, so the union of rules applies.

Enforcement is performed by the network plugin, not by Kubernetes itself. A CNI such as Calico or Cilium supports policies; a plugin that does not support them will accept the objects and silently do nothing. Always test that a blocked connection is truly blocked.

Default deny and explicit allows

The standard pattern is to create a default-deny policy in each namespace using an empty podSelector with policyTypes Ingress and Egress and no rules. Then add narrow allow policies: the frontend may reach the API on TCP 8080; the API may reach the database on TCP 5432; every pod may reach cluster DNS on UDP and TCP 53.

Remember egress. Forgetting to allow DNS is the classic reason an application suddenly fails after default deny. Restricting egress also limits data exfiltration and blocks pods from reaching the cloud metadata address 169.254.169.254, which is a valuable control against credential theft.

Selectors, namespaces and testing

Rules can match peers with podSelector, namespaceSelector or ipBlock. A common mistake is to write podSelector and namespaceSelector as separate list items, which means 'either', when a single item with both means 'and'. Label namespaces consistently so namespaceSelector rules are reliable.

Test with temporary pods: kubectl run with a busybox or curl image, and try the allowed and forbidden paths. Cilium and Calico offer flow logs and policy visualization to show what traffic is allowed or dropped. Treat policies as code reviewed in version control, and consider service mesh mutual TLS for identity-based controls in addition.

Putting it into practice

Roll out network policy namespace by namespace. First observe real traffic using flow logs from Cilium Hubble or Calico, then draft policies that match observed legitimate flows, apply them in a staging environment, and only then enforce in production. Keep a short runbook for emergencies, including how to temporarily relax a policy under change control. Document each allowed flow with its business purpose, because a policy set that nobody understands is soon replaced by a permissive rule.

CONFIG

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: api-allow-frontend, namespace: shop}
spec:
  podSelector: {matchLabels: {app: api}}
  policyTypes: [Ingress]
  ingress:
    - from: [{podSelector: {matchLabels: {app: frontend}}}]
      ports: [{protocol: TCP, port: 8080}]
```

WORKED EXAMPLE

In the shop namespace a default-deny policy is applied. The frontend suddenly cannot resolve names, so an egress rule is added allowing UDP and TCP 53 to the kube-system namespace. Next, the policy above lets only pods labeled app=frontend reach the api pods on TCP 8080. A test pod without that label fails to connect, showing that the segmentation works and that lateral movement from an unrelated pod is blocked.

KEY TERMS

NetworkPolicy: A Kubernetes object that restricts pod ingress and egress traffic.

Default deny: A policy that selects all pods and allows no traffic until further rules add it.

CNI plugin: The networking component, such as Calico or Cilium, that actually enforces policies.

podSelector: A label query that chooses which pods a rule applies to or matches.

Egress: Traffic leaving a pod toward other destinations.

REMEMBER

- Pods can talk freely by default; policies restrict it and need a supporting CNI.
- Apply default deny then allow specific flows, including DNS.
- Test connectivity and review selector logic carefully.

QUICK CHECK

1. With no NetworkPolicies, pod-to-pod traffic is:
 - A) Blocked
 - B) Allowed
 - C) Only allowed within a node
 - D) Encrypted
2. What happens if the CNI does not support NetworkPolicy?
 - A) Policies are enforced anyway
 - B) Objects are accepted but not enforced
 - C) The cluster stops
 - D) Pods are deleted
3. After applying default-deny egress, apps often fail because:
 - A) Images are missing
 - B) DNS traffic was not allowed
 - C) The API server is down
 - D) RBAC denies pods
4. Which is the standard first policy in a namespace?
 - A) Allow all
 - B) Default deny for ingress and egress
 - C) Delete all pods
 - D) Allow only ICMP

Answers: 1-B 2-B 3-B 4-B

1. Kubernetes networking is flat and open by default.
2. Enforcement depends on the plugin.
3. Name resolution needs egress to port 53 of cluster DNS.
4. Start from deny and add narrow allows.

Runtime security and secrets

Prevention will miss something. Runtime detection spots an active attack inside a container, and good secret handling ensures a compromise does not yield every credential in the cluster.

Runtime detection with Falco

Falco watches kernel events through eBPF or a kernel module and matches them against rules written in YAML. Each rule has a condition, an output message and a priority. Default rules flag a shell spawned in a container, writes below /etc, reads of sensitive files such as /etc/shadow, unexpected outbound connections, and the use of package managers in running containers.

A typical alert contains the rule name, time, container ID, image, Kubernetes namespace and pod, the process and its parent, and the user. Send alerts to a SIEM or chat through Falcosidekick. Tune rules to your workloads, because a legitimate debug session also spawns a shell, and decide a response such as isolating the pod with a network policy, capturing evidence, then replacing it.

seccomp and AppArmor

seccomp filters which system calls a process can make. The RuntimeDefault profile blocks dozens of rarely needed and risky calls, and custom profiles can be generated from observed behavior. AppArmor and SELinux are mandatory access control systems that restrict file, network and capability access by profile, so even a root process in a container is confined.

Enable RuntimeDefault seccomp through the pod securityContext, and apply AppArmor profiles with the pod annotation or field supported by your version. These controls reduce the kernel attack surface available to an attacker who has code execution, which directly limits container escape options.

Handling secrets safely

Kubernetes Secrets are only base64 encoded by default and are stored in etcd in clear text unless encryption at rest is configured with an EncryptionConfiguration, ideally with a KMS provider so keys are kept outside the cluster. Restrict RBAC on Secrets, avoid exposing them as environment variables where logs or crash dumps can leak them, and prefer mounted files.

External secret stores such as HashiCorp Vault, AWS Secrets Manager or cloud equivalents provide rotation, short-lived credentials and audit trails. The External Secrets Operator or the Secrets Store CSI driver syncs or mounts them into pods. Never commit secrets to Git; if using GitOps, use sealed or encrypted secrets and scan repositories for leaked values.

Putting it into practice

Connect detection to a response plan before you need it. Decide who receives Falco alerts, which alerts page a human and which only create tickets, and rehearse the quarantine procedure in a test cluster. Keep forensic collection simple: save pod logs, describe output, the Falco event and, where policy allows, a filesystem snapshot. After any incident, rotate every secret the affected pod could read and review whether a tighter seccomp or network policy would have contained the activity sooner.

LOG EXCERPT

```
14:02:11.402 Warning Terminal shell in container (user=root
  container=web-7d9 image=example/web:1.4
  k8s.ns=shop k8s.pod=web-7d9 shell=sh
  parent=node cmdline=sh -c id)
14:02:14.877 Notice Outbound connection to unexpected IP
  (dest=203.0.113.50:4444 proc=sh pod=web-7d9)
```

WORKED EXAMPLE

Falco reports a shell in pod web-7d9 followed by an outbound connection to 203.0.113.50:4444, which is unusual for this web service. The responder applies a quarantine NetworkPolicy to block egress, preserves logs and a snapshot, rotates the secrets that pod could read, then redeploys a patched image. The investigation reveals an application flaw, and a seccomp profile and read-only filesystem are added to limit repeat impact.

KEY TERMS

Falco: An open-source runtime security tool that detects suspicious behavior from kernel events.

seccomp: A Linux feature that restricts the system calls a process may make.

AppArmor: A Linux security module that confines programs using per-program profiles.

Encryption at rest: Encrypting data stored in etcd so Secrets are not readable from disk or backups.

External Secrets Operator: A controller that syncs secrets from external stores into Kubernetes.

REMEMBER

- Falco detects runtime behavior; seccomp and AppArmor limit what a process can do.
- Enable etcd encryption, ideally with KMS, and restrict Secret access.
- Use external secret stores with rotation and audit.

QUICK CHECK

1. By default, Kubernetes Secrets in etcd are:
 - A) Encrypted with AES
 - B) Only base64 encoded
 - C) Hashed
 - D) Stored in the registry
2. Which event would a default Falco rule commonly flag?
 - A) A pod restarting
 - B) A shell spawned inside a container
 - C) A DNS lookup of example.com
 - D) A pod being scheduled
3. What does a seccomp profile restrict?
 - A) Disk quota
 - B) System calls available to a process
 - C) Image size
 - D) DNS records
4. Best immediate response to a Falco alert of a reverse-shell-like connection?
 - A) Ignore it
 - B) Isolate the pod, preserve evidence, then replace it
 - C) Disable Falco
 - D) Delete the cluster

Answers: 1-B 2-B 3-B 4-B

1. Without EncryptionConfiguration they are stored unencrypted in etcd.
2. Interactive shells in containers are a standard detection.
3. seccomp is a syscall filter.
4. Contain first, keep evidence, then remediate and rotate secrets.

Scan an image

SCENARIO

You are a platform engineer in a lab. The image `example/web:1.4` is about to be promoted to production, and the pipeline requires zero critical vulnerabilities. You have a terminal with a Trivy-style scanner and Docker build tools.

OBJECTIVE

Find the critical vulnerability and rebuild the image on a patched base so the scan passes.

STEPS IN THE SIMULATOR

- 1 **Do:** Type: `trivy image --severity HIGH,CRITICAL example/web:1.4`

Simulator shows:

```
Total: 2 (HIGH: 1, CRITICAL: 1). libssl1 CVE-2024-0001 CRITICAL installed 1.1.1n-0 fixed 1.1.1w-0. zlib1g CVE-2024-0002 HIGH fixed 1.2.13.
```

- 2 **Do:** Type: `grep FROM Dockerfile`

Simulator shows:

```
FROM debian:11.9-slim
```

- 3 **Do:** Type: `docker pull debian:11.11-slim`

Simulator shows:

```
Status: Downloaded newer image for debian:11.11-slim
```

- 4 **Do:** Edit the Dockerfile so FROM reads `debian:11.11-slim`

Simulator shows:

```
Dockerfile updated.
```

- 5 **Do:** Type: `docker build -t example/web:1.5 .`

Simulator shows:

```
Successfully built. Tagged example/web:1.5
```

- 6 **Do:** Type: `trivy image --severity HIGH,CRITICAL example/web:1.5`

Simulator shows:

```
Total: 0 (HIGH: 0, CRITICAL: 0)
```

- 7 **Do:** Type: `docker inspect --format '{{index .RepoDigests 0}}' example/web:1.5`

Simulator shows:

```
example/web@sha256:3f1c...a9d2
```

ANSWER

The critical finding was CVE-2024-0001 in `libssl1`, fixed in `1.1.1w-0`, inherited from the old base image. Updating the FROM line to a patched base and rebuilding as 1.5 produced a scan with zero HIGH or CRITICAL results. Deploy using the digest rather than the tag.

Hint shown in the app: Compare the Installed and Fixed columns, then look at which line in the Dockerfile controls the package versions.

Harden a Dockerfile

SCENARIO

A developer submitted a Dockerfile for an internal API. It uses a large base image, runs as root, copies an .env file containing a password, and has no health check. Review it in the editor and fix it.

OBJECTIVE

Produce a Dockerfile that is minimal, non-root, secret-free and health-checked.

STEPS IN THE SIMULATOR

- 1 **Do:** Open Dockerfile and read the first line: FROM ubuntu:latest

Simulator shows:

Finding: large, unpinned base. Suggested: a slim or distroless image pinned by version or digest.

- 2 **Do:** Replace the FROM line with: FROM python:3.12-slim

Simulator shows:

Line updated. Base size drops from roughly 78 MB to 45 MB in this simulation.

- 3 **Do:** Find the line COPY .env /app/.env and delete it

Simulator shows:

Warning cleared: secret file no longer copied. Note: the old image layers still contain it and must be rotated.

- 4 **Do:** Add: RUN useradd -u 10001 -r app

Simulator shows:

User created in the build instructions.

- 5 **Do:** Add: USER 10001

Simulator shows:

Check passed: container no longer runs as root.

- 6 **Do:** Add: HEALTHCHECK CMD python -c "import urllib.request;urllib.request.urlopen('http://127.0.0.1:8080/health')"

Simulator shows:

Check passed: healthcheck present.

- 7 **Do:** Click Run lint

Simulator shows:

4 of 4 checks passed: base image, secrets, non-root user, healthcheck.

ANSWER

The hardened Dockerfile uses a pinned slim base, contains no copied .env file, creates a numeric non-root user and switches to it with USER, and defines a HEALTHCHECK. The exposed password must be rotated because it existed in a previous image layer.

Hint shown in the app: Search the file for COPY lines and USER; remember a deleted secret must still be rotated.

Fix over-broad RBAC

SCENARIO

The service account `shop:web` needs to read ConfigMaps in the `shop` namespace. A previous engineer bound it to `cluster-admin` with a ClusterRoleBinding named `web-admin`. You are editing the manifests in a config editor.

OBJECTIVE

Replace the `cluster-admin` binding with a namespaced least-privilege Role and RoleBinding.

STEPS IN THE SIMULATOR

- 1 **Do:** Open `web-admin.yaml` and read the `roleRef`

Simulator shows:

```
roleRef: ClusterRole cluster-admin. Subjects: ServiceAccount web in namespace shop. Risk: full cluster control.
```

- 2 **Do:** Delete `web-admin.yaml`

Simulator shows:

```
ClusterRoleBinding web-admin removed.
```

- 3 **Do:** Create Role `cm-reader` in namespace `shop` with `apiGroups [""], resources ["configmaps"], verbs ["get","list"]`

Simulator shows:

```
Role cm-reader created in namespace shop.
```

- 4 **Do:** Create RoleBinding `web-cm-reader` referencing Role `cm-reader` and subject ServiceAccount `shop:web`

Simulator shows:

```
RoleBinding created.
```

- 5 **Do:** Run: `kubectl auth can-i list configmaps --as=system:serviceaccount:shop:web -n shop`

Simulator shows:

```
yes
```

- 6 **Do:** Run: `kubectl auth can-i list secrets --as=system:serviceaccount:shop:web -n shop`

Simulator shows:

```
no
```

- 7 **Do:** Run: `kubectl auth can-i '*' '*' --as=system:serviceaccount:shop:web`

Simulator shows:

```
no
```

ANSWER

The correct end state is a namespaced Role with only `get` and `list` on `configmaps`, bound to `shop:web` through a RoleBinding, and the ClusterRoleBinding to `cluster-admin` removed. `can-i` shows `yes` for `configmaps` and `no` for `secrets` and for wildcard access.

Hint shown in the app: Use a Role, not a ClusterRole, and list only the resource and verbs the app needs.

Write a NetworkPolicy

SCENARIO

The shop namespace has three tiers: frontend (app=frontend), api (app=api listening on TCP 8080) and database. Currently any pod can reach any other. You must isolate the api tier.

OBJECTIVE

Create a default-deny ingress policy and a policy allowing only the frontend to reach the api on TCP 8080.

STEPS IN THE SIMULATOR

- 1 **Do:** Create default-deny with podSelector {} and policyTypes [Ingress]

Simulator shows:

NetworkPolicy default-deny created in namespace shop.

- 2 **Do:** Run test pod: `kubectl run t1 --image=busybox -n shop -- wget -qO- -T2 http://api:8080`

Simulator shows:

wget: download timed out (blocked by default-deny).

- 3 **Do:** Create policy api-allow-frontend with podSelector app=api

Simulator shows:

Policy created, selects 2 pods.

- 4 **Do:** Add ingress from podSelector app=frontend, ports TCP 8080

Simulator shows:

Rule added.

- 5 **Do:** Run test from a pod labeled app=frontend

Simulator shows:

HTTP/1.1 200 OK (allowed).

- 6 **Do:** Run test from an unlabeled pod

Simulator shows:

Timed out (denied).

- 7 **Do:** Click Validate

Simulator shows:

Result: segmentation correct. Note: egress is not restricted; consider an egress policy allowing DNS.

ANSWER

The solution is a namespace-wide default-deny ingress policy plus a policy selecting app=api that allows ingress only from pods labeled app=frontend on TCP 8080. Tests confirm the frontend succeeds and other pods are blocked.

Hint shown in the app: The allow policy's podSelector chooses the protected pods; the from block chooses who may connect.

Investigate a Falco alert

SCENARIO

Falco has raised alerts from the production cluster overnight. You are on call and have a log viewer open on the Falco output for the shop namespace.

OBJECTIVE

Determine whether the alert is a true compromise and choose a containment action.

STEPS IN THE SIMULATOR

- 1 **Do:** Filter the log by priority Warning and above

Simulator shows:

2 events: 'Terminal shell in container' at 14:02:11 and 'Outbound connection to unexpected IP' at 14:02:14.

- 2 **Do:** Open the first event details

Simulator shows:

pod=web-7d9, image=example/web:1.4, user=root, cmdline=sh -c id, parent=node (the app process).

- 3 **Do:** Check the change calendar entry for pod web-7d9

Simulator shows:

No maintenance or debugging session is scheduled.

- 4 **Do:** Open the second event

Simulator shows:

dest=203.0.113.50:4444 proc=sh. The destination is not in the egress allow-list.

- 5 **Do:** Search the log for other pods contacting 203.0.113.50

Simulator shows:

No other pods contacted it.

- 6 **Do:** Choose containment: apply quarantine NetworkPolicy to pod web-7d9

Simulator shows:

Policy applied. Outbound traffic from web-7d9 is now blocked; the pod is kept running for forensics.

- 7 **Do:** Choose follow-up: rotate secrets mounted in web-7d9 and redeploy a patched image

Simulator shows:

Ticket created with steps; secrets scheduled for rotation.

ANSWER

This is a likely true compromise: an interactive shell spawned by the application process, no approved change, then a connection to an unlisted external address. Contain by isolating the pod with network policy, keep it for evidence, rotate its secrets and redeploy after fixing the vulnerability.

Hint shown in the app: Ask whether a change record explains the shell and whether the destination is known.

Container and Kubernetes Security: 10 questions, 80% to pass

1. Why can a kernel vulnerability affect many containers?
 - A) Each container has its own kernel
 - B) They share the host kernel
 - C) Containers use hypervisors
 - D) Images contain the kernel
2. Which is the most effective way to avoid secrets leaking from image layers?
 - A) Delete them in a later layer
 - B) Never add them; use secret mounts or a store
 - C) Rename the file
 - D) Compress the image
3. Which command checks a service account's permission?
 - A) `kubectl auth can-i --as=system:serviceaccount:ns:sa`
 - B) `kubectl get nodes`
 - C) `kubectl cp`
 - D) `kubectl drain`
4. Which role should be avoided for ordinary workloads?
 - A) A namespaced Role with get on configmaps
 - B) cluster-admin
 - C) A Role with list on pods
 - D) A read-only Role
5. Which Pod Security Standard level is recommended for typical apps?
 - A) Privileged
 - B) Baseline only if unsure
 - C) Restricted
 - D) None
6. What does a default-deny NetworkPolicy do?
 - A) Allows all traffic
 - B) Selects pods and allows no traffic until other rules add it
 - C) Encrypts traffic
 - D) Deletes pods
7. After default-deny egress, what is commonly forgotten?
 - A) Allowing DNS
 - B) Allowing ICMP
 - C) Allowing SSH
 - D) Allowing FTP
8. Which tool detects a shell spawned in a running container?
 - A) Trivy
 - B) Falco
 - C) kube-bench
 - D) cosign
9. Kubernetes Secrets are stored in etcd how by default?
 - A) Encrypted with KMS
 - B) Unencrypted, base64 encoded
 - C) Hashed with bcrypt
 - D) In the kubelet only
10. What does pinning an image by digest protect against?
 - A) Slow pulls
 - B) Content changing behind a mutable tag
 - C) Network latency
 - D) RBAC errors

Answers: 1-B 2-B 3-A 4-B 5-C 6-B 7-A 8-B 9-B 10-B

1. Containers are processes on one shared kernel.
2. Layers are immutable, so secrets must never enter them.
3. can-i with --as tests the identity's rights.
4. cluster-admin grants unrestricted power.
5. Restricted reflects current hardening practice.
6. It begins from deny and requires explicit allows.
7. Name resolution needs egress to DNS.
8. Falco is a runtime detection tool; Trivy scans images.
9. Encryption at rest must be configured.
10. Digests identify exact content.

Infrastructure as Code Security

Advanced | 4 lessons | 3 simulated labs | checkpoint | about 4h

Infrastructure as Code lets teams build cloud environments from reviewed text files, which also means a single bad line can be copied to hundreds of resources. This module covers the common IaC misconfigurations and the risk of secrets in state files, how to scan Terraform and CloudFormation with tools such as Checkov, tfsec and cfn-nag, how to detect and respond to drift, and how to enforce guardrails with OPA, Sentinel and service control policies before and after deployment.

BY THE END YOU CAN

- + Scan Terraform and CloudFormation for misconfigurations
- + Detect and remediate infrastructure drift
- + Enforce guardrails with policy as code

IN THIS MODULE

Lesson 1 IaC security risks

Lesson 2 Scanning IaC templates

Lesson 3 Drift detection and state security

Lesson 4 Guardrails and policy as code

Lab 1 Fix a Terraform file (Code editor)

Lab 2 Triage drift (Terminal)

Lab 3 Write a guardrail (Config editor)

Checkpoint 10 questions, 80% to pass

IaC security risks

IaC turns infrastructure into software, so it inherits software risks at cloud scale. Finding a problem in a pull request is far cheaper than finding it in production.

Why IaC changes the risk picture

With Terraform, CloudFormation, Pulumi or Bicep, a few lines describe networks, storage, identities and firewalls. That brings speed, repeatability and review through version control, but also means a mistake is replicated every time the module is used. A public storage bucket in a shared module can appear in every environment. Attackers and auditors alike see the same weakness repeated, and the fix needs to be made once, in code, then rolled out through the pipeline rather than patched by hand in a console.

Common misconfigurations

Typical findings include storage buckets with public access or without encryption, security groups open to 0.0.0.0/0 on ports such as 22 and 3389, databases reachable from the internet, IAM policies using Action: '*' and Resource: '*', disabled logging such as CloudTrail or flow logs, missing versioning and backups, and unencrypted volumes. Compute resources may also lack IMDSv2 enforcement. These are rarely clever mistakes; they are defaults, copy-pasted examples and temporary shortcuts that were never removed. Because they are declarative, they can be detected reliably by static analysis.

Secrets and state files

Hard-coded credentials in .tf files, variables files or CI logs are an obvious problem and end up in Git history, where deleting the line does not remove them. Less obvious is the Terraform state file, which records resource attributes in plain text, including database passwords, generated keys and connection strings, even when variables are marked sensitive. Anyone who can read the state can read those values. Store state in a remote backend with encryption and tight access, never commit terraform.tfstate, and pull secrets from a secrets manager at runtime instead of passing them through code.

Shift-left scanning

Shift-left means moving checks earlier: in the editor, in pre-commit hooks, in pull requests and in the pipeline before apply. Static scanners read templates without deploying anything, so feedback arrives in seconds and fixes are cheap. A layered model is best: scan code early, evaluate the plan before apply, enforce policy at deployment, and detect drift and misconfiguration in the running cloud afterward. No single layer catches everything, since console changes and dynamic values can bypass static checks, but together they close most of the gaps.

Keep ownership clear: name a team that owns the shared modules, publish a short secure-by-default module catalog, and make the pipeline output explain each failure in plain language. When developers can fix a finding in one commit without opening a ticket, adoption rises and the number of risky resources falls quickly.

CODE EXCERPT

```
resource "aws_s3_bucket" "logs" {
  bucket = "example-logs-prod"
  acl = "public-read" # finding: public access
}
resource "aws_security_group_rule" "ssh" {
  type = "ingress"
  from_port = 22
  to_port = 22
  protocol = "tcp"
  cidr_blocks = ["0.0.0.0/0"] # finding: SSH open to world
}
```

WORKED EXAMPLE

A pull request adds the module above to create a log bucket and an SSH rule. A scanner running in CI flags the public-read ACL and the world-open SSH rule before anyone reviews the diff. The author changes the bucket to block public access with encryption and restricts SSH to the office range 198.51.100.0/24, or better, removes SSH in favor of a managed session service. The pipeline passes, and the weak pattern never reaches the cloud account.

KEY TERMS

Infrastructure as Code: Defining infrastructure in version-controlled text files that tools apply automatically.

Misconfiguration: A setting that is valid but unsafe, such as a public bucket.

State file: A record of deployed resources that can contain sensitive values in plain text.

Shift left: Moving security checks earlier in the development process.

Static analysis: Inspecting code or templates for problems without running them.

REMEMBER

- IaC multiplies both good and bad configuration.
- Watch for public storage, open ports, wide IAM, disabled logging and secrets in code and state.
- Layer checks: pre-commit, pull request, plan, deploy-time policy and runtime detection.

QUICK CHECK

1. Why can an IaC mistake be especially damaging?
 - A) IaC cannot be reviewed
 - B) One flawed module can be replicated across many resources
 - C) IaC disables logging
 - D) IaC is not versioned
2. Which item is a typical IaC misconfiguration?
 - A) Encrypted volumes
 - B) Security group allowing 0.0.0.0/0 to port 22
 - C) Least-privilege IAM
 - D) Enabled CloudTrail
3. Where can secrets be exposed even if variables are marked sensitive?
 - A) Terraform state file
 - B) Terraform version number
 - C) Provider name
 - D) Module comments only
4. What is the main benefit of scanning IaC in pull requests?
 - A) It replaces monitoring
 - B) Problems are found before deployment, when fixes are cheap
 - C) It encrypts the cloud
 - D) It removes the need for review

Answers: 1-B 2-B 3-A 4-B

1. Reusable code multiplies a defect everywhere it is applied.
2. World-open administrative ports are a classic finding.
3. State stores resource attributes in plain text.
4. Early feedback is faster and cheaper than remediation in production.

Scanning IaC templates

Scanners encode hundreds of known good practices so reviewers do not have to remember them. Using them well means reading findings sensibly and tuning them without hiding real risk.

The scanner landscape

Checkov (Python, Terraform, CloudFormation, Kubernetes, Dockerfiles and more), tfsec (now folded into Trivy), Trivy's config scanning, KICS and Terrascan examine Terraform and related formats. cfn-nag and cfn-lint target CloudFormation, and cfn-guard evaluates templates against rules. Each ships with a library of checks mapped to benchmarks such as the CIS Foundations Benchmarks. They differ in language coverage, custom rule syntax and how well they resolve variables and modules, so many teams pick one primary tool and test it on their own code.

Running a scan and reading results

A basic run is `checkov -d ./infra` or `trivy config ./infra`. Output lists each failed check with an ID such as CKV_AWS_18, a short description, the resource and the file and line range. Some tools also print a guide link and the passed checks. Output can be emitted as SARIF or JUnit so a code host can annotate pull requests directly. Start by sorting by severity, and look for repeated patterns that point to one shared module where a single fix removes many findings.

Fixing, suppressing and tuning

Fix findings in code wherever possible: add server-side encryption, enable block public access, narrow the CIDR range, scope IAM actions. When a finding is a real exception, suppress it narrowly with an inline comment such as `#checkov:skip=CKV_AWS_18:Reason and ticket`, or a baseline file, so the reason is visible in review. Never disable a check globally to make a build pass. Review suppressions on a schedule, because a skip written for a temporary case can outlive its purpose.

Pipeline integration and limits

Run scanners in pre-commit for fast feedback and in CI as an enforced gate. Begin in report-only mode, establish a baseline of existing findings, then fail builds on new high-severity issues while the backlog is paid down. Remember the limits: static scanners cannot always resolve values computed at apply time, cannot see changes made in the console, and may produce false positives. Complement them with plan-based scanning, which evaluates the resolved terraform plan output, and with runtime checks such as cloud security posture management.

Document each suppression in the same pull request that adds it, and set a recurring review so old skips are re-evaluated. Pair scanner results with a short list of the checks your organization treats as always blocking, such as public storage and world-open admin ports, so teams know which findings are negotiable.

TERMINAL OUTPUT

```
$ checkov -d ./infra --compact
Failed checks: 3, Passed checks: 41
Check: CKV_AWS_18: "Ensure S3 bucket has access logging"
  FAILED for resource: aws_s3_bucket.logs
  File: /s3.tf:1-4
Check: CKV_AWS_24: "Ensure no SG allows 0.0.0.0/0 to port 22"
  FAILED for resource: aws_security_group_rule.ssh
  File: /sg.tf:1-8
```

WORKED EXAMPLE

A team adds Checkov to CI in report-only mode and finds 120 failed checks. They group results by module and discover that 70 come from one shared bucket module. After fixing that module, the count drops by more than half. They then enable blocking for new critical findings, and record two justified skips, each with a ticket number and review date, for a bucket that must be public by design as a static website.

KEY TERMS

Checkov: An open-source policy-as-code scanner for Terraform, CloudFormation and other IaC formats.

cfn-nag: A tool that scans CloudFormation templates for insecure patterns.

SARIF: A standard JSON format for static analysis results used by code hosts.

Suppression: A documented exception that tells a scanner to skip a specific check.

Baseline: A recorded set of existing findings used so only new problems fail the build.

REMEMBER

- Use Checkov, Trivy or similar tools plus cfn-nag for CloudFormation.
- Fix in code, suppress narrowly and review suppressions.
- Gate pipelines gradually and add plan-based and runtime checks.

QUICK CHECK

1. What is the best way to handle a justified exception?
 - A) Disable the check globally
 - B) Add a narrow, documented suppression with a reason
 - C) Delete the template
 - D) Ignore the scanner
2. Which tool primarily targets CloudFormation?
 - A) cfn-nag
 - B) tfsec
 - C) kubectrl
 - D) docker scan
3. Why start with report-only mode and a baseline?
 - A) To hide findings
 - B) To avoid blocking all work while preventing new issues
 - C) To speed up the cloud
 - D) To skip reviews
4. A limitation of static template scanning is that it may not see:
 - A) Resource names
 - B) Console changes made outside code
 - C) File paths
 - D) Provider versions

Answers: 1-B 2-A 3-B 4-B

1. A scoped skip with a reason keeps the control active elsewhere.
2. cfn-nag checks CloudFormation templates.
3. A baseline lets you block new regressions while paying down old findings.
4. Manual changes in the cloud are not in the templates.

Drift detection and state security

The cloud as deployed often differs from the code. Drift can hide risky manual changes, and an unprotected state file can hand an attacker the keys to the environment.

What drift is and why it happens

Drift is a difference between the declared configuration in code and the actual resources in the cloud. Causes include emergency console edits during an incident, another automation tool changing the same resource, a team member testing a change, or an attacker deliberately weakening a control, such as opening a firewall rule. Drift matters twice: it breaks the guarantee that code is the source of truth, and it can silently introduce exposure that no scanner of the templates will ever see.

Detecting drift

For Terraform, `terraform plan` compares code and state to the real resources after refreshing, and shows changes marked with `~` for update in place, `+` for create, `-` for destroy. Scheduled plans, or a platform feature like Terraform Cloud drift detection, give regular reports. In AWS, CloudFormation drift detection reports resources as `IN_SYNC` or `MODIFIED`, and AWS Config evaluates resources against rules continuously and records configuration history. Alert on changes to security-critical resources and use CloudTrail to learn who made a manual change.

Responding: revert or codify

Triage each drift. If the change was unauthorized or unsafe, revert it by applying the code, and investigate how it happened. If the change was legitimate, such as a needed emergency fix, codify it by updating the code through a pull request so the next apply does not undo it. Never ignore it, because the next terraform apply may overwrite a good fix or fail unexpectedly. Reduce future drift by limiting console write access, using read-only roles for humans, and requiring changes through the pipeline.

Protecting remote state

Use a remote backend such as an encrypted S3 bucket with versioning, access limited to the pipeline role and administrators, and a locking mechanism (DynamoDB or native locking, or Terraform Cloud) so two applies cannot corrupt state. Enable server-side encryption with a customer-managed KMS key, block public access, log access, and keep backups. Reduce secrets in state by using references to secret managers and ephemeral values where supported. Treat the state bucket as highly sensitive, since read access can reveal credentials and write access can alter what gets deployed.

Build drift handling into routine operations: assign an owner for each stack, send drift reports to that owner, and set a service level for resolving them. Frequent drift in one stack usually points to a process gap, such as missing permissions in the pipeline, rather than careless users.

TERMINAL OUTPUT

```
$ terraform plan -refresh-only
# aws_security_group.web has changed
~ resource "aws_security_group" "web" {
  ~ ingress = [
    + {
      + cidr_blocks = ["0.0.0.0/0"]
      + from_port = 22
      + to_port = 22
    },
  ]
}
Plan: 0 to add, 1 to change, 0 to destroy.
```

WORKED EXAMPLE

A scheduled plan shows that the web security group gained a rule allowing SSH from 0.0.0.0/0. CloudTrail shows a user changed it in the console at 02:10 for troubleshooting and forgot to remove it. Because the rule is unsafe, the team reverts it by running apply, restricts that user's console write permissions, and adds an AWS Config rule that alerts on any security group open to the world on port 22.

KEY TERMS

Drift: A difference between declared infrastructure code and the real deployed resources.

State locking: A mechanism preventing concurrent operations from corrupting the state file.

AWS Config: A service that records resource configuration and evaluates it against rules.

Remote backend: Shared storage for Terraform state, such as an encrypted S3 bucket.

Codify: To bring a legitimate manual change into code so it is tracked and repeatable.

REMEMBER

- Drift is divergence between code and reality, whether accidental or malicious.
- Detect with plan, CloudFormation drift detection and AWS Config; decide revert or codify.
- Protect state: encryption, versioning, locking and strict access.

QUICK CHECK

1. A manual console change that is unsafe should usually be:
 - A) Left in place
 - B) Reverted through the pipeline and investigated
 - C) Hidden from the plan
 - D) Copied to every region
2. Which Terraform symbol in a plan indicates an in-place update?
 - A) +
 - B) -
 - C) ~
 - D) #
3. Why enable state locking?
 - A) To encrypt secrets
 - B) To prevent concurrent applies from corrupting state
 - C) To speed up scans
 - D) To disable drift
4. Why must state storage be tightly protected?
 - A) It is public by design
 - B) It can contain secrets and controls what is deployed
 - C) It stores only comments
 - D) It is deleted after each run

Answers: 1-B 2-C 3-B 4-B

1. Restore the declared secure state and learn how the change occurred.
2. The tilde marks a resource to be changed in place.
3. Locking serializes operations on the same state.
4. Read access leaks values and write access can change deployments.

Guardrails and policy as code

Scanners advise; guardrails enforce. Policy as code makes security rules testable, versioned and automatic so non-compliant infrastructure is blocked rather than merely reported.

Preventive versus detective controls

A detective control finds a problem after it exists, such as AWS Config flagging an open security group. A preventive control stops it from being created. Mature programs use both, in layers: policy checks in the pipeline before apply, organization-level restrictions that even administrators cannot bypass, and continuous detection in the running environment. Policy as code expresses these rules as machine-readable files in version control, so they get reviews, tests, change history and consistent enforcement across teams and accounts.

OPA, Conftest and Sentinel

Open Policy Agent (OPA) uses the Rego language to evaluate structured input and return allow or deny decisions. Conftest applies OPA policies to files such as Terraform plan JSON, Kubernetes manifests or CI configuration. HashiCorp Sentinel is the equivalent embedded in Terraform Cloud and Enterprise, evaluating plans between plan and apply. Policies commonly require encryption, approved regions, mandatory tags, and forbid public ingress to administrative ports. Evaluate the plan output, not just the source, so computed values and module expansions are included.

Service control policies and cloud-native guardrails

AWS Organizations service control policies (SCPs) set the maximum permissions for member accounts, regardless of local IAM. They can deny actions such as disabling CloudTrail, leaving the organization, or creating resources outside approved regions. Azure Policy and GCP Organization Policy offer similar effects. These apply even if IaC is bypassed, which makes them a strong backstop. They need careful testing, because an overbroad deny can break deployments, and an exception path should exist for approved break-glass activity.

Designing and testing guardrails

Write each policy with a clear message, an owner and a test suite with both passing and failing examples, so changes do not silently weaken rules. Roll out in advisory mode first, review the violations, then move to mandatory. Provide secure modules that already comply, because developers follow the easy path. Track exceptions with expiry. Measure outcomes: number of blocked deployments by rule, time to fix, and number of resources that bypassed checks. The goal is not to say no but to make the safe way the default and fast.

Keep guardrails understandable. A policy that fails with a cryptic message trains developers to look for workarounds, while one that names the resource, the rule and the approved alternative gets fixed quickly. Review blocked-deployment metrics monthly to retire rules that no longer matter and tighten ones that are often bypassed.

POLICY EXCERPT

```
package terraform.ssh
deny[msg] {
  rc := input.resource_changes[_]
  rc.type == "aws_security_group_rule"
  rc.change.after.type == "ingress"
  rc.change.after.from_port <= 22
  rc.change.after.to_port >= 22
  rc.change.after.cidr_blocks[_] == "0.0.0.0/0"
  msg := sprintf("%s opens SSH to the internet", [rc.address])
}
```

WORKED EXAMPLE

A platform team adds the Rego policy above to the pipeline using Conftest against terraform show -json output. A developer's plan that opens port 22 to everyone is rejected with the message naming the resource. They also deploy an SCP that denies cloudtrail:StopLogging and DeleteTrail in all member accounts. When someone later tries to disable logging with console access, the organization-level policy blocks it regardless of their local permissions.

KEY TERMS

Policy as code: Security and compliance rules written as versioned, testable, machine-enforced code.

OPA/Rego: An open policy engine and its declarative language for evaluating structured data.

Sentinel: HashiCorp's policy framework that checks Terraform plans before apply.

Service control policy: An AWS Organizations policy that sets maximum permissions for accounts.

Preventive control: A control that blocks a bad action before it happens.

REMEMBER

- Use preventive and detective controls in layers.
- OPA/Conftest and Sentinel check plans; SCPs and cloud policies block at account level.
- Test policies, provide secure modules, and track exceptions and outcomes.

QUICK CHECK

1. Which control type blocks a non-compliant deployment before it is created?

- A) Detective
- B) Preventive
- C) Corrective only
- D) Informational

2. Why evaluate the terraform plan JSON instead of only the source files?

- A) It is smaller
- B) It contains resolved values and module expansions
- C) It is encrypted
- D) It removes drift

3. What is the effect of an SCP?

- A) Grants permissions to users
- B) Sets the maximum permissions available in member accounts
- C) Encrypts state
- D) Scans templates

4. What is a good rollout approach for a new guardrail?

- A) Enforce immediately everywhere
- B) Advisory mode first, review violations, then enforce
- C) Never test it
- D) Apply it only to production without review

Answers: 1-B 2-B 3-B 4-B

1. Preventive controls stop the action; detective ones report it after.
2. The plan reflects what will actually be created.
3. SCPs restrict what any identity in the account can be allowed to do.
4. Advisory mode reveals impact before blocking work.

Fix a Terraform file

SCENARIO

A developer's Terraform file creates a storage bucket for application logs and a security group for a bastion host. The CI scanner fails with several findings. You are editing main.tf in the lab editor.

OBJECTIVE

Correct the bucket and security group so all scanner checks pass.

STEPS IN THE SIMULATOR

- 1 **Do:** Run scan: `checkov -f main.tf`

Simulator shows:

Failed: CKV_AWS_20 bucket ACL public-read; CKV_AWS_19 no server-side encryption; CKV_AWS_24 SG allows 0.0.0.0/0 on port 22.

- 2 **Do:** Change `acl = "public-read"` to `acl = "private"`

Simulator shows:

Line updated. CKV_AWS_20 now pending re-scan.

- 3 **Do:** Add an `aws_s3_bucket_public_access_block` resource with all four settings true

Simulator shows:

Resource added.

- 4 **Do:** Add `aws_s3_bucket_server_side_encryption_configuration` using `sse_algorithm = "aws:kms"`

Simulator shows:

Resource added.

- 5 **Do:** In the security group, replace `cidr_blocks ["0.0.0.0/0"]` with `["198.51.100.0/24"]`

Simulator shows:

Rule now limited to the office range.

- 6 **Do:** Run scan again: `checkov -f main.tf`

Simulator shows:

Failed checks: 0. Passed checks: 12.

- 7 **Do:** Review the file for any `#checkov:skip` comments

Simulator shows:

No suppression comments present; all findings were fixed in code.

ANSWER

The bucket must be private with public access blocked and server-side encryption enabled, and the SSH rule must be limited to a known range such as 198.51.100.0/24 rather than 0.0.0.0/0. After the fixes, the scan reports zero failed checks without any suppressions.

Hint shown in the app: Fix the cause in code before considering a skip; check the ACL, encryption and CIDR.

Triage drift

SCENARIO

The nightly pipeline ran terraform plan and produced a non-empty diff although no pull request was merged. You have a terminal on the pipeline host and read access to the cloud audit log.

OBJECTIVE

Decide for each drifted change whether to revert or codify it.

STEPS IN THE SIMULATOR

- 1 **Do:** Run: terraform plan -refresh-only

Simulator shows:

Two changes: aws_security_group.web ingress +22 from 0.0.0.0/0; aws_instance.api instance_type t3.medium -> t3.large.

- 2 **Do:** Run: cloudtrail lookup for AuthorizeSecurityGroupIngress

Simulator shows:

Event at 02:10 by user fake-user-a via console, source IP 203.0.113.25. No ticket reference.

- 3 **Do:** Run: cloudtrail lookup for ModifyInstanceAttribute

Simulator shows:

Event at 09:40 by user fake-user-b referencing incident INC-4411: CPU saturation.

- 4 **Do:** Decide on the security group change

Simulator shows:

Marked: UNSAFE, REVERT. SSH open to the internet has no approval.

- 5 **Do:** Decide on the instance size change

Simulator shows:

Marked: LEGITIMATE, CODIFY. Update instance_type to t3.large in code via pull request.

- 6 **Do:** Run: terraform apply on the revert branch

Simulator shows:

1 changed: aws_security_group.web ingress rule removed. Apply complete.

- 7 **Do:** Run: terraform plan

Simulator shows:

Only expected change: instance size matches once the codify PR merges. No other drift.

ANSWER

Revert the security group rule because it exposed SSH to the world without approval, and investigate the 02:10 console session. Codify the instance type increase, which has a legitimate incident reference, by updating the code through a pull request. Then restrict console write access to prevent recurrence.

Hint shown in the app: Check who made each change and whether a ticket or approval exists.

Write a guardrail

SCENARIO

The platform team wants to block any Terraform plan that opens port 22 to the internet. You will author a Rego policy in the editor and test it against two sample plans.

OBJECTIVE

Write and verify a policy that denies public ingress on port 22 and allows a restricted range.

STEPS IN THE SIMULATOR

- 1 Do:** Create policy file `ssh.rego` with package `terraform.ssh`
Simulator shows:
File created.
- 2 Do:** Add a deny rule matching `aws_security_group_rule` ingress where `from_port <= 22` and `to_port >= 22`
Simulator shows:
Rule saved. Syntax check: OK.
- 3 Do:** Add the condition `cidr_blocks[_] == "0.0.0.0/0"`
Simulator shows:
Condition saved.
- 4 Do:** Add `msg := sprintf("%s opens SSH to the internet", [rc.address])`
Simulator shows:
Message saved.
- 5 Do:** Run test against `bad-plan.json` (0.0.0.0/0 on 22)
Simulator shows:
FAIL: `aws_security_group_rule.ssh` opens SSH to the internet.
- 6 Do:** Run test against `good-plan.json` (198.51.100.0/24 on 22)
Simulator shows:
PASS: no violations.
- 7 Do:** Run test against plan with 0.0.0.0/0 on port 443
Simulator shows:
PASS: not matched, HTTPS is allowed.

ANSWER

The policy denies any ingress rule whose port range includes 22 and whose CIDR includes 0.0.0.0/0, with a message naming the resource. It fails the bad plan, passes the restricted range, and does not block unrelated port 443 rules, showing it is both effective and not overbroad.

Hint shown in the app: Test both a failing and a passing plan, and check the port range condition.

Infrastructure as Code Security: 10 questions, 80% to pass

1. Which is a typical secret exposure risk specific to Terraform?
 - A) Provider docs
 - B) Plain-text values in the state file
 - C) Module names
 - D) Resource tags
2. What does shift-left mean for IaC?
 - A) Scan only in production
 - B) Check configuration earlier, such as in pull requests
 - C) Stop using IaC
 - D) Delay reviews
3. Which tool family checks CloudFormation templates for insecure patterns?
 - A) cfn-nag
 - B) kubectf
 - C) nmap
 - D) Wireshark
4. What is the safest way to deal with a real, accepted exception to a check?
 - A) Disable the check globally
 - B) Use a narrow documented skip with reason and review date
 - C) Remove the scanner
 - D) Rename the resource
5. Which result in a Terraform plan shows an in-place change?
 - A) +
 - B) -
 - C) ~
 - D) !
6. A legitimate emergency console change should be:
 - A) Ignored
 - B) Codified in code through a pull request
 - C) Hidden
 - D) Overwritten silently
7. What does state locking prevent?
 - A) Encryption
 - B) Concurrent runs corrupting state
 - C) Tagging
 - D) Drift
8. Which control still applies even when IaC is bypassed through the console?
 - A) A pre-commit hook
 - B) An AWS service control policy
 - C) A code comment
 - D) A README
9. Why test policies with both passing and failing inputs?
 - A) To increase file size
 - B) To confirm they block bad cases without blocking good ones
 - C) To avoid reviews
 - D) To speed up plans
10. Which scanner input resolves computed values best?
 - A) Raw .tf source only
 - B) terraform plan JSON output
 - C) README files
 - D) Git commit messages

Answers: 1-B 2-B 3-A 4-B 5-C 6-B 7-B 8-B 9-B 10-B

1. State contains resource attributes including secrets.
2. Earlier checks are faster and cheaper to act on.
3. cfn-nag is built for CloudFormation.
4. Narrow, documented suppressions preserve the control elsewhere.
5. The tilde marks an update in place.
6. Codify it so apply does not undo it and it is tracked.
7. Locks serialize operations.
8. SCPs apply to all identities in the account.
9. Tests prevent silent weakening and false blocks.
10. Plan output includes resolved values.

Logging and Monitoring Architecture

Intermediate | 5 lessons | 4 simulated labs | checkpoint | about 4h 25m

Detection is only as good as the data behind it. This module shows how to choose log sources across identity, endpoint, network and cloud, design reliable collection pipelines, and parse and normalize events so searches work. You will write and tune detection rules mapped to MITRE ATT&CK, set retention and protect log integrity to meet regulatory needs, and measure the program with source health, detection time, false positive rate and coverage.

BY THE END YOU CAN

- + Design a log pipeline with the right sources and retention
- + Engineer SIEM detections and tune alerts
- + Define metrics for monitoring coverage

IN THIS MODULE

Lesson 1 Log sources and pipeline design

Lesson 2 SIEM engineering and parsing

Lesson 3 Detection and alert design

Lesson 4 Retention, integrity and compliance

Lesson 5 Metrics and detection coverage

Lab 1 Fix a parser (Log viewer)

Lab 2 Write a detection (Mock SIEM)

Lab 3 Tune a noisy alert (Mock SIEM)

Lab 4 Plan retention (Scenario)

Checkpoint 10 questions, 80% to pass

Log sources and pipeline design

If an event is never collected, no rule can detect it. A deliberate pipeline design decides what is visible during an investigation and how much it costs.

Choosing log sources by value

Start from the questions you must answer during an incident: who logged in, from where, what ran, what was accessed, and where data went. Identity logs come first: Active Directory security events (4624 logon, 4625 failed logon, 4720 account created), identity provider sign-in logs and VPN. Next are endpoint telemetry from EDR and process creation (Windows 4688 or Sysmon event 1), network sources such as firewall, DNS, proxy and NetFlow, and cloud audit logs such as AWS CloudTrail, Azure Activity and Entra sign-ins, plus application and email logs.

Collection methods

Common collection methods are agents (Elastic Agent, Beats, Splunk Universal Forwarder, Windows Event Forwarding), syslog (UDP 514 or, preferably, TCP 514 or TLS on 6514), API pulls from cloud and SaaS services, and file or message-queue ingestion. Agents offer buffering, filtering and encryption, while syslog suits network devices that cannot run agents. Prefer reliable, encrypted transport. UDP syslog drops messages silently under load, which can hide exactly the burst of events you need most.

Pipeline architecture

A robust pipeline has stages: collectors at the edge, a buffering layer such as Kafka or a managed queue to absorb spikes, a processing layer such as Logstash, Fluentd, Vector or Cribl that parses, enriches and filters, and a storage and search platform, either a SIEM or a data lake. Design for failure with retries and back-pressure, for scale with horizontal workers, and for security with TLS and authenticated senders. Keep the original raw event alongside the parsed version so parsers can be fixed and data replayed.

Filtering, cost and time

Ingest volume drives cost, so decide per source what is useful. Drop known noise, such as successful health checks, at the pipeline, but never drop the only copy of security-relevant events. Route high-value events to the SIEM and bulk data to cheaper storage. Time matters too: every device should sync with NTP, log in UTC or record the time zone, and the pipeline should store both event time and ingest time. Without consistent clocks, correlating events across systems becomes unreliable.

A short inventory sheet listing each source, owner, format, volume per day, transport and retention helps keep the design honest and reviewable.

Review the design with the people who will use it. Ask incident responders which questions took longest to answer in the last incident, and check whether the planned pipeline would have captured that evidence. Revisit the source list every quarter as new systems, SaaS tools and cloud accounts appear, so visibility grows with the business instead of falling behind it.

SYSLOG LINE

```
<134>1 2026-03-14T09:21:07Z fw01.example.com firewall - - - action=deny src=203.0.113.50 dst=192.0.2.10
dpt=3389 proto=tcp
# transport: syslog over TLS, port 6514
# pipeline: fw01 -> collector -> queue -> parser -> SIEM
```

WORKED EXAMPLE

A company with 500 staff is designing logging for the first time. It starts with identity (domain controller security events and the cloud sign-in logs), EDR telemetry, DNS and firewall deny logs, and CloudTrail. Firewalls send syslog over TLS to a collector, servers use an agent, and cloud logs are pulled by API. Everything flows through a queue so a SIEM outage does not lose events, and the raw message is stored with the parsed fields.

KEY TERMS

Log source: A system or application that produces events useful for security monitoring.

Syslog: A standard protocol and format for sending log messages over a network.

Agent: Software on a host that collects and forwards logs.

Buffer/queue: A layer such as Kafka that absorbs bursts and prevents data loss when downstream is slow.

NTP: Network Time Protocol, used to keep clocks consistent across systems.

REMEMBER

- Pick sources from the questions an investigator must answer: identity, endpoint, network, cloud.
- Use agents, TLS syslog and APIs with a buffering layer.
- Keep raw events, sync clocks and control volume without dropping critical data.

QUICK CHECK

1. Which log source best answers 'who logged in and from where'?

- A) Printer logs
- B) Identity provider and domain controller logs
- C) Disk SMART data
- D) Screensaver logs

2. Why is UDP syslog risky for security data?

- A) It is encrypted
- B) Messages can be dropped silently
- C) It is too reliable
- D) It needs agents

3. What is the benefit of a queue between collectors and the SIEM?

- A) It deletes duplicates
- B) It absorbs bursts and protects against data loss
- C) It replaces parsing
- D) It removes the need for NTP

4. Why store the raw event as well as parsed fields?

- A) To waste space
- B) So parsers can be corrected and data replayed
- C) Because SIEMs require it
- D) To avoid time sync

Answers: 1-B 2-B 3-B 4-B

1. Authentication logs record the account, time and source.
2. UDP has no delivery guarantee.
3. Buffering handles spikes and downstream outages.
4. Raw data lets you re-parse after fixing mistakes.

SIEM engineering and parsing

Parsing and normalization turn raw text into searchable fields. Broken parsers mean silent blind spots: the event exists but no rule can see its fields.

Parsing raw events

Raw logs arrive in many formats: key=value pairs, JSON, CSV, CEF, LEEF and free text. Parsers extract fields using methods such as Grok patterns, regular expressions, dissect or JSON decoding. A good parser names fields consistently, converts types (strings to integers, timestamps to dates), handles optional fields and survives format changes. Test parsers against a sample library that includes normal events, edge cases and malformed lines, and add a tag or error field for events that fail to parse so failures are visible.

Normalization with common schemas

Different vendors call the source address `src`, `SourceIp`, `client_ip` or `sourceAddress`. A common schema maps them to one name so a single rule works across products. Elastic Common Schema (ECS) uses fields such as `source.ip`, `destination.ip`, `user.name`, `event.action`, `event.outcome` and `host.name`. Splunk's Common Information Model (CIM) uses `src`, `dest`, `user` and `action` in data models. Other options include OCSF. Adopting a schema lets you reuse detection content and compare data from firewalls, proxies and cloud services consistently.

Time, enrichment and reliability

Parse the event timestamp and its time zone carefully; a parser that uses ingest time instead of event time misplaces events and breaks correlation. Enrich events with context: GeoIP for external addresses, asset inventory (owner, criticality), user directory attributes and threat intelligence. Enrichment at ingest speeds up searches but adds cost; enrichment at search time is flexible but slower. Monitor ingestion lag and parse failure rates, and keep parsers and mappings in version control with a review and rollback process.

Field quality and common failures

Typical failures include a vendor update that changes the log format, multiline events split into separate records, missing time zones, truncated messages and fields captured with extra whitespace or quotes. These do not generate errors; they just produce empty fields. Build checks that alert when a field that should always exist, such as `source.ip` in firewall logs, is suddenly null for more than a small percentage of events. Quality dashboards by source catch regressions before a missed detection reveals them.

Keep a small library of test events for every parser, including one valid sample, one with missing optional fields and one deliberately malformed line. Run these tests automatically whenever a parser or mapping changes, so that a well-meant edit for one data source does not break extraction for another source that shares the same pattern.

PARSER CONFIG

```
# raw
Mar 14 09:21:07 srv1 sshd[811]: Failed password for invalid user fake-admin from 203.0.113.50 port 51122
ssh2
# grok
%{SYSLOGTIMESTAMP:ts} %{HOSTNAME:host.name} sshd\[ %{INT:int}\]: Failed password for (invalid user
)?%{USERNAME:user.name} from %{IP:source.ip} port %{INT:source.port}
# result: event.action=logon-failed user.name=fake-admin source.ip=203.0.113.50
```

WORKED EXAMPLE

After a firewall firmware upgrade, a detection for port scans stops firing. Investigation shows the vendor changed `src=` to `source=` in the log, so the parser leaves `source.ip` empty. A field-quality dashboard would have shown the empty rate jump to 100 percent. The engineer updates the Grok pattern to accept both names, replays the stored raw logs from the week, and adds an alert for any critical field exceeding a 2 percent null rate.

KEY TERMS

Parsing: Extracting structured fields from raw log text.

Normalization: Mapping vendor-specific field names to a common schema.

ECS: Elastic Common Schema, a standard set of field names for events.

CIM: Splunk Common Information Model, used to normalize data for shared searches.

Enrichment: Adding context such as geolocation or asset owner to events.

REMEMBER

- Parse carefully with tests, and flag unparsed events.
- Normalize to ECS, CIM or OCSF so content is reusable.
- Fix time handling, enrich events and monitor field quality.

QUICK CHECK

1. What is the purpose of normalization?
 - A) Reducing disk space only
 - B) Letting one rule work across data from different vendors
 - C) Encrypting logs
 - D) Deleting duplicates
2. Which ECS field typically holds the source IP address?
 - A) src_ip_text
 - B) source.ip
 - C) ip_from_log
 - D) addr1
3. Why monitor parse failures and null field rates?
 - A) To find silent blind spots
 - B) To increase ingest
 - C) To avoid NTP
 - D) To disable rules
4. Using ingest time instead of event time causes:
 - A) Better correlation
 - B) Misplaced events and broken timelines
 - C) Faster searches
 - D) Smaller logs

Answers: 1-B 2-B 3-A 4-B

1. Common field names allow shared detections.
2. ECS uses source.ip.
3. Empty fields silently break detections.
4. Event order and correlation depend on true event times.

Detection and alert design

An alert is a request for a person's time. Well-designed detections catch real attacker behavior, carry context, and tell the responder what to do next.

From behavior to rule

Begin with a technique rather than an indicator. MITRE ATT&CK catalogs adversary tactics and techniques, such as T1110 Brute Force, T1078 Valid Accounts and T1059 Command and Scripting Interpreter. For each technique, identify the log evidence it leaves, express it as a query, and map the rule to the ATT&CK ID so coverage can be reported. Behavior-based rules last longer than indicator lists such as single IP addresses, which attackers change easily, though both have a place.

Thresholds, correlation and severity

Many events are only suspicious in aggregate or in sequence. A rule can count failed logons per account in a time window, such as 10 failures in 5 minutes, or correlate a burst of failures followed by a success from the same source, which suggests a guessed or sprayed password. Choose thresholds from baseline data, not guesses. Set severity from impact and confidence: a successful logon after failures on an administrator account is higher severity than failures on a service desk test account.

Context and runbooks

Every alert should include who, what, where and when, plus enrichment such as the asset owner, criticality and the user's usual locations. Attach a runbook: what the rule means, likely false positives, steps to validate, containment actions and who to escalate to. Write the rule description for a tired responder at 3 a.m. Alerts without a defined response should be reviewed, because an alert nobody can act on is noise and wastes analyst attention.

Lifecycle and testing

Treat detections as code: store them in version control, review changes, and test them against sample events and simulated activity, such as an Atomic Red Team test in a lab. Roll out in a low-severity or silent mode, review hits, then promote. Record each rule's owner, data source dependency, last review and measured precision. When a data source fails or a parser breaks, dependent rules go blind, so link detections to health checks of the sources they need.

Review detection quality in a regular meeting. Look at the five noisiest rules, the five rules that never fire, and any incident that was found by chance rather than by an alert. Each of those findings leads to a concrete action: tune a rule, test a rule with simulated activity, or write a new detection to cover the missed technique.

QUERY

```
index=auth action=failure OR action=success
| bin _time span=10m
| stats count(eval(action="failure")) AS fails count(eval(action="success")) AS wins by user, src, _time
| where fails >= 10 AND wins >= 1
| eval technique="T1110", severity="high"
# runbook: confirm with user, check MFA, reset password, block src
```

WORKED EXAMPLE

A team writes a rule for password guessing. It counts failures per source IP and user over 10 minutes and alerts at 10 or more failures followed by a success. It is mapped to T1110 and carries a runbook: contact the user, check MFA prompts, reset the credential and block the source 203.0.113.50 if confirmed. After a week in silent mode, the team sees which accounts, such as a misconfigured scanner service, trigger it and refines the rule.

KEY TERMS

MITRE ATT&CK: A knowledge base of adversary tactics and techniques used to structure detection.

Threshold: A count or rate that must be exceeded before an alert fires.

Correlation: Combining multiple events or sources to reveal a pattern.

Runbook: Documented steps for responding to a specific alert.

Detection as code: Managing rules in version control with review and testing.

REMEMBER

- Build rules from attacker behavior and map them to ATT&CK.
- Use baselines for thresholds and correlation, and set severity by impact and confidence.
- Attach context and runbooks; manage detections as code.

QUICK CHECK

1. Why map rules to ATT&CK techniques?
 - A) To lower log volume
 - B) To report coverage and structure detections by behavior
 - C) To remove thresholds
 - D) To avoid testing
2. Which rule is best at suggesting a successful password guess?
 - A) Single failed logon
 - B) Many failures followed by a success from the same source
 - C) One successful logon
 - D) DNS query count
3. What should every alert include?
 - A) Only a title
 - B) Context and a runbook for response
 - C) A random ID
 - D) No details
4. Why start new rules in silent or low-severity mode?
 - A) To hide them from the team
 - B) To measure false positives before paging people
 - C) To avoid logging
 - D) To reduce parsing

Answers: 1-B 2-B 3-B 4-B

1. Mapping shows what techniques are covered and which are gaps.
2. The sequence indicates guessing that eventually worked.
3. Responders need context and next steps.
4. Observation reveals noise before it harms responders.

Retention, integrity and compliance

Logs are evidence. How long you keep them, where, and how well they are protected decides whether you can investigate an old intrusion or satisfy an auditor.

Hot, warm and cold tiers

Storage tiers balance speed and cost. Hot storage on fast disks holds recent data, often 7 to 30 days, for quick searches and detections. Warm storage holds data for months at lower performance. Cold or archive storage, such as object storage with lifecycle rules, holds data for a year or more at low cost and is slower to restore. Index lifecycle management automates moving data between tiers. Many organizations keep raw logs in cheap storage and only selected, high-value data in the expensive searchable tier.

Regulatory and business minimums

Requirements vary by regulation and contract, so confirm with legal and compliance. PCI DSS requires retaining audit trail history for at least 12 months, with the most recent 3 months immediately available. HIPAA requires documentation retained for 6 years, and many organizations apply that to relevant audit records. Government frameworks and sector rules set other periods. Privacy laws such as GDPR add the opposite pressure: do not keep personal data longer than needed, so define retention per log type and justify it.

Protecting log integrity

Attackers who gain access often try to erase or alter logs. Defenses include forwarding logs off the host in near real time, write-once or immutable storage (object lock or WORM), restricted admin access with separation of duties so system administrators cannot delete security logs, hashing or signing log batches, and audit logging of the logging platform itself. Encrypt data in transit and at rest, and monitor for logging being stopped, such as Windows event 1102 for a cleared security log or CloudTrail StopLogging.

Practical retention planning

Create a retention schedule listing each log type, its purpose, regulatory driver, hot, warm and cold durations, and owner. Estimate daily volume and multiply by retention to forecast cost, then compare options such as compression, sampling of low-value data and tiering. Test restores from cold storage regularly, because an archive nobody can read is not retention. Define how legal holds suspend deletion, and document destruction at the end of the period so the process is defensible.

Be careful that privacy and security goals agree. Logs often contain usernames, IP addresses and sometimes sensitive values inside URLs or command lines. Mask or hash fields you do not need, restrict who can search the raw data, and record who accessed it. Doing so reduces legal exposure while keeping the data useful for investigations.

POLICY TABLE

Type	Hot	Warm	Cold	Driver
Auth/IdP	30d	90d	12mo	PCI, investigations
Firewall	14d	60d	12mo	investigations
EDR	30d	90d	12mo	threat hunting
DNS	7d	30d	6mo	cost
CloudTrail	30d	90d	7y	audit (object lock on)

WORKED EXAMPLE

A retailer must meet PCI DSS and control cost. Authentication, EDR and cloud audit logs keep 30 days hot and 90 days warm, then move to object storage with lock enabled for a total of 12 months or more. High-volume DNS logs keep a shorter window. A quarterly exercise restores one month of cold data to confirm that it can be queried, and an alert fires if a host clears its event log or CloudTrail stops delivering.

KEY TERMS

Hot storage: Fast, expensive storage for recent, frequently searched data.

WORM: Write once, read many storage that prevents alteration or deletion.

Retention schedule: A documented list of how long each log type is kept and why.

Legal hold: A requirement to preserve data relevant to a legal matter, overriding normal deletion.

Log integrity: Assurance that logs are complete and have not been altered.

REMEMBER

- Use hot, warm and cold tiers matched to value and regulation.
- Protect logs with off-host forwarding, immutability and separation of duties.
- Document a retention schedule, test restores and honor legal holds.

QUICK CHECK

1. Which control best protects logs from tampering by an attacker on the source host?

- A) Local storage only
- B) Forwarding to a separate, immutable store
- C) Disabling logging
- D) Short retention

2. PCI DSS audit trail history requirement is at least:

- A) 1 week
- B) 12 months
- C) 10 years
- D) No requirement

3. Why test restores from cold storage?

- A) To increase costs
- B) To confirm archived data is actually usable
- C) To delete logs
- D) To change schemas

4. Which event suggests an attacker may be hiding activity?

- A) A scheduled backup
- B) Windows Security log cleared (1102)
- C) User login
- D) DHCP lease

Answers: 1-B 2-B 3-B 4-B

1. Off-host immutable copies survive host compromise.
2. PCI DSS requires 12 months with 3 months immediately available.
3. Unrestorable archives fail an investigation or audit.
4. Clearing audit logs is a common defense-evasion step.

Metrics and detection coverage

You cannot improve what you do not measure. Metrics show whether monitoring is actually working and where to invest next.

Log source health

The first metric is whether data is arriving. Track, per source, the last event time, events per hour against a baseline, ingestion lag and parse failure rate. A source that goes silent is a high-priority alert, because everything depending on it is blind. Maintain a coverage list of the expected population, such as all domain controllers or every EDR-enabled laptop, and compare it to who actually reported in the last 24 hours. Report the percentage of assets logging, and name the owners of the gaps.

Time-based metrics: MTTD and MTTR

Mean time to detect (MTTD) measures the gap between the start of malicious activity and its detection; mean time to respond or remediate (MTTR) measures the gap between detection and containment or resolution. Use medians as well as means, since a few long incidents skew averages. Track also time to acknowledge an alert and time to triage. Purple team exercises give reliable start times for measuring MTTD, because the exercise team knows exactly when each action occurred.

Alert quality metrics

The false positive rate is the share of alerts judged benign. Precision is the share of alerts that are true positives, and analysts' closure reasons feed these numbers. Also track alert volume per analyst per shift, the percentage of alerts closed without action, and rules producing the most noise. Be careful with a single target: pushing false positives to zero can hide real attacks through over-tuning. Review high-noise rules for tuning and low-fire rules for validation that they still work.

ATT&CK coverage

Map detections to ATT&CK techniques and visualize them with the ATT&CK Navigator. Coverage means a rule exists and its data source is healthy and tested, not merely that a rule is written. Prioritize techniques used by threat groups relevant to your sector and techniques that your own incidents or red team exercises show to be gaps. Present a dashboard with source health, MTTD, false positive rate and coverage, and use it in regular reviews to drive decisions rather than as a static report.

Share metrics with the right audience. Executives need a short trend of coverage and detection time, engineers need the list of silent sources and noisy rules, and auditors need evidence that controls operate continuously. Automating the report from the platform itself keeps the numbers honest and removes the monthly scramble to assemble slides.

REPORT EXCERPT

```
Source Expected Reporting Last event Status
Domain ctrls 4 4 09:20Z OK
EDR laptops 520 498 - 22 silent
Firewalls 6 5 08:02Z fw03 SILENT 75m
Metric Value
Median MTTD 38 min
False pos rate 41% (target <30%)
ATT&CK covered 57 of 120 priority techniques
```

WORKED EXAMPLE

A monthly review shows that firewall fw03 has been silent for 75 minutes and 22 laptops are not reporting EDR data. A rule that detects credential dumping has a 41 percent false positive rate, driven by one backup tool, so it is tuned with an allow-list. Coverage is 57 of 120 priority techniques, and the team picks the five techniques most relevant to recent threat reports for new detections, assigning owners and due dates.

KEY TERMS

MTTD: Mean time to detect: the average time between malicious activity starting and detection.

MTTR: Mean time to respond or remediate after detection.

False positive rate: The share of alerts that turn out to be benign.

ATT&CK Navigator: A tool for visualizing coverage across ATT&CK techniques.

Silent source: A log source that has stopped sending expected data.

REMEMBER

- Monitor source health first: silence means blindness.
- Track MTTD, MTTR, false positive rate and alert volume with medians.
- Map detections to ATT&CK and verify data and tests before claiming coverage.

QUICK CHECK

1. What is the highest-priority log source health signal?
 - A) A source going silent
 - B) A new field name
 - C) A weekend
 - D) A larger disk
2. MTTD measures:
 - A) Time from attack start to detection
 - B) Time to buy tools
 - C) Disk speed
 - D) Number of rules
3. Why is pushing the false positive rate to zero risky?
 - A) It reduces coverage and can hide true attacks
 - B) It improves speed
 - C) It adds logs
 - D) It is impossible to measure
4. Detection coverage should count a technique as covered only when:
 - A) A rule exists
 - B) A tested rule exists and its data source is healthy
 - C) It is in a spreadsheet
 - D) A vendor says so

Answers: 1-A 2-A 3-A 4-B

1. A silent source means blindness in anything that depends on it.
2. It is the delay before the attack is noticed.
3. Over-tuning may suppress real malicious activity.
4. A rule is useless if its data is missing or untested.

Fix a parser

SCENARIO

A new web proxy was onboarded, but the normalized events show empty source.ip and user.name fields, so detections do not fire. You have a log viewer showing raw lines and the parsed result, with a parser editor.

OBJECTIVE

Correct the field extraction so the source IP and user appear in the normalized event.

STEPS IN THE SIMULATOR

- 1 **Do:** Open a raw event

Simulator shows:

```
ts=2026-03-14T09:21:07Z client=203.0.113.50 usr=fake-user-c url=https://example.com/a status=200
```

- 2 **Do:** Open the parsed event

Simulator shows:

```
source.ip: (empty), user.name: (empty), url.full: https://example.com/a, event.outcome: success
```

- 3 **Do:** Open the parser editor and view the current pattern

Simulator shows:

```
Pattern expects: src=%{IP:source.ip} user=%{WORD:user.name}
```

- 4 **Do:** Change src= to client=

Simulator shows:

```
Pattern updated: client=%{IP:source.ip}
```

- 5 **Do:** Change user= to usr= and use %{USERNAME:user.name}

Simulator shows:

```
Pattern updated: usr=%{USERNAME:user.name} (accepts hyphens)
```

- 6 **Do:** Click Test on the sample event

Simulator shows:

```
source.ip: 203.0.113.50, user.name: fake-user-c, url.full: https://example.com/a
```

- 7 **Do:** Click Replay last 24h raw events

Simulator shows:

```
Replayed 18,422 events. Parse failure rate: 0.0%.
```

ANSWER

The parser looked for src= and user=, but the proxy writes client= and usr=. After changing the field names and allowing hyphens in the username, the normalized event shows source.ip 203.0.113.50 and user.name fake-user-c, and replaying stored raw events backfilled history.

Hint shown in the app: Compare the raw key names with the names in the pattern.

Write a detection

SCENARIO

You are building a rule to catch password guessing that eventually succeeds. The SIEM has an auth index with fields user, src, action and a 10-minute window available. Sample data contains one attack and some normal users.

OBJECTIVE

Build a query for repeated failed logins followed by a success and set a threshold.

STEPS IN THE SIMULATOR

- 1 **Do:** Run: `index=auth action=failure | stats count by user, src`

Simulator shows:

```
fake-user-a 203.0.113.50 47; fake-user-d 198.51.100.7 3; fake-user-e 192.0.2.44 2.
```

- 2 **Do:** Pick a window: `bin _time span=10m`

Simulator shows:

```
Window applied. 3 groups remain.
```

- 3 **Do:** Add stats counting failures and successes per user and src

Simulator shows:

```
fake-user-a 203.0.113.50 fails=47 wins=1; others wins=1 fails<=3.
```

- 4 **Do:** Set threshold: `where fails >= 10`

Simulator shows:

```
1 result: fake-user-a from 203.0.113.50.
```

- 5 **Do:** Add: `where wins >= 1`

Simulator shows:

```
1 result remains. This sequence suggests a guessed password.
```

- 6 **Do:** Set severity high and tag technique T1110

Simulator shows:

```
Rule saved with tag T1110, severity high.
```

- 7 **Do:** Click Backtest on the last 7 days

Simulator shows:

```
3 hits: 1 true positive, 2 benign (a mistyped password at start of shift). Estimated precision 33%.
```

ANSWER

The rule counts failures and successes per user and source in a 10-minute window and alerts when there are at least 10 failures and at least one success. It flags fake-user-a from 203.0.113.50, mapped to T1110. Backtesting shows some benign hits, so the threshold or context should be refined.

Hint shown in the app: You need both counts in the same window: failures above the threshold and at least one success.

Tune a noisy alert

SCENARIO

The rule 'Suspicious PowerShell Encoded Command' fires 200 times per day. Analysts close most as benign. You must reduce noise without losing real detections. You can view alert samples and edit the rule.

OBJECTIVE

Reduce false positives with allow-lists and context while keeping true positives.

STEPS IN THE SIMULATOR

- 1 **Do:** Open 24h alert breakdown by host and parent process

Simulator shows:

180 alerts: host mgmt01, parent=ccmclient.exe (software deployment). 15 alerts: various user laptops. 5 alerts: parent=winword.exe.

- 2 **Do:** Inspect mgmt01 events

Simulator shows:

Command line signed by the IT deployment tool, run by account svc-deploy on a fixed schedule.

- 3 **Do:** Add allow-list: host=mgmt01 AND user=svc-deploy AND parent=ccmclient.exe

Simulator shows:

Rule updated. Estimated alerts per day: 20.

- 4 **Do:** Inspect the 5 winword.exe events

Simulator shows:

Encoded command downloading from 203.0.113.80. Confirmed suspicious by analyst notes.

- 5 **Do:** Add context: raise severity to critical when parent is an Office application

Simulator shows:

Severity logic saved.

- 6 **Do:** Run backtest on 7 days of data

Simulator shows:

Hits: 138 -> 21. All 5 known true positives retained.

- 7 **Do:** Set the allow-list entry to expire in 90 days with owner IT-Ops

Simulator shows:

Expiry set. Reminder ticket created.

ANSWER

The noise came from an approved deployment tool on one host and account. A narrow allow-list on host, user and parent process removed about 90 percent of alerts, while a severity rule for Office-spawned PowerShell kept the five true positives. The allow-list has an owner and expiry so it is reviewed.

Hint shown in the app: Allow-list the specific, verified behavior rather than the entire rule or host.

Plan retention

SCENARIO

A company must satisfy PCI DSS, keep investigation ability for a year, and cut its SIEM storage bill by 30 percent. You are given four log types with daily volumes and must set hot, warm and cold periods.

OBJECTIVE

Assign retention periods to log types that meet compliance and cost goals.

STEPS IN THE SIMULATOR

- 1 Do:** Review the table: Auth 5 GB/day, EDR 40 GB/day, DNS 60 GB/day, CloudTrail 8 GB/day
Simulator shows:
Current plan: everything hot for 365 days. Cost index 100.

- 2 Do:** Set Auth: 30 hot, 90 warm, 12 months total
Simulator shows:
Compliance check: PCI audit trail OK. Cost index 92.

- 3 Do:** Set CloudTrail: 30 hot, 90 warm, then cold with object lock for 7 years
Simulator shows:
Compliance check: audit OK. Integrity: immutable.

- 4 Do:** Set EDR: 30 hot, 90 warm, 12 months total
Simulator shows:
Cost index 70.

- 5 Do:** Set DNS: 7 hot, 30 warm, 6 months cold
Simulator shows:
Warning: shorter than 12 months. Justification needed: low investigative value. Cost index 61.

- 6 Do:** Add control: restore test from cold every quarter
Simulator shows:
Control recorded.

- 7 Do:** Click Evaluate plan
Simulator shows:
Compliance: PASS. Cost reduction: 39% (target 30%). Note: document the DNS exception.

ANSWER

Keep authentication, EDR and cloud audit data for at least 12 months with short hot periods and immutable cold storage for audit logs, and keep only high-volume DNS for a shorter documented period. This meets the PCI requirement, exceeds the 30 percent saving, and includes quarterly restore tests.

Hint shown in the app: Move most data to cheaper tiers quickly and shorten only low-value, high-volume sources with a documented reason.

Logging and Monitoring Architecture: 10 questions, 80% to pass

1. Which source type answers 'who authenticated'?
 - A) Identity provider logs
 - B) Printer logs
 - C) Backup logs only
 - D) Disk logs
2. Why prefer TCP or TLS syslog to UDP?
 - A) It is slower
 - B) It reduces silent loss and can encrypt
 - C) It avoids parsing
 - D) It needs no network
3. Which ECS field name holds a source address?
 - A) source.ip
 - B) ipSrc
 - C) srcAddrText
 - D) from
4. A parser silently outputs empty fields after a firmware upgrade. Best control?
 - A) Ignore it
 - B) Alert on null rates for critical fields
 - C) Remove the source
 - D) Lower retention
5. Which approach to rule design lasts longest against attackers?
 - A) Single IP indicators only
 - B) Behavior-based rules mapped to techniques
 - C) Random keywords
 - D) Alerting on every event
6. A good alert includes:
 - A) Only a title
 - B) Context plus a runbook
 - C) No owner
 - D) A password
7. Which control best protects against log deletion by an admin on the source host?
 - A) Off-host immutable storage
 - B) Longer hot retention
 - C) More parsers
 - D) Larger disks
8. PCI DSS audit history minimum is:
 - A) 30 days
 - B) 12 months
 - C) 5 years
 - D) Not specified
9. What does a rising median MTTD indicate?
 - A) Detection is getting faster
 - B) Detection is getting slower
 - C) More logs
 - D) Cheaper storage
10. Which counts as real ATT&CK coverage?
 - A) A rule name in a spreadsheet
 - B) A tested rule with a healthy data source
 - C) A vendor logo
 - D) An untested query

Answers: 1-A 2-B 3-A 4-B 5-B 6-B 7-A 8-B 9-B 10-B

1. Authentication events show accounts and sources.
2. UDP can drop messages without notice.
3. ECS uses source.ip.
4. Field-quality monitoring catches silent regressions.
5. Behaviors are harder to change than indicators.
6. Responders need context and next steps.
7. Immutable remote copies cannot be altered from the source.
8. PCI DSS requires at least 12 months.
9. A larger MTTD means longer time before attacks are noticed.
10. Coverage needs data and validated logic.

Vulnerability and Patch Programs

Intermediate | 5 lessons | 4 simulated labs | checkpoint | about 4h 25m

A vulnerability program turns scanning into reduced risk. This module covers building an asset inventory with owners, planning scans so coverage is real without breaking services, and prioritizing findings with CVSS, EPSS, the CISA Known Exploited Vulnerabilities catalog, exposure and asset criticality. You will then define remediation SLAs, report on aging, and run exception processes with expiry, approvals and compensating controls.

BY THE END YOU CAN

- + Build an asset inventory that drives scanning
- + Prioritize vulnerabilities with CVSS, EPSS and KEV
- + Run SLA and exception processes

IN THIS MODULE

Lesson 1 Asset inventory and attack surface

Lesson 2 Scanning cadence and coverage

Lesson 3 Risk-based prioritization

Lesson 4 Remediation SLAs and reporting

Lesson 5 Exceptions and compensating controls

Lab 1 Reconcile inventory (File explorer)

Lab 2 Prioritize the backlog (Scenario)

Lab 3 Read a scan report (Log viewer)

Lab 4 Handle an exception (Chat sim)

Checkpoint 10 questions, 80% to pass

Asset inventory and attack surface

You cannot patch what you do not know exists. An accurate inventory with owners decides whether scans cover everything and whether a finding reaches someone who can fix it.

Why inventory comes first

Every vulnerability program rests on knowing what assets exist, what they do and who owns them. Missing assets are never scanned or patched, and the forgotten server, test database or cloud instance is a favorite entry point for attackers. Inventory also supplies context for prioritization: a flaw on an internet-facing payment server matters more than the same flaw on an isolated lab machine. Aim to track hardware, operating systems, software, cloud resources, containers, SaaS applications and network devices.

Combining data sources

No single source is complete. A configuration management database (CMDB) holds intended assets and ownership but is often stale. Cloud provider APIs list instances, buckets and databases accurately in real time. Endpoint agents, EDR and MDM report managed devices. Directory services show domain-joined machines. Network discovery scans, DHCP leases and NetFlow reveal what is actually on the wire. Reconcile these sources by a stable key such as hostname, MAC address, cloud instance ID or serial number, and treat differences as findings.

Owners, criticality and attack surface

Each asset needs an owner team, a business criticality rating, a data classification and an environment tag (production, test). Without an owner, tickets go nowhere. External attack surface management searches from the outside: domains, subdomains, open ports, certificates and cloud storage visible to the internet, including shadow IT that teams created without notification. Compare that external view to the internal inventory. Anything reachable from the internet that is not in the inventory deserves prompt investigation.

Keeping it accurate

Automate reconciliation on a schedule and measure the results: the percentage of assets with an owner, the number of unmanaged devices, and the number of assets seen by scanners but absent from the CMDB. Tag cloud resources at creation through policy so ownership is enforced from the start. Retire records when assets are decommissioned. Set an expectation that new systems register before going live, and use discovery as a safety net rather than the primary process.

Review stale records: an asset not seen for 30 to 60 days is either gone or hiding, and either answer needs action.

RECONCILIATION OUTPUT

```
Source Count
CMDB 1,204
Cloud API 1,311
Scanner 1,289
EDR 1,150
In scanner, not in CMDB: 94 (61 no owner)
In CMDB, not scanned 30d: 9
External: vpn-old.example.com 203.0.113.60 (not in inventory)
```

WORKED EXAMPLE

A reconciliation job compares scanner results with the CMDB and finds 94 hosts that scanned successfully but are not in the CMDB, 61 with no owner. It also finds an old VPN gateway visible from the internet at 203.0.113.60 that no one has listed. The team assigns the cloud tagging policy to enforce an owner tag, opens tickets with the likely owning teams, and schedules the old gateway for emergency review because it could be unpatched.

KEY TERMS

CMDB: Configuration management database that records assets and their relationships.

Asset owner: The person or team accountable for an asset and its remediation.

Attack surface: The set of systems and entry points reachable by an attacker.

Shadow IT: Systems or services used without formal approval or registration.

Reconciliation: Comparing multiple data sources to find gaps and mismatches.

REMEMBER

- Inventory must cover hosts, cloud, containers and SaaS, with owners and criticality.
- Combine CMDB, cloud APIs, agents and discovery, then reconcile.
- Include external attack surface and measure inventory quality.

QUICK CHECK

1. Why is an inventory the first step of a vulnerability program?

- A) It speeds scanners
- B) Unknown assets are never scanned or patched
- C) It replaces patching
- D) It is required by TCP

2. Which source is generally most current for cloud resources?

- A) A spreadsheet
- B) The cloud provider API
- C) A printed list
- D) Last year's CMDB export

3. An internet-facing host not in the inventory should be:

- A) Ignored
- B) Investigated promptly
- C) Assumed safe
- D) Scanned only yearly

4. Why record an owner for each asset?

- A) For decoration
- B) So findings can be routed to someone who can fix them
- C) To reduce disk use
- D) For DNS

Answers: 1-B 2-B 3-B 4-B

1. Coverage depends on knowing what exists.
2. APIs reflect the real, current state.
3. Unknown exposed assets are high risk.
4. Without owners remediation tickets stall.

Scanning cadence and coverage

Scans are your measurement tool. The method, frequency and credentials you choose determine whether results are accurate and whether scanning itself causes outages.

Authenticated versus unauthenticated scans

An unauthenticated scan probes from the network, as an outsider would, and sees open ports, banners and exposed services. It is good for the perimeter view but misses installed software and missing patches. An authenticated (credentialed) scan logs in using SSH, WinRM or SMB with a dedicated read-only account and inspects the registry, package lists and file versions. It finds far more vulnerabilities and gives fewer false positives. Use authenticated scans internally and unauthenticated scans for the external view.

Agents, containers and cloud

Network scans struggle with laptops that are off the corporate network, ephemeral cloud servers and containers. Agents installed on endpoints report vulnerabilities wherever the device is, and are good for remote workers. Cloud workloads can be assessed by snapshot-based scanning or agentless platform tools, and container images should be scanned in the registry and pipeline. Web applications need dynamic testing (DAST) and dependency analysis (SCA), which network vulnerability scanners do not replace.

Cadence and scan windows

Common practice is to scan internet-facing assets daily or weekly, internal servers weekly, and workstations through agents continuously, with a full discovery scan at least monthly. Compliance frameworks, such as PCI DSS, require internal and external scans at least quarterly and after significant changes. Schedule heavy scans in maintenance windows, throttle the number of concurrent hosts and requests, and exclude fragile systems such as legacy industrial controllers from intrusive checks. Coordinate with owners so an unexpected scan is not mistaken for an attack.

Quality and coverage metrics

Measure scan coverage as the percentage of inventory assets scanned successfully within the cadence, and authentication success as the share of assets where credentials worked. A failed login means the host was only partly assessed, which creates a false sense of safety. Use dedicated, vaulted scan credentials with least privilege, allow-list scanner IPs in firewalls and EDR, and verify that scanner traffic is not blocked. Keep scanner plugins updated so new vulnerabilities are recognized.

Review scans that time out repeatedly, since they often hide unstable or overloaded systems that deserve attention.

SCAN SUMMARY

```
Scan: internal-weekly Targets: 820 Completed: 781
Auth success: 702 Auth failed: 79 Unreachable: 39
Top auth failures: SSH key rejected (41), WinRM refused (28), timeout (10)
Scan window: Sat 01:00-05:00 UTC, max 20 hosts concurrent, source 192.0.2.5
```

WORKED EXAMPLE

A weekly scan reports 781 of 820 targets complete, but authentication failed on 79 hosts. The team finds that 41 Linux hosts rejected the scan key after a rotation and that 28 Windows hosts block WinRM from the scanner subnet. They fix credentials and firewall rules, then rerun only the failed targets. Coverage then rises from 86 to 98 percent authenticated, and the 39 unreachable hosts are checked against the inventory to see which are decommissioned.

KEY TERMS

Authenticated scan: A scan that logs in to a host to inspect installed software and patches.

Agent-based scanning: Using software on the endpoint to report vulnerabilities from wherever it is.

Scan window: A scheduled period when scans may run, chosen to limit impact.

DAST: Dynamic application security testing that probes a running web app.

Scan coverage: The percentage of assets successfully scanned within the required cadence.

REMEMBER

- Use authenticated scans internally and unauthenticated scans for the external view.
- Add agents, cloud and container scanning where network scans cannot reach.
- Track coverage and authentication success, not just scan completion.

QUICK CHECK

1. What does an authenticated scan provide that an unauthenticated one usually cannot?
 - A) Public DNS names
 - B) Installed software and patch levels
 - C) Router MAC addresses
 - D) Faster results only
2. Which assets are best covered by agents?
 - A) Remote laptops often off the network
 - B) A firewall in a data center
 - C) A printer
 - D) A switch
3. A scan shows many authentication failures. This means:
 - A) Hosts are safe
 - B) Hosts were only partly assessed
 - C) Scanner works perfectly
 - D) Coverage is 100%
4. Why throttle scans and use windows?
 - A) To make results longer
 - B) To avoid disrupting fragile services
 - C) To avoid inventory
 - D) To hide results

Answers: 1-B 2-A 3-B 4-B

1. Credentialed access reveals local package and patch data.
2. Agents report from wherever the device is.
3. Without a login, local vulnerabilities are not detected.
4. Heavy scanning can slow or crash systems.

Risk-based prioritization

Thousands of findings cannot be fixed at once. Prioritizing by real-world risk lets a small team reduce the most danger first, instead of chasing every high CVSS number.

CVSS and its limits

The Common Vulnerability Scoring System rates the technical severity of a vulnerability from 0.0 to 10.0, with bands: Low 0.1-3.9, Medium 4.0-6.9, High 7.0-8.9, Critical 9.0-10.0. The base score reflects attack vector, complexity, privileges required, user interaction and impact on confidentiality, integrity and availability. It does not measure whether anyone is exploiting the flaw or how important your asset is. Treating all scores of 7 and above as equally urgent produces long queues, because a large share of high-scoring vulnerabilities are never exploited.

EPSS and the KEV catalog

The Exploit Prediction Scoring System (EPSS) estimates the probability, from 0 to 1, that a vulnerability will be exploited in the wild within the next 30 days, using data on exploit activity and vulnerability characteristics. The CISA Known Exploited Vulnerabilities (KEV) catalog lists CVEs with confirmed active exploitation and is binding, with due dates, for US federal agencies. A KEV listing is a strong signal to move that vulnerability to the front of the queue regardless of its CVSS score.

Exposure and asset criticality

Context completes the picture. Is the vulnerable service reachable from the internet, or only from a restricted network? Does a public exploit or a working proof-of-concept exist? Is a compensating control, such as a web application firewall rule or EDR protection, in place? How critical is the asset: does it hold regulated data, support revenue, or act as a domain controller? Combine these factors into a priority tier, as frameworks such as SSVC (Stakeholder-Specific Vulnerability Categorization) formalize with decisions such as Track, Attend, Act.

A practical ranking approach

A simple model is to rank first by KEV status, then by EPSS and exposure, then by asset criticality, using CVSS only as a tiebreaker. For example, a KEV-listed flaw on an internet-facing server is an emergency. A critical CVSS flaw with EPSS 0.01 on an internal, low-value host can wait for the normal cycle. Document the rules so decisions are consistent and defensible, and review them periodically. Remember that prioritization chooses order, not who is exempt: everything still needs a plan.

Re-score continuously, because EPSS and KEV change as exploitation evidence appears.

PRIORITY TABLE

CVE	CVSS	EPSS	KEV	Exposed	Asset	Priority
CVE-2026-1111	9.8	0.02	no	no	lab-vm	P3
CVE-2026-2222	7.5	0.91	yes	yes	web-gw	P1 (act now)
CVE-2026-3333	8.8	0.40	no	yes	crm-app	P2
CVE-2026-4444	5.3	0.01	no	no	intranet	P4

WORKED EXAMPLE

A backlog lists CVE-2026-1111 with CVSS 9.8 on an isolated lab VM and CVE-2026-2222 with CVSS 7.5 on an internet-facing gateway. The second has EPSS 0.91 and appears in KEV. Despite the lower score, it is the first to be fixed, with an emergency change. The lab VM flaw is scheduled in the normal cycle, and the team documents why. Within two days the exploited flaw is patched and the gateway is rescanned to confirm.

KEY TERMS

- CVSS:** A standard scoring system for the technical severity of vulnerabilities, from 0.0 to 10.0.
- EPSS:** A model estimating the probability that a vulnerability will be exploited in the next 30 days.
- KEV:** CISA's catalog of vulnerabilities known to be exploited in the wild.
- SSVC:** A decision framework that prioritizes vulnerabilities using context and outcomes.
- Compensating control:** A safeguard that reduces risk when a vulnerability cannot be patched immediately.

REMEMBER

- CVSS measures severity; EPSS measures likelihood; KEV confirms exploitation.
- Add exposure, compensating controls and asset criticality.
- Use a documented ranking and re-score as data changes.

QUICK CHECK

1. What does EPSS estimate?
 - A) Impact on confidentiality
 - B) Probability of exploitation in the near term
 - C) Patch size
 - D) CVSS vector
2. A CVE listed in CISA KEV means:
 - A) It has never been exploited
 - B) Exploitation in the wild is confirmed
 - C) It is low risk
 - D) It has no patch
3. Why is CVSS alone a poor prioritization method?
 - A) It ignores exploitation and asset context
 - B) It is not numeric
 - C) It is secret
 - D) It only covers Linux
4. Which finding is typically most urgent?
 - A) Critical CVSS, isolated lab, EPSS 0.01
 - B) Medium CVSS, internet-facing, in KEV
 - C) Low CVSS, offline
 - D) Informational banner

Answers: 1-B 2-B 3-A 4-B

1. EPSS predicts the chance of exploitation in the next 30 days.
2. KEV lists vulnerabilities with known active exploitation.
3. CVSS measures severity, not likelihood or business importance.
4. Active exploitation plus exposure drives urgency.

Remediation SLAs and reporting

Prioritization means little unless fixes happen on time. SLAs, tickets and clear reporting create accountability across teams.

Defining SLAs

A remediation SLA sets the maximum time to fix or mitigate a vulnerability by priority. A typical example is: emergency (KEV or actively exploited and exposed) within 48 hours to 7 days, critical within 15 days, high within 30 days, medium within 60 to 90 days, low within 180 days or the next cycle. Align timings with obligations such as CISA's KEV due dates or PCI DSS, which expects critical patches within a month of release. Count the clock from detection or vendor release, and say clearly which.

Tickets, ownership and workflow

Findings must reach owners as actionable tickets, not as raw scanner exports. Group by asset or by fix, such as 'update package X on these 40 hosts', to reduce the number of tickets. Each ticket includes the affected assets, the CVE, the priority, the due date, the fix guidance and the validation step. Integrate scanners with the ticketing system, such as Jira or ServiceNow, and automate closure when a rescan confirms the fix. Work with patch and change management so approved maintenance windows exist.

Metrics and aging

Track mean and median time to remediate by severity, SLA compliance percentage, the count of open findings by age band (for example 0-30, 31-60, 61-90, 90+ days), and the number of overdue critical items. Show trends rather than one-time counts, and normalize by asset count so large teams are not unfairly compared. Watch for the vulnerability half-life, the time taken to remove half of new findings, as an indicator of process health. Report KEV items separately because they have the strongest risk signal.

Reporting to different audiences

Engineers need lists of what to fix and by when. Team leads want a view of their team's overdue items and trends. Executives need a short summary: overall risk trend, SLA compliance, top exposed systems and decisions required, such as resourcing for a legacy platform. Use dashboards that update automatically and a monthly review with owners of the worst-performing areas. Escalate overdue critical items through a defined path, and celebrate teams who improve, since recognition sustains cooperation better than blame alone.

Use root-cause analysis for repeated misses, for instance a base image that is never rebuilt.

DASHBOARD EXCERPT

```
Priority SLA Open Overdue Median TTR
P1 (KEV) 7d 3 1 4d
Critical 15d 41 9 19d
High 30d 212 44 33d
Medium 90d 640 58 61d
Aging (critical+high): 0-30d 140 | 31-60d 62 | 61-90d 31 | 90d+ 20
```

WORKED EXAMPLE

A monthly report shows critical SLA compliance at 78 percent. Drilling down, nine overdue critical findings are mostly in one team's container images that have not been rebuilt in months. Instead of chasing nine tickets, the security team works with the team to automate weekly base-image rebuilds. They report the change to leadership, and in the following month overdue critical items fall to two.

KEY TERMS

SLA: A service level agreement setting the time allowed to remediate by priority.

Time to remediate: The elapsed time from detection to verified fix.

Aging: How long open findings have been unresolved, grouped in bands.

SLA compliance: The percentage of findings fixed within their SLA.

Rescan validation: Scanning again to confirm that a fix worked before closing a ticket.

REMEMBER

- Set SLAs by priority and align with regulations and KEV due dates.
- Deliver actionable tickets, grouped by fix, to named owners.
- Report aging, SLA compliance and trends to each audience, and fix root causes.

QUICK CHECK

1. Which finding class should normally have the shortest SLA?

- A) Low severity internal
- B) Actively exploited and exposed (KEV)
- C) Informational
- D) Documentation issues

2. Why group findings by fix?

- A) It increases tickets
- B) It reduces ticket volume and effort
- C) It hides risk
- D) It removes SLAs

3. What does aging show?

- A) Hardware age
- B) How long findings have stayed open
- C) Network speed
- D) Employee tenure

4. Best way to close a vulnerability ticket?

- A) When the owner says done
- B) After a rescan confirms the fix
- C) After 30 days
- D) When priority changes

Answers: 1-B 2-B 3-B 4-B

1. Active exploitation on exposed assets demands the fastest response.
2. One action can resolve many findings.
3. Aging bands reveal stuck or neglected items.
4. Verification prevents false closure.

Exceptions and compensating controls

Some vulnerabilities cannot be fixed on time. A controlled exception process manages that risk openly instead of leaving it as silent, permanent debt.

When exceptions are legitimate

Reasons include legacy systems that vendors no longer patch, patches that break critical applications, validated medical or industrial devices that cannot be changed without recertification, and short-term business constraints such as a year-end freeze. An exception is a formal, documented decision to accept a known risk for a limited time. It is not a way to avoid work: someone with authority must consciously accept the risk, and the vulnerability remains on the books.

What a good exception record contains

Each request lists the vulnerability and affected assets, the business reason, the risk if exploited, the compensating controls, the requester and risk owner, the approver, the start date and, importantly, an expiry date, typically 30, 90 or at most 180 days. It also defines a plan for permanent remediation, such as replacement or vendor upgrade, with a target date. Approval levels should scale with risk: a low-risk exception can be approved by a manager, while an exposed critical flaw needs a senior risk owner.

Compensating controls

A compensating control reduces the likelihood or impact of exploitation when the fix is not applied. Examples include network segmentation or a firewall rule limiting access to a few hosts, a web application firewall or IPS virtual patch for a known exploit pattern, disabling the vulnerable feature or service, enhanced monitoring with specific detection rules, stronger authentication, and removing internet exposure. A good compensating control addresses the same attack path, is verified to work, and is monitored. A promise to be careful is not a control.

Running the process

Maintain an exception register with owners and expiry dates, send reminders before expiry, and require re-approval with fresh evidence rather than automatic renewal. Re-evaluate when new information arrives, for example when the CVE is added to KEV or an exploit is published, because that may invalidate the original risk acceptance. Report the number of active exceptions, those expiring soon and those renewed repeatedly. Repeated renewals signal a systemic problem that needs funding or architectural change, not another extension.

Test that compensating controls still work, such as checking the firewall rule or WAF signature after changes.

EXCEPTION RECORD

```
ID: EXC-0192 Status: Approved
Vuln: CVE-2026-5555 on erp-legacy (192.0.2.80), CVSS 8.1, EPSS 0.12, not in KEV
Reason: vendor patch needs 6-week migration
Compensating: access limited to 192.0.2.0/28 via firewall; IPS signature 90112 enabled; alert rule
ERP-EXPLOIT-01
Risk owner: fake-owner (Finance IT) Expires: 2026-09-30
Plan: upgrade to v12 by 2026-09-15
```

WORKED EXAMPLE

The finance team asks to delay patching erp-legacy for six weeks because the upgrade needs testing. The security analyst agrees to a time-boxed exception only if access is restricted to a small admin subnet, an IPS signature is enabled, and a detection rule is added. The risk owner signs, the register records the expiry date, and a reminder is set. Two weeks later the CVE is added to KEV, so the analyst reopens the decision and shortens the exception.

KEY TERMS

Risk acceptance: A formal decision by an accountable owner to tolerate a known risk.

Exception expiry: The date when an exception ends and must be remediated or re-approved.

Virtual patching: Using IPS or WAF rules to block exploitation without changing the vulnerable software.

Exception register: A tracked list of all approved exceptions with owners and expiry dates.

Risk owner: The business person accountable for a risk and its acceptance.

REMEMBER

- Exceptions are documented, approved, time-boxed risk acceptance.
- Compensating controls must address the attack path and be verified.
- Keep a register, reassess on new intelligence and escalate repeated renewals.

QUICK CHECK

1. An exception should always include:
 - A) No end date
 - B) An expiry date and a remediation plan
 - C) Only a verbal approval
 - D) No compensating controls
2. Which is a valid compensating control?
 - A) Hoping nobody attacks
 - B) Network restriction plus an IPS signature and monitoring
 - C) Ignoring the CVE
 - D) Renaming the host
3. A CVE in an exception is newly added to KEV. What should happen?
 - A) Nothing
 - B) Re-evaluate and likely shorten or end the exception
 - C) Extend it a year
 - D) Delete the register
4. Repeated renewals of the same exception indicate:
 - A) Good process
 - B) A systemic problem needing real remediation
 - C) A scanner bug
 - D) Perfect security

Answers: 1-B 2-B 3-B 4-B

1. Time-boxing and a plan prevent permanent risk debt.
2. It must address the attack path and be verifiable.
3. New exploitation evidence invalidates the old assessment.
4. Constant extensions mean the root issue is not being solved.

Reconcile inventory

SCENARIO

You have three exported files: cmdb.csv, scanner.csv and cloud.csv. Your job is to compare them and report on gaps before the quarterly audit. All hosts use fictional names and 192.0.2.x or 198.51.100.x addresses.

OBJECTIVE

Find unmanaged assets and assets without owners by comparing scanner results to the CMDB.

STEPS IN THE SIMULATOR

1 Do: Open cmdb.csv

Simulator shows:

1,204 rows. Columns: hostname, ip, owner, criticality, env.

2 Do: Open scanner.csv

Simulator shows:

1,289 rows. Columns: hostname, ip, os, last_scan.

3 Do: Run Compare: scanner minus CMDB by hostname

Simulator shows:

94 hosts in scanner but not in CMDB, e.g. build-07 (192.0.2.115), kiosk-03 (198.51.100.22).

4 Do: Filter the 94 by owner tag in cloud.csv

Simulator shows:

33 have an owner tag in the cloud; 61 have no owner anywhere.

5 Do: Run Compare: CMDB minus scanner (last_scan older than 30 days)

Simulator shows:

9 hosts, e.g. old-fs01 (192.0.2.60).

6 Do: Open external.csv from attack surface scan

Simulator shows:

vpn-old.example.com 203.0.113.60 open on 443, not in CMDB or scanner.

7 Do: Click Summarize

Simulator shows:

Unmanaged: 94. No owner: 61. Stale: 9. Exposed unknown: 1 (vpn-old.example.com).

ANSWER

Findings: 94 assets seen by the scanner but absent from the CMDB, 61 with no owner anywhere, 9 CMDB assets not scanned in 30 days, and one unknown internet-exposed host, vpn-old.example.com at 203.0.113.60. Assign owners, add or retire records, and investigate the exposed host first.

Hint shown in the app: Subtract one list from the other, then check which results have no owner tag.

Prioritize the backlog

SCENARIO

You have 10 CVEs from this week's scans. Each has CVSS, EPSS, KEV status, exposure and asset criticality. The patch team can handle three emergency items this week. Decide the order and justify it.

OBJECTIVE

Rank the ten CVEs and choose the top three with reasons.

STEPS IN THE SIMULATOR

- 1 **Do:** View the table of 10 CVEs

Simulator shows:

CVE-A 9.8/0.02/no/internal/low; CVE-B 7.5/0.91/KEV/internet/high; CVE-C 8.8/0.40/no/internet/high; CVE-D 9.1/0.85/KEV/internal/high; ... (10 rows).

- 2 **Do:** Sort by KEV status

Simulator shows:

KEV items: CVE-B (internet, web-gw) and CVE-D (internal, domain controller).

- 3 **Do:** Rank KEV plus internet exposure first

Simulator shows:

Rank 1: CVE-B. Reason: exploited, exposed, high-value.

- 4 **Do:** Rank the KEV internal item next

Simulator shows:

Rank 2: CVE-D. Reason: exploited and the asset is a domain controller.

- 5 **Do:** Pick the next by EPSS and exposure

Simulator shows:

Rank 3: CVE-C. Reason: EPSS 0.40, internet exposed, CRM data.

- 6 **Do:** Place CVE-A and the rest by criticality

Simulator shows:

CVE-A (CVSS 9.8, EPSS 0.02, lab, internal) moves to the normal cycle.

- 7 **Do:** Submit order with justification

Simulator shows:

Accepted: top three B, D, C. Note: CVE-A deferred with documented rationale.

ANSWER

Patch CVE-B first (KEV, internet-facing, high value), then CVE-D (KEV on a domain controller), then CVE-C (high EPSS, exposed). CVE-A has the highest CVSS but low exploitation probability and no exposure, so it follows the normal SLA with documented reasoning.

Hint shown in the app: Start with KEV, then exposure and EPSS; use CVSS last.

Read a scan report

SCENARIO

An authenticated scan of a web server (192.0.2.30) produced a long report. Several findings may be false positives because of backported patches or tool quirks. You have the log viewer and the host's package details.

OBJECTIVE

Separate false positives from real findings and decide what to fix.

STEPS IN THE SIMULATOR

- 1 **Do:** Open finding 1: Apache 2.4.37 vulnerable to CVE-2026-6001

Simulator shows:

Detected by banner version. Authenticated check: package httpd-2.4.37-65.el8 with vendor backport RHSA-2026:1234.

- 2 **Do:** Compare with the vendor advisory

Simulator shows:

The advisory lists 2.4.37-65 as fixed. Classification: FALSE POSITIVE (backported).

- 3 **Do:** Open finding 2: OpenSSL 1.1.1k CVE-2026-6002

Simulator shows:

Package openssl-1.1.1k-9.el8 is older than the fixed 1.1.1k-12. Classification: REAL.

- 4 **Do:** Open finding 3: SSH weak MAC algorithms

Simulator shows:

sshd_config allows hmac-sha1. Real configuration issue, medium.

- 5 **Do:** Open finding 4: Missing patch KB5000000 on Linux host

Simulator shows:

Plugin targets Windows only. Classification: FALSE POSITIVE (wrong OS plugin). Report to scanner team.

- 6 **Do:** Mark false positives with evidence

Simulator shows:

2 false positives recorded with justification and expiry.

- 7 **Do:** Create tickets for real findings

Simulator shows:

Tickets: update openssl package (P2); change MACs (P3).

ANSWER

Findings 1 and 4 are false positives: one is a backported fix that a banner check misreads and the other is a Windows plugin firing on a Linux host. Findings 2 and 3 are real: an outdated OpenSSL package and weak SSH MACs. Record evidence for false positives and create tickets for the real ones.

Hint shown in the app: Check the package release and vendor advisory, not just the banner version.

Handle an exception

SCENARIO

You are a security analyst. The owner of erp-legacy (192.0.2.80), a finance manager, asks to postpone patching a high-severity vulnerability for six months because of a busy audit season. You chat with them to agree on terms.

OBJECTIVE

Negotiate and document a time-boxed exception with a compensating control.

STEPS IN THE SIMULATOR

- 1 Do:** Type: Thanks for explaining. Six months is too long. What is the earliest feasible date?
Simulator shows:
Owner: The vendor upgrade needs a 6-week migration. We can do it by Sept 15 if we start now.
- 2 Do:** Type: I can support a 90-day exception if we add compensating controls. Is access limited to a few admins?
Simulator shows:
Owner: Only the finance admin subnet needs it. Everyone else uses the web front end.
- 3 Do:** Type: Please restrict port 443 on that server to 192.0.2.0/28 and let me enable an IPS signature.
Simulator shows:
Owner: Agreed. Network team will change the firewall today.
- 4 Do:** Type: I also need a detection rule and you as the named risk owner with a 90-day expiry.
Simulator shows:
Owner: OK. I will sign as risk owner. Expiry 90 days.
- 5 Do:** Fill the exception form: reason, controls, owner, expiry, plan
Simulator shows:
Form saved: EXC-0192, expiry 90 days, plan: upgrade to v12 by Sept 15.
- 6 Do:** Click Submit for approval
Simulator shows:
Approved by security manager. Reminder set 14 days before expiry.
- 7 Do:** Type: If the CVE is added to KEV, I will reopen this decision.
Simulator shows:
Owner: Understood.

ANSWER

The agreed outcome is a 90-day exception, not six months, with compensating controls: access restricted to 192.0.2.0/28, an IPS signature and a detection rule. It names the risk owner, includes a remediation plan with a target date, has an expiry, and is reopened if exploitation evidence such as KEV listing appears.

Hint shown in the app: Offer a shorter time-box plus concrete controls and a named owner, rather than refusing outright.

Vulnerability and Patch Programs: 10 questions, 80% to pass

1. Which approach best finds assets missing from the CMDB?
 - A) Reading the README
 - B) Comparing scanner and cloud data to the CMDB
 - C) Rebooting hosts
 - D) Ignoring DHCP
2. An authenticated scan generally gives:
 - A) Fewer details
 - B) More accurate local patch and software findings
 - C) Only port lists
 - D) No findings
3. Which asset type is best covered by agents?
 - A) Remote laptops
 - B) Data center firewall
 - C) Switch
 - D) Printer
4. CVSS Critical range is:
 - A) 1.0-3.9
 - B) 4.0-6.9
 - C) 7.0-8.9
 - D) 9.0-10.0
5. What does EPSS predict?
 - A) Patch time
 - B) Likelihood of exploitation within 30 days
 - C) Asset value
 - D) Network latency
6. Which should usually go first?
 - A) CVSS 9.8, isolated, not in KEV
 - B) CVSS 7.5, KEV-listed, internet-facing
 - C) CVSS 4.0, internal
 - D) CVSS 3.1, offline
7. A good remediation ticket includes:
 - A) Only a CVE ID
 - B) Assets, priority, due date, fix and validation
 - C) A random ID
 - D) Nothing
8. What does aging reporting reveal?
 - A) Staff age
 - B) How long findings remain open
 - C) Disk age
 - D) DNS TTL
9. An exception must have:
 - A) No expiry
 - B) An expiry date, owner and compensating controls
 - C) Only an email
 - D) No risk owner
10. Which is a false positive example?
 - A) Outdated package confirmed by version
 - B) Banner version flagged although the vendor backported the fix
 - C) Weak cipher in config
 - D) Open admin port

Answers: 1-B 2-B 3-A 4-D 5-B 6-B 7-B 8-B 9-B 10-B

1. Reconciliation across independent sources exposes gaps.
2. Credentialed access reveals installed software and patches.
3. Agents report from wherever the endpoint is.
4. Critical is 9.0 to 10.0.
5. EPSS gives an exploitation probability.
6. Known exploitation and exposure drive urgency.
7. Owners need actionable, verifiable detail.
8. Aging bands show backlog health.
9. These make risk acceptance time-boxed and accountable.
10. Backported fixes often fool banner-based checks.

Endpoint and Email Security

Intermediate | 5 lessons | 4 simulated labs | checkpoint | about 4h 25m

This module covers the controls that sit closest to users: endpoint agents, device management, email defenses and browsers. You will plan an EDR pilot and measure coverage, enforce encryption and compliance with MDM and conditional access, and tune an email gateway with a reporting workflow. You will also move a domain from DMARC p=none to reject without breaking mail, and harden browsers with policy and isolation. These are the areas where phishing and malware most often begin, so careful rollout matters as much as the tools.

BY THE END YOU CAN

- + Roll out EDR and MDM in phases
- + Deploy SPF, DKIM and DMARC without breaking mail
- + Harden browsers and email gateways

IN THIS MODULE

Lesson 1 EDR selection and rollout

Lesson 2 MDM and device compliance

Lesson 3 Email gateway and phishing defense

Lesson 4 SPF, DKIM and DMARC rollout

Lesson 5 Browser security

Lab 1 Triage EDR coverage (Mock SIEM)

Lab 2 Fix DMARC records (Config editor)

Lab 3 Analyze a phish (Log viewer)

Lab 4 Set MDM policy (Config editor)

Checkpoint 10 questions, 80% to pass

EDR selection and rollout

Endpoints are where most intrusions finally execute, so an EDR agent that is deployed badly, or only on half the fleet, gives a false sense of safety.

What an EDR does and how to choose one

Endpoint detection and response (EDR) software runs a lightweight sensor on each laptop, workstation and server. It records process starts, command lines, file writes, registry changes, network connections and logon events, sends that telemetry to a cloud or on-premises console, and applies detections based on behavior rather than only file signatures. When selecting a product, compare telemetry depth, detection quality, response actions, operating system coverage (Windows, macOS, Linux, sometimes mobile), API and SIEM integration, and sensor impact on CPU and memory. Public evaluations such as MITRE ATT&CK Evaluations show how a product detects specific techniques, but they are one input. Always test against your own workloads in a proof of concept.

Pilot first, then roll out in rings

Never install a new sensor on every machine in one weekend. Start with an inventory taken from your asset database, directory and network scans, because you cannot cover devices you do not know about. Then deploy in rings: the security and IT teams first, then a pilot group of roughly 5 percent that includes each hardware model and key application, then wider business groups, and servers last with change approval. Run the sensor in detect-only or audit mode during the pilot so that a false positive does not block a payroll application. Move each ring forward only after a defined soak period, often one to two weeks, with no unresolved performance or compatibility tickets.

Coverage, exclusions and tamper protection

Coverage is a measurable outcome: divide hosts reporting a healthy sensor in the last 24 hours by hosts in the asset inventory, and chase the gap. Watch for sensors in reduced functionality mode, which happens when an operating system version is unsupported or a kernel component fails to load. Exclusions deserve special care. Every excluded path or process is a place where an attacker can hide, so scope exclusions narrowly by full path, signing certificate and process, document the owner and reason, and review them quarterly. Enable tamper protection so that stopping, uninstalling or modifying the agent requires a console-issued token or password, and alert when a sensor goes silent.

Response actions and operating the tool

The value of EDR shows in the response features: isolating a host from the network while keeping the console channel open, killing a process tree, quarantining a file, collecting a triage package, and opening a remote shell for live investigation. Write down who may use each action. Isolating a laptop is routine, while isolating a domain controller or a production database server needs approval and a runbook. Use role-based access control and log every console action. Forward EDR alerts to the SIEM and ticketing system so they are triaged with other signals. Finally, plan upgrades, since sensors need version updates, and test those in the same rings you used for the first deployment.

COVERAGE CHECK (PSEUDO-QUERY)

```
assets = 1,240 hosts in inventory
reporting = 1,118 sensors seen in last 24h
reduced_fn = 31 sensors in reduced mode
unmanaged = 122 + 31 = 153 gaps
coverage = 1118 / 1240 = 90.2% (target 98%+)
```

WORKED EXAMPLE

Example Corp buys an EDR and wants it live in six weeks. The team exports 1,240 hosts from the asset database and starts with 25 IT laptops in audit mode. The pilot reveals that a legacy accounting program is flagged, so a narrow exclusion by signed publisher and path is added with an owner and review date. Ring two covers 60 mixed users, ring three the remaining laptops, and servers follow after change approval. Weekly coverage is 90 percent after ring three, and the gap list shows 31 hosts in reduced functionality mode due to an old OS build that is then upgraded.

KEY TERMS

EDR: Endpoint detection and response: a sensor plus console that records endpoint behavior, detects threats and enables response.

Sensor coverage: Share of inventoried hosts reporting a healthy EDR agent.

Exclusion: A path, process or hash the agent is told to ignore, which creates a potential blind spot.

Tamper protection: Controls that stop users or malware from disabling or removing the agent.

Host isolation: A response action that cuts a host off from the network except the management channel.

Reduced functionality mode: A state where the sensor runs with fewer protections, often due to an unsupported OS or failed driver.

REMEMBER

- Test EDR in a pilot using real workloads and audit mode before enforcing.
- Measure coverage against the asset inventory and fix sensors in reduced mode.
- Keep exclusions narrow, enable tamper protection and restrict who can isolate hosts.

QUICK CHECK

1. What is the safest way to start an EDR deployment?
 - A) Install on every host overnight with blocking on
 - B) A pilot ring in detect-only mode with varied hardware and applications
 - C) Install only on servers
 - D) Skip the pilot to save time
2. Why should exclusions be narrow and reviewed?
 - A) Exclusions make the sensor faster in every case
 - B) Exclusions are required by law
 - C) Excluded files are automatically scanned twice
 - D) Each exclusion is a place malware can run unseen
3. What does tamper protection do?
 - A) Requires a token or password to stop, remove or alter the agent
 - B) Encrypts the disk
 - C) Blocks all USB devices
 - D) Hides the agent in task manager
4. Which metric best shows deployment health?
 - A) Number of alerts per day
 - B) Console logins per week
 - C) Hosts with a healthy sensor divided by hosts in the asset inventory
 - D) Size of the sensor installer

Answers: 1-B 2-D 3-A 4-C

1. A small, varied pilot in audit mode surfaces compatibility problems before they affect everyone.
2. Overbroad exclusions create blind spots attackers can abuse.
3. It prevents attackers and users from disabling the sensor.
4. Coverage against a trusted inventory shows how many endpoints are actually protected.

MDM and device compliance

Phones and laptops hold company data wherever their owners carry them, and mobile device management is how an organization sets and proves a minimum security state.

What MDM controls

Mobile device management (MDM) enrolls a device with a management service so the organization can push configuration and read device state. Platforms such as Apple's MDM protocol, Android Enterprise and the Windows MDM channel expose settings for encryption, screen lock, application installation, VPN and Wi-Fi profiles, and remote actions. MDM is usually combined with mobile application management (MAM), which protects data inside specific apps without controlling the whole device. Typical remote actions are lock, passcode reset, selective wipe that removes only corporate data, and full wipe. Enrollment can be automated for company devices through programs such as Apple Business Manager or Windows Autopilot, so a device is configured from first power-on.

Compliance baselines

A compliance policy is a list of conditions a device must meet to be called healthy. Common rules include storage encryption on (FileVault, BitLocker, or default Android and iOS encryption), a passcode of at least six digits or a complex password with a short lock timeout, a minimum operating system version, no jailbreak or root, a healthy EDR or mobile threat defense signal, and a firewall enabled. Choose rules that match the data being protected. A grace period of a few days lets users fix problems, such as installing an update, before access is affected. Write the baseline once, test it against a pilot group, and report compliance percentages to owners.

Conditional access

Compliance data becomes powerful when identity systems use it. In conditional access, a sign-in to email or a business application is allowed only if the user passes multifactor authentication and the device is marked compliant or enrolled. A non-compliant device is blocked, or given limited access such as web-only without download. This stops a stolen password from working on an unmanaged machine. Design the policies with report-only mode first to see who would be blocked, keep a break-glass administrator account excluded and monitored, and publish clear instructions for users on how to remediate. Conditional access moves enforcement to the moment of access, which is when it matters.

BYOD and privacy

Personally owned devices need a lighter model. With bring your own device (BYOD), organizations often use app protection policies or work profiles: iOS User Enrollment and the Android work profile separate corporate apps and data from personal ones. The company can require a PIN for the work apps, block copy and paste to personal apps, and perform a selective wipe, but it cannot see personal photos, messages or browsing. State this boundary in writing, since trust affects adoption. Pick a minimum OS version and device age that keeps old, unpatched phones out. Lost and stolen device procedures should be quick: report, lock, selective wipe and credential reset.

EXAMPLE COMPLIANCE RULES

```
Platform: iOS / Android (BYOD group)
  require passcode, length >= 6
  require device encryption
  block jailbroken or rooted
  min OS: iOS 17.x / Android 13
  grace period: 3 days
Conditional access:
  apps: Mail, Files
  grant: MFA + compliant device
  noncompliant: block
```

WORKED EXAMPLE

A fictional firm lets staff read email on personal phones. The security engineer creates a BYOD group and a work-profile policy: PIN of six digits, device encryption, no root, and a minimum OS version. Conditional access allows Mail and Files only with MFA and a compliant status. The rule is first run in report-only mode for two weeks, which shows 40 users on old OS versions. They receive guidance, and after the grace period non-compliant phones are blocked. When an employee loses a phone, the help desk performs a selective wipe, and personal data is untouched.

KEY TERMS

MDM: Mobile device management: enrolling devices to push settings and issue remote actions.

MAM: Mobile application management: controlling and protecting data at the app level.

Compliance policy: Set of conditions a device must meet to be considered healthy.

Conditional access: Access decisions that combine identity, device state and context.

Selective wipe: Removing only corporate data and apps from a device.

BYOD: Bring your own device: using personal hardware for work with limited management.

REMEMBER

- MDM enforces a baseline and enables remote lock and wipe.
- Compliance status feeds conditional access so only healthy devices reach data.
- BYOD uses work profiles and selective wipe to respect privacy.

QUICK CHECK

1. Which action removes only company data from a personal phone?
 - A) Full factory reset
 - B) Selective wipe
 - C) Remote lock only
 - D) Passcode removal
2. What does a conditional access rule requiring a compliant device achieve?
 - A) Replaces passwords with usernames
 - B) Disables MFA
 - C) Encrypts email in transit
 - D) Blocks access from devices that fail MDM health checks even with a valid password
3. Why use report-only mode first?
 - A) To see who would be blocked before enforcing
 - B) To turn off logging
 - C) To speed up sign-in
 - D) To avoid writing a policy
4. Which setting is a typical compliance rule?
 - A) Wallpaper color
 - B) Preferred browser tab count
 - C) Storage encryption enabled and minimum OS version
 - D) Number of installed games

Answers: 1-B 2-D 3-A 4-C

1. Selective wipe deletes managed apps and data and leaves personal content.
2. Identity checks are combined with device posture.
3. It reveals impact and lets you fix problems before enforcement.
4. Encryption and patch level are core baseline controls.

Email gateway and phishing defense

Email remains the most common delivery path for phishing and malware, so layered filtering plus user reporting cuts risk before anyone clicks.

The filtering pipeline

A secure email gateway, or the filtering built into a cloud mail platform, inspects each message before delivery. It checks the sending IP and domain reputation, authentication results (SPF, DKIM, DMARC), content for spam and phishing traits, and attachment types. Messages receive a verdict such as deliver, junk, quarantine or reject. Tune the policy by risk: quarantine high-confidence phishing and known malware, send bulk mail to the junk folder, and block risky file types such as executables, script files and macro-enabled Office documents from external senders. Add banners that mark external email, and use impersonation protection for executive display names and lookalike domains such as `examp1e.com`.

Attachment sandboxing and link protection

Signature scanning misses new malware, so gateways detonate attachments in a sandbox, a controlled virtual machine where behavior is observed, before the message is released. Sandboxing adds a delay of a minute or two, and some products deliver the body first and the attachment afterward. For links, URL rewriting sends the click through the vendor's proxy so the destination is checked at the time of click, which catches sites that were clean at delivery and turned malicious later. Time-of-click checks also protect against delayed attacks. Content disarm and reconstruction removes active content from documents and delivers a safe copy.

User reporting and response workflow

Filters will miss some messages, so give users a one-click Report Phishing button in the mail client. A reported message goes to a mailbox or to automation that extracts the sender, URLs and attachment hashes, checks reputation, and classifies it. If it is malicious, the security team searches every mailbox for the same message and purges it, blocks the sender and domain, and adds the URL to the blocklist. Thank the reporter, because feedback keeps reporting rates high. Track metrics such as report rate, median time from delivery to removal, and the number of users who clicked, rather than punishing people for failing a simulation.

Reading a suspicious message

Teach staff and analysts to look beyond the display name. Check the real sender address, mismatched reply-to, urgency and requests for payment, and hover over links to see the true destination. Analysts read full headers: the Received lines show the path of the message, and the Authentication-Results header shows spf, dkim and dmarc outcomes. A message claiming to be from your finance team yet showing `spf=fail` and `dmarc=fail` from an unrelated IP is a strong spoofing sign. Be aware of newer tactics too, such as QR codes embedded in images, links in shared document notifications from legitimate services, and multi-stage attacks that harvest session tokens. Layer controls instead of relying on one.

HEADER EXCERPT (FICTIONAL)

```
From: "Accounts" <billing@example.com>
Reply-To: pay@mail.example.net
Received: from unknown (HELO mail.example.net) [203.0.113.45]
Authentication-Results: mx.example.com;
  spf=fail smtp.mailfrom=example.com;
  dkim=none; dmarc=fail header.from=example.com
Subject: Urgent invoice - action today
```

WORKED EXAMPLE

A finance clerk at Example Corp receives an invoice email with a PDF. The gateway holds it for sandbox analysis, which finds the file opens a link to a credential page, and the message is quarantined for most recipients. One copy slipped through to a user who had received it earlier through a different route. She presses Report Phishing. Automation extracts the URL, the analyst confirms it is malicious, and a mailbox search purges four more copies within 20 minutes. The domain is added to the block list and a short note tells staff what to look for.

KEY TERMS

Secure email gateway: Service that filters inbound and outbound email for spam, phishing and malware.

Sandbox: Isolated environment used to run a file and observe its behavior.

URL rewriting: Replacing links with proxy links so the destination is checked at click time.

Quarantine: A holding area for suspicious messages pending review.

Impersonation protection: Detection of messages that mimic trusted people or domains.

Report Phishing button: A client feature that sends a suspicious message to security for analysis.

REMEMBER

- Use layered filtering: reputation, authentication, content, sandbox and time-of-click link checks.
- Make reporting easy and automate purge, block and feedback.
- Read headers and authentication results to spot spoofing.

QUICK CHECK

1. Why does URL rewriting help?

- A) It deletes all links
- B) The link is checked when clicked, catching sites that turn malicious after delivery
- C) It makes the link shorter only
- D) It blocks internal mail

2. What does attachment sandboxing do?

- A) Encrypts the attachment
- B) Changes the file name
- C) Scans only the file extension
- D) Runs the file in an isolated VM to observe malicious behavior

3. What should happen after a user reports a confirmed phish?

- A) Search all mailboxes, purge copies and block the sender and URL
- B) Do nothing
- C) Reboot the mail server
- D) Delete the user's account

4. Which header result suggests spoofing?

- A) spf=pass and dkim=pass aligned
- B) Message size of 40 KB
- C) spf=fail and dmarc=fail for the claimed domain
- D) Received header present

Answers: 1-B 2-D 3-A 4-C

1. Time-of-click checks catch delayed attacks.
2. Behavioral detonation finds malware with no known signature.
3. Removal and blocking limit exposure for everyone else.
4. Failed authentication for the From domain is a spoofing indicator.

SPF, DKIM and DMARC rollout

Without these three DNS-based controls anyone can send mail that appears to come from your domain, and a rushed DMARC policy can also block your own legitimate mail.

SPF and DKIM

Sender Policy Framework (SPF, RFC 7208) is a DNS TXT record listing the servers allowed to send mail for a domain. The receiver checks the connecting IP against the record of the envelope sender (MAIL FROM) domain. Keep it under the limit of 10 DNS lookups, and end with ~all during testing or -all when you are confident. DomainKeys Identified Mail (DKIM, RFC 6376) signs selected headers and the body with a private key, and publishes the public key in DNS at selector._domainkey.example.com. The receiver verifies the signature, proving the content was not altered and that the signing domain authorized it. SPF breaks on forwarding, while DKIM usually survives it.

DMARC and alignment

DMARC (RFC 7489) ties these together. It requires that SPF or DKIM passes and that the passing domain aligns with the visible From domain, then tells receivers what to do on failure and where to send reports. The record is a TXT at _dmarc.example.com with tags: v, p (policy), rua (aggregate report address), ruf (forensic), pct (percentage) and adkim/aspf (strict or relaxed alignment). The policy has three values: none (monitor only), quarantine (treat as suspicious, usually spam folder) and reject (refuse the message). Alignment matters because an attacker can pass SPF for their own domain while showing yours in the From header.

The staged rollout

Start with p=none and an rua mailbox or a DMARC reporting service. Aggregate reports are daily XML files from receivers showing source IPs, volumes and pass or fail results. Inventory every legitimate sender: your mail platform, marketing platforms, ticketing, HR and payment systems. For each, add it to SPF or, better, configure custom-domain DKIM signing. When legitimate sources pass consistently, move to p=quarantine with pct=10, then 25, 50 and 100, reviewing reports at each step. Finally set p=reject. The process often takes several months, and a pause at each stage is normal. Also publish sp= for subdomains and a p=reject record for parked domains that never send mail.

Pitfalls and ongoing care

Common failures include exceeding the SPF 10-lookup limit with many include statements, signing with a DKIM key under a different domain than the From address (misalignment), forgetting third-party senders, mailing lists that modify messages and break DKIM, and forwarders that break SPF. ARC can help preserve results through forwarding. Rotate DKIM keys at least yearly, using 2048-bit keys where supported, and remove old selectors. Monitor reports continuously, since a new SaaS tool can start sending as your domain without anyone telling security. Layer on MTA-STS and BIMI later if desired. A reject policy is the goal, because it is the setting that actually stops spoofing.

DNS RECORDS (DOCUMENTATION DOMAIN)

```
example.com. TXT "v=spf1 ip4:192.0.2.25 include:_spf.mailer.example.net -all"
sel1._domainkey.example.com. TXT "v=DKIM1; k=rsa; p=MIIBIjANBgkq..."
_dmarc.example.com. TXT "v=DMARC1; p=none; rua=mailto:dmarc-reports@example.com; adkim=r; aspf=r"
# later stage:
_dmarc.example.com. TXT "v=DMARC1; p=quarantine; pct=25; rua=mailto:dmarc-reports@example.com"
```

WORKED EXAMPLE

Example Corp publishes p=none with reports. After four weeks the XML shows 92 percent of volume passing, plus a failing source at 198.51.100.77 that turns out to be the newsletter platform. The admin enables custom DKIM signing for that platform and adds its include to SPF while staying under 10 lookups. Two weeks later every legitimate source aligns. The policy moves to quarantine at pct=10, then 50, then 100, and after another month of clean reports to reject. A spoof attempt from an unknown IP is now refused at the receiver.

KEY TERMS

SPF: DNS record listing IPs or services allowed to send mail for a domain.

DKIM: Cryptographic signature on mail verified with a public key in DNS.

DMARC: Policy that requires aligned SPF or DKIM and sets handling and reporting.

Alignment: Match between the authenticated domain and the visible From domain.

Aggregate report (rua): Daily XML summary of authentication results sent by receivers.

p=reject: Policy telling receivers to refuse messages that fail DMARC.

REMEMBER

- SPF authorizes sending IPs, DKIM signs messages, DMARC requires alignment and sets policy.
- Roll out none, then quarantine with rising pct, then reject, guided by aggregate reports.
- Inventory third-party senders and watch the SPF lookup limit.

QUICK CHECK

1. What is the safest first DMARC policy?
 - A) p=reject immediately
 - B) p=none with reporting enabled
 - C) No record at all
 - D) p=quarantine with pct=100
2. What does DMARC alignment require?
 - A) The message is under 1 MB
 - B) DNS TTL is 300
 - C) The sender uses TLS
 - D) The domain passing SPF or DKIM matches the visible From domain
3. Which common SPF mistake breaks evaluation?
 - A) More than 10 DNS lookups
 - B) Using a TXT record
 - C) Listing an IPv4 address
 - D) Ending with -all
4. What is the purpose of the pct tag?
 - A) Set key length
 - B) Choose the report format
 - C) Apply the policy to only a percentage of failing mail during rollout
 - D) Limit message size

Answers: 1-B 2-D 3-A 4-C

1. Monitoring first reveals legitimate senders before any mail is affected.
2. Alignment stops a passing attacker domain from covering a spoofed From.
3. RFC 7208 limits lookups to 10, causing a permerror.
4. It lets you ramp enforcement gradually.

Browser security

The browser is the front door to cloud applications and the web, so managing it centrally removes many drive-by and phishing risks.

Managed browser policy

Modern browsers such as Chrome, Edge and Firefox can be configured centrally through group policy, MDM profiles or a cloud management console. Useful settings include forcing automatic updates, turning on Safe Browsing or SmartScreen, blocking pop-ups, disabling the built-in password manager for corporate accounts when you provide an enterprise vault, requiring HTTPS-first mode, restricting developer tools, and controlling which sites may use the camera, microphone or notifications. Set a baseline from a trusted benchmark, such as the CIS Benchmarks for browsers, then add exceptions with documented owners. Keep browser versions current, because many web attacks target recently disclosed flaws that vendors have already fixed.

Extension control

Extensions run with deep access to pages and can read or change data on the sites you visit, so they are a supply-chain risk. Attackers buy popular extensions, push a malicious update, or publish lookalikes. Use an allowlist approach: block everything by default and approve extensions by ID after a review of permissions, publisher, update history and business need. Force-install the few you require, such as a password manager. Review permissions like read and change all data on all websites, and rerun the review when an update asks for new permissions. Report on installed extensions through the management console and remove stale ones.

Safe browsing, isolation and DNS filtering

Layered web protection reduces exposure to malicious sites. Reputation services such as Safe Browsing and secure web gateways block known phishing and malware domains, and DNS filtering can stop lookups for newly registered domains and command-and-control hosts. For risky content, remote browser isolation (RBI) runs the page in a disposable container in the cloud and streams only a safe rendering to the user, so exploit code never reaches the endpoint. Use isolation for uncategorized sites, links from email and high-risk roles such as finance. Download controls, such as blocking executables from the web or scanning them first, add another barrier, and TLS inspection may be needed where policy and privacy rules allow it.

Identity, data and monitoring

Many attacks now steal session cookies after a successful login, so pair browser hardening with phishing-resistant MFA such as FIDO2 passkeys and short session lifetimes, and bind sessions to device state through conditional access. Enterprise browsers and data loss prevention can restrict upload, copy and print on sensitive web apps, and watermark screens. Separate profiles for personal and work reduce cross-contamination. Send browser telemetry, extension inventories and blocked-site events to the SIEM. Provide users with an easy way to report warnings. Finally, test policies with a pilot group before enforcing, since over-strict settings drive staff toward unmanaged browsers.

EXAMPLE MANAGED-BROWSER SETTINGS

```
AutoUpdate : enforced
SafeBrowsing : enhanced protection
HttpsOnlyMode : enabled
ExtensionInstallBlocklist : *
ExtensionInstallAllowlist : <approved extension IDs>
ExtensionInstallForcelist : <password manager ID>
PasswordManagerEnabled: false (use enterprise vault)
DownloadRestrictions : block dangerous files
```

WORKED EXAMPLE

A finance team at Example Corp reports slow pages and strange pop-ups. An audit of the browser console shows a coupon extension with permission to read all site data that was updated to a new publisher. The admin removes it by ID, switches the policy to block all extensions except an allowlist, and enables isolation for uncategorized links in email. Two weeks later a click on a newly registered phishing domain is blocked by DNS filtering. Because the session lifetime is short and tied to a compliant device, a stolen cookie would also be of little use.

KEY TERMS

Browser policy: Centrally managed settings applied to browsers by GPO, MDM or console.

Extension allowlist: Approved list of extension IDs; all others are blocked.

Safe Browsing: Reputation service that warns about or blocks phishing and malware sites.

Remote browser isolation: Running web content in a remote disposable container and streaming a safe view.

HTTPS-first mode: Setting that upgrades connections to HTTPS and warns when it is unavailable.

Session hijacking: Stealing a valid session cookie or token to impersonate a user.

REMEMBER

- Manage browsers by policy: updates, safe browsing, HTTPS-first and download controls.
- Use an extension allowlist and review permissions regularly.
- Isolate risky content and protect sessions with strong MFA and device state.

QUICK CHECK

1. What is the safest extension strategy?

- A) Allow any extension with 4 stars
- B) Block by default and allow reviewed extension IDs
- C) Let users decide
- D) Allow only extensions older than a year

2. What does remote browser isolation prevent?

- A) All phishing emails
- B) Password reuse
- C) Slow DNS
- D) Web exploit code from executing on the user's endpoint

3. Why keep browsers updated automatically?

- A) Attacks target recently fixed flaws
- B) Updates remove all extensions
- C) Updates change the theme
- D) Updates disable TLS

4. Which control best reduces stolen-cookie risk?

- A) Larger bookmark folder
- B) Disabling pop-ups only
- C) Phishing-resistant MFA, short sessions and device-bound conditional access
- D) Using dark mode

Answers: 1-B 2-D 3-A 4-C

1. An allowlist limits the supply-chain risk of extensions.

2. The page runs elsewhere and only a safe rendering reaches the user.

3. Patching closes known vulnerabilities quickly.

4. These controls make a stolen session less useful and harder to obtain.

Triage EDR coverage

SCENARIO

Example Corp's security lead asks for an EDR coverage review before the board meeting. In the SIEM simulator you have the asset inventory feed and sensor heartbeat events for the last 24 hours. Several hosts either never reported a heartbeat or report that the sensor is in reduced functionality mode. Your task is to produce a prioritized list of gaps and a remediation plan that distinguishes servers, laptops and unsupported systems.

OBJECTIVE

Identify hosts without a healthy sensor and plan fixes by priority.

STEPS IN THE SIMULATOR

- 1 **Do:** Run a search comparing the asset inventory with sensor heartbeat events from the last 24 hours.

Simulator shows:

1,240 assets, 1,118 hosts with heartbeats, so 122 hosts are missing.

- 2 **Do:** Filter heartbeat events where sensor_mode equals reduced.

Simulator shows:

31 hosts listed, mostly Windows 7 and an old Linux kernel.

- 3 **Do:** Add the host role field and sort by criticality.

Simulator shows:

9 of the missing hosts are servers, including FS-02 and DB-05.

- 4 **Do:** Check last_seen for the missing hosts.

Simulator shows:

Most laptops were last seen 20 days ago, suggesting they are offline or retired.

- 5 **Do:** Cross-check missing hosts against the CMDB status field.

Simulator shows:

40 hosts are marked decommissioned and should leave the inventory.

- 6 **Do:** Assign actions: install on new servers, upgrade OS on reduced hosts, retire stale records.

Simulator shows:

Plan shows coverage rising from 90.2 percent to about 98 percent.

- 7 **Do:** Confirm tamper protection status for the healthy hosts.

Simulator shows:

Four hosts show tamper protection off and are added to a fix list.

ANSWER

122 hosts lack a healthy sensor and 31 run reduced mode. Prioritize the 9 servers, upgrade or replace unsupported operating systems, remove the 40 decommissioned records, and enable tamper protection on four hosts. Expected coverage is about 98 percent.

Hint shown in the app: Compare inventory to heartbeats, then split the gap by server criticality and cause.

Fix DMARC records

SCENARIO

Example Corp publishes DMARC at p=none and has collected four weeks of aggregate reports. The reports show the newsletter platform at 198.51.100.77 failing both SPF and DKIM alignment, while corporate mail from 192.0.2.25 passes. The SPF record also exceeds 10 lookups. In the config editor you can change the DNS zone for example.com. Fix the authentication so the domain can move toward enforcement without blocking legitimate mail.

OBJECTIVE

Edit SPF, DKIM and DMARC records so legitimate senders align, then raise policy safely.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the zone file and read the current TXT records for example.com.

Simulator shows:

SPF has five include statements and ends with ~all.

- 2 **Do:** Run the lookup counter in the editor.

Simulator shows:

Count is 12, above the limit of 10, causing permerror.

- 3 **Do:** Remove an obsolete include for a retired survey tool and flatten one include to its IP range.

Simulator shows:

Lookup count drops to 8.

- 4 **Do:** Add the DKIM selector record for the newsletter platform with its public key.

Simulator shows:

Validator shows the selector news1._domainkey.example.com resolves.

- 5 **Do:** Change SPF ending from ~all to -all only after confirming all sources are listed.

Simulator shows:

Validator passes with all known senders included.

- 6 **Do:** Update DMARC to p=quarantine; pct=10 keeping the rua address.

Simulator shows:

Record valid; the simulated replay shows legitimate mail passing.

- 7 **Do:** Replay the aggregate report data against the new records.

Simulator shows:

Newsletter now passes DKIM with alignment; unknown sender 203.0.113.88 fails.

ANSWER

Reduce SPF to 10 or fewer lookups, add custom DKIM for the newsletter platform so it aligns, keep reporting, and move DMARC to quarantine at pct=10 first. The unknown sender at 203.0.113.88 should be treated as spoofing and will be quarantined.

Hint shown in the app: Fix lookups and alignment first; ramp the DMARC policy gradually with pct.

Analyze a phish

SCENARIO

A user at Example Corp forwarded a suspicious message that claims to be from the finance department and asks for an urgent gift card purchase. You have the raw message headers in the log viewer. Your job is to trace the delivery path, interpret the authentication results, identify the signs of spoofing, and decide what actions the security team should take for the mailbox and the domain.

OBJECTIVE

Read headers to find spoofing and the failing authentication results.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the raw headers and read the Received lines from bottom to top.

Simulator shows:

The first hop is mail.example.net at 203.0.113.45, not an Example Corp server.

- 2 **Do:** Find the Authentication-Results header.

Simulator shows:

spf=fail, dkim=none, dmarc=fail for example.com.

- 3 **Do:** Compare the From address with the Return-Path and Reply-To.

Simulator shows:

From shows cfo@example.com, but Reply-To is pay@mail.example.net.

- 4 **Do:** Check the Date and Message-ID.

Simulator shows:

Message-ID domain is mail.example.net, which does not match the claimed sender.

- 5 **Do:** Extract URLs and sender IP and look them up in the reputation panel.

Simulator shows:

Domain registered 3 days ago; IP flagged as bulk sender.

- 6 **Do:** Choose containment actions.

Simulator shows:

Options include purge, block sender, block URL, notify user.

- 7 **Do:** Select purge from all mailboxes, block domain and IP, and notify recipients.

Simulator shows:

Case closed as confirmed spoofing; DMARC reject is recommended.

ANSWER

The message came from 203.0.113.45 via mail.example.net, failed SPF, had no DKIM and failed DMARC for example.com, with a mismatched Reply-To. It is spoofed. Purge copies, block the domain and IP, warn users, and move DMARC toward reject.

Hint shown in the app: Compare the claimed From domain with the first external hop and authentication results.

Set MDM policy

SCENARIO

Example Corp wants employees in the Sales group to read email and files on personal phones. Your task in the config editor is to write the compliance policy and conditional access rule for this BYOD group. The business wants strong protection without reading personal data. You must pick sensible settings, set a grace period, and make sure non-compliant devices cannot access corporate apps while giving users a path to fix problems.

OBJECTIVE

Configure compliance rules and conditional access for a BYOD group.

STEPS IN THE SIMULATOR

- 1 **Do:** Create a new compliance policy named BYOD-Sales for iOS and Android.

Simulator shows:

Policy editor opens with empty rules.

- 2 **Do:** Require a passcode of at least 6 digits and a lock timeout of 5 minutes.

Simulator shows:

Rule saved and shown as required.

- 3 **Do:** Require device encryption and block jailbroken or rooted devices.

Simulator shows:

Both rules are enabled.

- 4 **Do:** Set minimum OS versions to iOS 17 and Android 13.

Simulator shows:

Preview shows 12 of 150 devices would be non-compliant.

- 5 **Do:** Set a 3-day grace period and a notification message with update instructions.

Simulator shows:

Users receive a notice with steps to fix.

- 6 **Do:** Create a conditional access rule for Mail and Files requiring MFA and a compliant device, in report-only mode.

Simulator shows:

Report-only shows 12 would be blocked; no break-glass account is affected.

- 7 **Do:** Enable the rule after the grace period.

Simulator shows:

Non-compliant devices are blocked; compliant ones work normally.

ANSWER

Use a BYOD compliance policy with passcode, encryption, no root or jailbreak, and a minimum OS version, with a 3-day grace period. Add a conditional access rule requiring MFA and compliant device for Mail and Files, tested in report-only mode first.

Hint shown in the app: Start with report-only, grace period and a work profile that protects privacy.

Endpoint and Email Security: 10 questions, 80% to pass

1. Which metric best shows EDR deployment health?
 - A) Number of alerts closed
 - B) Healthy sensors divided by inventoried hosts
 - C) Installer file size
 - D) Console license count
2. Why is a pilot ring recommended for EDR?
 - A) To find compatibility and false-positive issues before wide rollout
 - B) To avoid buying licenses
 - C) To skip testing servers
 - D) To disable tamper protection
3. Which MDM compliance rule is typical?
 - A) Wallpaper must be corporate
 - B) No camera ever
 - C) Encryption on, passcode set, minimum OS version
 - D) Only one app installed
4. What does conditional access do with MDM data?
 - A) Creates user accounts
 - B) Patches the OS
 - C) Scans email attachments
 - D) Allows access only from compliant devices with MFA
5. Which DMARC order is safest?
 - A) reject, then none
 - B) none, then quarantine with rising pct, then reject
 - C) quarantine at 100 first
 - D) No monitoring phase
6. What does DMARC alignment check?
 - A) That TLS is enabled
 - B) That the sender has an MX record
 - C) That the authenticated domain matches the From domain
 - D) That the message is signed twice
7. What does a sandbox do for attachments?
 - A) Detonates the file in isolation to observe behavior
 - B) Renames the file
 - C) Deletes macros only
 - D) Signs it with DKIM
8. Which header result most strongly shows spoofing?
 - A) dkim=pass aligned
 - B) Received count of 3
 - C) Message size 20 KB
 - D) spf=fail and dmarc=fail for the claimed domain
9. Which browser control reduces extension risk?
 - A) Allow all extensions from stores
 - B) Allowlist of reviewed extension IDs
 - C) Disable updates
 - D) Let users add any extension
10. What does remote browser isolation protect against?
 - A) Weak passwords
 - B) Lost phones
 - C) Web exploit code running on the endpoint
 - D) Slow networks

Answers: 1-B 2-A 3-C 4-D 5-B 6-C 7-A 8-D 9-B 10-C

1. Coverage against inventory shows true protection.
2. A small varied pilot limits the impact of mistakes.
3. These are baseline device controls.
4. Device posture becomes an access condition.
5. Monitor first, enforce gradually.
6. Alignment prevents spoofed From headers hiding behind another domain's pass.
7. Behavior analysis finds unknown malware.
8. Authentication failures for the visible domain indicate forgery.
9. Allowlisting limits unreviewed code.
10. The page executes remotely and only a safe view is streamed.

Security Automation and Scripting

Advanced | 7 lessons | 5 simulated labs | checkpoint | about 6h 25m

This module teaches you to turn repetitive security work into reliable code. You will write Bash and Python that parse logs and call REST APIs, enrich indicators and score alerts, design SOAR playbooks with approval gates, add guardrails such as dry-run and rollback to automated containment, and write tests that prove hardening stays in place. Automation multiplies a small team's reach, but badly built automation can also cause outages, so safety and verification are treated as first-class skills throughout.

BY THE END YOU CAN

- + Write Python and Bash that call security APIs
- + Build SOAR playbooks that automate triage and response
- + Write infrastructure tests for security controls

IN THIS MODULE

Lesson 1 Bash for security tasks

Lesson 2 Python for security engineering

Lesson 3 Working with REST APIs

Lesson 4 Automating enrichment and triage

Lesson 5 SOAR playbooks

Lesson 6 Automated response safely

Lesson 7 Infrastructure and security tests

Lab 1 Parse auth logs (Code editor)

Lab 2 jq pipeline (Terminal)

Lab 3 Paginate an API (Code editor)

Lab 4 Build a playbook (Diagram builder)

Lab 5 Test a hardening rule (Code editor)

Checkpoint 10 questions, 80% to pass

Bash for security tasks

Bash is on nearly every server, so it is the fastest way to triage a host or reshape log data without installing anything.

Pipelines and the core tools

The Unix philosophy is small tools chained with pipes. `grep` selects lines, `awk` splits fields and does arithmetic, `sort` and `uniq -c` count, `cut` and `tr` reshape text, and `find` walks directories by name, owner, permission or modification time. A classic triage pipeline is: `grep 'Failed password' /var/log/auth.log | awk '{print $(NF-3)}' | sort | uniq -c | sort -rn | head`. Each stage reads standard input and writes standard output, so you can build and test a pipeline one stage at a time.

For structured data use `jq`, which parses JSON properly instead of guessing with regular expressions. `jq -r '.alerts[] | select(.severity=="high") | .host'` prints the host of every high severity alert, and `-r` removes the JSON quotes. Prefer `jq` to `grep` for JSON because field order, whitespace and nesting can change between tool versions and silently break a text match.

Quoting and safe expansion

Most Bash security bugs are quoting bugs. An unquoted variable is split on whitespace and expanded as a glob, so a file named 'a b' becomes two arguments, and a name beginning with a dash may be read as an option. Always write "\$var", use "\${arr[@]}" for arrays, and put `--` before filenames when a command supports it. Never pass untrusted text to `eval` or build a command string from log content, because an attacker who controls a username or User-Agent can control your script.

Use `[[ ]]` for tests, `$(...)` instead of backticks, and read files with `while IFS= read -r line; do ... done < file` so backslashes and leading spaces survive. When you must handle odd filenames, `find -print0` together with `xargs -0` or `read -d "` keeps newlines in names from splitting records.

Strict mode and loops

By default Bash keeps running after an error, which is dangerous in automation. Start scripts with `set -euo pipefail`: `-e` exits on a failing command, `-u` treats unset variables as errors, and `pipefail` makes a pipeline fail if any stage fails, not only the last. Add a trap on `ERR` or `EXIT` to clean temporary files, create them with `mktemp`, and set `umask 077` so scratch data is private.

Loops let you act on many hosts or files, for example `for ip in $(cat blocklist.txt)` is fragile, while `while read -r ip` is safe. Validate every input against an expected pattern before using it, log what the script did with a timestamp, and test with `shellcheck`, which catches many quoting and portability mistakes before they reach production.

Where Bash stops

Bash is excellent for glue and quick triage, but it is weak at nested data, error handling and testing. When a script grows past about a hundred lines, needs real data structures, or must call several APIs, move it to Python. Keep the shell version only as a thin wrapper that validates arguments and calls the tool.

TERMINAL OUTPUT

```
$ grep 'Failed password' auth.log | awk '{print $(NF-3)}' | sort | uniq -c | sort -rn | head -3
212 198.51.100.23
41 203.0.113.9
6 192.0.2.77
```

WORKED EXAMPLE

An analyst needs the top sources of failed SSH logins from `auth.log`. The pipeline above extracts the source address field, counts duplicates and sorts descending. The script wrapping it begins with `set -euo pipefail`, quotes the log path in "\$1", and exits with an error if the file does not exist. The top address, 198.51.100.23, with 212 failures, becomes a candidate for a firewall rule after the analyst confirms it is not a legitimate scanner.

KEY TERMS

pipefail: A Bash option that makes a pipeline return failure if any command in it fails.

jq: A command line JSON processor used to filter and reshape structured output.

Quoting: Wrapping expansions in double quotes so values are not word-split or globbed.

Globbering: Shell expansion of wildcard patterns such as *.log into file names.

shellcheck: A static analysis tool that reports common shell scripting errors.

REMEMBER

- Chain small tools with pipes and build pipelines stage by stage.
- Quote every expansion and never eval untrusted text.
- Use set -euo pipefail, mktemp, traps and shellcheck.

QUICK CHECK

1. What does set -o pipefail change?

- A) A pipeline fails if any stage fails
- B) Errors are hidden
- C) Variables become read-only
- D) Pipes run in parallel

2. Why quote "\$file"?

- A) It makes it uppercase
- B) It stops word splitting and globbing on odd names
- C) It encrypts the value
- D) It speeds up the script

3. Best tool to filter fields in JSON?

- A) jq
- B) wc
- C) tr
- D) grep

4. What does set -u do?

- A) Unsets all variables
- B) Treats unset variables as errors
- C) Uses UTF-8
- D) Updates the shell

Answers: 1-A 2-B 3-A 4-B

1. Without it only the last command's status is returned.
2. Unquoted values are split on whitespace and expanded as globs.
3. jq parses the structure instead of matching text.
4. It catches typos in variable names that would expand to empty strings.

Python for security engineering

Python handles messy data, APIs and larger tools better than shell, and it is the common language of security automation.

Reading files and parsing logs

Open files with a context manager, with `open(path, encoding='utf-8', errors='replace')` as `f`, and iterate line by line so a multi gigabyte log never has to fit in memory. Use `collections.Counter` to count events and `defaultdict` to group them. For structured formats prefer the `json`, `csv` and `ipaddress` modules over string slicing; `ipaddress.ip_address()` both validates and normalizes an address, and lets you test membership in a network with `ip` in `ip_network('192.0.2.0/24')`.

`Pathlib` gives portable path handling, and `datetime` with timezone aware objects avoids the classic bug of comparing local and UTC timestamps. Parse timestamps once, convert to UTC, and keep them as `datetime` objects until output.

Regular expressions that stay safe

The `re` module extracts fields from semi structured logs. Compile patterns once with `re.compile`, use named groups such as `(?P<ip>\d{1,3}(\.\d{1,3}){3})`, and always handle the case where `match()` returns `None`. Anchor patterns and avoid nested quantifiers like `(a+)+`, which can cause catastrophic backtracking and let a crafted log line freeze your parser, a denial of service known as ReDoS.

A regex proves shape, not meaning: `999.999.999.999` matches the pattern above but is not an address. Validate extracted values afterward with `ipaddress` or a proper parser, and treat all log content as untrusted input.

Building real tools

Turn scripts into tools with `argparse`, which gives help text, typed arguments and clear errors, and use the logging module rather than `print` so verbosity can be controlled. Isolate dependencies with `python3 -m venv .venv` and pin versions in `requirements.txt`, ideally with hashes, so a tool behaves the same on every analyst machine and a poisoned package update is easier to notice.

Never use `subprocess` with `shell=True` on untrusted input, never use `pickle` or `yaml.load` on data you did not create, and keep secrets in environment variables or a secrets manager, not in source code. Write small functions that take data and return data so they can be unit tested with `pytest`.

Habits that prevent mistakes

Handle bad input as the normal case. Logs contain truncated lines, odd encodings and attacker supplied text, so wrap parsing in `try` and `except`, count the lines you could not parse, and report that number so silent data loss is visible. Keep configuration such as thresholds in arguments or a file, write one `pytest` case per parsing rule, and review dependencies before adding them, since every package is code you now trust.

PYTHON

```
import re, argparse
from collections import Counter
rx = re.compile(r'Failed password .* from (?P<ip>\S+)')
c = Counter()
for line in open(args.log, errors='replace'):
    m = rx.search(line)
    if m:
        c[m.group('ip')] += 1
print(c.most_common(5))
```

WORKED EXAMPLE

You receive a 2 GB authentication log and must list brute force sources. Reading line by line with a compiled regex and a `Counter` keeps memory flat. After counting, you validate each captured string with `ipaddress.ip_address` and skip failures, then print only sources above a threshold of 20. Arguments for log path and threshold come from `argparse`, so a colleague can reuse the tool without editing code, and a `pytest` case with three sample lines confirms the counts.

KEY TERMS

Counter: A dictionary subclass in collections that counts hashable items.

argparse: The standard library module for command line option parsing.

venv: A virtual environment that isolates a project's Python packages.

ReDoS: A denial of service caused by a regex with catastrophic backtracking.

ipaddress: A standard module that validates and manipulates IP addresses and networks.

REMEMBER

- Stream files, count with Counter, parse times as UTC.
- Use anchored regex, then validate extracted values.
- Package tools with argparse, logging, venv and tests.

QUICK CHECK

1. Why iterate a file line by line?

- A) It keeps memory use low on large logs
- B) It sorts the data
- C) It is encrypted
- D) It disables errors

2. What is a risk of subprocess with shell=True?

- A) Slower output
- B) Command injection from untrusted input
- C) Python 2 only
- D) No stdout

3. Why use a virtual environment?

- A) To speed up CPU
- B) To isolate and pin dependencies
- C) To avoid logging
- D) To hide code

4. A regex match of 999.999.999.999 shows what?

- A) Python bug
- B) Pattern is wrong
- C) Regex checks shape not validity
- D) The IP is real

Answers: 1-A 2-B 3-B 4-C

1. Streaming avoids loading the whole file.
2. The shell interprets metacharacters in the input.
3. It keeps packages consistent and separate.
4. Validate with ipaddress after extracting.

Working with REST APIs

Almost every security product exposes a REST API, and correct handling of auth, paging and limits separates a toy script from a dependable one.

Requests, authentication and errors

REST APIs use HTTP methods on resources: GET reads, POST creates, PUT or PATCH updates, DELETE removes, and responses are usually JSON. In Python the requests library makes this concise: `requests.get(url, headers=..., params=..., timeout=10)`. Always set a timeout, because the default is none and a hung connection will stall your automation forever. Check `r.status_code` or call `r.raise_for_status()` and handle 401 (bad credentials), 403 (not permitted), 404, 429 (rate limited) and 5xx (server trouble) differently.

Common auth schemes are API keys in a header, bearer tokens as `Authorization: Bearer <token>`, and OAuth 2.0 client credentials that exchange an ID and secret for a short lived token. Read secrets from environment variables, never print them, and never commit them. Use a `requests.Session` to reuse connections and share headers.

Pagination and rate limits

APIs return large result sets in pages. Offset or page-number paging is simple but can skip or duplicate items if data changes mid scan; cursor paging, where each response includes a next token or a Link header, is more reliable. The loop is: request, process items, read the next cursor, and stop only when it is absent. A script that stops after the first response silently misses data, which in security means missed alerts.

Rate limits protect servers. A 429 response often includes a `Retry-After` header, and many APIs report remaining quota in headers such as `X-RateLimit-Remaining`. Honor them instead of hammering the service, since persistent violations can get your key blocked.

Retries and idempotency

Retry only transient failures: timeouts, connection resets, 429 and 502, 503 or 504. Use exponential backoff with jitter, for example waiting about 1, 2, 4 and 8 seconds plus a random fraction, and cap the number of attempts. Do not retry 400, 401 or 404, because repeating a bad request cannot fix it.

Be careful retrying POST requests that create or change things. GET, PUT and DELETE are idempotent by definition, so repeating them is safe, while a repeated POST can open duplicate tickets or isolate a host twice. Where supported, send an idempotency key, and log every request ID so failures can be traced.

Reading API documentation

Before coding, read the vendor documentation for four facts: the authentication method, the paging style, the rate limit and the error format. Test calls with curl first, redact tokens from any saved output, and use a read only key while developing. Store a sample response in your repository so tests can run offline with a mocked session instead of calling the live service on every run.

PYTHON

```
url = 'https://api.example.com/v1/alerts'
params = {'limit': 100}
while url:
    r = s.get(url, params=params, timeout=10)
    if r.status_code == 429:
        time.sleep(int(r.headers.get('Retry-After', 5))); continue
    r.raise_for_status()
    body = r.json(); handle(body['items'])
    url, params = body.get('next'), None
```

WORKED EXAMPLE

A script pulls alerts from a fictional API at `api.example.com` but only ever shows 100 results, and it crashes during busy periods. The fix is to follow the next link until it is empty, so every page is read, and to sleep for the `Retry-After` value on a 429 instead of failing. Adding a 10 second timeout and capping retries at five prevents a hang, and the API token comes from an environment variable named `EXAMPLE_TOKEN`.

KEY TERMS

Pagination: Splitting a large result set into pages fetched with offsets, page numbers or cursors.

429: The HTTP status code meaning too many requests, often with a Retry-After header.

Exponential backoff: Increasing the wait between retries, usually with random jitter.

Idempotent: An operation that has the same effect when repeated.

Bearer token: A credential sent in the Authorization header to prove access.

REMEMBER

- Set timeouts, check status codes, keep secrets out of code.
- Follow cursors until exhausted and honor Retry-After.
- Retry transient errors with backoff; beware repeating POST.

QUICK CHECK

1. Which status means rate limited?

- A) 500
- B) 401
- C) 429
- D) 403

2. Why set a timeout in requests?

- A) It is required by HTTP
- B) It adds headers
- C) Default is none and calls can hang forever
- D) It encrypts traffic

3. Which is safe to retry blindly?

- A) POST isolate host
- B) GET a resource
- C) POST create ticket
- D) POST send email

4. A loop that reads only page one most likely...

- A) Misses data
- B) Is secure
- C) Is faster and correct
- D) Retries forever

Answers: 1-C 2-C 3-B 4-A

1. 429 is Too Many Requests.

2. Without a timeout a stalled server blocks the script.

3. GET is idempotent.

4. It ignores the next cursor so later items are missed.

Automating enrichment and triage

Analysts waste hours looking up the same facts by hand; enrichment and scoring give them context in seconds so they can focus on judgment.

What enrichment adds

Enrichment attaches context to an indicator of compromise (IOC) such as an IP, domain, URL or file hash. Typical sources are threat intelligence services (reputation and malware family), passive DNS, WHOIS registration age, GeoIP and ASN data, and internal data like asset owner, user role and earlier alerts for the same host. A domain registered two days ago and seen in no other network is more suspicious than one that is ten years old and popular.

Hashes are the cleanest lookups because they identify a file exactly. IPs and domains change hands quickly, so always record the time of the lookup and the source, and treat any single verdict as one signal. Never submit private or customer data, such as an unreleased document, to a public service, because uploads may become visible to others.

Scoring and prioritizing alerts

A simple scoring model adds weighted points: base severity from the detection rule, plus reputation hits, plus asset criticality, plus whether the user is an administrator, minus points for known benign tags such as a scanner on an approved list. Keep weights in a config file so they can be tuned, and show the contributing factors beside the number so an analyst can see why the score is 82.

Decide thresholds in advance: above 80 page the on-call analyst, 40 to 80 queue for review, below 40 auto-close with a note. Measure the result by sampling closed alerts for missed true positives, and compare false positive rates before and after, otherwise automation may quietly hide real incidents.

Building it reliably

Enrichment depends on external services, so design for failure. Cache results with a time to live to cut cost and stay inside rate limits, run lookups concurrently with limits, and when a service is down mark the field unknown rather than guessing clean. Normalize every provider's response into one schema, such as {indicator, type, source, verdict, confidence, fetched_at}.

Write the output back where analysts work, usually as ticket fields or case comments, and keep an audit trail of what was queried. Defang IOCs in notes, for example `hxxp://bad[.]example[.]com`, so nobody clicks a live link by accident.

Keeping humans in the loop

Automated scoring should assist judgment, not replace it. Let analysts override a score with a reason, record those overrides, and review them monthly to improve weights. Periodically sample auto-closed alerts and replay known attack simulations to confirm the pipeline still raises them. Document every data source and its limits, so a reader knows that a clean verdict means only that no service had seen the indicator.

PYTHON

```
score = rule_severity # 0-40
score += 25 if intel['malicious'] >= 5 else 0
score += 15 if asset['tier'] == 1 else 0
score += 10 if user['is_admin'] else 0
score -= 30 if ip in APPROVED_SCANNERS else 0
action = 'page' if score >= 80 else 'queue' if score >= 40 else 'close'
```

WORKED EXAMPLE

An alert fires for outbound traffic from a finance server to 203.0.113.50. The script queries a reputation API and finds 7 vendors flag the address, fetches the asset record showing a tier 1 system, and sees the user is not an admin. The score is 35 for the rule, plus 25 and 15, totalling 75, so it is queued as high priority with the three factors listed in the ticket. An analyst confirms in minutes instead of researching from scratch.

KEY TERMS

IOC: An observable such as a hash, IP or domain that indicates possible compromise.

Enrichment: Adding context from other sources to an alert or indicator.

Passive DNS: A historical record of which names resolved to which addresses.

Defanging: Altering a URL or IP so it cannot be clicked or resolved accidentally.

TTL cache: Stored results that expire after a set time to limit repeat lookups.

REMEMBER

- Enrich with reputation, age, asset and user context.
- Score with transparent weights and pre-set thresholds.
- Cache, normalize, fail safe and audit every lookup.

QUICK CHECK

1. Why are file hashes reliable IOCs?
 - A) They never change owners
 - B) They are secret
 - C) They identify a specific file exactly
 - D) They are short
2. If an intel API is down, the script should...
 - A) Delete the alert
 - B) Mark unknown and continue
 - C) Mark clean
 - D) Retry forever
3. Why show score factors?
 - A) Required by HTTP
 - B) Looks nicer
 - C) Analysts can see why and tune it
 - D) To save storage
4. Why defang IOCs in notes?
 - A) To encrypt them
 - B) To prevent accidental clicks
 - C) To block email
 - D) To shorten text

Answers: 1-C 2-B 3-C 4-B

1. A cryptographic hash is unique to file content.
2. Failure must not be treated as a clean verdict.
3. Transparency enables trust and tuning.
4. Defanged links are not live.

SOAR playbooks

SOAR platforms codify response procedures so routine incidents get fast, consistent handling and humans spend time on decisions.

Anatomy of a playbook

SOAR stands for security orchestration, automation and response. A playbook is a workflow with a trigger, a series of actions, decision points and an end state. Triggers can be a SIEM alert, a reported phishing email, a webhook or a schedule. Actions call integrations: query an EDR, look up a hash, search mailboxes, open a ticket or post to chat. Decisions branch on data, for example if the sender domain is younger than 30 days or the URL is rated malicious.

Good playbooks are small and single purpose, such as 'phishing triage' or 'compromised account'. Reuse sub playbooks for common steps like enrichment, and version them in source control so changes are reviewed like code.

Branching, approvals and ordering

Order steps from cheap and reversible to costly and disruptive. First gather evidence and enrich, then decide, then contain. Branch on confidence: high confidence with low impact can be automated, while anything touching executives, servers or production data should pause for human approval. An approval step presents the evidence, the proposed action and its blast radius, then waits for a named approver with a timeout that escalates rather than proceeds.

Include parallel branches where safe, for example enriching URLs and attachments at the same time, and always define what happens on failure of each step. A playbook that silently stops half way is worse than no playbook.

Choosing what to automate

Start with high volume, low risk, well understood tasks: phishing triage, duplicate alert closure, IOC enrichment and ticket creation. Document the manual procedure first, because you cannot automate a process nobody has defined. Measure time saved and error rate, and review which cases analysts override.

Safe response actions include quarantining an email, disabling a single user session, blocking a hash or URL at the proxy, and isolating a non-critical endpoint. Riskier ones, such as disabling accounts organization wide or blocking entire address ranges, need approval. Record every action, input and decision in the case so the incident can be reconstructed.

Testing and maintaining playbooks

Test playbooks with recorded sample alerts in a sandbox before connecting them to production tools. Give each step a timeout and an error branch, track execution metrics such as success rate and median run time, and review failed runs weekly. Integrations change when vendors update APIs, so assign an owner to each playbook and re-test after upgrades. Retire playbooks nobody uses, because stale automation is an unreviewed risk.

PLAYBOOK OUTLINE

```
Trigger: user-reported phish (mailbox phish@example.com)
1 Extract URLs, attachments, headers
2 Enrich in parallel: URL rep, hash rep, domain age
3 Decision: any malicious? -> yes: step 4, no: step 7
4 Search mail for same message; list recipients
5 Approval: analyst confirms purge
6 Quarantine copies, block URL at proxy
7 Reply to reporter, close or escalate case
```

WORKED EXAMPLE

A user forwards a suspicious email. The playbook extracts a link and attachment hash, finds the link rated malicious and the domain eleven days old, then searches mailboxes and finds 14 more recipients. An analyst receives an approval prompt listing the 14 recipients, approves, and the playbook quarantines the messages, blocks the URL and updates the ticket. The reporter receives a thank-you note. Total handling time drops from about 40 minutes to 5, and every step is logged.

KEY TERMS

SOAR: Tooling that orchestrates security products and automates response workflows.

Playbook: A defined workflow of triggers, actions and decisions for a type of incident.

Approval gate: A step that pauses automation until a human approves.

Blast radius: The scope of systems or users affected by an action.

Integration: A connector that lets a playbook call another product's API.

REMEMBER

- Playbooks combine triggers, actions, decisions and end states.
- Order cheap and reversible first; gate risky steps.
- Automate well defined, high volume tasks first and log everything.

QUICK CHECK

1. What starts a playbook?

- A) A backup
- B) Only a human
- C) A trigger such as an alert or webhook
- D) A firewall rule only

2. Which step should come first?

- A) Isolate the server
- B) Disable all users
- C) Gather and enrich evidence
- D) Delete mailboxes

3. When is an approval gate best?

- A) For logging
- B) For high impact actions on critical assets
- C) For duplicate closure
- D) Never

4. Why version playbooks in Git?

- A) Faster execution
- B) Reviewable, reversible changes
- C) Encryption
- D) Licensing

Answers: 1-C 2-C 3-B 4-B

1. Triggers include alerts, emails, webhooks and schedules.
2. Cheap, reversible steps go before disruptive ones.
3. Impactful actions need human judgment.
4. Treat playbooks as code.

Automated response safely

Automation acts at machine speed, so a single flawed rule can lock out an entire company; guardrails keep speed from becoming damage.

Guardrails and scope limits

The first guardrail is an allowlist of assets and accounts that automation must never touch, such as domain controllers, break-glass accounts, executives' accounts and critical production servers. Next come rate limits: if a playbook is about to isolate more than five hosts in ten minutes, it should stop and page a human, because that pattern usually means a bad detection, not five simultaneous compromises. Add confidence thresholds, so low scoring alerts never trigger containment.

Use least privilege for the automation account. It may need permission to isolate an endpoint but not to delete one, and its credentials should be scoped, rotated and monitored like any privileged identity.

Dry run, logging and approvals

A dry-run or 'what if' mode performs every step except the final change and prints what it would have done. Run new playbooks in dry-run against live alerts for a week and compare with analyst decisions before enabling enforcement. Make dry-run the default and require an explicit flag such as `--apply` to act.

Log the who, what, when, why and result of every action: the triggering alert, the evidence, the target, the API request ID and the outcome. Send logs to a separate system the automation cannot alter. Require human approval for destructive or wide actions, and make the approval record part of the case.

Rollback and fail-safe design

Every containment action needs a defined undo. Before blocking an address, record the previous rule state; before disabling an account, record its group memberships and sessions. Prefer reversible actions such as quarantine, temporary block with an expiry, or network isolation that leaves management access intact, over deletion. Expiring blocks after 24 hours prevents stale rules from accumulating.

Design for failure. If an API call errors halfway, the playbook should detect partial state and either complete or roll back, and it should notify a human. Include a kill switch, a single flag that disables all automated enforcement instantly, and test it. Finally, review automation actions regularly and track false containment events as a metric.

Governance of automation

Treat response automation as a change controlled system. Every new action needs a documented owner, a risk rating, a tested rollback and sign-off from the teams whose assets it may touch, such as IT operations. Run periodic game days where you trigger the playbook on a test asset and confirm that logging, approval, expiry and the kill switch all work as designed.

SCRIPT EXCERPT

```
if host in PROTECTED: raise Skip('protected asset')
if recent_isolations() >= 5: raise Halt('rate limit, page oncall')
log.info('plan isolate %s alert=%s', host, alert_id)
if not args.apply: return 'dry-run: no change'
save_state(host); edr.isolate(host, expires='24h')
log.info('isolated %s id=%s', host, req_id)
```

WORKED EXAMPLE

A new rule flags beaconing and the playbook is set to isolate hosts automatically. In dry-run for a week it proposes isolating 38 hosts, but analysts judge only 3 legitimate, revealing that a backup agent causes the pattern. The team tunes the rule, adds the backup server to the allowlist, requires approval for tier 1 assets, and adds the five-per-ten-minute limiter. Only then do they enable enforcement, with automatic 24 hour expiry and a documented rollback command.

KEY TERMS

Allowlist: A list of assets or accounts exempt from automated action.

Dry run: Executing logic without making the final change, to preview effects.

Kill switch: A control that instantly disables automated enforcement.

Rollback: Restoring the state that existed before an action.

Rate limiter: A cap on how many actions may occur in a time window.

REMEMBER

- Protect critical assets with allowlists and limits.
- Default to dry-run and log every action immutably.
- Make actions reversible with expiry, rollback and a kill switch.

QUICK CHECK

1. Best default for a new response script?

- A) Dry-run unless --apply is given
- B) Run as root
- C) No logs
- D) Apply immediately

2. Why a rate limit on isolations?

- A) Mass actions often signal a bad rule
- B) Avoid licenses
- C) Hide actions
- D) Save bandwidth

3. What makes an action reversible?

- A) Using no logs
- B) Running faster
- C) Deleting data
- D) Saving prior state and using expiry

4. Why least privilege for automation accounts?

- A) Limits damage if abused or buggy
- B) Cheaper
- C) Faster API
- D) Required by Python

Answers: 1-A 2-A 3-D 4-A

1. Safe by default, explicit to enforce.
2. Many simultaneous hits suggest a false positive pattern.
3. Recorded state and expiry allow undo.
4. Smaller permission means smaller blast radius.

Infrastructure and security tests

Hardening decays as systems change; automated tests turn security requirements into checks that fail loudly when something regresses.

Why test security controls

A hardening standard written in a document is a wish; a test is evidence. Infrastructure tests assert desired state, for example that SSH root login is disabled, a port is closed, a file has mode 0600 or a package is absent. Running them in a CI pipeline and on a schedule detects configuration drift, the slow divergence between what you intended and what exists, as well as regressions introduced by a new image or a well meaning change.

Good tests describe the requirement in plain terms and check the effective setting, not just the file. For SSH, the effective value is shown by `sshd -T`, which accounts for defaults and included files, whereas grepping `sshd_config` may miss an override.

Tools: testinfra, InSpec and pytest

Testinfra is a pytest plugin with fixtures for hosts: `host.file('/etc/ssh/sshd_config')`, `host.service('sshd')`, `host.socket('tcp://0.0.0.0:23').is_listening`. InSpec, from Chef, uses a readable Ruby DSL with controls such as `describe sshd_config do its('PermitRootLogin') { should eq 'no' } end`, and ships profiles that implement CIS and DISA STIG benchmarks. Plain pytest also works well for cloud checks by calling APIs, for example asserting no storage bucket policy allows public reads.

Pick the tool your team can maintain. Whichever you use, keep tests in version control, tag them with the standard they implement, such as CIS 5.2.10, and run them against a representative test host before production.

Writing tests that stay useful

Write a test to fail first on a known bad configuration so you know it can detect the problem, then fix and watch it pass. Keep each test focused on one control, give failures a clear message naming the host and setting, and avoid tests that always pass because they inspect the wrong thing. Treat exceptions explicitly: a documented, time limited waiver is better than deleting the test.

Integrate results into the pipeline so a failing control blocks promotion of a base image, and export results as evidence for audits. Add negative tests too, such as confirming that an unauthorized role cannot read a secret, because a control that blocks nothing is not a control. Review failure trends over time to find recurring drift sources.

Shifting tests left

Run the same checks earlier in the lifecycle. Scan infrastructure as code with policy tools before deployment, test container images in the build pipeline, and run the host checks against every golden image. Failing early is cheap: a rejected pull request costs minutes, while discovering an open management port in production costs an incident. Track the pass rate per control so the team can see whether hardening is improving.

PYTHON TEST (TESTINFRA)

```
def test_ssh_root_login_disabled(host):
    out = host.check_output('sshd -T')
    assert 'permitrootlogin no' in out.lower()

def test_telnet_not_listening(host):
    assert not host.socket('tcp://0.0.0.0:23').is_listening
```

WORKED EXAMPLE

A team bakes server images with a CIS aligned SSH config. A later change to a configuration management role accidentally sets `PermitRootLogin yes` on one environment. The nightly test run executes `sshd -T` on each host, the assertion for `'permitrootlogin no'` fails on three servers, and the pipeline sends an alert naming them. Engineers fix the role, rebuild the image and rerun, and the passing report is stored as audit evidence.

KEY TERMS

Configuration drift: Gradual deviation of a system from its intended configuration.

Testinfra: A pytest based framework for testing server state.

InSpec: A compliance and testing framework with a readable DSL and benchmark profiles.

CIS Benchmark: A consensus hardening guide with numbered configuration recommendations.

Regression: A previously fixed problem that reappears after a change.

REMEMBER

- Turn hardening rules into automated assertions.
- Check effective state with tools like `sshd -T`.
- Run in CI and on a schedule; keep results as evidence.

QUICK CHECK

1. What does `sshd -T` show?
 - A) Effective SSH daemon settings
 - B) Users
 - C) Only the file text
 - D) Open ports
2. Why run tests on a schedule?
 - A) Avoid logging
 - B) Detect drift over time
 - C) Rotate keys
 - D) Make hosts faster
3. A good first step when writing a test is to...
 - A) Confirm it fails on a bad config
 - B) Disable it
 - C) Make it always pass
 - D) Skip failing
4. Which is an InSpec feature?
 - A) Packet capture
 - B) Disk encryption
 - C) DNS hosting
 - D) Readable DSL controls and benchmark profiles

Answers: 1-A 2-B 3-A 4-D

1. It prints the active, merged configuration.
2. Settings change after deployment.
3. A test that cannot fail proves nothing.
4. InSpec profiles map to CIS and STIG.

Parse auth logs

SCENARIO

A fictional host, `srv01.example.com`, wrote an `auth.log` with thousands of SSH events. Several lines show failed logins from addresses in `198.51.100.0/24` and `203.0.113.0/24`. You must write a Python script that counts failed logins per source address and flags any above a threshold of 10.

OBJECTIVE

Count failures per IP and print those above the threshold.

STEPS IN THE SIMULATOR

- 1 **Do:** Open `auth.log` in the editor and view the first lines

Simulator shows:

```
Lines like 'Jan 12 03:14:07 srv01 sshd[411]: Failed password for invalid user admin from 198.51.100.23 port 51022 ssh2'
```

- 2 **Do:** Add a compiled regex with a named group for the source address

Simulator shows:

```
rx = re.compile(r'Failed password .* from (?P<ip>\S+)') accepted
```

- 3 **Do:** Loop over lines, count with Counter

Simulator shows:

```
Counter updated; 3 lines skipped as non-matching
```

- 4 **Do:** Validate each captured IP with `ipaddress.ip_address`

Simulator shows:

```
0 invalid addresses; counter has 4 sources
```

- 5 **Do:** Filter counts greater than 10 and print them sorted

Simulator shows:

```
198.51.100.23 212
203.0.113.9 41
```

- 6 **Do:** Run the tests

Simulator shows:

```
2 passed: counts correct, 192.0.2.77 (6 failures) not flagged
```

ANSWER

The script flags `198.51.100.23` (212 failures) and `203.0.113.9` (41 failures). `192.0.2.77` with 6 failures stays below the threshold. The solution streams the file, uses a Counter and validates addresses before reporting.

Hint shown in the app: Use `Counter.most_common` or a dictionary comprehension that keeps counts greater than the threshold.

jq pipeline

SCENARIO

You have alerts.json, output from a fictional detection tool, holding an array of alerts with fields id, severity, host, rule and tags. Management wants the hosts that have high severity alerts, without duplicates, and a count per rule.

OBJECTIVE

Use jq to extract hosts with high severity alerts and count alerts per rule.

STEPS IN THE SIMULATOR

- 1 **Do:** Run: `jq length alerts.json`

Simulator shows:
42

- 2 **Do:** Run: `jq -r '[] | select(.severity=="high") | .host' alerts.json`

Simulator shows:
web01.example.com
db02.example.com
web01.example.com
vpn03.example.com

- 3 **Do:** Add sort -u to remove duplicates

Simulator shows:
db02.example.com
vpn03.example.com
web01.example.com

- 4 **Do:** Run: `jq -r '[] .rule' alerts.json | sort | uniq -c | sort -rn`

Simulator shows:
17 Suspicious PowerShell
9 SSH Brute Force
6 New Admin Account

- 5 **Do:** Try `jq '[] | select(.tags | index("vip"))' alerts.json`

Simulator shows:
2 alerts shown, both for host vpn03.example.com

- 6 **Do:** Save results to high_hosts.txt

Simulator shows:
high_hosts.txt written with 3 lines

ANSWER

Three unique hosts have high severity alerts: db02.example.com, vpn03.example.com and web01.example.com. Rule counts are 17, 9 and 6, with Suspicious PowerShell the most frequent. select filters, -r removes quotes, and sort -u removes duplicates.

Hint shown in the app: Use `select(.severity=="high")` then pipe the raw output into `sort -u`.

Paginate an API

SCENARIO

A script fetches alerts from the fictional `https://api.example.com/v1/alerts`. It returns only the first 100 items though 340 exist, and it crashes when the server answers HTTP 429. Fix both faults.

OBJECTIVE

Make the script follow pagination and honor Retry-After on 429.

STEPS IN THE SIMULATOR

- 1 **Do:** Run the script as given

Simulator shows:
Fetched 100 alerts. Done.

- 2 **Do:** Inspect the JSON response

Simulator shows:
`{'items': [...100], 'next': 'https://api.example.com/v1/alerts?cursor=abc123'}`

- 3 **Do:** Wrap the request in a while loop that follows `body['next']`

Simulator shows:
Fetched 200 alerts; then `HTTPError 429 Too Many Requests`

- 4 **Do:** Add a branch: if status is 429, sleep Retry-After seconds and continue

Simulator shows:
Sleeping 3s... resumed

- 5 **Do:** Add `timeout=10` and cap retries at 5

Simulator shows:
Config accepted; retry counter shown

- 6 **Do:** Run again

Simulator shows:
Fetched 340 alerts in 4 pages, 1 retry. Done.

ANSWER

The fixed script loops until next is empty, retrieving all 340 alerts across 4 pages, sleeps for the Retry-After value on 429, uses a timeout and limits retries to avoid an endless loop.

Hint shown in the app: The loop condition should depend on the next cursor, not on a fixed number of requests.

Build a playbook

SCENARIO

A user reports a suspicious email to phish@example.com. You must assemble a SOAR playbook from available blocks: Trigger, Extract artifacts, Enrich, Decision, Approval, Quarantine and block, Notify and close.

OBJECTIVE

Order the blocks into a safe phishing response playbook.

STEPS IN THE SIMULATOR

- 1 **Do:** Drag Trigger: reported email to the canvas

Simulator shows:

Node 1 placed; output: email object

- 2 **Do:** Connect Extract artifacts

Simulator shows:

Outputs: 2 URLs, 1 attachment hash

- 3 **Do:** Connect Enrich after extraction

Simulator shows:

URL rating: malicious; hash: unknown; domain age: 11 days

- 4 **Do:** Add a Decision on verdict

Simulator shows:

Branches: malicious or unknown vs. clean

- 5 **Do:** On the malicious branch add Approval before Quarantine and block

Simulator shows:

Approval prompt shows 14 recipients

- 6 **Do:** Add Notify and close at the end of both branches

Simulator shows:

Validation passed: all paths end, containment gated by approval

ANSWER

Correct order: Trigger, Extract artifacts, Enrich, Decision, then on the malicious branch Approval followed by Quarantine and block, and finally Notify and close on every path. Containment comes after enrichment and approval.

Hint shown in the app: Gather evidence first, and put the disruptive action behind a human gate.

Test a hardening rule

SCENARIO

Servers must have SSH root login disabled. A test host in the lab has an sshd configuration you cannot see. Write a pytest or testinfra check that fails when root login is enabled and passes when it is disabled.

OBJECTIVE

Write a test that fails when PermitRootLogin is enabled.

STEPS IN THE SIMULATOR

- 1 **Do:** Open test_ssh.py

Simulator shows:

Empty file with import pytest

- 2 **Do:** Write a test using host.check_output('sshd -T')

Simulator shows:

Function def test_root_login(host) created

- 3 **Do:** Assert 'permitrootlogin no' appears in the lowercased output

Simulator shows:

Assertion added

- 4 **Do:** Run against the vulnerable host

Simulator shows:

FAILED: assert 'permitrootlogin no' in '... permitrootlogin yes ...'

- 5 **Do:** Fix the configuration and reload sshd on the test host

Simulator shows:

sshd reloaded; sshd -T shows permitrootlogin no

- 6 **Do:** Run the test again

Simulator shows:

1 passed

ANSWER

The test runs sshd -T and asserts permitrootlogin no. It fails on the host configured with yes and passes after remediation, proving the check can detect the regression.

Hint shown in the app: Check the effective setting from sshd -T, not just the config file text.

Security Automation and Scripting: 10 questions, 80% to pass

1. Which Bash option makes a pipeline fail if any stage fails?
 - A) set -f
 - B) set -x
 - C) set -e
 - D) set -o pipefail
2. Why prefer jq to grep for JSON?
 - A) It is encrypted
 - B) It parses structure reliably
 - C) It is shorter
 - D) It sorts
3. What is the safest way to pass a user supplied filename to a command in Bash?
 - A) Via backticks
 - B) Via eval
 - C) Quoted, after --
 - D) Unquoted
4. What can a nested quantifier in a regex cause?
 - A) Faster matching
 - B) Syntax highlighting
 - C) Memory encryption
 - D) ReDoS
5. An API returns 429 with Retry-After: 7. You should...
 - A) Change the key
 - B) Wait 7 seconds then retry
 - C) Give up
 - D) Retry immediately
6. Which request type is risky to retry blindly?
 - A) HEAD
 - B) GET
 - C) POST that isolates a host
 - D) GET with params
7. If an enrichment source is unreachable, the field should be...
 - A) Unknown
 - B) Clean
 - C) Deleted
 - D) Malicious
8. Best place for an approval gate in a phishing playbook?
 - A) After closing
 - B) Before reading headers
 - C) Before purging mail from many mailboxes
 - D) Before logging
9. Which feature lets you undo containment automatically after a period?
 - A) Faster API
 - B) Alert tags
 - C) Expiry on the block
 - D) Shorter logs
10. What does a test using sshd -T verify?
 - A) Password strength
 - B) Effective SSH settings
 - C) Network latency
 - D) File timestamps

Answers: 1-D 2-B 3-C 4-D 5-B 6-C 7-A 8-C 9-C 10-B

1. pipefail returns the first nonzero status.
2. Regex on JSON breaks with format changes.
3. Quoting and -- avoid splitting and option injection.
4. Catastrophic backtracking can freeze a parser.
5. Honor the server's rate limit instruction.
6. Non-idempotent actions can repeat their effect.
7. Never convert failure into a verdict.
8. Wide, disruptive actions need human approval.
9. Temporary blocks that expire reduce stale rules.
10. It reports the active merged configuration.

Incident Readiness and Resilience

Intermediate | 5 lessons | 4 simulated labs | checkpoint | about 4h 25m

This module prepares an organization for the day something goes wrong. You will build an incident response plan around NIST SP 800-61, test it with tabletop exercises, design backups and disaster recovery that meet stated RTO and RPO targets, run a business impact analysis and plan for ransomware, and validate detections with purple team exercises. Readiness is not a document on a shelf; it is practiced, measured and corrected before a real attacker forces the test.

BY THE END YOU CAN

- + Write and exercise an incident response plan
- + Design backup and DR to meet RTO and RPO
- + Run purple team exercises to validate controls

IN THIS MODULE

Lesson 1 IR plan and roles

Lesson 2 Tabletop exercises

Lesson 3 Backups and disaster recovery

Lesson 4 Business continuity and ransomware resilience

Lesson 5 Purple team exercises

Lab 1 Tabletop a ransomware case (Scenario)

Lab 2 Pick backup strategy (Config editor)

Lab 3 Recovery order (Diagram builder)

Lab 4 Purple team detection gap (Mock SIEM)

Checkpoint 10 questions, 80% to pass

IR plan and roles

When an incident starts, nobody has time to decide who is in charge; a written, rehearsed plan removes confusion at the worst moment.

The lifecycle and the plan

NIST SP 800-61 Revision 2 describes four phases: preparation; detection and analysis; containment, eradication and recovery; and post-incident activity. The newer Revision 3 aligns incident response with the NIST Cybersecurity Framework 2.0 functions, but the same ideas remain. An IR plan states the mission, scope, definitions of event versus incident, severity levels, and the authority of the response team to act, including taking systems offline. Executive sign-off matters, because responders will need to make disruptive decisions quickly.

Roles and escalation

A plan names roles, not only people, because people go on holiday. Typical roles are the incident commander who owns decisions, a technical lead for investigation, a communications lead, a legal and privacy contact, a scribe who keeps the timeline, and liaisons from IT operations, HR and the business. Escalation criteria tie severity to who is notified and how fast: for example a severity 1 incident, such as confirmed data theft, pages the commander within 15 minutes and informs the executive team within an hour. Keep a current contact list with out-of-band methods.

Communications and legal duties

Communication has three audiences: the response team, internal stakeholders and external parties such as customers, regulators, insurers and law enforcement. Assume the corporate chat and email may be compromised, so define a fallback channel. Legal duties differ by region and sector; GDPR requires notifying the supervisory authority within 72 hours of becoming aware of a personal data breach where required, and many contracts set shorter windows. Route outside statements through one approved spokesperson and use templates prepared in advance.

Playbooks, tools and evidence

Beneath the plan sit playbooks for common scenarios such as ransomware, business email compromise, lost laptop and web application compromise. Each lists triggers, first actions, decision points and who to call. Prepare a jump kit: forensic tools, clean workstations, write blockers, access to logs and a documented chain of custody process so evidence stays usable. Review the plan at least yearly and after every incident or major infrastructure change, and keep a printed or offline copy for when systems are unavailable.

PLAN EXCERPT

Severity 1: confirmed data loss or active ransomware
Notify: Incident Commander within 15 min
Notify: CISO, Legal within 60 min
Decision rights: IC may isolate any system
Backup channel: phone bridge 555-0100 (fictional)
Regulatory clock: privacy officer assesses 72h duty

WORKED EXAMPLE

A fictional company, Example Corp, detects encrypted files on a file server at 02:10. The on-call analyst declares severity 1 under the plan, which automatically pages the incident commander. The commander opens the phone bridge because email is suspect, assigns a scribe, tasks IT to isolate the file server's VLAN, and asks legal to start the 72 hour breach assessment clock. Because roles, thresholds and contact details were already written down, the first hour is spent acting, not arguing.

KEY TERMS

Incident commander: The person with overall authority for decisions during an incident.

Severity level: A classification that determines urgency, escalation and notification.

Playbook: A scenario specific checklist of response actions.

Chain of custody: Documentation of who handled evidence and when.

Out-of-band channel: A communication path independent of the possibly compromised network.

REMEMBER

- Follow the NIST lifecycle and get executive authority in writing.
- Define roles, severity levels, escalation and contact paths.
- Keep scenario playbooks, an evidence process and an offline plan copy.

QUICK CHECK

1. Which are the four NIST SP 800-61 Rev 2 phases?
 - A) Preparation; detection and analysis; containment, eradication and recovery; post-incident activity
 - B) Plan, Do, Check, Act
 - C) Scan, Exploit, Persist, Exfiltrate
 - D) Identify, Protect, Detect, Respond
2. Why define roles rather than names?
 - A) Cheaper
 - B) Law requires it
 - C) People may be unavailable
 - D) Names are secret
3. Why a backup communications channel?
 - A) Faster email
 - B) Lower cost
 - C) Normal channels may be compromised
 - D) Fewer people
4. The scribe's job is to...
 - A) Fix servers
 - B) Talk to press
 - C) Keep the timeline of events and decisions
 - D) Approve budget

Answers: 1-A 2-C 3-C 4-C

1. That is the Rev 2 lifecycle.
2. Roles can be filled by deputies.
3. Attackers may read corporate chat or email.
4. A timeline supports analysis and later review.

Tabletop exercises

A plan only proves its worth when people rehearse it; tabletops expose gaps cheaply, without risking production.

What a tabletop is

A tabletop exercise is a facilitated discussion in which participants walk through a realistic scenario and describe what they would do. No systems are touched. It differs from a full simulation or a technical red team test, and it is the cheapest way to test decisions, roles, communication and the plan itself. Include not only security staff but also IT operations, legal, communications, HR, a business owner and an executive sponsor, because most real incident delays come from decisions, approvals and handoffs rather than from technology.

Designing scenarios and injects

Start with an objective, such as testing the ransomware playbook or the breach notification decision. Write a believable scenario drawn from your threat landscape, then script injects, the new pieces of information fed in at timed intervals: 'Backups appear to be encrypted', 'A journalist emails asking about a leak', 'The attacker posts sample data'. Good injects force trade-offs, create time pressure and introduce ambiguity. Prepare facilitator notes with expected actions, so the discussion can be compared against the plan, and keep the scenario fictional or anonymized.

Running the session

A facilitator keeps time, reads injects, asks open questions such as 'who decides that, and how do you reach them at 3 a.m.?' and stops arguments that drift into blame. An observer records decisions, delays and assumptions. Sessions typically last two to three hours. Use a no-fault rule: the purpose is to find weaknesses in the process, and people who admit they do not know something are helping. Rotate scenarios so different teams and plans, including cloud and third-party failures, are tested over the year.

Capturing and closing gaps

Immediately after, hold a hot wash to gather impressions, then write a short report listing findings, evidence from the notes, a severity, a named owner and a due date. Track actions in the same ticket system used for engineering work and review them at a regular meeting until closed. Examples include an outdated contact list, no authority to shut down a production line, or no template for customer notice. Measure improvement by repeating a similar scenario later and checking that earlier gaps no longer appear.

INJECT CARD

T+00 Alert: file server shows .locked extensions
 T+20 Inject: backups console unreachable
 T+45 Inject: ransom note demands payment, 72h deadline
 T+70 Inject: customer asks if their data is affected
 Questions: who isolates? who informs legal? who speaks?

WORKED EXAMPLE

During a tabletop, the group reaches the inject about customer data and discovers nobody knows who may approve a public statement. The legal lead assumes communications decides, while communications assumes the CEO decides. The observer records a gap titled 'No approval path for external statements', assigns it to the communications director, and sets a due date four weeks out. At the next quarterly exercise the group checks that a one page approval flow exists and that it works.

KEY TERMS

Inject: A scripted piece of new information introduced during an exercise.

Hot wash: A quick debrief right after an exercise.

Facilitator: The person who guides the exercise and keeps it on track.

No-fault rule: A ground rule that the exercise tests process, not individuals.

Action item: A tracked task with an owner and due date that closes a gap.

REMEMBER

- Tabletops are low cost discussions that test the plan.
- Use timed injects, mixed participants and a no-fault rule.

- Record gaps as owned, dated actions and re-test later.

QUICK CHECK

1. What is an inject?
 - A) A log type
 - B) A software patch
 - C) A firewall rule
 - D) New information fed during the exercise
2. A tabletop primarily tests...
 - A) Encryption strength
 - B) Network speed
 - C) Decisions, roles and communication
 - D) Antivirus signatures
3. Why include legal and communications?
 - A) Many incident delays involve decisions and notices
 - B) Cost sharing
 - C) They write code
 - D) Tradition
4. Best way to treat findings?
 - A) Owner, due date and tracking
 - B) Email once
 - C) Remember them
 - D) Ignore minor ones

Answers: 1-D 2-C 3-A 4-A

1. Injects drive the scenario forward.
2. It is a discussion based exercise.
3. Incidents are business events, not only technical.
4. Tracked actions get closed.

Backups and disaster recovery

Backups are the last line of defense against ransomware and failure, but only if they are intact, isolated and proven to restore.

3-2-1-1-0 and immutability

The classic 3-2-1 rule says keep three copies of data, on two different media types, with one copy off site. The modern extension 3-2-1-1-0 adds one copy that is offline, air-gapped or immutable, and zero errors after verification. Immutable storage uses write once, read many features such as object lock in compliance mode, so even an administrator with stolen credentials cannot delete or encrypt backups before the retention period ends. Separate backup credentials from the domain, and require multi person approval to change retention.

RPO and RTO

Two numbers drive design. The recovery point objective (RPO) is the maximum tolerable data loss measured in time: an RPO of one hour means backups or replication must occur at least hourly. The recovery time objective (RTO) is the maximum tolerable time to restore service: an RTO of four hours means the restore process, bandwidth and staffing must be capable of that. Tighter targets cost more, for example continuous replication versus nightly backup, so business owners should set them based on the cost of downtime and data loss.

Backup types and retention

A full backup copies everything; an incremental copies changes since the last backup of any type; a differential copies changes since the last full. Incrementals are small and fast to take but need the chain to restore, while differentials restore with two sets. Snapshots are quick but usually live on the same storage and do not replace backups. Retention should cover both operational needs and hidden compromise: if an attacker lived in the network for 60 days, you may need clean restore points older than that. Encrypt backups and protect the keys separately.

Restore testing and DR sites

A backup that has never been restored is only a hope. Schedule restore tests, automate integrity checks, and time a full recovery of a critical system against its RTO. Disaster recovery sites come as hot (running, near instant failover), warm (partly provisioned) or cold (space and hardware only), trading cost against recovery time. In the cloud, use separate accounts and regions for copies, and document runbooks so that recovery does not depend on the knowledge of one person. Record every test result and fix failures.

BACKUP POLICY

```
dataset: erp-db
rpo: 1h rto: 4h
schedule: full weekly, incremental hourly
copies: 3 media: disk + object storage
offsite: region-b immutable: 35 days (object lock)
restore_test: monthly, result logged
```

WORKED EXAMPLE

A finance database must lose no more than one hour of data and be back within four hours. The team takes hourly incrementals with a weekly full, keeps a copy in another region with object lock for 35 days, and tests restores monthly. A drill shows the restore takes six hours because of slow transfer. That exceeds the RTO, so they stage the latest full backup in the recovery region and retest, bringing the time to three and a half hours.

KEY TERMS

RPO: The maximum acceptable amount of data loss measured in time.

RTO: The maximum acceptable time to restore a service.

Immutable backup: A copy that cannot be altered or deleted until retention expires.

Air gap: Isolation of a copy from the network.

Hot site: A fully equipped recovery site that can take over almost immediately.

REMEMBER

- Follow 3-2-1-1-0 with an immutable or offline copy.
- Set RPO and RTO with the business and design to meet them.
- Test restores on a schedule and keep recovery runbooks.

QUICK CHECK

1. What does the final 0 in 3-2-1-1-0 mean?
 - A) Zero cost
 - B) Zero encryption
 - C) Zero copies on site
 - D) Zero errors after verification
2. An RPO of 1 hour means...
 - A) Restore takes 1 hour
 - B) At most 1 hour of data loss is tolerable
 - C) Backups last 1 hour
 - D) 1 hour of downtime per year
3. Why are snapshots on the same array not enough?
 - A) They are slow
 - B) They are encrypted
 - C) They share the failure and attack domain
 - D) They are too big
4. What proves a backup is usable?
 - A) Compression
 - B) A large file size
 - C) Green job status
 - D) A tested restore

Answers: 1-D 2-B 3-C 4-D

1. Backups are verified, with no errors.
2. RPO measures data loss.
3. If storage or credentials fail, snapshots go too.
4. Only restore testing proves recoverability.

Business continuity and ransomware resilience

Recovery is not only about restoring files; it is about restoring the right systems, in the right order, while the business keeps functioning.

Business impact analysis

A business impact analysis (BIA) identifies critical business processes and measures the effect of their interruption over time: lost revenue, regulatory penalties, safety impacts and reputational harm. From it you derive the maximum tolerable downtime, then an RTO and RPO for each process and supporting system. Interview process owners, not only IT, because the people who run payroll or order fulfilment know what breaks first. Rank services into tiers, such as tier 0 for identity and networking, tier 1 for revenue systems and tier 2 for internal tools.

Dependencies and recovery order

Systems depend on others, and recovery must follow that graph. Almost everything needs identity services (Active Directory or an identity provider), DNS, DHCP, networking and time before applications can work, so those come first. Next are databases and storage, then application servers, then front ends, then user facing integrations. Map third-party dependencies such as payment providers and SaaS. Record the order in a recovery runbook with owners and estimated times, and note any circular dependency, for example a backup system that needs the very directory it must restore.

Ransomware scenarios

Modern ransomware often combines data theft with encryption, so recovery from backup solves only part of the problem; you may still face extortion and notification duties. Plan for loss of the domain, hypervisors and backup consoles at once. Prepare an isolated clean room to rebuild and scan restored systems before reconnecting, and keep offline copies of golden images, configurations and the runbook itself. Decide in advance who may authorize negotiations or payment, and consult legal counsel and law enforcement guidance, since sanctions and policy considerations apply.

Continuity planning and testing

Business continuity plans cover people, premises, process and technology: manual workarounds such as paper forms, alternative offices, a staff call tree and priority customer communication. Define the threshold for declaring a disaster and activating the plan. Exercise it with both tabletop and technical recovery tests, and update it when systems change. Cyber insurance may require specific controls and prompt notice, so know the policy requirements ahead of time. Success means critical services return within tolerance, not that every server is restored.

BIA TABLE

Process	MTD	RTO	RPO	Depends on
Payroll	72h	24h	4h	AD, DB, HR app
Online orders	4h	2h	15m	DNS, DB, API, payments
Intranet	7d	3d	24h	AD, web server
Tier order: identity/DNS -> databases -> apps -> front ends				

WORKED EXAMPLE

After a fictional ransomware attack, Example Corp finds the domain controllers encrypted. The runbook says identity comes first, so the team restores an offline copy of a domain controller into the clean room, validates it, and brings up DNS. Only then are the order database, API servers and payment integration restored, matching the BIA tiers. Online ordering resumes in six hours, inside its tolerance of eight, while the intranet waits two days, exactly as the business had agreed beforehand.

KEY TERMS

BIA: An analysis of how interruptions to business processes affect the organization over time.

MTD: Maximum tolerable downtime before the impact becomes unacceptable.

Recovery tier: A ranking that sets restoration priority.

Clean room: An isolated environment used to rebuild and inspect systems safely.

Double extortion: Ransomware that both encrypts and steals data for leverage.

REMEMBER

- Run a BIA to set tiers, RTO and RPO per process.

- Map dependencies and restore identity and networking first.
- Plan for ransomware with a clean room and offline runbooks.

QUICK CHECK

1. What does a BIA produce?
 - A) Exploit list
 - B) Process criticality and recovery targets
 - C) Firewall rules
 - D) Password policy
2. Which is usually restored first?
 - A) Intranet
 - B) Identity, DNS and networking
 - C) Test systems
 - D) Reports
3. Why is restoring from backup alone not enough against double extortion?
 - A) Backups are slow
 - B) Data may also have been stolen
 - C) Encryption is weak
 - D) Backups are secret
4. Why include process owners in a BIA?
 - A) Cheaper
 - B) They know business impact and workarounds
 - C) Required by TCP
 - D) They approve patches

Answers: 1-B 2-B 3-B 4-B

1. It sets priorities and tolerances.
2. Most services depend on them.
3. Stolen data creates extortion and notification duties.
4. IT alone cannot judge business impact.

Purple team exercises

Purple teaming joins attackers and defenders to measure which behaviors your controls actually detect, then fixes the gaps on the spot.

Red, blue and purple

A red team emulates an adversary, a blue team defends and detects, and a purple team exercise has both work together in real time rather than in secret. The red side runs a technique, the blue side checks whether the right telemetry and alert appeared, and together they tune until it does. The goal is not to win but to improve detection coverage. This is far faster than a classic red team engagement, where findings arrive in a report weeks later.

MITRE ATT&CK and Atomic Red Team

MITRE ATT&CK is a knowledge base of adversary tactics and techniques, each with an ID such as T1059.001 (PowerShell) or T1003.001 (LSASS memory credential dumping). Atomic Red Team is an open library of small, scripted tests mapped to those technique IDs, designed to run safely on a lab system so defenders can verify detection. Invoke-AtomicTest T1059.001 runs the tests for that technique, and a cleanup step reverses changes. Always run tests only on authorized lab or test systems, with written approval and rollback.

Running an exercise

Choose techniques based on threat intelligence about groups that target your sector, and agree scope and safety rules. For each test, record the time and expected data source, execute it, and look in the SIEM or EDR. Classify the outcome: logged and alerted, logged but no alert, or no telemetry. Each result suggests a fix: enable a missing log source such as Sysmon or PowerShell script block logging, write a detection rule, or adjust response. Re-run the same test to confirm the new rule fires.

Measuring and sustaining

Track detection coverage as a share of tested techniques with working alerts, mean time to detect for the simulated activity, and the false positive rate of new rules. Store rules and test mappings in version control and replay the tests automatically after changes so that detections do not quietly break, a practice often called detection as code. Rotate through ATT&CK tactics over the year, and feed lessons into the IR plan and tabletop scenarios. A single run is a snapshot; regular repetition builds confidence.

TEST AND RESULT

```
> Invoke-AtomicTest T1059.001 -TestNumbers 1 (lab host ws-lab01)
12:04:10 executed, encoded PowerShell command
SIEM: event 4104 present, rule 'Suspicious PowerShell' = no alert
Gap: logged but not alerted
Fix: add rule for -EncodedCommand with long base64
12:41:55 re-run -> alert fired
```

WORKED EXAMPLE

A purple team picks T1059.001 because a fictional threat group relevant to Example Corp uses encoded PowerShell. The red operator runs an Atomic test on a lab host, and the blue analyst sees script block event 4104 in the SIEM but no alert. Together they write a rule that matches long encoded commands from non-admin parent processes, test it against 24 hours of benign data to keep false positives low, deploy it, and re-run the test. The alert fires, and the coverage chart improves by one technique.

KEY TERMS

Purple team: A collaborative exercise where attack emulation and defense tune detections together.

ATT&CK: A knowledge base of adversary tactics and techniques.

Atomic Red Team: A library of small tests mapped to ATT&CK techniques.

Detection gap: An attack behavior that produces no useful alert.

Detection as code: Managing detection rules and tests in version control with automated checks.

REMEMBER

- Purple teams test and tune detections together.

- Map tests to ATT&CK and classify results as alerted, logged or blind.
- Fix gaps, re-run to confirm and automate replay.

QUICK CHECK

1. What distinguishes purple from classic red teaming?

- A) Attackers and defenders collaborate in real time
- B) It uses no tools
- C) It is only reporting
- D) It is illegal

2. What is a technique ID like T1003.001?

- A) Hash
- B) ATT&CK technique identifier
- C) Port
- D) CVE number

3. Logged but not alerted means...

- A) Test failed
- B) Rule fires
- C) Telemetry exists, a rule is missing
- D) No data exists

4. Where may Atomic tests be run?

- A) The internet
- B) Customer laptops
- C) Authorized lab or test systems
- D) Any production server

Answers: 1-A 2-B 3-C 4-C

1. Immediate feedback speeds tuning.
2. ATT&CK uses T-prefixed IDs.
3. The fix is a detection rule.
4. They simulate real behavior and need approval.

Tabletop a ransomware case

SCENARIO

Example Corp, a fictional firm, faces a ransomware outbreak. You are the incident commander and will receive timed injects. At each step you must decide on containment, communication and recovery, balancing business impact against risk. The plan names a legal lead and a communications lead, and backups are in an unknown state.

OBJECTIVE

Make sound containment, communication and recovery decisions across timed injects.

STEPS IN THE SIMULATOR

- 1 **Do:** Inject T+00: file server shows .locked extensions. Choose an action.

Simulator shows:

Options: isolate VLAN, power off, ignore. You choose isolate VLAN. Feedback: good, preserves memory and stops spread.

- 2 **Do:** Inject T+20: backup console is unreachable. Choose.

Simulator shows:

Options: assume backups fine, check immutable copy, rebuild blindly. You check the immutable copy. Feedback: object lock copy verified from 03:00.

- 3 **Do:** Inject T+45: ransom note with a 72 hour deadline. Choose who to notify.

Simulator shows:

Choose legal, CISO and insurer. Feedback: correct, regulatory clock and policy notice start.

- 4 **Do:** Inject T+70: customer asks if their data is affected. Choose a response.

Simulator shows:

Options: deny, say nothing, holding statement approved by legal. You choose the holding statement. Feedback: correct.

- 5 **Do:** Inject T+120: attacker posts sample data online. Choose.

Simulator shows:

Confirm data theft; privacy officer starts breach assessment. Feedback: recovery alone does not close the incident.

- 6 **Do:** Choose the recovery order.

Simulator shows:

Identity, DNS, databases, apps. Feedback: matches BIA tiers. Score: 5 of 5 decisions on plan.

ANSWER

Best decisions: isolate the affected VLAN, verify the immutable backups, notify legal, the CISO and the insurer, send an approved holding statement, treat the data posting as a breach, and restore identity and DNS before applications.

Hint shown in the app: Think containment first, evidence second, communications through one approved voice.

Pick backup strategy

SCENARIO

A logistics company needs its order database to meet an RPO of 30 minutes and an RTO of 4 hours, and must survive ransomware that steals admin credentials. You edit a backup configuration for schedule, retention, immutability and testing.

OBJECTIVE

Configure backups that meet the stated RPO and RTO and resist credential theft.

STEPS IN THE SIMULATOR

1 Do: Open backup.yaml

Simulator shows:

Fields: schedule, retention_days, immutable, offsite, restore_test. Current: daily full, 7 days, immutable false.

2 Do: Set schedule to incremental every 15 minutes plus nightly full

Simulator shows:

Validator: RPO 15m satisfies target 30m

3 Do: Set offsite to true with a second region

Simulator shows:

Validator: copy count 3, offsite present

4 Do: Set immutable true, 35 days, compliance mode

Simulator shows:

Validator: deletion blocked until expiry, ransomware resistant

5 Do: Set restore_test to monthly full restore with timing

Simulator shows:

Validator: expected restore 3h10m, under RTO 4h

6 Do: Save and run validation

Simulator shows:

All checks passed: RPO ok, RTO ok, immutability ok

ANSWER

Use 15 minute incrementals with a nightly full, an off-site copy in another region, 35 day immutable retention in compliance mode, and monthly timed restore tests showing recovery under 4 hours.

Hint shown in the app: The schedule interval must be no longer than the RPO, and immutability defeats stolen credentials.

Recovery order

SCENARIO

After a BIA at Example Corp, you must sequence the restoration of seven systems following a ransomware event: identity service, DNS and DHCP, order database, payment gateway, web front end, payroll app and intranet. Drag them into the correct order on the dependency diagram.

OBJECTIVE

Sequence recovery by dependency and business criticality.

STEPS IN THE SIMULATOR

- 1 **Do:** Drag Identity service to position 1

Simulator shows:

Accepted: no dependencies

- 2 **Do:** Drag DNS and DHCP to position 2

Simulator shows:

Accepted: needs network only

- 3 **Do:** Drag Order database to position 3

Simulator shows:

Accepted: depends on identity and DNS

- 4 **Do:** Drag Payment gateway to position 4

Simulator shows:

Accepted: depends on database

- 5 **Do:** Drag Web front end to position 5

Simulator shows:

Accepted: depends on API and payments

- 6 **Do:** Place Payroll app then Intranet last

Simulator shows:

Accepted: lower tiers. Validation passed: all dependencies satisfied before each step

ANSWER

Correct order: identity service, DNS and DHCP, order database, payment gateway, web front end, payroll app, intranet. Foundational services precede dependent ones, and higher tier revenue systems precede internal tools.

Hint shown in the app: Restore what everything else depends on before the applications.

Purple team detection gap

SCENARIO

In a lab, the red operator will run an Atomic Red Team test for T1059.001 (encoded PowerShell) on host ws-lab01. You monitor the SIEM and must determine whether the technique is detected and, if not, write the missing rule.

OBJECTIVE

Find the undetected technique and create a detection that fires on re-test.

STEPS IN THE SIMULATOR

- 1 **Do:** Start the simulated technique

Simulator shows:

Red: executed T1059.001 test 1 at 12:04:10

- 2 **Do:** Search SIEM: host=ws-lab01 earliest=-15m

Simulator shows:

3 events: process create, event 4104 script block, network connect

- 3 **Do:** Check alerts for the same window

Simulator shows:

0 alerts. Coverage: T1059.001 = not detected

- 4 **Do:** Open event 4104 and note fields

Simulator shows:

Fields: ScriptBlockText, powershell.exe -EncodedCommand JAB..., ParentImage winword.exe

- 5 **Do:** Write rule: powershell with -EncodedCommand, base64 length over 100, parent not an admin tool

Simulator shows:

Rule saved; backtest on 24h: 0 false positives

- 6 **Do:** Ask red to re-run the test

Simulator shows:

Alert fired: Suspicious Encoded PowerShell, ws-lab01, 12:41:55

ANSWER

The technique was logged (event 4104) but not alerted. The fix is a rule on encoded PowerShell with long base64 content and an unusual parent process, validated by a backtest and a re-run that produced an alert.

Hint shown in the app: Telemetry existed, so the gap is a rule, not a log source.

Incident Readiness and Resilience: 10 questions, 80% to pass

1. Which document defines incident response phases in NIST SP 800-61?

- A) Rev 2 lifecycle
- B) ISO 9001
- C) PCI DSS
- D) RFC 791

2. Why define severity levels in an IR plan?

- A) Billing
- B) Backups
- C) To set urgency, escalation and notification
- D) Branding

3. A tabletop exercise primarily...

- A) Discusses a scenario to test decisions and roles
- B) Changes production systems
- C) Encrypts disks
- D) Scans ports

4. What is the best output of a tabletop?

- A) A longer plan
- B) Applause
- C) Findings with owners and due dates
- D) Fewer participants

5. What does 3-2-1-1-0 add to 3-2-1?

- A) Two more sites
- B) An offline or immutable copy and zero verification errors
- C) Cloud only
- D) Daily deletion

6. An RTO of 4 hours means...

- A) Service must be restored within 4 hours
- B) Backups kept 4 hours
- C) Up to 4h of data loss
- D) 4 backups

7. What does a BIA help determine?

- A) Critical processes and recovery priorities
- B) IP ranges
- C) Password length
- D) Exploits

8. Which service typically comes back first after a total outage?

- A) Identity, DNS and networking
- B) Intranet
- C) Reports
- D) Wiki

9. Which ID format names an ATT&CK technique?

- A) CVE-2024-1
- B) SHA256
- C) T1059.001
- D) TCP/443

10. An event is logged but no alert fires. The fix is...

- A) Reboot
- B) Turn off the SIEM
- C) Write or tune a detection rule
- D) Delete the log

Answers: 1-A 2-C 3-A 4-C 5-B 6-A 7-A 8-A 9-C 10-C

1. 800-61 defines the IR lifecycle.
2. They drive who is told and how quickly.
3. No systems are touched.
4. Tracked actions close the gaps.
5. The extra 1 is offline or immutable, 0 is errors.
6. RTO is time to restore.
7. It ranks processes and sets tolerances.
8. Other systems depend on these.
9. Techniques use T-numbers.
10. Telemetry exists; detection logic is missing.

GRC for Engineers

Intermediate | 7 lessons | 5 simulated labs | checkpoint | about 5h 35m

Governance, risk and compliance can look like paperwork, but it is how an organization proves that security is deliberate. This module teaches you to assess risk with likelihood, impact and annualized loss expectancy, to separate policies from standards and procedures, to map one control across frameworks, and to understand ISO/IEC 27001, SOC 2, PCI DSS and NIST SP 800-53 and the CSF. You will also learn to gather audit evidence efficiently and to respond calmly to findings.

BY THE END YOU CAN

- + Perform a risk assessment and map controls to frameworks
- + Explain ISO 27001, SOC 2, PCI DSS and NIST SP 800-53 requirements
- + Collect audit evidence efficiently

IN THIS MODULE

Lesson 1 Risk assessment methods

Lesson 2 Policies, standards and control mapping

Lesson 3 ISO/IEC 27001

Lesson 4 SOC 2

Lesson 5 PCI DSS

Lesson 6 NIST SP 800-53 and CSF

Lesson 7 Audits and evidence collection

Lab 1 Score a risk register (Scenario)

Lab 2 Map controls (Diagram builder)

Lab 3 Define PCI scope (Diagram builder)

Lab 4 Gather evidence (File explorer)

Lab 5 Respond to an audit finding (Chat sim)

Checkpoint 10 questions, 80% to pass

Risk assessment methods

Security budgets are finite; risk assessment tells you which problems deserve money first and which can be accepted.

Likelihood, impact and the register

Risk is the chance that a threat exploits a vulnerability to harm an asset. A common formula is $\text{risk} = \text{likelihood} \times \text{impact}$. A risk assessment identifies assets, threats and vulnerabilities, estimates both factors, and records each risk in a risk register with an owner, a score, a treatment decision and a review date. Likelihood considers threat capability, exposure and existing controls; impact considers financial loss, downtime, legal penalties, safety and reputation. Inherent risk is before controls and residual risk is what remains after them.

Qualitative versus quantitative

Qualitative methods rate risks on scales, for example likelihood 1 to 5 and impact 1 to 5, plotted on a heat map. They are quick and easy to explain but subjective. Quantitative methods use money. Single loss expectancy (SLE) equals asset value times exposure factor; annualized rate of occurrence (ARO) is how often per year the loss is expected; annualized loss expectancy is $\text{ALE} = \text{SLE} \times \text{ARO}$. If a database worth 400,000 would lose 50 percent in a breach expected once every four years, SLE is 200,000, ARO is 0.25 and ALE is 50,000.

Treatment options

Four standard responses exist. Mitigate (treat) by adding or improving controls. Transfer or share by buying insurance or contracting a provider, although legal responsibility often stays with you. Avoid by stopping the risky activity, for example retiring a legacy service. Accept when the residual risk is within appetite, with documented approval from someone who has the authority. A control is cost justified when its annual cost is lower than the reduction in ALE it delivers.

Making assessments credible

Use consistent scales with written definitions, such as 'impact 4 means loss above 1 million or a regulatory fine'. Involve asset owners, use incident history and threat intelligence where possible, and avoid false precision, since an ALE built on guesses is still a guess. Frameworks such as ISO/IEC 27005, NIST SP 800-30 and FAIR give structure. Reassess after major changes and at least yearly, and report trends so that leadership sees whether risk is going up or down.

RISK REGISTER ROW

```
ID: R-07 Asset: customer DB Threat: ransomware
Likelihood: 4 Impact: 5 Inherent: 20
Controls: immutable backups, EDR Residual: 12
SLE 200000 ARO 0.25 ALE 50000
Treatment: mitigate (add MFA on admin) Owner: CIO Review: 2026-12-01
```

WORKED EXAMPLE

A company values its customer database at 400,000 and estimates that a breach would destroy half of that value, with such a breach likely once every four years. ALE is 50,000. A proposed control costs 15,000 per year and would cut breach likelihood in half, reducing ALE to 25,000. The saving of 25,000 exceeds the 15,000 cost, so mitigation is justified. The risk owner records the decision and the expected residual risk in the register.

KEY TERMS

Risk register: A maintained list of risks with scores, owners and treatments.

Residual risk: The risk that remains after controls are applied.

ALE: Annualized loss expectancy, SLE multiplied by ARO.

Risk appetite: The level of risk an organization is willing to accept.

Risk transfer: Shifting financial impact to another party, such as an insurer.

REMEMBER

- Risk is likelihood times impact, tracked in a register.
- Quantify with SLE, ARO and ALE where data allows.
- Choose mitigate, transfer, avoid or accept and document owners.

QUICK CHECK

1. ALE equals...
 - A) $SLE \times ARO$
 - B) ARO / SLE
 - C) Impact + likelihood
 - D) Cost / value
2. Which treatment is buying cyber insurance?
 - A) Transfer
 - B) Avoid
 - C) Accept
 - D) Ignore
3. A control is worth it when...
 - A) Its cost is below the ALE reduction
 - B) It is new
 - C) It is expensive
 - D) It is popular
4. What is a qualitative weakness?
 - A) Needs money
 - B) Subjectivity
 - C) Needs math
 - D) Too precise

Answers: 1-A 2-A 3-A 4-B

1. Annualized loss expectancy multiplies loss per event by yearly frequency.
2. Insurance shifts financial impact.
3. Compare cost with risk reduction.
4. Scales rely on judgment.

Policies, standards and control mapping

Without a clear hierarchy of documents, rules become inconsistent; without mapping, teams repeat the same audit work for every framework.

The document hierarchy

A policy is a short, high level statement of management intent, approved by leadership, for example 'Access to systems is granted on a need-to-know basis'. A standard sets mandatory, measurable requirements that implement the policy, such as 'Administrator accounts require phishing-resistant MFA and 12 character minimum passwords elsewhere'. A procedure describes the steps to carry out a task, and a guideline offers non-binding advice. Policies change rarely; procedures change often. Each document has an owner, version, approval date and review cycle.

Controls and their types

A control is a safeguard that reduces risk. By function they are preventive, detective, corrective, deterrent or compensating; by nature they are administrative, technical or physical. A well written control states what must happen, how often and who is responsible, so it can be tested: 'Quarterly, the IT manager reviews all privileged accounts and removes unneeded access, with evidence stored in the ticket system'. Vague controls such as 'access is managed appropriately' cannot be audited.

Mapping one control to many frameworks

Frameworks overlap heavily. A single quarterly access review can satisfy ISO/IEC 27001 Annex A control 5.18 on access rights, SOC 2 criterion CC6.2 and CC6.3 on access provisioning and removal, and PCI DSS requirement 7 on restricting access by business need. Build a unified control set, a table in which each internal control is linked to the requirements it supports. This lets teams do the work once, collect evidence once and reuse it across audits.

Keeping mappings honest

Mappings are judgments, not exact equivalents; a control may satisfy part of a requirement but not all of it, so note gaps. Use official crosswalks, such as published mappings between NIST SP 800-53 and ISO 27001, as a starting point and review them with auditors. Store the control library in a GRC tool or a controlled spreadsheet, record the owner, frequency and evidence source for each, and update the mapping when a framework releases a new version, for example PCI DSS 4.0 replacing 3.2.1.

CONTROL MAPPING

```
Internal control AC-04: Quarterly privileged access review
ISO/IEC 27001:2022 A.5.18 Access rights
SOC 2 CC6.2, CC6.3
PCI DSS 4.0 7.2.4 (review of user accounts)
NIST SP 800-53 AC-2(j)
Owner: IT Manager Evidence: review ticket + export
```

WORKED EXAMPLE

Example Corp used to run three separate access reviews, one each for ISO, SOC 2 and PCI auditors. The GRC analyst creates one internal control, AC-04, a quarterly review of privileged accounts, and maps it to all three frameworks. The IT manager performs the review once, saves the export and approval ticket, and the same evidence is supplied to each auditor. The company saves several days of work per audit cycle and removes inconsistent answers.

KEY TERMS

Policy: A high level statement of management intent.

Standard: A mandatory, specific requirement supporting a policy.

Procedure: Step by step instructions for a task.

Compensating control: An alternative safeguard used when a requirement cannot be met directly.

Crosswalk: A table relating requirements of one framework to another.

REMEMBER

- Policy, standard, procedure and guideline have different roles.

- Write controls that are specific, owned and testable.
- Map each control to several frameworks and note any gaps.

QUICK CHECK

1. Which document sets mandatory, measurable requirements?

- A) Memo
- B) Poster
- C) Guideline
- D) Standard

2. Why map controls across frameworks?

- A) Increase cost
- B) Avoid audits
- C) Do work and evidence collection once
- D) More paperwork

3. What makes a control auditable?

- A) Clear action, frequency, owner
- B) Length
- C) Secrecy
- D) Vague wording

4. A detective control example is...

- A) Firewall block
- B) Training poster
- C) Lock
- D) Log monitoring alert

Answers: 1-D 2-C 3-A 4-D

1. Standards specify requirements.
2. Overlap reduces duplicate effort.
3. Testers need something specific.
4. Detective controls find events after they occur.

ISO/IEC 27001

ISO/IEC 27001 is the best known international standard for an information security management system and is often required by customers.

The ISMS and clauses 4 to 10

ISO/IEC 27001 specifies requirements for an information security management system (ISMS), a managed, continually improving set of policies, processes and controls. Its mandatory clauses 4 to 10 cover the context of the organization and scope, leadership and policy, planning including risk assessment and treatment, support such as resources and awareness, operation, performance evaluation through monitoring, internal audit and management review, and improvement through corrective action. The standard follows the plan, do, check, act idea and is risk-based, so the controls you need depend on your own risks.

Annex A and the Statement of Applicability

Annex A of the 2022 edition lists 93 controls in four themes: organizational (37), people (8), physical (14) and technological (34). Supporting guidance is in ISO/IEC 27002. After the risk assessment you produce a Statement of Applicability (SoA) that lists every Annex A control, whether it is applicable, whether it is implemented, and the justification for including or excluding it. The SoA is a key audit document, and excluded controls need a sound reason tied to risk or scope, not convenience.

Certification audits

Certification is performed by an accredited certification body. Stage 1 reviews documentation and readiness; stage 2 tests whether the ISMS is implemented and effective. Certificates last three years with annual surveillance audits and a recertification audit at the end of the cycle. Findings are classed as major nonconformities, which block certification until fixed, and minor ones, which need a corrective action plan. Auditors sample records, interview staff and look for evidence of internal audits and management review.

Running an ISMS well

Define scope carefully, since it determines cost and what the certificate means; a certificate for 'the cloud platform and supporting offices' tells customers more than a vague scope. Keep the risk treatment plan and SoA current, run internal audits by independent staff, hold management reviews with real decisions and minutes, and track corrective actions to closure. The 2022 revision of the standard moved from 114 to 93 controls and reorganized them, and organizations certified to the 2013 version had a transition period to update.

SOA ROW

```
Control: A.8.13 Information backup
Applicable: Yes
Implemented: Yes
Justification: Risk R-07 (ransomware) treatment
Evidence: backup policy, restore test log

Control: A.7.4 Physical security monitoring
Applicable: Yes (office), No (cloud, covered by provider)
```

WORKED EXAMPLE

Example Corp wants ISO 27001 certification for its SaaS product. It scopes the ISMS to the platform team and the two offices, runs a risk assessment, and builds an SoA covering all 93 Annex A controls with justifications. Stage 1 finds the internal audit missing, a major gap, so the company performs one and holds a management review before stage 2. The auditors then sample access reviews and backup tests, find no major issues, and the certificate is issued for three years with yearly surveillance.

KEY TERMS

ISMS: A management system for planning, operating and improving information security.

Statement of Applicability: A document listing Annex A controls with applicability and justification.

Surveillance audit: A yearly audit during the certification cycle.

Nonconformity: A failure to meet a requirement, rated major or minor.

ISO/IEC 27002: The guidance document explaining the Annex A controls.

REMEMBER

- An ISMS is a risk-based management system with clauses 4 to 10.
- Annex A lists 93 controls, with applicability recorded in the SoA.
- Certification involves stage 1, stage 2 and annual surveillance.

QUICK CHECK

1. How many controls are in Annex A of ISO 27001:2022?

- A) 114
- B) 14
- C) 200
- D) 93

2. What does the SoA document?

- A) Network maps
- B) Staff salaries
- C) Control applicability and justification
- D) Passwords

3. How long does an ISO certificate usually last?

- A) 3 years
- B) 1 year
- C) 10 years
- D) Forever

4. A major nonconformity means...

- A) Certification is blocked until fixed
- B) A cheaper audit
- C) Praise
- D) Ignore

Answers: 1-D 2-C 3-A 4-A

1. The 2022 edition has 93 controls in four themes.
2. It records which controls apply and why.
3. Three years with annual surveillance.
4. Major findings must be resolved.

SOC 2

SOC 2 reports are the standard way SaaS and service companies prove their controls to customers in North America and beyond.

Trust Services Criteria

SOC 2 is an attestation framework from the AICPA, performed by a licensed CPA firm. It evaluates controls against the Trust Services Criteria: Security, which is mandatory and called the common criteria (CC series), plus optional Availability, Processing Integrity, Confidentiality and Privacy. The company chooses which criteria are in scope based on its services and customer promises. The report describes the system, management's assertion and the auditor's opinion on whether controls are suitably designed and, for Type II, operating effectively.

Type I versus Type II

A Type I report examines the design of controls at a single point in time. A Type II report examines both design and operating effectiveness over a period, commonly three to twelve months. Customers usually prefer Type II because it shows controls working consistently, not just existing on paper. Many companies start with a Type I to establish a baseline, then run a Type II observation window. The report is restricted to customers and prospects under NDA, unlike a public certificate.

Common control tests

Auditors test samples of the population. Typical tests include whether new hires received background checks and security training, whether terminated users lost access within a defined time, whether changes were peer reviewed and approved before production, whether vulnerability scans ran and critical findings were fixed within the SLA, whether backups were restored, whether incidents were logged and handled, and whether vendors were reviewed. For each control the auditor samples items from the period, for example 25 closed tickets, and looks for exceptions.

Preparing and handling exceptions

Start with a readiness assessment against the criteria, fix gaps, and begin collecting evidence automatically so the observation window is clean. Define each control precisely, because auditors test what you say you do. If a sample fails, for example one leaver kept access for ten days against a 24 hour rule, the report notes an exception with management's response; one exception is not necessarily a qualified opinion, but patterns are. Complementary user entity controls describe what customers must do themselves, and subservice organizations such as the cloud provider are carved out or included.

SAMPLE TEST RESULT

Control CC6.3: Access removed within 24h of termination
Population: 41 leavers in period Sample: 25
Result: 24 pass, 1 exception (removed after 10 days)
Management response: offboarding ticket now auto-creates
revocation tasks; confirmed on 3 later leavers

WORKED EXAMPLE

Example Corp sells a fictional analytics service. After a three month Type I, it opens a six month Type II window covering Security and Availability. In the audit, 25 of 41 leavers are sampled, and one account stayed active ten days. The auditor lists an exception with management's remediation, and the opinion remains unqualified because the control otherwise operated. Customers read the report, see the exception and fix, and gain confidence from the honest disclosure.

KEY TERMS

Trust Services Criteria: The AICPA criteria categories used in SOC 2.

Type II: A report on control design and operating effectiveness over a period.

Common criteria: The mandatory Security criteria in SOC 2.

Exception: A sampled instance where a control did not operate as stated.

Attestation: An independent auditor's opinion on a subject matter.

REMEMBER

- SOC 2 is tested against Trust Services Criteria with Security mandatory.
- Type I tests design at a date; Type II tests operation over time.
- Auditors sample evidence; exceptions are disclosed with responses.

QUICK CHECK

1. Which criterion is mandatory in SOC 2?
 - A) Security
 - B) Availability
 - C) Privacy
 - D) Confidentiality
2. What does Type II add over Type I?
 - A) Public posting
 - B) Operating effectiveness over a period
 - C) A logo
 - D) Price
3. Who performs SOC 2?
 - A) NIST
 - B) A licensed CPA firm
 - C) A vendor
 - D) Any employee
4. What is a sample exception?
 - A) A tool
 - B) A new rule
 - C) A tested item where the control failed
 - D) A holiday

Answers: 1-A 2-B 3-B 4-C

1. Security (common criteria) is always included.
2. Type II tests effectiveness across time.
3. It is an AICPA attestation.
4. It records a deviation from the control.

PCI DSS

Any organization that stores, processes or transmits payment card data must meet PCI DSS, and good scoping drastically reduces cost and risk.

Cardholder data and scope

The Payment Card Industry Data Security Standard is maintained by the PCI Security Standards Council and enforced through card brands and acquiring banks. It protects cardholder data, which includes the primary account number (PAN), cardholder name, expiry date and service code, and sensitive authentication data such as the full track, CVV and PIN, which must never be stored after authorization. The cardholder data environment (CDE) is made up of systems that store, process or transmit that data, and systems that can affect its security are also in scope.

Segmentation and scope reduction

Network segmentation, using firewalls and VLANs or cloud security groups, isolates the CDE so unrelated systems fall out of scope, but it must be tested, for example by annual penetration tests of segmentation controls. Tokenization replaces the PAN with a token, point-to-point encryption protects data from the card reader, and hosted payment pages sent to a PCI compliant provider keep card numbers away from your servers entirely. Reducing scope is usually cheaper and safer than securing a sprawling environment.

The twelve requirements

PCI DSS has twelve principal requirements in six goals: build and maintain a secure network with firewalls and secure configuration; protect account data with storage minimization and strong cryptography and encrypt transmission over public networks; maintain a vulnerability management program with anti-malware and secure development; implement strong access control through need-to-know, unique IDs and MFA, and restrict physical access; monitor and test networks with logging and regular scanning; and maintain an information security policy. Version 4.0 became mandatory, with future-dated requirements, in 2025, and v4.0.1 is the current edition.

Validation and compensating controls

Validation depends on volume and role: large merchants and service providers use a Qualified Security Assessor and a Report on Compliance, while smaller ones complete a Self-Assessment Questionnaire. Quarterly external vulnerability scans come from an Approved Scanning Vendor. When a requirement cannot be met as written, a compensating control may be used if it meets the intent, goes beyond other controls, and is documented in a worksheet and approved by the assessor. It is a temporary or exceptional measure, not a shortcut.

DATA ELEMENT RULES

```
Element Store allowed? Must be protected?
PAN Yes Render unreadable (encrypt, token)
Cardholder name Yes Protect with PAN
Expiry date Yes Protect with PAN
CVV / CVC NO after auth Never store
Full track data, PIN NO after auth Never store
```

WORKED EXAMPLE

Example Shop, a fictional retailer, finds its web servers, billing database and internal wiki all in scope because the network is flat. It moves card entry to a hosted payment page from a compliant provider, stores only tokens, and segments the remaining order system into its own VLAN. A segmentation penetration test confirms the wiki cannot reach the card systems. The number of in-scope systems drops from 120 to 9, cutting audit effort and the impact of a future breach.

KEY TERMS

PAN: The primary account number on a payment card.

CDE: The cardholder data environment of in-scope systems and processes.

Tokenization: Replacing a PAN with a surrogate value that has no exploitable meaning.

QSA: A Qualified Security Assessor authorized to perform PCI assessments.

Compensating control: An approved alternative control that meets a requirement's intent.

REMEMBER

- Know which card data may be stored and which may not.
- Scope is the CDE plus systems that could affect it.
- Reduce scope with segmentation, tokens and hosted payment pages.

QUICK CHECK

1. Which data may never be stored after authorization?
 - A) Cardholder name
 - B) PAN
 - C) Expiry date
 - D) CVV
2. What defines the CDE?
 - A) Only the website
 - B) Only databases
 - C) All servers
 - D) Systems that store, process or transmit card data and those that affect their security
3. What reduces PCI scope?
 - A) More users
 - B) Longer passwords only
 - C) Segmentation, tokenization and hosted payment pages
 - D) More servers
4. Who performs quarterly external scans?
 - A) Any developer
 - B) An Approved Scanning Vendor
 - C) A QSA only
 - D) The bank manager

Answers: 1-D 2-D 3-C 4-B

1. Sensitive authentication data must not be kept.
2. Connected systems also fall in scope.
3. Less card data in fewer systems.
4. ASV scans are required quarterly.

NIST SP 800-53 and CSF

NIST publications give a detailed control catalog and a simple outcome framework that together can structure an entire security program.

800-53 families and controls

NIST SP 800-53 Revision 5 is a catalog of security and privacy controls for federal systems, widely used in industry. Controls are grouped into 20 families, each with a two letter ID, such as AC (Access Control), AU (Audit and Accountability), CM (Configuration Management), IA (Identification and Authentication), IR (Incident Response), RA (Risk Assessment), SC (System and Communications Protection) and SI (System and Information Integrity). A control such as AC-2 Account Management has a base statement, supplemental guidance and optional enhancements like AC-2(1).

Baselines and tailoring

SP 800-53B defines baselines, pre-selected sets of controls for low, moderate and high impact systems, determined by categorizing the system under FIPS 199 for confidentiality, integrity and availability. Organizations then tailor: scope the baseline to the system, apply compensating controls, and add overlays for special cases. This feeds the Risk Management Framework in SP 800-37: prepare, categorize, select, implement, assess, authorize and monitor. FedRAMP is a well known program that builds on these baselines for cloud services.

The Cybersecurity Framework

The NIST Cybersecurity Framework 2.0, released in 2024, has six functions: Govern, Identify, Protect, Detect, Respond and Recover. The new Govern function covers strategy, roles, policy and supply chain risk. The framework is outcome based and does not list detailed controls; instead it uses categories and subcategories such as 'PR.AA: identity management and authentication', with informative references to catalogs such as 800-53. Organization profiles describe the current state and a target state, and implementation tiers describe how rigorous risk management practices are.

Using them together

A practical pattern is to use the CSF as the shared language with executives and to measure maturity, then use 800-53 or another catalog for the specific controls behind each outcome. Perform a gap analysis by scoring each subcategory today versus the target profile, turn the largest gaps into a roadmap, and report progress by function. Because both are free and widely mapped to ISO 27001, SOC 2 and CIS Controls, they lower the cost of switching between audits and give a defensible structure when the board asks how risk is managed.

CSF PROFILE GAP

Function	Subcategory	Current	Target
GOVERN	GV.RM risk strategy	2	3
IDENTIFY	ID.AM asset inventory	2	4
PROTECT	PR.AA identity and access	3	4
DETECT	DE.CM continuous monitoring	3	4
RESPOND	RS.MA incident management	2	3
RECOVER	RC.RP recovery plan	1	3

WORKED EXAMPLE

A fictional hospital assesses itself against the CSF and finds the lowest score in Recover, where backups exist but nobody has tested restoration. Using a gap analysis, it targets a score of 3 in twelve months, picks 800-53 controls CP-4 (contingency plan testing) and CP-9 (system backup) as the detailed requirements, and adds them to the roadmap. The CISO reports progress to the board by function rather than by tool, so directors can follow the story.

KEY TERMS

- Control family:** A group of related controls in 800-53 identified by a two letter prefix.
- Baseline:** A predefined set of controls for low, moderate or high impact systems.
- Overlay:** A tailored set of control adjustments for a specific community or technology.
- CSF Profile:** A description of current or target cybersecurity outcomes.
- Govern:** The CSF 2.0 function covering strategy, roles and oversight.

REMEMBER

- 800-53 offers 20 families of detailed controls with baselines.
- CSF 2.0 has Govern, Identify, Protect, Detect, Respond and Recover.
- Use profiles and gap analysis to build a roadmap.

QUICK CHECK

1. How many functions does CSF 2.0 have?
 - A) Five
 - B) Six
 - C) Twelve
 - D) Seven
2. What does AC mean in 800-53?
 - A) Asset Catalog
 - B) Authentication Code
 - C) Access Control
 - D) Audit Controls
3. What sets which 800-53B baseline applies?
 - A) Age
 - B) System impact level under FIPS 199
 - C) Company size
 - D) Budget
4. The CSF is mainly...
 - A) An outcome based framework
 - B) A scanner
 - C) A control catalog
 - D) A law

Answers: 1-B 2-C 3-B 4-A

1. Govern was added to the original five.
2. AC is the Access Control family.
3. Low, moderate or high impact guides selection.
4. It lists outcomes, with references to catalogs.

Audits and evidence collection

Audits go smoothly or painfully depending on how well evidence has been prepared; automation turns a scramble into routine reporting.

Types of audit and the audit lifecycle

Audits may be internal, external by a customer or regulator (a second-party audit), or third-party by an independent firm. The lifecycle is planning and scoping, requesting evidence, testing and sampling, interviews, drafting findings, management response and a final report. Preparation starts months earlier: know the scope, name an audit coordinator, understand what each control owner must provide and keep a calendar of recurring activities such as quarterly reviews. Auditors judge what is documented and evidenced, so an undocumented control is treated as non-existent.

What good evidence looks like

Good evidence is relevant, complete, authentic and dated. Examples include system generated reports with timestamps, configuration exports, screenshots showing the user and date, tickets with approvals, signed policies with version history, meeting minutes and log samples. Evidence should show the control operating during the whole period, not only on the day of the audit. Avoid editing exports, and record who produced them and how, so an auditor can trust provenance. Never share secrets or unnecessary personal data; redact what the auditor does not need.

Automating collection and sampling

Automation reduces effort and error: scripts or GRC platforms can pull IAM user lists, MFA status, cloud configuration snapshots, vulnerability scan results and ticket data on a schedule and store them in a controlled evidence repository. Auditors choose samples from a population, for example 25 of 300 changes, so be ready to supply the full population listing quickly and consistent artifacts for each item. Continuous controls monitoring tests controls daily and alerts when a control drifts, so exceptions are fixed before the audit period ends.

Handling findings

Findings are classed by severity, commonly observation, minor and major, or deficiency, significant deficiency and material weakness in financial audits. Respond by confirming facts, agreeing the root cause, and proposing a corrective action plan with an owner and a realistic date, instead of arguing or over-promising. Track actions to completion, provide evidence of closure, and look for similar issues elsewhere. A calm, factual relationship with auditors, and honesty about gaps, builds credibility, and repeated findings are treated more severely than new ones.

EVIDENCE REQUEST

```
Request R-12: Show MFA enforced for all admins (CC6.1)
Provided: iam_users_2026-09-30.csv (export, 14 admins)
         column mfa_enabled = true for 14 of 14
Provided: policy_mfa_v3.2.pdf (approved 2026-03-14)
Provider: J. Rivera (IT), pulled by script evidence_pull.py
```

WORKED EXAMPLE

An auditor asks Example Corp for proof that all administrators use multi-factor authentication. Instead of screenshots, the security engineer runs the evidence script, which exports the identity provider's admin list with the MFA status column and a timestamp, and attaches the approved MFA policy. The export shows 14 of 14 admins enrolled. Because the script also ran quarterly, the engineer supplies the earlier three exports to show the control worked throughout the period.

KEY TERMS

Evidence: Records that demonstrate a control exists and operated.

Population: The full set of items from which an auditor draws a sample.

Corrective action plan: An agreed set of tasks, owners and dates that fixes a finding.

Continuous controls monitoring: Automated, frequent testing of controls.

Provenance: The documented origin and handling of an evidence item.

REMEMBER

- Evidence must be relevant, complete, authentic and dated.

- Automate collection and keep populations ready for sampling.
- Answer findings with root cause, owner and date.

QUICK CHECK

1. Which evidence is strongest?
 - A) An old email
 - B) A rumor
 - C) Verbal claim
 - D) A dated system export
2. What is a sampling population?
 - A) All items a sample is drawn from
 - B) Customers
 - C) The company staff
 - D) Auditors
3. When responding to a finding you should...
 - A) Delete evidence
 - B) Hide it
 - C) Argue
 - D) Agree root cause and give owner and date
4. Why automate evidence collection?
 - A) It removes controls
 - B) It avoids audits
 - C) Consistency, lower effort, and continuous coverage
 - D) Looks modern

Answers: 1-D 2-A 3-D 4-C

1. System generated, dated, complete.
2. The sample is a subset of the population.
3. A corrective plan shows control.
4. Scheduled pulls show operation across the period.

Score a risk register

SCENARIO

You are the analyst at Example Corp reviewing six risks: unpatched VPN appliance, lost laptop, phishing of finance staff, cloud storage bucket misconfiguration, legacy FTP server, and supplier outage. For each you rate likelihood and impact on a 1 to 5 scale and choose treat, transfer, accept or avoid.

OBJECTIVE

Rate six risks and assign a sensible treatment to each.

STEPS IN THE SIMULATOR

1 Do: Rate the unpatched VPN appliance

Simulator shows:

Likelihood 4, impact 5, score 20. Treatment: treat (patch and restrict). Feedback: correct.

2 Do: Rate the lost laptop

Simulator shows:

Likelihood 4, impact 2 with disk encryption, score 8. Treatment: accept or treat with MDM wipe. Feedback: acceptable.

3 Do: Rate phishing of finance staff

Simulator shows:

Likelihood 5, impact 4, score 20. Treatment: treat (MFA, mail filtering, training). Feedback: correct.

4 Do: Rate the public cloud bucket

Simulator shows:

Likelihood 3, impact 5, score 15. Treatment: treat (block public access, policy). Feedback: correct.

5 Do: Rate the legacy FTP server

Simulator shows:

Likelihood 4, impact 3, score 12. Treatment: avoid (retire service). Feedback: correct, unneeded.

6 Do: Rate the supplier outage

Simulator shows:

Likelihood 2, impact 4, score 8. Treatment: transfer (contract SLA, insurance). Feedback: correct.

ANSWER

Highest risks are the VPN appliance and phishing at 20 each, then the bucket at 15. Treat the top three, avoid the legacy FTP server by retiring it, transfer supplier outage risk through contract and insurance, and accept or lightly treat the encrypted laptop risk.

Hint shown in the app: Multiply likelihood by impact, then match the response to cost and whether the activity is still needed.

Map controls

SCENARIO

Example Corp is preparing for ISO 27001, SOC 2 and PCI DSS audits at the same time. The team has one control: quarterly review of privileged user accounts. Connect the control to the requirements it supports so evidence can be reused.

OBJECTIVE

Link one control to ISO, SOC 2 and PCI requirements to cut duplicate work.

STEPS IN THE SIMULATOR

- 1 **Do:** Place control AC-04 Quarterly privileged access review on the canvas

Simulator shows:

Node created, owner IT Manager

- 2 **Do:** Connect it to the ISO/IEC 27001 control list

Simulator shows:

Suggested: A.5.18 Access rights. Accepted.

- 3 **Do:** Connect it to the SOC 2 criteria

Simulator shows:

Suggested: CC6.2 and CC6.3. Accepted.

- 4 **Do:** Connect it to PCI DSS requirements

Simulator shows:

Suggested: 7.2.4 account review. Accepted.

- 5 **Do:** Try connecting it to PCI Requirement 3 (stored account data)

Simulator shows:

Rejected: not related, dotted line removed.

- 6 **Do:** Attach one evidence node: review ticket and account export

Simulator shows:

Validation: one evidence set serves 3 frameworks

ANSWER

AC-04 maps to ISO 27001 A.5.18, SOC 2 CC6.2 and CC6.3, and PCI DSS 7.2.4. One review and one evidence set satisfy all three; mapping it to PCI Requirement 3 is wrong because that is about stored data.

Hint shown in the app: Choose requirements about access rights, not about data storage or encryption.

Define PCI scope

SCENARIO

Example Shop has a web store, a payment page, an order database that stores tokens and PANs, an internal wiki, a reporting server connected to the order database, and a marketing site. Classify each system as in scope, connected-to (in scope for security impact), or out of scope.

OBJECTIVE

Mark systems correctly for the cardholder data environment.

STEPS IN THE SIMULATOR

- 1 **Do:** Mark the order database storing PANs

Simulator shows:

Marked: in scope (CDE)

- 2 **Do:** Mark the payment page that receives card numbers

Simulator shows:

Marked: in scope (CDE)

- 3 **Do:** Mark the reporting server with network access to the database

Simulator shows:

Marked: connected-to, in scope for security

- 4 **Do:** Mark the internal wiki on a separate VLAN with no route to the CDE

Simulator shows:

Marked: out of scope, segmentation validated

- 5 **Do:** Mark the marketing site with no card data

Simulator shows:

Marked: out of scope

- 6 **Do:** Check scope summary

Simulator shows:

3 in scope or connected, 2 out of scope. Reminder: test segmentation annually.

ANSWER

In scope: order database and payment page. Connected, so security-impacting: reporting server. Out of scope: wiki and marketing site, provided segmentation is verified and no route exists to the CDE.

Hint shown in the app: Ask whether the system touches card data or can reach or affect those that do.

Gather evidence

SCENARIO

An SOC 2 auditor sends five control requests to Example Corp: access removal for leavers, change approval, vulnerability remediation, backup restore, and security training completion. The evidence share holds many files, some correct and some distractors such as outdated drafts. Choose the right artifact for each request.

OBJECTIVE

Pick appropriate evidence artifacts for five SOC 2 control requests.

STEPS IN THE SIMULATOR

- 1 **Do:** Request 1: leavers lost access. Open /evidence/hr/

Simulator shows:

Pick terminations_Q3.csv with removal timestamps. Rejected: org_chart.pdf

- 2 **Do:** Request 2: changes approved. Open /evidence/change/

Simulator shows:

Pick change_tickets_export_Q3.csv with approver column. Rejected: change_policy_draft.docx

- 3 **Do:** Request 3: vulns fixed within SLA. Open /evidence/vuln/

Simulator shows:

Pick scan_results_and_ticket_aging_Q3.xlsx. Rejected: scanner_marketing_brochure.pdf

- 4 **Do:** Request 4: backup restore. Open /evidence/backup/

Simulator shows:

Pick restore_test_report_2026-08.pdf signed. Rejected: backup_job_status.png

- 5 **Do:** Request 5: training done. Open /evidence/training/

Simulator shows:

Pick lms_completion_export.csv (97 percent). Rejected: training_slides.pptx

- 6 **Do:** Submit package

Simulator shows:

5 of 5 accepted: system generated, dated, within period

ANSWER

Correct artifacts are the terminations export with removal timestamps, the change ticket export with approvers, the scan and ticket aging report, the signed restore test report, and the LMS completion export. Policies and drafts do not prove operation.

Hint shown in the app: Prefer dated system exports over policies, slides or screenshots without context.

Respond to an audit finding

SCENARIO

An external auditor reports that 3 of 25 sampled leavers kept access to the ticketing system for 6 to 10 days, against your 24 hour policy. You chat with the auditor to confirm the facts, state the root cause and agree on a remediation plan with a date.

OBJECTIVE

Agree factual findings and a credible remediation plan and date.

STEPS IN THE SIMULATOR

- 1 **Do:** Ask the auditor to confirm sample details

Simulator shows:

Auditor: users e1, e2, e3, removed 6, 8 and 10 days after termination.

- 2 **Do:** Verify and acknowledge the facts

Simulator shows:

Auditor: thank you, I will record the exception as stated.

- 3 **Do:** State the root cause: HR notifications did not trigger the ticketing system deprovisioning

Simulator shows:

Auditor: understood. What will you change?

- 4 **Do:** Propose: automate deprovisioning via the identity provider and add a weekly leaver reconciliation

Simulator shows:

Auditor: reasonable. What date?

- 5 **Do:** Commit to a date: automation live by 2026-11-15, reconciliation starting next week

Simulator shows:

Auditor: acceptable. I will need closure evidence.

- 6 **Do:** Agree to provide evidence: configuration export and three leaver samples after the change

Simulator shows:

Auditor: recorded as corrective action CA-03, owner IT Manager

ANSWER

Acknowledge the exception, give the root cause (missing deprovisioning link), propose automated removal and weekly reconciliation, commit to a dated plan of 2026-11-15, and agree to supply a configuration export and post-fix samples as closure evidence.

Hint shown in the app: Be factual, give root cause, owner and date, and offer closure evidence.

GRC for Engineers: 10 questions, 80% to pass

1. A 2 million asset with 25 percent exposure has SLE of...
 - A) 500000
 - B) 250000
 - C) 2000000
 - D) 25000
2. If ARO is 0.5 and SLE is 100000, ALE is...
 - A) 500000
 - B) 100000
 - C) 200000
 - D) 50000
3. Which treatment retires a risky legacy service?
 - A) Accept
 - B) Ignore
 - C) Avoid
 - D) Transfer
4. Which document sets mandatory measurable requirements?
 - A) Standard
 - B) Email
 - C) Guideline
 - D) Poster
5. ISO 27001:2022 Annex A has how many controls?
 - A) 93
 - B) 20
 - C) 12
 - D) 114
6. SOC 2 Type II reports on...
 - A) Price
 - B) Design only at a date
 - C) Design and operating effectiveness over a period
 - D) Tokens
7. Which is never stored after card authorization?
 - A) Name
 - B) Token
 - C) CVV
 - D) Expiry date
8. How many functions does NIST CSF 2.0 have?
 - A) 5
 - B) 8
 - C) 6
 - D) 20
9. A single control mapped to several frameworks helps...
 - A) Avoid evidence entirely
 - B) Skip audits
 - C) Hide gaps
 - D) Reuse work and evidence
10. Best response to an audit finding?
 - A) Delay
 - B) Root cause, owner, date and closure evidence
 - C) Dispute everything
 - D) Delete logs

Answers: 1-A 2-D 3-C 4-A 5-A 6-C 7-C 8-C 9-D 10-B

1. $SLE = \text{value} \times \text{exposure factor} = 500000$.
2. $ALE = SLE \times ARO$.
3. Avoid removes the activity.
4. Standards define required specifics.
5. Four themes, 93 controls.
6. Type II tests operation over time.
7. Sensitive authentication data is prohibited.
8. Govern plus the original five.
9. Mapping reduces duplication.
10. A concrete corrective action plan is expected.

Security Metrics and Program Management

Advanced | 3 lessons | 2 simulated labs | checkpoint | about 2h 55m

This module teaches how to measure, plan and explain security as a business discipline. You will choose KPIs and KRIs such as MTTD, MTTR and patch SLA compliance while avoiding vanity metrics that look good but show nothing. You will build a risk-based roadmap, prioritize initiatives by risk reduction and cost, use maturity models, and justify a budget. Finally you will learn to report to executives and boards in business terms, with concise dashboards and a clear ask that supports a decision.

BY THE END YOU CAN

- + Select KPIs that show security outcomes
- + Build a roadmap and budget tied to risk
- + Communicate security status to leadership

IN THIS MODULE

Lesson 1 Security KPIs and KRIs

Lesson 2 Roadmaps and budgets

Lesson 3 Reporting to leadership

Lab 1 Pick the right KPIs (Scenario)

Lab 2 Brief the executive (Chat sim)

Checkpoint 10 questions, 80% to pass

Security KPIs and KRIs

Leaders fund and trust what they can measure, so the numbers you choose decide whether security is seen as a cost center or as a managed reduction of risk.

KPIs, KRIs and outcomes

A key performance indicator (KPI) measures how well a process is working, such as how quickly critical patches are installed. A key risk indicator (KRI) signals that exposure is rising, such as the count of internet-facing systems with known exploited vulnerabilities. Good measures describe outcomes, meaning a change in risk or capability, not just activity. Compare the number of firewall rules reviewed with the time an attacker stays undetected: the first shows effort, the second shows result. Each metric should have a definition, data source, owner, target and review frequency, so that two people calculating it get the same answer.

Core operational metrics

Several measures are widely used. Mean time to detect (MTTD) is the average time from the start of malicious activity to its detection, and mean time to respond or recover (MTTR) is the time from detection to containment or restoration. Patch SLA compliance is the share of vulnerabilities fixed within target windows, for example critical within 7 days and high within 30. Others include EDR and MFA coverage, percentage of privileged accounts with phishing-resistant MFA, phishing report rate, backup restore test success, and percentage of critical assets with an owner. Use medians or percentiles alongside means, because a few extreme incidents can distort an average.

Vanity metrics and gaming

Vanity metrics look impressive but do not show risk change. Examples are the number of blocked attacks on the firewall, total vulnerabilities scanned, number of alerts generated, or hours of training delivered. Blocking a million automated probes says little about whether a targeted attack would succeed. A related trap is Goodhart's law: when a measure becomes a target it ceases to be a good measure. If analysts are rewarded for closing tickets quickly, they close tickets without investigating. Pair each metric with a quality check, such as sampling closed alerts, and prefer ratios and trends over raw counts.

Choosing and presenting a set

Select a small set, often five to ten, that maps to the main risks and to business goals such as ransomware resilience. Tie each to a framework when helpful, for example NIST CSF functions or CIS Controls. Show trend over time, a target and a status color, and define thresholds that trigger action. State data quality honestly: a coverage metric based on an incomplete inventory is overstated. Revisit the set every year and retire measures that no longer drive a decision. If a number would not change what anyone does, it probably does not belong on the dashboard.

METRIC DEFINITION CARD

Name: Critical patch SLA compliance
 Definition: critical vulns fixed within 7 days / critical vulns due
 Source: vuln scanner + ticketing, weekly
 Owner: IT operations lead
 Target: >= 95% Current: 81% Trend: up 6 pts
 Action: below 85% -> escalate to CIO

WORKED EXAMPLE

An Example Corp security manager inherits a monthly report showing 2.1 million blocked connections, 9,400 vulnerabilities found and 98 percent training completion. None of it shows whether risk is falling. She replaces it with six measures: median time to detect, median time to contain, critical patch SLA compliance, MFA coverage for admins, EDR coverage and restore test success. Detect time falls from 14 days to 3 over two quarters, and patch compliance rises from 62 to 88 percent. The board can now see direction and decide where to invest next.

KEY TERMS

KPI: Key performance indicator: measures how well a process or control is performing.

KRI: Key risk indicator: signals a rising or falling level of risk exposure.

MTTD: Mean time to detect: average time from compromise start to detection.

MTTR: Mean time to respond or recover after detection.

Vanity metric: A number that looks impressive but does not reflect risk or decisions.

Goodhart's law: When a measure becomes a target, it stops being a reliable measure.

REMEMBER

- Choose outcome measures such as MTTD, MTTR and patch SLA compliance.
- Avoid vanity metrics and pair each metric with a quality check.
- Keep a small set with owners, targets, trends and honest data caveats.

QUICK CHECK

1. Which is an outcome metric?
 - A) Number of training slides produced
 - B) Median time to detect an intrusion
 - C) Number of firewall rules
 - D) Total scan reports generated
2. Why is blocked-attack count a poor board metric?
 - A) It is illegal to report
 - B) It is too hard to collect
 - C) It measures only patching
 - D) It does not show whether a targeted attack would succeed
3. What does Goodhart's law warn about?
 - A) Targets can distort behavior and make a measure misleading
 - B) Budgets always grow
 - C) KPIs must be secret
 - D) Logs must be encrypted
4. What should every metric have?
 - A) A bright color only
 - B) A vendor logo
 - C) A definition, data source, owner and target
 - D) A secret formula

Answers: 1-B 2-D 3-A 4-C

1. Detection time shows a security result, not just activity.
2. High block counts mostly reflect automated noise.
3. Rewarding a number invites gaming of it.
4. Clear definitions make results consistent and actionable.

Roadmaps and budgets

Limited money and staff mean not every risk can be addressed at once, so security leaders need a defensible way to rank initiatives and justify spend.

Starting from risk

A security roadmap is a time-phased plan of initiatives, usually covering 12 to 36 months. Begin with a risk assessment that lists the top risks, such as ransomware, business email compromise, cloud misconfiguration and third-party breach, with likelihood and impact estimates. Tie each initiative to the risk it reduces and to a business objective. A roadmap built from a list of tools tends to buy products that nobody operates, while one built from risks can explain why each project exists. Include regulatory obligations and audit findings as fixed items, since they have deadlines you cannot choose.

Prioritizing by risk reduction and cost

Rank initiatives by the risk reduction they provide per unit of cost and effort. Quantitative approaches estimate annualized loss expectancy: single loss expectancy multiplied by annual rate of occurrence, then compare the loss avoided against the cost of the control. Many teams use simpler scoring with impact, likelihood reduction, cost, effort and dependencies, and plot results on a quadrant of value and effort. Quick wins such as enforcing MFA for administrators, tested offline backups and patching exposed systems usually come first. Foundations such as asset inventory and logging go early, because later projects depend on them.

Maturity models and benchmarks

Maturity models describe levels of capability and help show where you are and where you should be. Examples are the NIST Cybersecurity Framework implementation tiers and profiles, the CIS Controls implementation groups, and the Cybersecurity Capability Maturity Model. Assess each area on a consistent scale, set a target level based on risk appetite rather than aiming for the top everywhere, and show the gap. Peer benchmarks, such as spend as a percentage of IT budget, can inform a conversation but should not replace risk reasoning. Maturity scores are most credible when backed by evidence, such as test results and audit reports.

Building and defending the budget

Split the budget into run costs (licenses, staff, managed services) and change costs (new projects), and show the three-year picture, because licenses recur. For each request, state the risk addressed, expected reduction, cost, owner, success measure and the consequence of not funding it. Offer options at different levels, such as minimum, recommended and full, so leaders can choose. Reuse existing tools before buying new ones, and count staff time and operating effort, not only license price. Report progress against the roadmap each quarter and adjust when threats or the business change. A roadmap is a living document, not a one-time proposal.

INITIATIVE SCORING (FICTIONAL)

Initiative	Risk cut	Cost	Effort	Score
Admin MFA enforcement	High	Low	Low	9
Offline backup + test	High	Med	Med	8
Patch internet-facing	High	Low	Med	8
SIEM tuning	Med	Med	Med	5
New sandbox appliance	Low	High	High	2

ALE example: $SLE\ 600,000 \times ARO\ 0.2 = 120,000$ per year

WORKED EXAMPLE

Example Corp has 400,000 dollars for security next year and nine requests. The manager scores each by risk reduction, cost and effort. Administrator MFA, tested offline backups and patching of internet-facing servers rank at the top, together costing about 150,000. A new sandbox appliance scores low because the cloud mail platform already provides sandboxing. Using a ransomware estimate of 600,000 per incident and a yearly likelihood of 20 percent, the expected loss is 120,000 per year, which supports funding the backup project. She presents minimum and recommended options and a maturity chart showing the target gap.

KEY TERMS

Roadmap: A time-phased plan of security initiatives tied to risk and business goals.

ALE: Annualized loss expectancy: expected yearly loss, equal to SLE times ARO.

Risk appetite: The amount of risk an organization is willing to accept.

Maturity model: A scale describing levels of capability for a security domain.

Quick win: A low-cost, low-effort action that reduces significant risk fast.

Run versus change cost: Ongoing operating spend versus one-time project spend.

REMEMBER

- Build the roadmap from top risks, regulatory needs and dependencies.
- Rank by risk reduction per cost and effort, doing foundations and quick wins first.
- Use maturity targets and clear budget options with consequences of not funding.

QUICK CHECK

1. How should initiatives primarily be prioritized?
 - A) By vendor popularity
 - B) By risk reduction relative to cost and effort
 - C) By alphabetical order
 - D) By who asked first
2. What is ALE?
 - A) Annual license expense
 - B) Average log entries
 - C) Asset location estimate
 - D) Single loss expectancy multiplied by annual rate of occurrence
3. What should a target maturity level be based on?
 - A) Risk appetite and business need
 - B) The highest level everywhere
 - C) A competitor's logo
 - D) The newest tool
4. Why offer minimum, recommended and full options?
 - A) It hides costs
 - B) It avoids reporting
 - C) Leaders can choose with a clear view of risk trade-offs
 - D) It removes the need for a roadmap

Answers: 1-B 2-D 3-A 4-C

1. Value per cost links spending to risk.
2. ALE equals SLE times ARO.
3. Targets should match risk, not perfection.
4. Options make the decision transparent.

Reporting to leadership

Executives and boards decide on money and risk acceptance, and they can only do so if security is explained in their terms: impact, likelihood, cost and decisions.

Know the audience

Boards and executives are not security specialists, and they have limited time, often 10 to 20 minutes. They care about business impact: revenue, customer trust, regulatory fines, operational downtime and legal exposure. Replace jargon with plain statements. Instead of saying that a critical remote code execution flaw exists in the VPN appliance, say that an attacker could take control of the system that remote staff use, and that fixing it takes two days. Learn what the audience already knows, how they judge other risks, and which decisions are theirs. Boards also have oversight duties, and in many jurisdictions disclosure rules make cyber risk a governance matter.

Translate risk into business terms

Express risk with impact and likelihood, ideally in money or operational terms. A statement such as a ransomware event could stop order processing for about five days, costing roughly 1.5 million dollars, with a moderate likelihood this year, is easier to act on than a heat-map color. Use ranges and be clear about uncertainty instead of false precision. Frameworks such as FAIR support quantitative estimates, while simple tiers also work if applied consistently. Compare to risk appetite: show whether the current level is inside or outside what leaders agreed to accept, and which options reduce it.

Dashboards and structure

Use a concise structure: headline status, top risks, trends against targets, major incidents and lessons, progress on the roadmap, and the ask. A one-page dashboard with five to eight indicators and arrows for trend works better than forty charts. Lead with the conclusion, then the evidence. Show change over time, not isolated snapshots, and explain red items with the cause and the plan. Keep detailed appendices for those who want them. Be honest about bad news early, as surprises damage trust far more than a bad number. For incidents, give status, impact, what was done, and what will change.

Making the ask

Every report should end with a clear request, such as approve 180,000 dollars for offline backups and recovery testing, accept the risk of delaying an upgrade until Q3, or assign an executive owner for a vendor risk program. State the impact statement: the risk addressed, the reduction expected, the cost, the timeline, and what happens if the request is declined. Offer options and a recommendation. Record decisions and risk acceptances with an owner and review date, then follow up on results at the next meeting. Trust grows when you deliver what you promised and report outcomes, including misses.

EXECUTIVE ONE-PAGER OUTLINE

```
Status: AMBER (improving)
Top risk: Ransomware -> 5 days downtime, ~1.5M, likelihood medium
Trend: MTTR 14d -> 3d | Patch SLA 62% -> 88% | Admin MFA 100%
Incident: 1 phishing event, contained in 6h, no data loss
Ask: Approve 180k for offline backups + restore tests
If no: downtime risk remains outside appetite
```

WORKED EXAMPLE

At the quarterly board meeting, the Example Corp security lead has ten minutes. She opens with one sentence: the company is at amber, improving, and the largest exposure is ransomware stopping order processing for about five days. A single slide shows three trend lines and the one incident of the quarter, handled in six hours. She asks the board to approve 180,000 dollars for offline backups and restore testing, explaining that without it the risk stays above the stated appetite. The board approves and sets a review for next quarter.

KEY TERMS

Risk appetite: The level of risk leaders are willing to accept to meet business goals.

Impact statement: A short description of consequences and cost if a risk occurs or an ask is declined.

FAIR: Factor Analysis of Information Risk, a model for quantifying risk in financial terms.

Risk acceptance: A formal decision by an accountable owner to live with a risk.

Dashboard: A concise display of key indicators, trends and status.

Heat map: A grid that rates risks by likelihood and impact using colors.

REMEMBER

- Speak in business terms: impact, likelihood, cost and risk appetite.
- Use a concise dashboard with trends and honest status, leading with the conclusion.
- End with a clear ask, options and a recorded decision.

QUICK CHECK

1. How should a technical flaw be presented to a board?
 - A) With CVE numbers only
 - B) As business impact, likelihood, cost and a decision needed
 - C) With packet captures
 - D) With raw scanner output
2. What belongs at the end of a leadership report?
 - A) A list of tool names
 - B) A thank you only
 - C) An appendix of logs
 - D) A clear ask with impact if declined
3. Why show trends instead of single snapshots?
 - A) Direction over time shows whether risk is improving
 - B) Snapshots are forbidden
 - C) Trends are easier to fake
 - D) Boards dislike numbers
4. What should be done with a risk acceptance?
 - A) Hide it
 - B) Delete it after approval
 - C) Record it with an accountable owner and review date
 - D) Let IT sign anonymously

Answers: 1-B 2-D 3-A 4-C

1. Boards decide on business risk, not technical detail.
2. A request with consequences lets leaders decide.
3. Trends reveal progress and drift.
4. Documented ownership and review keep acceptance accountable.

Pick the right KPIs

SCENARIO

The board of Example Corp has asked for a one-page security report next month. The security manager has a list of twelve candidate metrics gathered from tools and teams. Some show real changes in risk, and others are activity counts that look impressive but cannot guide decisions. In the scenario simulator you must choose the six metrics that belong in the board report, discard the rest, and give a short reason for each decision.

OBJECTIVE

Select outcome metrics for a board report and discard vanity metrics.

STEPS IN THE SIMULATOR

- 1 **Do:** Read the list of twelve candidate metrics.

Simulator shows:

Items include MTTD, blocked connections, patch SLA compliance, alerts generated, MFA coverage and training hours.

- 2 **Do:** Mark 'Blocked firewall connections: 2.1 million' as discard.

Simulator shows:

Feedback: reflects automated noise, not risk change.

- 3 **Do:** Mark 'Median time to detect: 3 days' as keep.

Simulator shows:

Feedback: outcome metric with trend and target.

- 4 **Do:** Mark 'Critical patch SLA compliance: 88%' as keep.

Simulator shows:

Feedback: shows control performance against a defined target.

- 5 **Do:** Discard 'Total alerts generated' and 'Hours of training delivered'.

Simulator shows:

Feedback: activity measures with no link to outcome.

- 6 **Do:** Keep 'Admin MFA coverage', 'EDR coverage' and 'Restore test success rate'.

Simulator shows:

Feedback: coverage and resilience measures tied to ransomware risk.

- 7 **Do:** Submit the selection.

Simulator shows:

Score: 6 of 6 correct, with a note to add targets and owners to each metric.

ANSWER

Keep median time to detect, median time to contain, critical patch SLA compliance, admin MFA coverage, EDR coverage and restore test success. Discard blocked connections, alert counts, scan totals and training hours because they measure activity, not changes in risk.

Hint shown in the app: Ask whether the number would change a decision or show a change in risk.

Brief the executive

SCENARIO

The fictional chief financial officer of Example Corp messages you before a budget meeting. She asks how likely a ransomware breach is, what it would cost, and whether a requested 180,000 dollars for backups and recovery testing is justified. You are the security lead. In the chat simulator you must answer in business terms, avoid jargon, give a range with uncertainty, and finish with a clear ask and an impact statement that explains the consequence of refusing.

OBJECTIVE

Answer leadership questions on breach risk and request budget with a clear impact statement.

STEPS IN THE SIMULATOR

- 1 **Do:** Read the CFO's first message asking how worried she should be.

Simulator shows:

Chat offers three reply styles: technical, vague, business-focused.

- 2 **Do:** Choose the business-focused reply stating that ransomware could halt order processing for about five days.

Simulator shows:

CFO asks for a cost estimate.

- 3 **Do:** Give a range of 1.0 to 1.8 million dollars with a medium likelihood this year.

Simulator shows:

CFO accepts the range and asks what reduces it.

- 4 **Do:** Explain that offline backups and tested restores cut recovery from five days to about one.

Simulator shows:

CFO asks what the 180,000 buys.

- 5 **Do:** State the cost breakdown: storage, tooling, and two restore exercises per year.

Simulator shows:

CFO asks what happens if the request is declined.

- 6 **Do:** Deliver the impact statement: risk stays above appetite, with likely downtime of five days.

Simulator shows:

CFO asks for a recommendation and options.

- 7 **Do:** Offer minimum, recommended and full options and recommend the middle one.

Simulator shows:

CFO approves and asks for a quarterly progress report.

ANSWER

Describe the risk as about five days of downtime costing 1.0 to 1.8 million dollars with medium likelihood. Request 180,000 dollars for offline backups and restore tests, which cut recovery to about one day. If declined, the risk stays outside appetite. Offer options and recommend one.

Hint shown in the app: Use impact, likelihood, cost, a recommendation and the consequence of saying no.

Security Metrics and Program Management: 10 questions, 80% to pass

1. Which is an outcome metric?
 - A) Alerts generated
 - B) Median time to detect
 - C) Training hours delivered
 - D) Firewall rule count
2. What is MTTR?
 - A) Mean time to respond or recover after detection
 - B) Mean time to review rules
 - C) Maximum time to restart routers
 - D) Mean test request rate
3. Which is a vanity metric?
 - A) Critical patch SLA compliance
 - B) Restore test success rate
 - C) Admin MFA coverage
 - D) Number of blocked automated connections
4. What does Goodhart's law imply?
 - A) Metrics must be secret
 - B) Budgets must be fixed
 - C) Targeted measures can be gamed and lose value
 - D) All KPIs are meaningless
5. How should initiatives be ranked?
 - A) By popularity
 - B) By risk reduction relative to cost and effort
 - C) By newest technology
 - D) By team size
6. What is ALE?
 - A) SLE multiplied by annual rate of occurrence
 - B) Annual license fee
 - C) Alert log estimate
 - D) Asset lifecycle expense
7. How should maturity targets be set?
 - A) Always at the highest level
 - B) Copied from a vendor
 - C) Based on risk appetite and business need
 - D) Left undefined
8. How should risk be explained to a board?
 - A) With technical jargon
 - B) With raw logs
 - C) With tool screenshots
 - D) In impact, likelihood, cost and decisions
9. What should a leadership report end with?
 - A) A list of vendors
 - B) A clear ask with options and impact if declined
 - C) An apology
 - D) Raw data
10. What should a recorded risk acceptance include?
 - A) An accountable owner and a review date
 - B) No names
 - C) Only the vendor name
 - D) A technical log file

Answers: 1-B 2-A 3-D 4-C 5-B 6-A 7-C 8-D 9-B 10-A

1. It reflects security result over time.
2. It measures speed of containment or recovery.
3. Blocked probe counts do not show risk change.
4. Pair metrics with quality checks.
5. This ties spend to risk.
6. ALE gives expected yearly loss.
7. Not every area needs top maturity.
8. Boards need business terms.
9. A decision request makes the report actionable.
10. Accountability keeps accepted risk visible.

Capstone: Secure a Company

Advanced | 3 lessons | 2 simulated labs | checkpoint | about 2h 55m

This capstone brings the whole path together. You will act as an engineer reviewing a fictional company, Harbor Goods, from the outside in. You will threat model its architecture, find gaps in identity, network, cloud and data, and rank the findings by risk, effort and owner. Finally you will turn the plan into a 12-month roadmap with metrics and present it to leadership, including the risk that remains. These are the skills employers expect from a security engineer who can move from analysis to action.

BY THE END YOU CAN

- + Review an architecture for security gaps
- + Prioritize findings into a hardening plan
- + Present a risk-based roadmap

IN THIS MODULE

Lesson 1 Design review of the company

Lesson 2 Hardening plan and prioritization

Lesson 3 Roadmap and presentation

Lab 1 Review the architecture (Diagram builder)

Lab 2 Build the plan (Scenario)

Checkpoint 10 questions, 80% to pass

Design review of the company

A capstone starts by looking at the whole system the way an attacker and an auditor would, before anyone proposes a single fix.

Scope and threat model

A design review begins with scope. Our fictional company, Harbor Goods, sells outdoor gear through a web shop at shop.example.com, runs a small office network, stores customer data in a cloud database and employs 120 people. Draw a data flow diagram showing users, the web tier, the application, the database, the office, and the identity provider. Mark trust boundaries, the lines where data crosses from a less trusted zone to a more trusted one, such as internet to web tier. Then apply STRIDE to each element and flow: spoofing, tampering, repudiation, information disclosure, denial of service and elevation of privilege. The output is a list of plausible threats, not yet a list of fixes.

Identity gaps

Identity is usually the richest source of findings. At Harbor Goods, the review finds that multi-factor authentication covers only email, while the VPN and the cloud console accept passwords alone. Three administrators share one cloud root-level account. Former contractors still have active accounts, and a service account used by the build pipeline has an access key that never expires and holds administrator rights. Each gap maps to a known failure: weak authentication, no accountability, stale access and excessive privilege. Record the evidence for each finding, such as an export of users with last-login dates, so that the claim can be checked rather than argued.

Network and cloud gaps

On the network side the office uses one flat segment, so a compromised laptop can reach printers, file servers and the badge system. A firewall rule allows RDP on TCP 3389 from any address to a jump host. In the cloud, a storage bucket holding product exports is publicly readable, security groups allow SSH on TCP 22 from 0.0.0.0/0, and virtual machines still permit the older metadata service version that is vulnerable to request-forgery abuse. Outbound traffic is unrestricted, so malware could reach any server on the internet. Compare each setting with a baseline such as the CIS benchmarks for your cloud provider.

Data gaps

Data findings ask where sensitive information lives, how it is protected and whether it can be recovered. Harbor Goods has never classified its data, so payment-related logs, customer addresses and internal pricing sit in the same folders. Database snapshots are unencrypted, secrets such as API tokens appear in a code repository history, and nightly backups are stored in the same account as production, where a stolen admin key could delete them. Logging exists but is kept for only seven days. Note the regulatory angle too: card data brings PCI DSS duties, and customer personal data may bring privacy law obligations.

FINDING RECORD FORMAT

```
ID: F-07
Area: Identity
Finding: Cloud admin account shared by 3 people
Evidence: IAM export 2026-03-02, 1 user, 3 owners
Threat: Spoofing / no accountability
Impact: High Likelihood: Medium
Tag: access-control, cloud
```

WORKED EXAMPLE

A reviewer walks through the Harbor Goods diagram with the engineering lead. At the arrow from the internet to the web tier she asks what authenticates the caller, and at the arrow from the web tier to the database she asks what limits the queries. The second answer is that the application connects as a database owner. She writes two findings: excessive privilege on the database account, and a public bucket seen in the cloud inventory. Each finding carries evidence, an affected asset and a STRIDE category, ready for the prioritization step.

KEY TERMS

Trust boundary: A line in a design where data moves between zones of different trust and controls must be enforced.

STRIDE: A threat categorization: spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege.

Data flow diagram: A drawing of processes, stores, actors and the data moving between them, used for threat modeling.

Least privilege: Giving an account only the access it needs for its task.

Flat network: A network with little or no segmentation, letting one compromised host reach many others.

Baseline: An agreed secure configuration used as a yardstick, such as a CIS benchmark.

REMEMBER

- Start with scope, a data flow diagram and trust boundaries.
- Walk identity, network, cloud and data areas looking for gaps against a baseline.
- Record every finding with evidence, asset, threat and impact.

QUICK CHECK

1. What is the main purpose of marking trust boundaries on a data flow diagram?
 - A) To show where controls must be enforced as data changes trust level
 - B) To count network cables
 - C) To choose the backup schedule
 - D) To list every employee
2. Which finding best matches the STRIDE category of elevation of privilege?
 - A) A missing company logo
 - B) A web page that loads slowly
 - C) A pipeline service account holding administrator rights it does not need
 - D) A printer out of toner
3. Why should each finding include evidence?
 - A) Because auditors forbid opinions
 - B) To make the report longer
 - C) So the claim can be verified and not just argued
 - D) To avoid assigning owners
4. A bucket holding exports is publicly readable. Which goal is most directly at risk?
 - A) Confidentiality
 - B) Availability of the printers
 - C) Password length
 - D) Physical safety

Answers: 1-A 2-C 3-C 4-A

1. Boundaries identify points where authentication, validation and filtering are needed.
2. Excess rights let a compromised account do far more than intended.
3. Evidence such as exports and screenshots makes findings credible and reproducible.
4. Public read access exposes data to anyone, which is a confidentiality failure.

Hardening plan and prioritization

A long list of findings helps nobody until it is ranked, owned and scheduled against real budget and staff limits.

Scoring findings

Turn each finding into a risk by judging impact and likelihood. A simple three-level scale, low, medium and high, is often enough, while larger programs use CVSS for vulnerabilities and a numeric matrix such as 1 to 5 on each axis. Impact asks what is lost if it happens: money, regulated data, uptime or reputation. Likelihood asks how easy it is to exploit and whether the weakness is exposed to the internet. A public bucket with customer exports is high impact and high likelihood. A missing sticker on a server room door is neither. Write the reasoning down so reviewers can challenge the score.

Effort, owners and quick wins

Next add effort and an owner. Effort can be hours, days or quarters, and an owner is a named person or team accountable for delivery, not a vague department. The most valuable group is quick wins: high risk reduction for low effort. At Harbor Goods these include closing public access on the bucket, restricting SSH and RDP to known addresses such as 203.0.113.10, enforcing multi-factor authentication on the VPN and cloud console, and disabling stale accounts. Larger items, such as network segmentation or a new logging platform, need projects with milestones and dependencies, so they are scheduled rather than rushed.

Sequencing under constraints

Sequence work by risk reduction per unit of cost, then adjust for dependencies. You cannot detect misuse well without logging, and you cannot segment safely before you know which systems talk to each other. Also consider controls that cover several findings at once: a central identity provider with MFA fixes weak authentication, shared accounts and stale access together. Budget and staff are fixed, so something will be deferred. State the trade-off openly: for example, defer the new endpoint tool a quarter in order to fund immutable backups that protect against ransomware, the threat with the largest impact.

Defense in depth and accepting risk

A good plan layers controls so that one failure is not fatal: prevent with hardening, detect with logging and alerts, and recover with tested backups. Some risks will not be fixed. Leadership may accept, transfer or avoid a risk, but only through a documented decision with an owner and an expiry date for review. A risk accepted silently is just an unmanaged risk. Plans should also define how to verify each fix, such as a repeat scan or configuration check, so that a ticket closed means a weakness truly removed.

PRIORITIZED PLAN EXCERPT

Rank	Risk	Effort	Owner	Item
1	High	2 h	CloudOps	Block public access on bucket
2	High	1 d	NetOps	Limit SSH/RDP to 203.0.113.10
3	High	3 d	IT	MFA on VPN and console
4	High	2 w	Platform	Immutable offsite backups
5	Med	1 q	NetOps	Segment office network

WORKED EXAMPLE

The Harbor Goods team has a budget for three projects this quarter. The findings list has twenty-two items. The reviewer groups them: four quick wins that need under a week in total, three medium projects, and several long ones. She funds all quick wins first, then selects immutable backups and central MFA because they reduce the most risk across several findings. Network segmentation is scheduled for the next quarter, with its dependency, an asset inventory, started now. The deferral and its reason are written into the plan.

KEY TERMS

Risk: The combination of how likely a harmful event is and how bad its impact would be.

Quick win: A fix with large risk reduction for little effort.

Risk acceptance: A documented decision to live with a risk, with an owner and review date.

Dependency: Work that must be done before another task can safely start.

Residual risk: The risk that remains after controls are applied.

CVSS: A standard scoring system for the severity of vulnerabilities.

REMEMBER

- Score by impact and likelihood and record the reasoning.
- Add effort and owner, fund quick wins first, then high-leverage projects.
- Document deferred items and accepted risks and verify each fix.

QUICK CHECK

1. Which finding is the best quick win?

- A) Blocking public access on a bucket holding customer exports
- B) Replacing the whole network
- C) Buying a new SIEM
- D) Writing a five-year strategy

2. Why does a plan need a named owner for each item?

- A) Owners are required by TCP
- B) Owners set the risk scores
- C) To lengthen the document
- D) So someone is accountable for delivery

3. Why might logging be scheduled before segmentation?

- A) Segmentation is illegal
- B) Segmentation depends on knowing which systems communicate, which logs help reveal
- C) Logging removes vulnerabilities
- D) Logs are cheaper than firewalls in every case

4. What makes a risk acceptance valid?

- A) Closing the ticket without a fix
- B) A verbal comment in chat
- C) A documented decision with owner and review date
- D) An engineer ignoring the ticket

Answers: 1-A 2-D 3-B 4-C

1. It cuts a high risk in hours with little cost.
2. Without accountable owners, items stall.
3. Dependencies shape sequence; visibility informs safe segmentation.
4. Formal acceptance keeps the risk visible and reviewed.

Roadmap and presentation

Findings only change a company when leadership understands them, so the final skill is communicating risk, progress and a realistic timeline.

Know the audience

Executives, engineers and auditors need different views of the same plan. Leaders want to know what could hurt the business, what you recommend, what it costs and what decision you need. Open with a one-page summary: top three risks in business terms, such as 'a stolen admin password could let an attacker delete our backups and halt sales for days', then the recommended spend. Avoid unexplained acronyms. Engineers receive the detailed finding records and technical steps. Auditors want evidence and control mapping, for example to ISO 27001 or the NIST Cybersecurity Framework functions of identify, protect, detect, respond and recover.

Metrics that mean something

Choose a small set of metrics that show risk falling, not just activity. Useful examples are the percentage of accounts protected by MFA, the count of internet-exposed administrative ports, the share of critical vulnerabilities patched within 14 days, mean time to detect and respond, and the proportion of backups tested by a successful restore. Give each a baseline and target, for instance MFA coverage from 40 percent to 100 percent. Avoid vanity numbers such as the count of blocked scans, which rise with internet noise and say little about safety. Report trends quarterly so the board sees direction.

The 12-month roadmap

A roadmap groups work in time horizons. Months one to three hold quick wins and urgent fixes: exposed storage, remote-access hardening, MFA and admin cleanup. Months four to six deliver foundations such as central logging with alerts, immutable backups, and an asset inventory. Months seven to nine address structural change, including network segmentation and secrets management. Months ten to twelve cover maturity: an incident response exercise, a repeat of the design review and a penetration test. Show dependencies, owners and milestones, and keep a reserve for surprises, because new findings will arrive during the year.

Residual risk and honest limits

End by stating what remains. After the roadmap, Harbor Goods will still face phishing, supplier compromise and unknown flaws. Present residual risk plainly, with the controls that limit its impact and the decisions leaders must accept. Anticipate questions: why this order, what if budget is cut, how will we know it worked. Prepare a fallback ranking if funds shrink. Close with the ask, a date for the first review and the metrics you will report. A credible presentation neither promises zero risk nor frightens people into inaction; it shows a managed path.

METRIC TABLE SAMPLE

Metric	Now	Target(12m)
MFA coverage (all users)	40%	100%
Exposed admin ports	4	0
Critical vulns patched in 14d	55%	95%
Backup restores tested/qtr	0	4
Mean time to detect	9 d 1 d	

WORKED EXAMPLE

The Harbor Goods review lead presents for ten minutes to the leadership team. Slide one states that a single stolen password could halt sales and names three risks in plain words. Slide two shows the roadmap in four quarters with the quick wins in the first. Slide three shows five metrics with baselines and targets. The final slide lists residual risks, one accepted risk with an owner, and the request to approve the budget by Friday. The finance director asks what is cut if funds fall short, and the fallback list is ready.

KEY TERMS

Roadmap: A time-phased plan showing what will be delivered, in what order and by whom.

Residual risk: Risk remaining after planned controls are in place.

Mean time to detect: The average time between an incident beginning and its discovery.

NIST CSF: A framework organizing security outcomes into identify, protect, detect, respond and recover.

Baseline metric: The starting measurement used to show improvement.

Vanity metric: A number that looks impressive but does not reflect real risk reduction.

REMEMBER

- Tailor the message to leaders, engineers and auditors.
- Use baseline-and-target metrics that show falling risk.
- Present a phased 12-month roadmap, residual risk and a clear ask.

QUICK CHECK

1. What should an executive summary lead with?
 - A) Top risks in business terms and the decision needed
 - B) Every CVE number
 - C) A list of tools
 - D) Raw port scan output
2. Which is the strongest metric of improvement?
 - A) Number of scans blocked
 - B) Pages in the report
 - C) Number of meetings held
 - D) MFA coverage rising from 40 to 100 percent
3. Where would segmentation likely appear on the roadmap?
 - A) Months seven to nine as structural change after inventory and logging
 - B) Never
 - C) After the audit ends
 - D) Only in month one
4. Why present residual risk?
 - A) It reduces budget
 - B) It cancels the plan
 - C) It shows honestly what remains and what leaders accept
 - D) It replaces metrics

Answers: 1-A 2-D 3-A 4-C

1. Leaders need impact, recommendation and ask.
2. It measures an actual control and has a baseline and target.
3. It depends on earlier visibility work and needs more time.
4. No plan removes all risk; transparency builds trust.

Review the architecture

SCENARIO

You are given the Harbor Goods architecture diagram: internet, web tier, application servers, a cloud database, an office network with a VPN, a backup store and an identity provider. Several components hide weaknesses such as public exposure, shared accounts and missing segmentation. Your task is to inspect each component and flag problems directly on the diagram, then tag each flag with a risk level so that the later planning exercise has a ranked starting list.

OBJECTIVE

Mark at least six weaknesses on the diagram and tag each as high, medium or low risk.

STEPS IN THE SIMULATOR

- 1 **Do:** Open the diagram and list its components.

Simulator shows:

Nine nodes and the trust boundaries appear.

- 2 **Do:** Select the storage bucket node and read its settings.

Simulator shows:

Public read access is enabled.

- 3 **Do:** Add a weakness marker on the bucket with category data.

Simulator shows:

A marker appears and asks for a risk tag.

- 4 **Do:** Tag the bucket marker high.

Simulator shows:

The marker turns red.

- 5 **Do:** Inspect the VPN node and the identity provider.

Simulator shows:

VPN accepts password only; MFA covers email only.

- 6 **Do:** Mark the VPN as identity weakness, tag high.

Simulator shows:

A second red marker appears.

- 7 **Do:** Mark the flat office segment as medium and unencrypted snapshots as medium.

Simulator shows:

Two amber markers appear.

- 8 **Do:** Submit the diagram.

Simulator shows:

Score shows markers matched against the hidden weakness list.

ANSWER

High: public bucket, password-only VPN and cloud console, backups in production account. Medium: flat office network, unencrypted snapshots, short log retention. Low: missing asset labels. Tag by impact and exposure, with internet-facing data and authentication weaknesses ranked highest.

Hint shown in the app: Look first at anything reachable from the internet and at every login that lacks MFA.

Build the plan

SCENARIO

Harbor Goods has a fixed budget of ten work units this quarter. A list of remediation items is offered, each with cost, risk reduction and dependencies: bucket lockdown, SSH and RDP restriction, MFA rollout, immutable backups, network segmentation, central logging and a new endpoint tool. You must choose and order items within budget and write a short justification for what is deferred. The simulator scores risk reduced and the quality of your reasoning.

OBJECTIVE

Select and sequence remediation items within ten units and justify the deferred items.

STEPS IN THE SIMULATOR

- 1 Do:** Read the item list with costs and risk reductions.

Simulator shows:

Seven items show cost between 1 and 5 units.

- 2 Do:** Select bucket lockdown (1 unit).

Simulator shows:

Budget used: 1 of 10.

- 3 Do:** Select SSH and RDP restriction (1 unit).

Simulator shows:

Budget used: 2 of 10.

- 4 Do:** Select MFA rollout (2 units).

Simulator shows:

Budget used: 4 of 10; identity risk bar drops.

- 5 Do:** Select immutable backups (3 units).

Simulator shows:

Budget used: 7 of 10; ransomware impact drops.

- 6 Do:** Try adding segmentation (5 units).

Simulator shows:

Warning: over budget and depends on an inventory.

- 7 Do:** Select central logging (3 units) instead.

Simulator shows:

Budget used: 10 of 10.

- 8 Do:** Write the justification for deferring segmentation and the endpoint tool.

Simulator shows:

Plan accepted; score shown.

ANSWER

Fund the four quick wins and immutable backups, then logging, for ten units. Defer segmentation, which depends on an inventory and exceeds budget, and the endpoint tool, which has lower risk reduction. Justify by risk reduction per cost and dependencies, and schedule deferred items for next quarter.

Hint shown in the app: Fund cheap high-risk fixes first, then pick the large item that reduces the worst impact.

Capstone: Secure a Company: 10 questions, 80% to pass

1. In a design review, what is the first step before listing fixes?
 - A) Run an exploit
 - B) Rewrite policies
 - C) Define scope and draw the data flow with trust boundaries
 - D) Buy security tools
2. A review finds three admins sharing one cloud account. Which problems does this cause?
 - A) Faster backups
 - B) Lower attack surface
 - C) No accountability and weak access control
 - D) Better logging
3. Which firewall rule is the most concerning on a jump host?
 - A) RDP 3389 allowed from any address
 - B) SSH allowed from 203.0.113.10
 - C) HTTPS from the load balancer
 - D) DNS to the resolver
4. Which backup design best resists ransomware?
 - A) Immutable copies stored offsite in a separate account
 - B) Backups with no restore tests
 - C) One copy on a shared drive
 - D) Backups on the same server
5. How should risk be scored in a simple plan?
 - A) By alphabetical order
 - B) Impact and likelihood with written reasoning
 - C) By the length of the finding
 - D) By who found it
6. What defines a quick win?
 - A) Lowest cost regardless of value
 - B) High risk reduction for low effort
 - C) Items finished last
 - D) Any item with a vendor
7. Why should a risk acceptance have an expiry date?
 - A) So it can be forgotten
 - B) So it is reviewed again as conditions change
 - C) To raise the risk score
 - D) To satisfy a network protocol
8. Which metric best shows reduced exposure?
 - A) Pages of policy
 - B) Number of staff trained in a year only
 - C) Count of internet-exposed administrative ports falling to zero
 - D) Total alerts generated
9. What belongs in an executive summary?
 - A) Business-language risks, recommendation, cost and decision needed
 - B) Source code
 - C) Every ticket
 - D) Packet captures
10. Why present residual risk at the end of a roadmap?
 - A) No plan removes all risk and leaders must understand what remains
 - B) To hide findings
 - C) It is optional vanity
 - D) To undo the plan

Answers: 1-C 2-C 3-A 4-A 5-B 6-B 7-B 8-C 9-A 10-A

1. Scope and a model of the system make findings systematic.
2. Shared accounts cannot be attributed to a person.
3. Open RDP from anywhere is a common attack path.
4. Separation and immutability stop an attacker deleting copies.
5. Both factors are needed and reasoning allows challenge.
6. Quick wins offer the best return early.
7. Conditions change and accepted risk must be revisited.
8. It measures a concrete exposure.
9. Leaders need clear decisions.
10. Honest residual risk supports informed decisions.